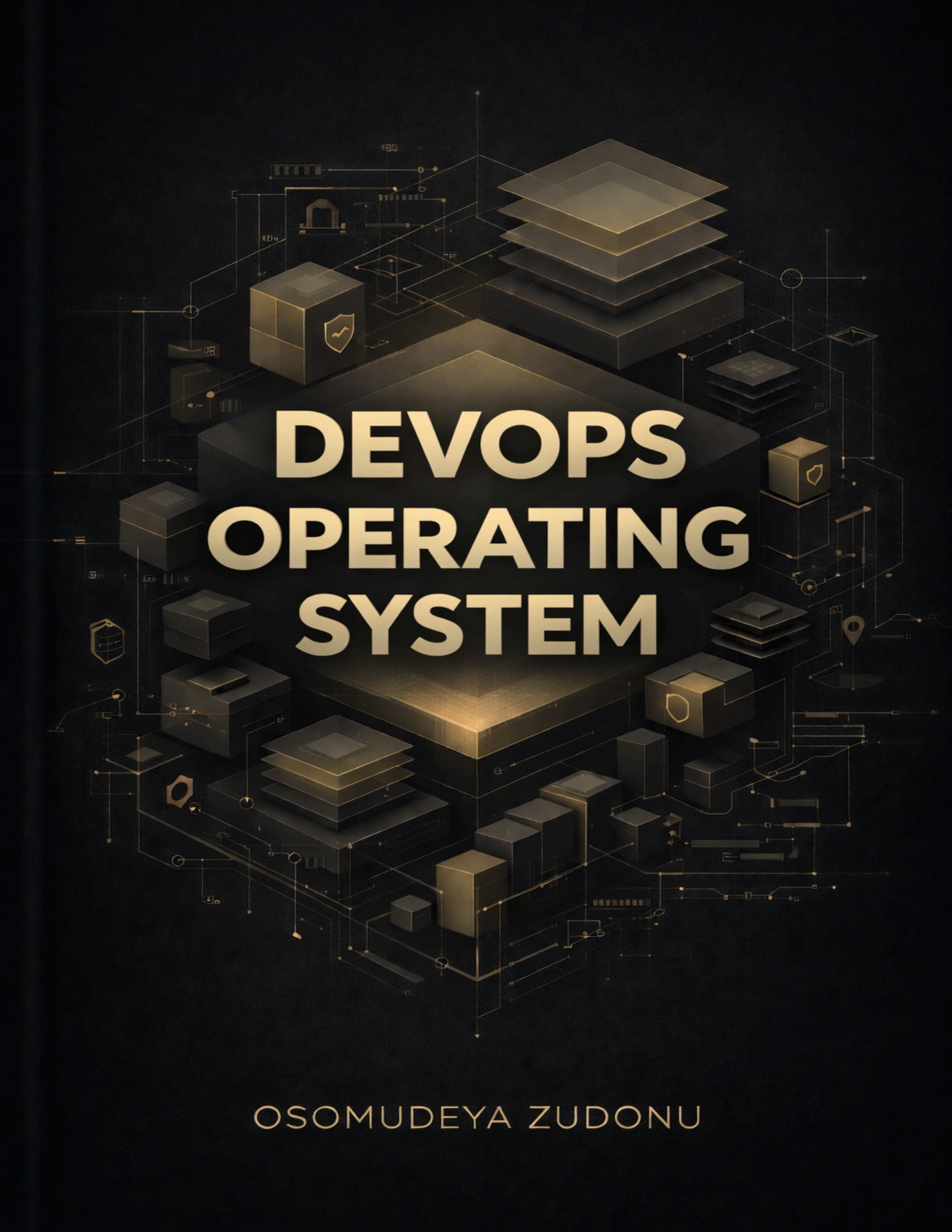




DEVOPS OPERATING SYSTEM

OSOMUDEYA ZUDONU

Before You Begin

Code Disclaimer

This ebook contains codes, Terraform configurations, and deployment scripts. While tested and verified at the time of writing:

- **Version differences:** may cause compatibility issues
- **AWS service changes:** may require configuration updates
- **Your environment:** may have different constraints or permissions
- **Regional differences:** may affect service availability

If code doesn't work:

1. Check the error message - it usually tells you what's wrong
2. Verify versions match (or update to latest stable)
3. Review AWS documentation for service-specific changes
4. Check your AWS account limits and permissions
5. Adapt the code to your specific needs

The goal is learning, not copy-paste: These examples teach concepts. Understanding WHY something works is more valuable than having it work perfectly on the first try.

SECTION 5

Terraform & Infrastructure as Code

You've spent weeks building Humor-Game on a single EC2 instance with MicroK8s. You've learned Kubernetes fundamentals, deployed databases, configured caching, and built a complete three-tier application.

But there's a problem.

Your entire application runs on one server. If that EC2 instance or your local machine goes off or fails say at night time, everything goes down. There's no automatic failover, no geographic distribution, no managed backups. You're manually managing PostgreSQL and Redis when AWS offers managed versions that handle backups, failover, and scaling automatically.

You've learned how to think. Now it's time to build like a pro.

This section transitions you from "learning infrastructure" to "production infrastructure." You'll use Terraform to provision a complete AWS environment:

- **EKS (Elastic Kubernetes Service)** - Managed Kubernetes replacing MicroK8s
- **RDS (Relational Database Service)** - Managed PostgreSQL replacing your StatefulSet
- **ElastiCache** - Managed Redis replacing your Deployment
- **VPC, Subnets, Security Groups** - Enterprise networking
- **Application Load Balancer** - Production-grade traffic routing
- **ECR (Elastic Container Registry)** - Private Docker registry
- **Auto Scaling Groups** - Nodes scale based on demand

All defined as code. All reproducible. All version controlled.

Prerequisites

Before starting Section 5, ensure you have:

Required Skills:

- ☐ Completed Section 4 (K1-K9) - Kubernetes fundamentals
- ☐ AWS account created and configured
- ☐ Basic Linux command-line knowledge

Required Tools:

- ☐ AWS CLI installed and configured
- ☐ Terraform installed (version 1.0+)
- ☐ kubectl installed
- ☐ Git installed

Recommended Tools:

- IDE/Code Editor (VS Code, Cursor, or similar)
 - Makes writing Terraform code much easier
 - Syntax highlighting, auto-completion, error detection
 - See [5.0-terraform-fundamentals.md](#) for IDE setup guide

Note: Tasks show `cat > file.tf` commands for compatibility, but you can use your IDE instead! See the "Choosing Your Editor" section in Task 5.0 for details.

Remote State Configuration

This section uses remote state from the start (configured in T1 Step 0).

Benefits:

- Production-ready from day 1
- Team collaboration enabled
- State backup and recovery
- Required for bootcamp-level competency

What is remote state?

Think of Terraform state as a notebook that records everything Terraform has created for you: servers, networks, databases, and their exact settings. If that notebook lives only on your laptop, you're the only one who has the latest copy. If a teammate runs Terraform from their own laptop, they might use an outdated copy and accidentally overwrite or break existing infrastructure.

By storing the state in S3, that notebook is kept in a shared, central location in the cloud. Everyone on the team reads and writes to the same up-to-date record, so Terraform always knows the real current state of your infrastructure. This also protects you if your laptop crashes, because the state is safely stored remotely.

The DynamoDB table acts like a lock on that notebook. When one person is making changes, Terraform "locks" the state so nobody else can edit it at the same time. This prevents two people from applying changes simultaneously and corrupting the state.

The result is that every infrastructure change is recorded in one reliable place, and multiple team members can safely manage the same infrastructure without stepping on each other's changes.

Cost: ~\$0.50/month (S3 storage + DynamoDB Pay-per-request)

Setup: Configured in T1 Step 0 (before creating any infrastructure)

Note: If you want to use local state for learning (not recommended), you can skip T1 Step 0. However, this is not production-ready and prevents team collaboration.

Let's Talk About Cost Before You Start

A full, always running production setup on AWS would normally cost around **\$183 per month**. That's the "leave everything on **24/7** like a real company" scenario.

But for learning, you don't need to run everything all the time.

With some smart optimizations:

- Use Spot instances to cut compute cost in half
- Shut everything down at night and on when you are done
- Pick the smallest instance sizes that still work
- Take advantage of the AWS free tier credits on new accounts

your real cost for a 4-week learning period drops to about **\$80–120 total**, which is roughly **\$3–4 per day**.

How the Weekly Spend Looks

You don't pay everything at once. As you add more components each week, the cost grows gradually:

- **Week 1:** about \$20 for networking and base setup
- **Week 2:** about \$35 when the EKS control plane is created
- **Week 3:** about \$30 after adding the database and cache
- **Week 4:** about \$25 for load balancer and container registry
- **Final optimization week:** about \$15

That brings the whole section to around **\$80–120**.

Where the Money Actually Goes

The main fixed cost is the managed Kubernetes control plane. AWS runs and maintains the Kubernetes brain for you, so you don't manage it yourself.

Worker nodes (the actual machines running your pods) are the next big part, but using Spot instances cuts this roughly in half. If you only run them while you are actively learning, the weekly cost drops even more.

A small PostgreSQL database and a small Redis cache are inexpensive in single-AZ mode, which is perfectly fine for learning. Multi-AZ high availability is skipped because that doubles the price and isn't necessary for practice.

The load balancer has a small fixed monthly fee plus tiny data transfer costs.

The NAT gateway can be expensive if you create one per availability zone, so for learning you use a single NAT gateway instead.

Remote Terraform state stored in S3 and locked with DynamoDB costs almost nothing because the files are tiny and access is minimal.

The Key Idea: Only Pay While You're Using It

You are not forced to keep the infrastructure running all day. When you finish studying for the day, you can delete everything. When you start again, you recreate it.

Destroying and recreating takes a few minutes, but saves most of the cost.

If everything is destroyed, your running cost is effectively **zero per hour**.

Extra Cost-Saving Choices

- One NAT gateway instead of three
- Single-AZ database and cache instead of multi-AZ
- Spot instances for worker nodes
- Turn everything off when you're done for the day

These changes are what bring the bill down from \$183/month to about \$100 for the whole learning period.

Is It Worth Spending Around \$100?

What you get in return is hands-on experience with:

- Terraform
- AWS production-style infrastructure
- EKS, VPC, RDS and networking
- real Infrastructure as Code workflows

These are exactly the keywords and skills most DevOps job descriptions ask for.

Spending about \$100 to gain skills that can increase your salary by thousands is an extremely high return on investment. One job offer would cover this cost many times over.

How You'll Run This

All Terraform commands run from your own laptop. Terraform talks to AWS and creates resources there.

This setup is separate from any previous EC2 or MicroK8s labs. You are building fresh, real AWS infrastructure from scratch.

You Stay in Control of the Bill

At any time you can check your usage and delete everything.

When you destroy the infrastructure, nothing is running and you stop paying.

You decide when it exists and when it doesn't. If that sounds reasonable, the next step is simple: start building real production-style infrastructure and turn it on only when you need it.

The One Thing That Makes This Click

Infrastructure as Code isn't about learning AWS Console. It's about making infrastructure reproducible.

MANUAL THINKING:

- "I need infrastructure"
- "I'll click through AWS Console"
- "Hope I remember what I clicked"
- "If it breaks... panic and guess"

TERRAFORM THINKING:

- "I need infrastructure"
- "I'll write code describing what I want"
- "Git tracks every change I make"
- "If it breaks: terraform apply (done)"

That's it. **That's Infrastructure as Code.**

Everything else (syntax, providers, state files) is just details. The core concept is:

"If it's not in code, it doesn't exist."

The Mental Model Shift:

BEFORE Section 5:

- "Infrastructure = clicking buttons in AWS Console"
- "My EC2 instance is a unique snowflake"
- "If it dies, I manually rebuild and hope I remember everything"
- "No one else can recreate my exact setup"

AFTER Section 5:

- "Infrastructure = .tf files in Git repository"
- "My setup is 100% reproducible from code"
- "If it dies: terraform apply (recreated in 15 minutes)"
- "Anyone can recreate my exact setup: git clone + terraform apply"

This is the difference between "systems admin" and "DevOps engineer."

Example:

Scenario: Production database deleted by accident

Without Terraform:

- |— "What was the configuration?"
- |— "Which subnet was it in?"
- |— "What security groups did it have?"
- |— "What were the backup settings?"
- |— Try to remember from memory
- |— Takes hours
- |— Probably not identical to before

With Terraform:

- |— `git log database.tf` (see exact configuration)
- |— `terraform apply` (recreate identically)
- |— Takes 15 minutes
- |— 100% identical to before

Now let's see how Terraform works...

Bridge from K9: Why Terraform Now?

Remember K9's core concept? **Desired state.**

Kubernetes (K1-K9):

```
You write: deployment.yaml (desired state)
You run: kubectl apply
Kubernetes ensures: 3 Pods always running
If Pod crashes: Kubernetes recreates it

Control loop: Watch → Compare → Act → Repeat
```

Terraform (Section 5):

```
You write: main.tf (desired state)
You run: terraform apply
AWS ensures: Infrastructure matches your code
If you change code: Terraform updates AWS

Control loop: Plan → Compare → Apply → Track
```

Same pattern. Different layer.

LAYER 3: Applications (Your code)

- └ Frontend (React)
- └ Backend (Node.js)
- └ Business logic

LAYER 2: Platform (Kubernetes - K1-K9)

- └ Pods, Services, Deployments
- └ ConfigMaps, Secrets
- └ Ingress, HPA
- └ kubectl apply = desired state for apps

LAYER 1: Infrastructure (Terraform - Section 5)

- └ VPC, EKS, RDS, Subnets
- └ Security Groups, Load Balancers
- └ EC2, ElastiCache, ECR
- └ terraform apply = desired state for infra

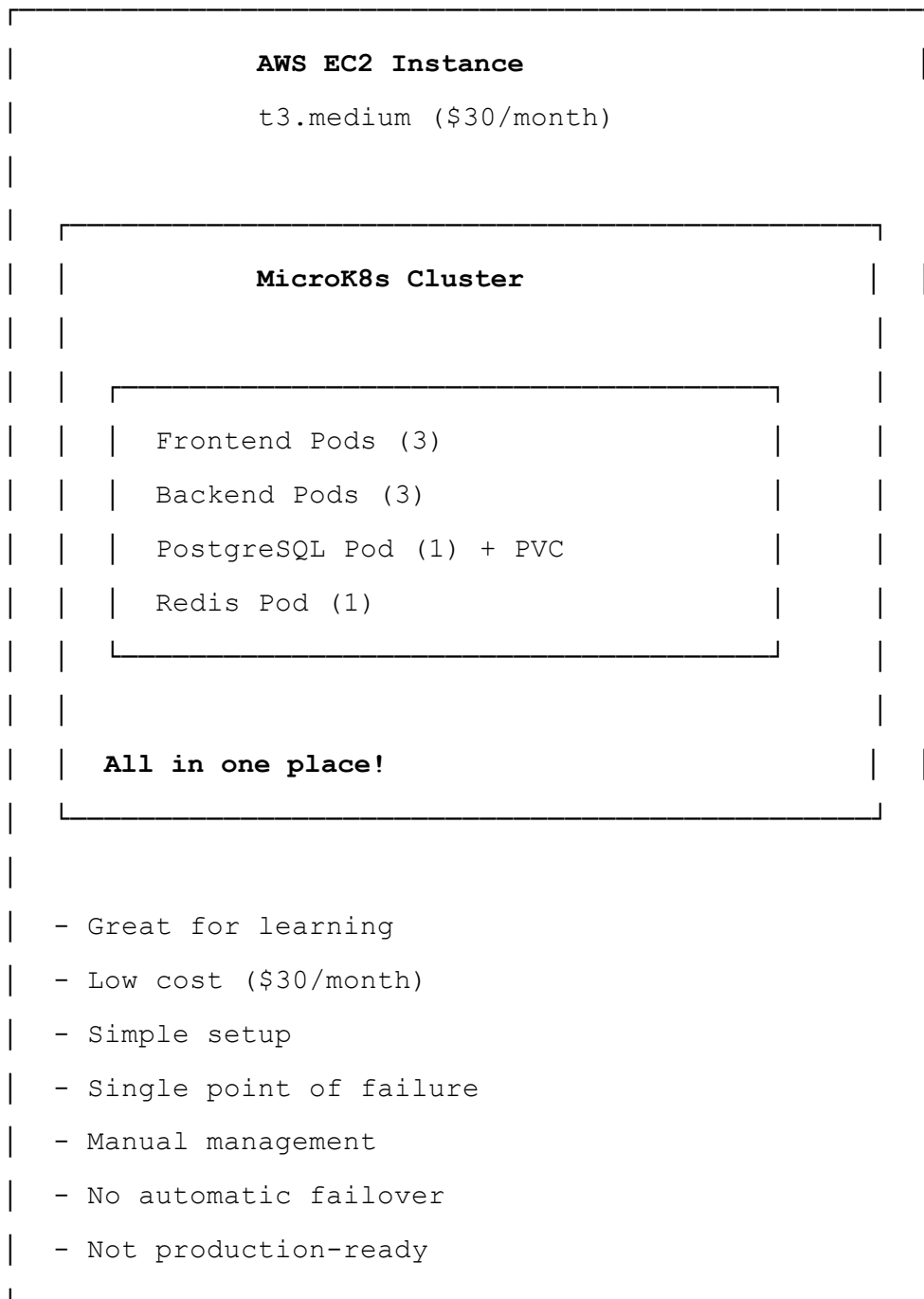
You already understand desired state management from K9.

Section 5 applies the same concept to infrastructure instead of applications.

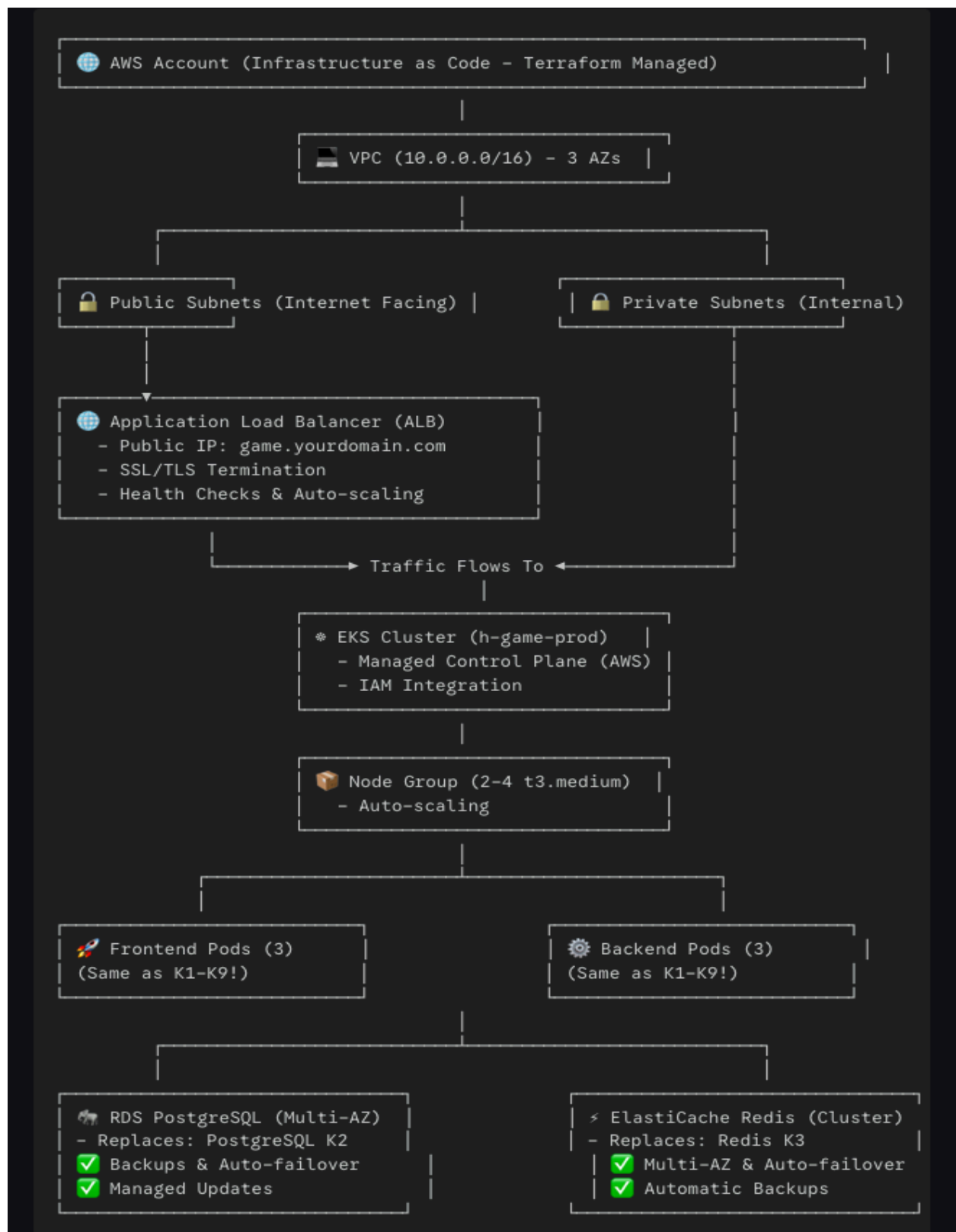
Kubernetes maintains your Pods. Terraform maintains your VPCs.

What Changes (Visual Comparison)

Your Current Setup (Weeks 1-9):



Production Setup (After Section 5):



Key Benefits (Production-Grade Architecture):

High Availability: Multi-AZ for all critical components.

Managed Services: AWS handles operations for EKS, RDS, and ElastiCache.

Auto-Scaling: Nodes and ALB scale based on load.

IaC/DR: Terraform for reproducibility and disaster recovery.

Learning cost: ~\$100 for 4 weeks (with optimizations)

Production cost: ~\$180/month (or less with Reserved Instances)

What's different:

Component	K1-K9 (Learning)	Section 5 (Production)
Kubernetes	MicroK8s (self-managed)	EKS (AWS managed)
Database	PostgreSQL StatefulSet	RDS (managed, Multi-AZ)
Cache	Redis Deployment	ElastiCache (managed)
Networking	Default EC2 network subnets	VPC with public/private subnets
Load Balancer	Ingress (local)	ALB (AWS managed)
Registry	localhost:32000	ECR (private, secure)
Scaling	Manual/HPA	Auto Scaling Groups
Backups	Manual pg_dump	Automated (RDS, ElastiCache)
Failover	None	Automatic (Multi-AZ)
Cost	\$30/month	\$100/month (learning), \$180 (prod)

What stays the same:

- Your K1-K9 Kubernetes manifests (no changes needed!)
- kubectl commands (same as before)
- Application code (no changes)
- Docker images (same containers)
- ConfigMaps, Secrets (same configuration)

Only the infrastructure underneath changes.

What is Terraform?

Terraform is a tool that lets you **write down your cloud setup instead of clicking around dashboards/consoles**.

It builds servers, networks, and databases **the same way every time** from that description.

Without Terraform, people rely on memory and clicks, which leads to mistakes and differences between environments.

With Terraform, your infrastructure becomes a **repeatable recipe**, like following the same pizza recipe every time.

If something breaks, you can **rebuild everything fast and correctly** from the same instructions.



The Problem it solves: Manual Infrastructure

Imagine setting up AWS manually:

Day 1: You click around the AWS console

- └ Create VPC (10 clicks)
- └ Create subnets (30 clicks, 6 subnets)
- └ Create Internet Gateway (5 clicks)
- └ Create NAT Gateways (15 clicks, 3 gateways)
- └ Create route tables (20 clicks)
- └ Create security groups (50 clicks for rules)
- └ Create EKS cluster (30 clicks, wait 15 minutes)
- └ Configure IAM roles (40 clicks)
- └ Set up RDS (25 clicks, 20 configuration options)
- └ Set up ElastiCache (20 clicks)
- └ Create ALB (25 clicks)
- └ Takes 4-6 hours of clicking (and you're not sure it's right)

Day 30: Your boss says, "Build staging environment"

- └ Try to remember what you clicked 30 days ago
- └ Forget something critical (different security group rule)
- └ Staging is slightly different from production (configuration drift)
- └ Debug for hours why staging doesn't work
- └ Takes 5-8 hours (and you're not confident it matches prod)

Day 60: Disaster! Production VPC deleted by accident

- └ How was it configured?
- └ What were the CIDR blocks?
- └ Which subnets were public vs private?
- └ What security group rules existed?
- └ You can't remember. Data might be gone.
- └ Panic. Resume. LinkedIn.

The Solution: Infrastructure as Code

With Terraform:

```
# infrastructure/main.tf (just text in a file!)

# VPC - Your private network in AWS
resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16" # IP address range for your network

  tags = {
    Name       = "h-game-vpc"
    Environment = "production"
  }
}

# Public Subnet - Can reach the internet directly
resource "aws_subnet" "public_1" {
  vpc_id            = aws_vpc.main.id # Belongs to VPC above
  cidr_block        = "10.0.1.0/24"   # Subset of VPC IP range
  availability_zone = "us-east-1a"    # Physical data center location

  tags = {
    Name = "h-game-public-1"
  }
}

# EKS Cluster - Managed Kubernetes
resource "aws_eks_cluster" "main" {
  name      = "h-game-prod"
  role_arn = aws_iam_role.eks_cluster.arn # Permissions for cluster
  version  = "1.28" # Kubernetes version

  # Network configuration
  vpc_config {
    subnet_ids = [
      aws_subnet.private_1.id, # Private subnet 1
      aws_subnet.private_2.id, # Private subnet 2
      aws_subnet.private_3.id, # Private subnet 3
    ]
  }
}

# ... (more configuration in code)
```

Now:

```
# Day 1: Create production
terraform apply
# Everything created in 15 minutes
#  Exactly as defined in code

# Day 30: Create staging (exact copy)
terraform workspace new staging
terraform apply -var="environment=staging"
# Perfect replica in 15 minutes
# Same code, different variables

# Day 60: Disaster recovery
git log main.tf # See exact configuration
terraform apply
# Restore everything exactly as it was
# 15 minutes from disaster to recovery

# Day 90: What's in production?
cat main.tf
# Every resource documented in code
# Git history shows who changed what and when

# Day 120: Audit for compliance
git log --all -- security-groups.tf
# Complete audit trail of all security changes
# Know exactly who opened port 22 and when
```

The Restaurant Analogy

Manual AWS Console = Cooking Without a Recipe:

You make pizza:

- Try to remember: How much flour? 2 cups? 3 cups?
- What temperature? 400°F? 450°F?
- How long? 15 minutes? 20 minutes?
- Different each time
- Can't easily teach someone else
- Forget an ingredient → Disaster
- **Result:** Inconsistent pizza

Attempts to make the same pizza again:

- "I think it was 2.5 cups of flour"
- "Was it 425°F or 450°F?"
- "Did I use olive oil or vegetable oil?"
- **Result:** Different every time (configuration drift)

Terraform = Following a Recipe:

Pizza recipe (pizza.tf):

- Flour: 3 cups
- Water: 1.5 cups
- Yeast: 2 tsp
- Temperature: 450°F
- Time: 18 minutes
- **Result:** Perfect pizza every time

Benefits:

- Version control recipe (Git)
- Anyone can follow recipe
- Change recipe → All future pizzas change
- Share recipe → Everyone makes same pizza
- **Result:** Consistent, reproducible pizza

Attempts to make same pizza again:

- Read recipe.txt
- Follow exactly
- **Result:** Identical every time

Terraform is the recipe for your infrastructure.

Why This Matters for Your Career

The Job Market Reality

Entry-level DevOps job posting Actual LinkedIn Examples:

DevOps Engineer - Junior Level

Salary: \$75,000 - \$95,000

Requirements:

- ☐ Docker & Kubernetes (you have this from Section 4!)
- ☐ Terraform or CloudFormation (Section 5 teaches this!)
- ☐ AWS (EC2, EKS, RDS, VPC) (Section 5 teaches this!)
- ☐ CI/CD pipelines (Section 6 will teach this)
- ☐ Infrastructure as Code (Section 5 teaches this)
- ☐ 1-2 years experience (but portfolio projects count!)

Without Terraform + AWS:

- └ Resume filtered out by ATS (Applicant Tracking System)
- └ "Thanks, we'll keep your resume on file"
- └ No interview

With Terraform + AWS (after Section 5):

- └ Resume passes ATS
- └ Recruiter sees: "Built production AWS infrastructure with Terraform"
- └ "Can you interview Tuesday?"
- └ Job offer

Salary comparison:

Skillset	Average Salary	Job Availability
-----	-----	-----
Docker only	\$60,000	5,000 jobs
Docker + Kubernetes	\$75,000	12,000 jobs
Docker + K8s + AWS	\$85,000	18,000 jobs
Docker + K8s + AWS + Terraform	\$95,000	25,000 jobs (Section 5!)
Full stack (above + CI/CD)	\$110,000	30,000+ jobs (Section 6)
Source: LinkedIn Job Search, Indeed, Glassdoor (2024)		

Section 5 Learning Path

Quick Overview (Matches K1-K9 Structure)

5.0: Terraform Fundamentals (30 min)
"Orientation - how Terraform works"

T0: Your First Terraform Resource (30 min, \$0)
"Quick win - build confidence"

T1: VPC Foundation (4 hours)
"Build the network foundation"

T2: Security Groups & NAT (3 hours)
"Add security and private networking"

T3: EKS Cluster - Control Plane (4 hours)
"Replace MicroK8s with managed Kubernetes"

T4: EKS Node Group - Worker Nodes (3 hours)
"Add worker nodes, deploy H-Game"

T5: RDS PostgreSQL Migration (5 hours)
"Replace PostgreSQL StatefulSet with managed RDS"

T6: ElastiCache Redis Migration (3 hours)
"Replace Redis Deployment with managed ElastiCache"

T7: Application Load Balancer (5 hours)
"Production-grade ingress"

T8: ECR Container Registry (2 hours)
"Private Docker registry"

T9: Auto-Scaling & Production Hardening (4 hours)
"Make infrastructure resilient"

T10: Infrastructure Testing & Documentation (3 hours)
"Ensure reproducibility"

Total: 40 hours over 4 weeks
Cost: \$80-120 total

Section 5 Success Metrics

By the end of Week 14, you'll have:

Infrastructure (Visible to Employers)

- ☐ Complete production AWS architecture
- ☐ VPC with multi-AZ high availability
- ☐ EKS cluster (managed Kubernetes)
- ☐ RDS PostgreSQL (managed database)
- ☐ ElastiCache Redis (managed cache)
- ☐ Application Load Balancer
- ☐ ECR (private container registry)
- ☐ Auto-scaling configured

Code (GitHub Portfolio)

- ☐ **Complete Terraform configuration (.tf files)**
- ☐ Variables files (terraform.tfvars)
- ☐ Remote state backend (S3)
- ☐ Complete documentation (README.md)
- ☐ Architecture diagrams
- ☐ Runbooks for common operations

Application (Production-Ready)

- ☐ H-Game running on EKS
- ☐ Backend → RDS (managed database)
- ☐ Backend → ElastiCache (managed cache)
- ☐ Frontend → ALB (production ingress)
- ☐ All monitoring working
- ☐ Zero downtime deployments tested

Skills (Interview-Ready)

- ☐ Can explain Infrastructure as Code
- ☐ Can provision AWS infrastructure with Terraform
- ☐ Can explain VPC networking

- ☐ Can explain EKS vs self-managed Kubernetes
- ☐ Can migrate applications to the cloud
- ☐ Can estimate AWS costs
- ☐ Can implement disaster recovery
- ☐ Can document infrastructure

Career Impact

- ☐ Resume shows: "Built production AWS infrastructure with Terraform."
- ☐ Portfolio shows: Complete Terraform codebase
- ☐ Can answer: "How would you set up production infrastructure?"
- ☐ Ready for: Junior DevOps Engineer roles (\$75k-\$95k)

Ready to Start?

Cost: \$80-120 total (4 weeks)

Time: 40 hours (10 hours/week)

Outcome: Production AWS infrastructure + Terraform skills

Career impact: \$5,000-\$10,000 salary increase

Next: 5.0 - Terraform Fundamentals (30 min orientation)

Then: TASK To - Your First Terraform Resource (30 min, \$0 cost)

Let's build production infrastructure.

Terraform Fundamentals

Time: 30 minutes | Orientation | **Cost:** \$0

Mission Brief

Learn the technical fundamentals of Terraform before you start building infrastructure. This quick orientation covers the core concepts and workflow you'll use in every task.

Outcomes:

- Understand Terraform's core components (provider, resource, state)
- Learn the Terraform workflow (init, plan, apply, destroy)
- Understand variables and outputs
- See how Terraform connects to your Kubernetes knowledge

Prerequisites: Complete Section 4 (K1-K9) - Kubernetes tasks

Setup Note: You'll run Terraform commands from your local machine. Terraform creates resources in AWS (EC2, EKS, RDS, etc.). See Section 5 Overview for general setup details and motivation.

Note: For the "why" behind Terraform (motivation, mental model, comparison to manual setup), see Section 5 Overview. This focuses on the "how", the technical concepts you need to start using Terraform.

Choosing Your Editor: IDE vs Command Line

Throughout Section 5, you'll see commands like:

```
cat > vpc.tf << 'EOF'
resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"
}
EOF
```

This creates files using the command-line. But you can use your favorite IDE instead!

Recommended: Use an IDE (VS Code, Cursor, etc.)

Why IDEs are better for learning:

- Syntax highlighting (easier to read colored code)
- Auto-completion (faster typing, fewer typos)
- Error detection (catch mistakes before `terraform plan`)
- File navigation (manage multiple .tf files easily)
- Git integration (track changes visually)

How to use IDE instead of `cat` commands:

Instead of running:

```
cat > vpc.tf << 'EOF'
# Terraform code here
EOF
```

Do this:

1. Open your Terraform folder in VS Code/Cursor

```
cd terraform/
code .          # VS Code
# or
cursor .       # Cursor
```

2. Create new file in IDE

- Right-click → "New File" → Name it `vpc.tf`

3. Copy the Terraform code from task (everything between `cat > file.tf << 'EOF'` and `EOF`)

4. Paste into your IDE and save (Ctrl+S / Cmd+S)

Run terraform commands (same as task):

```
terraform init
terraform plan
terraform apply
```

Result: Same outcome, easier editing!

Quick Setup Guide

VS Code / Cursor (Recommended):

1. Install:

- VS Code: <https://code.visualstudio.com/>
- Cursor: <https://cursor.sh/>

2. Install Terraform extension:

- Open VS Code/Cursor
- Extensions (Ctrl+Shift+X / Cmd+Shift+X)
- Search "HashiCorp Terraform"
- Click "Install"

3. Open your Terraform folder:

```
code terraform/    # VS Code
cursor terraform/  # Cursor
```

4. Start creating .tf files in IDE!

Keyboard shortcuts:

- Ctrl+S / Cmd+S - Save file
- Ctrl+P / Cmd+P - Quick file open
- Ctrl+F / Cmd+F - Find in file
- Ctrl+/ / Cmd+/ - Toggle comment

Alternative: Command-Line Editors (vim, nano)

If you prefer command-line:

- Works everywhere (SSH, remote servers)
- Lightweight and fast
- Steeper learning curve

The tasks show `cat` commands for compatibility, but use whichever editor you're comfortable with!

Pro Tip: Throughout Section 5, when you see:

```
cat > file.tf << 'EOF'
[code]
EOF
```

Just **create** `file.tf` in your IDE and paste the `[code]` section. Same result, easier workflow!

For complete IDE setup guide, keyboard shortcuts, and troubleshooting, see [IDE-GUIDE.md](#)

All code files available at [Repo](#)

How Terraform Works

The Core Components

1. Provider

```
# Tells Terraform which cloud to use
provider "aws" {
  region = "us-east-1" # Which AWS region
}
```

What it does: Connects Terraform to AWS (or GCP, Azure, etc.)

2. Resource

```
# Defines what you want to create
resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"

  tags = {
    Name = "h-game-vpc"
  }
}
```

What it does: Describes one piece of infrastructure (VPC, EC2, RDS, etc.)

3. State File

```
terraform.tfstate (created automatically)
```

What it does: Remembers what Terraform created (like Kubernetes etcd)

4. Configuration Files

```
main.tf      # Main resources
variables.tf  # Input variables
outputs.tf   # Output values
```

```
terraform.tfvars # Variable values
```

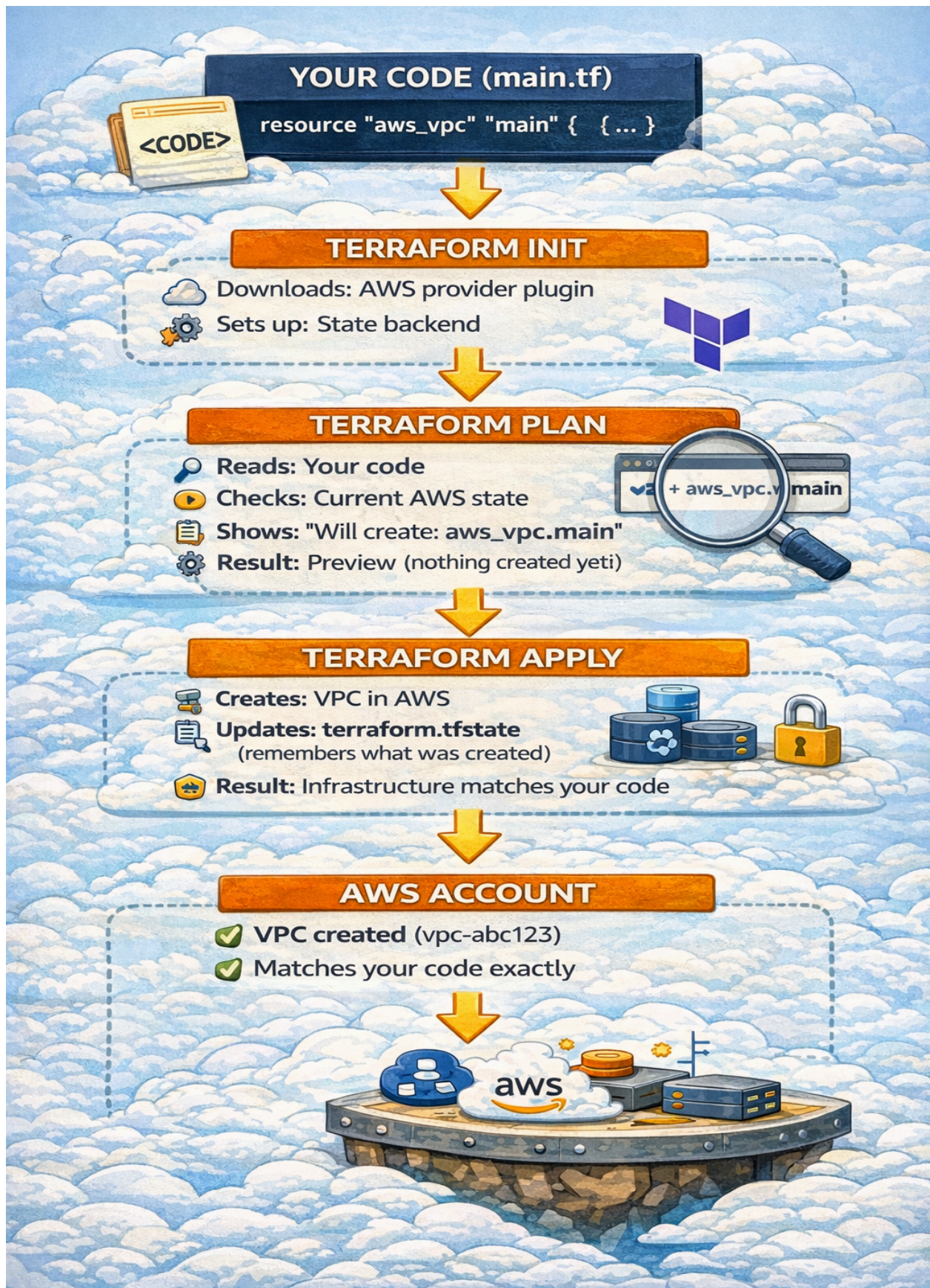
What they do: Organize your infrastructure code

The Terraform Workflow

Step-by-Step Process

TERRAFORM WORKFLOW	
1. WRITE CODE	
└> Create .tf files describing what you want	
2. TERRAFORM INIT	
└> Downloads provider plugins (AWS, etc.)	
└> Sets up backend (where state is stored)	
3. TERRAFORM PLAN	
└> Reads your code	
└> Checks current state	
└> Shows what will change (preview)	
└> No changes made yet!	
4. TERRAFORM APPLY	
└> Creates/updates/deletes resources in AWS	
└> Updates state file	
└> Infrastructure now matches your code	
5. TERRAFORM DESTROY (optional)	
└> Deletes all resources	
└> Cleans up (saves money!)	

Visual Flow



How Terraform Connects to Kubernetes

You already understand desired state management from Kubernetes!

KUBERNETES (K1-K9):

You write: deployment.yaml (desired state)	
You run: kubectl apply	
Kubernetes ensures: 3 Pods always running	

TERRAFORM (Section 5):

You write: main.tf (desired state)	
You run: terraform apply	
AWS ensures: Infrastructure matches your code	

Same pattern. Different layer.

- Kubernetes: Maintains desired state for Pods, Services, Deployments
- Terraform: Maintains desired state for VPCs, EKS, RDS, Security Groups

For the complete layer diagram and deeper explanation, see Section Overview above: "Bridge from K9: Why Terraform Now?"

Key Concepts

1. Provider

What: A Plugin that connects Terraform to a cloud provider

Examples:

```
provider "aws" {           # AWS cloud
  region = "us-east-1"
}

provider "azurerm" {       # Azure cloud
  # Azure configuration
}

provider "google" {        # Google Cloud
  # GCP configuration
}
```

Why: Different clouds have different APIs. Providers translate Terraform code to cloud API calls.

2. Resource

What: One piece of infrastructure (VPC, EC2, RDS, etc.)

Format:

```
resource "provider_resource_type" "local_name" {
  argument1 = "value1"
  argument2 = "value2"
}
```

Example:

```
# Creates an EC2 instance
resource "aws_instance" "web_server" {
  ami          = "ami-12345"      # Which OS image
  instance_type = "t3.micro"      # Size of server

  tags = {
    Name = "my-web-server"      # Label for identification
  }
}
```

Why: Resources are the building blocks. You combine them to build a complete infrastructure.

3. State

What: File that remembers what Terraform created

Location: `terraform.tfstate` (local) or S3 (remote)

Contains:

- What resources exist
- Their IDs (e.g., `vpc-abc123`)
- Their configuration
- Relationships between resources

Why: Terraform needs to know what already exists to decide what to create/update/delete.

Example:

```
{
  "resources": [
    {
      "type": "aws_vpc",
      "name": "main",
      "instances": [
        {
```

```

        "attributes": {
            "id": "vpc-abc123",
            "cidr_block": "10.0.0.0/16"
        }
    ]
}

```

4. Variables

What: Input values that make your code reusable

Example:

```

# variables.tf
variable "environment" {
    description = "Environment name"
    type        = string
    default     = "dev"
}

# main.tf
resource "aws_vpc" "main" {
    cidr_block = "10.0.0.0/16"

    tags = {
        Name           = "h-game-vpc"
        Environment    = var.environment # Uses variable
    }
}

# terraform.tfvars
environment = "production" # Value for variable

```

Why: If Same code works for dev, staging, production (just change variables).

5. Outputs

What: Values you want to see after Terraform runs

Example:

```
# outputs.tf
output "vpc_id" {
  description = "ID of the VPC"
  value       = aws_vpc.main.id
}

# After terraform apply:
# vpc_id = "vpc-abc123"
```

Why: You need the VPC ID to create other resources (subnets, etc.).

Quick Reference: Terraform vs Manual

For detailed comparison and motivation, see [Section Overview](#).

Quick summary:

- **Manual:** Click through AWS Console (4-6 hours), hard to reproduce, no version control
- **Terraform:** Write code (15 minutes), **terraform apply** recreates everything, Git tracks changes

Terraform = Infrastructure as Code (IaC)

Ready to Start?

You now understand:

- Terraform's core components (provider, resource, state, variables, outputs)
- The Terraform workflow (init, plan, apply, destroy)
- How Terraform connects to Kubernetes (desired state management)

Next: TASK To Your First Terraform Resource

In To, you'll:

- Install Terraform
- Write your first `.tf` file
- Run `terraform init, plan, apply`
- Create a real AWS resource (S3 bucket)
- Destroy it (cost: \$0)

Time: 30 minutes | **Cost:** \$0

Let's build your first infrastructure!

TASK To: Your First Terraform Resource

Time: 30 minutes | **Priority:** High | **Ticket:** DEVOPS-5100 | **Cost:** \$0

Mission Brief

Create your first AWS resource using Terraform. This builds confidence and proves the Terraform workflow works before you build complex infrastructure.

Outcomes:

- Install Terraform
- Configure AWS credentials
- Write your first Terraform configuration
- Run the complete Terraform workflow (init, plan, apply, destroy)
- Understand how Terraform state works

Prerequisites: Complete 5.0 Terraform Fundamentals

Setup Note: Run Terraform from your local machine. This task creates an S3 bucket in AWS (see Section 5 Overview for general setup details).

Cost Note: This task costs \$0. We'll create an S3 bucket (free) and destroy it at the end.

The One Thing That Will Make This Click

Terraform workflow = Write code → See plan → Apply → Verify

MANUAL WAY:

- "I need an S3 bucket"
- "I'll click through AWS Console"
- "Hope I remember the name I used"
- "If I need another one... click again"

TERRAFORM WAY:

- "I need an S3 bucket"
- "I'll write: resource 'aws_s3_bucket' 'my_bucket' { ... }"
- "terraform plan (see what will happen)"
- "terraform apply (create it)"
- "terraform destroy (delete it, \$0 cost)"

That's it. That's the Terraform workflow.

Everything else (syntax, providers, state) is just details. The core workflow is:

Write code → Plan → Apply → Verify

Now let's see it work...

Phase 1: Setup (10 min)

Step 1: Install Terraform

Check if Terraform is already installed:

```
# Check Terraform version
terraform version

# If you see: "Terraform v1.x.x" → Already installed!
# If you see: "command not found" → Need to install
```

Install Terraform (if needed):

macOS:

```
# Using Homebrew (recommended)
brew install terraform

# Verify installation
terraform version

# Should show: Terraform v1.6.0 (or similar)
```

Linux:

```
# Download Terraform
wget
https://releases.hashicorp.com/terraform/1.6.0/terraform_1.6.0_linux_
amd64.zip

# Unzip
unzip terraform_1.6.0_linux_amd64.zip

# Move to PATH
sudo mv terraform /usr/local/bin/

# Verify installation
terraform version
```

Windows:

```
# Using Chocolatey
choco install terraform

# Or download from: https://www.terraform.io/downloads
# Extract and add to PATH

# Verify installation
terraform version
```

Checkpoint: Did it work?

- `terraform version` shows version number
- Terraform command is available

Terraform installed!

Step 2: Configure AWS Credentials

Terraform needs permission to create resources in AWS.

Option 1: AWS CLI (Recommended - if you already have it)

```
# Check if AWS CLI is configured
aws sts get-caller-identity

# If you see your AWS account ID → Already configured!
# If you see error → Need to configure
```

Configure AWS CLI (if needed):

```
# Configure AWS credentials
aws configure

# You'll be asked for:
# AWS Access Key ID: [Your access key]
# AWS Secret Access Key: [Your secret key]
# Default region: us-east-1 (or your preferred region)
# Default output format: json

# Verify it worked
aws sts get-caller-identity
# Should show: Account ID, User ARN
```

Option 2: Environment Variables

```
# Set AWS credentials as environment variables
export AWS_ACCESS_KEY_ID="your-access-key"
export AWS_SECRET_ACCESS_KEY="your-secret-key"
export AWS_DEFAULT_REGION="us-east-1"

# Verify
aws sts get-caller-identity
```

Where to get AWS credentials:

1. **AWS Console** → IAM → Users → Your user → Security credentials
2. **Create Access Key** → Download CSV (save securely!)
3. **Use the keys** in `aws configure` or environment variables

Security Note: Never commit AWS credentials to Git! Use environment variables or AWS CLI configuration.

Checkpoint: Did it work?

- `aws sts get-caller-identity` shows your account
- AWS credentials are configured

AWS credentials configured!

Phase 2: Your First Terraform Resource (15 min)

Step 1: Create Project Directory

```
# Create directory for Terraform project
mkdir -p ~/projects/h-game-terraform
cd ~/projects/h-game-terraform

# This will be your Terraform workspace
```

Step 2: Configure Variables

Why we're doing this: Variables separate configuration from code, making infrastructure reusable and keeping sensitive data secure.

Create variables file:

```
# Create variables.tf
cat > variables.tf << 'EOF'
# =====
# TERRAFORM VARIABLES
# =====
# Variables make your code reusable and configurable
# Values are set in terraform.tfvars (not committed to Git)

variable "aws_region" {
  description = "AWS region for resources"
  type        = string
  default     = "us-east-1"
}

variable "bucket_name_prefix" {
  description = "Prefix for S3 bucket name (must be globally unique)"
  type        = string
  default     = "h-game-terraform-learning"
}

variable "environment" {
  description = "Environment name (e.g., learning, dev, production)"
  type        = string
```

```

    default      = "learning"
}
EOF

```

Create terraform.tfvars (your actual values - DO NOT COMMIT TO GIT!)

```

cat > terraform.tfvars << 'EOF'
# =====
# TERRAFORM VARIABLE VALUES
# =====
# ⚠ IMPORTANT: Add terraform.tfvars to .gitignore!
# This file contains your actual values and should NOT be committed to Git

aws_region      = "us-east-1"
bucket_name_prefix = "h-game-terraform-learning" # Change this to make it
unique!
environment     = "learning"
EOF

```

Create .gitignore to prevent committing sensitive files

```

cat > .gitignore << 'EOF'
# =====
# TERRAFORM SENSITIVE FILES
# =====
# ⚠ NEVER commit these files - they contain sensitive data!

# Variable files with actual values
terraform.tfvars
*.tfvars
!terraform.tfvars.example # Allow example file

# State files (contain passwords, endpoints, etc.)
terraform.tfstate
terraform.tfstate.backup
*.tfstate
*.tfstate.*

# Terraform directories
.terraform/

```

```
# Lock file - KEEP THIS (ensures version consistency)
# NOTE: .terraform.lock.hcl SHOULD be committed to Git
# It's already allowed (not ignored) - ensures everyone uses same provider versions

# Crash Logs
crash.log
crash.*.log

# Override files (for local testing)
override.tf
override.tf.json
*_override.tf
*_override.tf.json

# CLI configuration
.terraformrc
terraform.rc

# IDE files
.vscode/
.idea/
*.swp
*.swp
*~

# OS files
.DS_Store
Thumbs.db
EOF
```

Variables configured!

Step 3: Write Your First Terraform Configuration ([all code files in repo](#))

Create **main.tf**:

```
# Create main.tf file
cat > main.tf << 'EOF'
# =====
# TERRAFORM CONFIGURATION
# =====
# This file defines what infrastructure to create

# Provider: Tells Terraform which cloud to use
provider "aws" {
  region = "us-east-1" # AWS region (change to your preferred region)
  # Credentials come from AWS CLI or environment variables
}

# Resource: An S3 bucket (simple storage)
# This is your FIRST Terraform resource!
resource "aws_s3_bucket" "my_first_bucket" {
  # Bucket name must be globally unique
  # Using timestamp to ensure uniqueness
  bucket = "h-game-terraform-learning-${formatdate("YYYYMMDDhhmmss", timestamp())}"

  # Tags: Labels for organization
  tags = {
    Name       = "My First Terraform Bucket"
    Environment = "learning"
    CreatedBy  = "Terraform"
    Task       = "T0"
  }
}

# Output: Value to display after creation
output "bucket_name" {
  description = "Name of the S3 bucket we created"
  value       = aws_s3_bucket.my_first_bucket.bucket
}

output "bucket_arn" {
  description = "ARN (Amazon Resource Name) of the bucket"
  value       = aws_s3_bucket.my_first_bucket.arn
}
EOF
```

What this does:

- **Provider:** Connects to AWS
- **Resource:** Creates an S3 bucket (simple storage)
- **Output:** Shows bucket name after creation

Why S3 bucket?

- Simple resource (no complex configuration)
- Free (no cost)
- Quick to create/delete
- Perfect for learning workflow

Variables Note: You've created `variables.tf` (definitions) and `terraform.tfvars` (values). The `.gitignore` file prevents committing `terraform.tfvars` to Git, keeping your values secure. Always use variables for configurable values!

Step 4: Format and Validate Code

Format Terraform code:

```
# Format Terraform code (fixes indentation, spacing)
terraform fmt
# Automatically formats your code:
# - Fixes indentation (2 spaces)
# - Aligns equals signs
# - Sorts attributes
# - Makes code readable

# Validate Terraform syntax and configuration
terraform validate
# Checks for errors:
# - Syntax errors (missing brackets, quotes)
# - Invalid attribute names
# - Type mismatches
# - Missing required attributes
# Should show: "Success! The configuration is valid."
```

Code formatted and validated!

Step 5: Initialize Terraform

What **terraform init** does:

- Downloads AWS provider plugin
- Sets up backend (where state is stored)
- Prepares Terraform to work

```
# Initialize Terraform
terraform init
```

Expected output:

```
Initializing the backend...

Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v5.30.0...
- Installed hashicorp/aws v5.30.0 (signed by HashiCorp)

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory.
```

What happened:

- Terraform downloaded AWS provider plugin
- Created **.terraform** directory (stores plugins)
- Ready to use Terraform!

Checkpoint: Did it work?

- **terraform init** completed successfully
- No errors in output
- **.terraform** directory created

Terraform initialized!

Step 6: Plan (Preview Changes)

What **terraform plan** does:

- Reads your code
- Checks what exists in AWS (nothing yet)
- Shows what will be created
- **Doesn't create anything yet!**

```
# Preview what Terraform will do
terraform plan
```

Expected output:

```
Terraform used the selected providers to generate the following execution plan.
Resource actions are indicated with the following symbols:
```

```
+ create
```

```
Terraform will perform the following actions:
```

```
# aws_s3_bucket.my_first_bucket will be created
+ resource "aws_s3_bucket" "my_first_bucket" {
  + arn                = (known after apply)
  + bucket              = "h-game-terraform-learning-20240115143000"
  + bucket_domain_name = (known after apply)
  + bucket_regional_domain_name = (known after apply)
  + force_destroy      = false
  + hosted_zone_id     = (known after apply)
  + id                 = (known after apply)
  + region             = (known after apply)
  + request_payer      = "BucketOwner"
  + tags               = {
    + CreatedBy = "Terraform"
    + Environment = "learning"
    + Name      = "My First Terraform Bucket"
    + Task      = "T0"
  }
  + tags_all = {
    + CreatedBy = "Terraform"
    + Environment = "learning"
    + Name      = "My First Terraform Bucket"
    + Task      = "T0"
  }
}
```

```
Plan: 1 to add, 0 to change, 0 to destroy.
```


What this tells you:

- **+ create** = Will create 1 resource
- **Plan: 1 to add** = 1 resource will be created
- **Bucket name** = What the bucket will be called
- **Tags** = Labels that will be applied

Key insight: Terraform shows you EXACTLY what will happen before it happens!

Checkpoint: Did it work?

- **terraform plan** shows 1 resource to create
- No errors
- You can see what will be created

Plan successful!

Step 7: Apply (Create the Resource)

What **terraform** **apply** does:

- Creates the resources shown in plan
- Updates state file (remembers what was created)
- Shows outputs (bucket name, ARN)

```
# Create the S3 bucket
terraform apply
```

Terraform will ask for confirmation:

```
Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

Enter a value:
```

Type **yes** and press Enter:

Expected output:

```
aws_s3_bucket.my_first_bucket: Creating...
aws_s3_bucket.my_first_bucket: Creation complete after 2s
[id=h-game-terraform-learning-20240115143000]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

bucket_name = "h-game-terraform-learning-20240115143000"
bucket_arn  = "arn:aws:s3:::h-game-terraform-learning-20240115143000"
```

What happened:

- S3 bucket created in AWS!
- State file updated (Terraform remembers it exists)
- Outputs shown (bucket name and ARN)

Verify in AWS Console:

1. Go to AWS Console → S3
2. You should see your bucket: **h-game-terraform-learning-20240115143000**
3. Click on it → See it's empty (that's fine!)

Or verify with AWS CLI:

```
# List S3 buckets
aws s3 ls

# You should see your bucket in the list!
```

Checkpoint: Did it work?

- **terraform apply** completed successfully
- Bucket exists in AWS Console
- Outputs show bucket name and ARN

Your first Terraform resource created!

Step 8: Verify State File

Terraform created a state file to remember what exists:

```
# View state file
cat terraform.tfstate

# Or use Terraform command (better)
terraform show
```

What state file contains:

- What resources exist
- Their IDs (bucket name)
- Their configuration
- Relationships

Why this matters:

- If you run `terraform plan` again, Terraform checks state file
- Sees bucket already exists → No changes needed
- If you delete bucket manually → Terraform thinks it still exists (state drift)

Checkpoint: Did it work?

- State file exists (`terraform.tfstate`)
- `terraform show` displays bucket information

State file working!

Step 9: Destroy (Clean Up - \$0 Cost)

What **terraform destroy** does:

- Deletes all resources created by Terraform
- Updates state file (removes resources)
- **This is why this task costs \$0!**

```
# Delete the S3 bucket
terraform destroy
```

Terraform will ask for confirmation:

```
Do you really want to destroy all resources?
Terraform will destroy all your managed infrastructure.
This action cannot be undone.

Enter a value:
```

Type **yes** and press Enter:

Expected output:

```
aws_s3_bucket.my_first_bucket: Refreshing state...
[id=h-game-terraform-learning-20240115143000]
aws_s3_bucket.my_first_bucket: Destroying...
aws_s3_bucket.my_first_bucket: Destruction complete after 1s

Destroy complete! Resources: 1 destroyed.
```

What happened:

- S3 bucket deleted from AWS
- State file updated (resource removed)
- **Cost: \$0** (bucket was free, now deleted)

Verify it's gone:

```
# List S3 buckets
aws s3 ls

# Your bucket should NOT be in the list anymore!
```

Checkpoint - Did it work?

- **terraform destroy** completed successfully
- Bucket no longer exists in AWS
- State file updated

Cleanup complete!

Phase 3: Understanding What Happened (5 min)

The Complete Workflow

You just completed the full Terraform workflow:

1. **WRITE CODE (main.tf)**
 - └─> Described what you want (S3 bucket)
2. **TERRAFORM INIT**
 - └─> Downloaded AWS provider
 - └─> Set up backend
3. **TERRAFORM PLAN**
 - └─> Previewed what would happen
 - └─> Showed: "Will create 1 resource"
4. **TERRAFORM APPLY**
 - └─> Created S3 bucket in AWS
 - └─> Updated state file
5. **TERRAFORM DESTROY**
 - └─> Deleted S3 bucket
 - └─> Updated state file

This is the same workflow you'll use for everything!

Key Insights

1. Terraform is Declarative

- You describe WHAT you want (S3 bucket)
- Terraform figures out HOW to create it
- Same as Kubernetes: You describe desired state, Kubernetes makes it happen

2. Plan Before Apply

- `terraform plan` shows you what will happen
- No surprises! You see changes before they happen
- Always run `plan` before `apply` in production

3. State is Critical

- State file remembers what Terraform created
- If state is lost, Terraform doesn't know what exists
- We'll use remote state (S3) in later tasks

4. Destroy is Powerful

- `terraform destroy` deletes everything
- Perfect for cleanup (saves money!)
- Use carefully in production!

Checklist: Did You Complete Everything?

Before moving on, check these boxes:

Setup Phase

- ☐ Terraform installed (`terraform version` works)
- ☐ AWS credentials configured (`aws sts get-caller-identity` works)
- ☐ Project directory created

Implementation Phase

- ☐ Created `main.tf` with S3 bucket resource
- ☐ Ran `terraform init` successfully
- ☐ Ran `terraform plan` (saw preview)
- ☐ Ran `terraform apply` (created bucket)
- ☐ Verified bucket in AWS Console
- ☐ Ran `terraform destroy` (deleted bucket)
- ☐ Verified bucket is gone

Understanding Phase

- ☐ Understand Terraform workflow (init → plan → apply → destroy)
- ☐ Understand what state file does
- ☐ Understand why plan is important

Git

- ☐ Committed `main.tf` to Git
- ☐ Did NOT commit `terraform.tfstate` (add to `.gitignore`)

Key Takeaways

The Big Picture

What you learned in this task:

TERRAFORM WORKFLOW MASTERED

- Write code (describe what you want)
- terraform init (download providers)
- terraform plan (preview changes)
- terraform apply (create resources)
- terraform destroy (clean up)

You can now create infrastructure with code!

Critical Concepts

1. Terraform Workflow:

- **init** - Download providers, set up backend
- **plan** - Preview what will happen (no changes)
- **apply** - Create/update resources
- **destroy** - Delete all resources

2. State File:

- Remembers what Terraform created
- Used to compare desired vs actual state
- Critical for Terraform to work correctly

3. Declarative Approach:

- Describe WHAT you want
- Terraform figures out HOW
- Same pattern as Kubernetes

4. Cost Control:

- `terraform destroy` deletes everything
- Perfect for learning (no ongoing costs)
- Use carefully in production!

What's Next?

Congratulations! You've created your first infrastructure with Terraform!

You now have:

- Terraform, installed and configured
- Understanding of Terraform workflow
- Confidence that Terraform works
- First infrastructure created with code

Next: TASK T1 - VPC Foundation

In T1, you'll learn:

- How to create a VPC (private network)
- How to create subnets (network segments)
- How to configure networking
- How resources connect

But most importantly: You now understand the Terraform workflow. Everything else is just more resources!

Proceed to TASK T1: VPC Foundation

TASK T1: VPC Foundation

Time: 4 hours | **Priority:** High | **Ticket:** DEVOPS-5101 | **Cost:** \$0

Mission Brief

Build the network foundation for your production infrastructure. Create a VPC (Virtual Private Cloud) with public and private subnets across multiple availability zones. This is the foundation that all other infrastructure will be built upon.

Outcomes:

- Understand what a VPC is and why it matters
- Create a VPC with proper CIDR blocks
- Create public and private subnets across 3 availability zones
- Configure Internet Gateway for public access
- Understand how networking fits into the complete system

Prerequisites: Complete Task To (Your First Terraform Resource)

Setup Note: VPC will be created in AWS (see Section 5 Overview for general setup details).

Cost Note: This task costs \$0. VPCs, subnets, and Internet Gateways are free. Only data transfer and NAT Gateways cost money (we'll add NAT in T2).

IDE Users: You'll see `cat > vpc.tf << 'EOF'` commands throughout this task.

You can use your IDE instead!

1. Create `vpc.tf` in VS Code/Cursor
2. Copy the Terraform code (between EOF markers)
3. Paste into your IDE
4. Save and run terraform commands

See `5.0-terraform-fundamentals.md` for complete IDE setup guide.

The One Thing You Must Understand

A VPC is your own private network inside AWS, just like the Wi-Fi network in your house.

Without a VPC, everything sits in a shared default network where it's hard to control access or stay organized.

With a VPC, you choose your own IP range and decide exactly **who can talk to** who using security rules.

This gives you isolation, better security, and a clean, professional setup instead of chaos.

That's it. That's a VPC.

Everything else (subnets, route tables, gateways) is just organizing your network. The core concept is:

"Your own private network in AWS where you control everything."

The Mental Model Shift:

BEFORE T1:

- "AWS is just a cloud"
- "Resources exist somewhere in AWS"
- "No control over networking"

AFTER T1:

- "AWS has my private network (VPC)"
- "Resources live in my network (10.0.0.0/16)"
- "I control traffic with security groups and route tables."

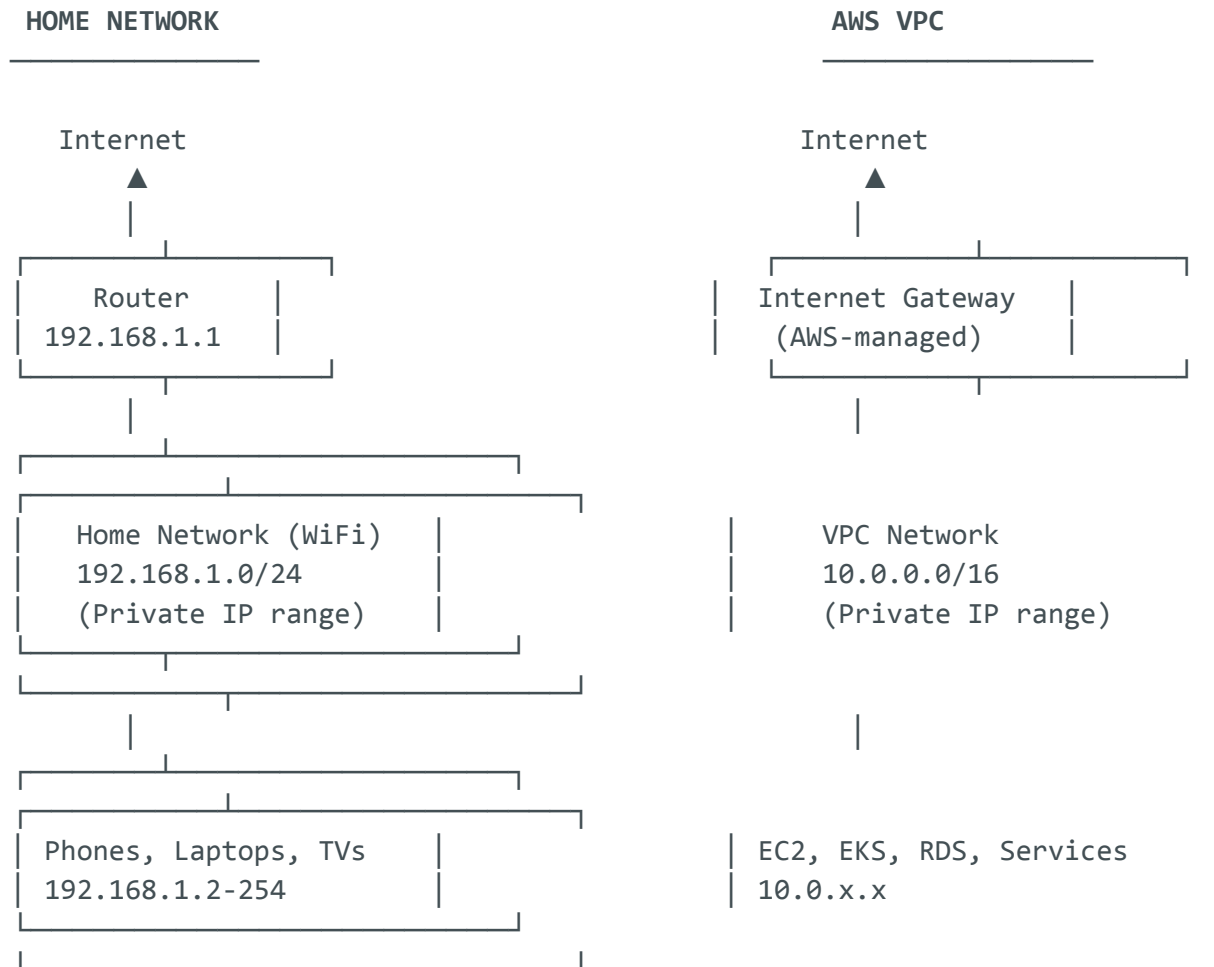
This is the difference between **"default AWS"** and **"production infrastructure."**

Now let's see how VPCs work...

Phase 1: Understanding VPCs (30 min)

What is a VPC?

The Home Network Analogy 🏠



WHAT YOU CONTROL

HOME NETWORK

- Who can connect
- Internet access
- Firewall rules

AWS VPC

- Which resources can connect
- Internet access
- Security Groups & Route Tables

Key insight: VPC = Your own private network in AWS, just like your home network.

Why VPCs Matter

- Complete control over network
- Organize resources logically
- Control traffic with security groups
- Isolate projects (dev, staging, prod)
- Professional, production ready setup

Exercise 1: Why Multiple Availability Zones?

Scenario: You need to create subnets for your VPC.

Question: Should you create subnets in:

- A) One availability zone (one data center)
- B) Three availability zones (three data centers)

Think about:

- What happens if one data center fails?
- How does this affect high availability?
- What do production systems do?

Your answer: _____

Answer Below 

Answer: B) Three availability zones

Why multiple AZs matter:

SINGLE AZ (One Data Center):

Availability Zone: us-east-1a └ Subnet 1 (Public) └ Subnet 2 (Private) └ All resources here	
Problem: If AZ fails → Everything down	
Result: No high availability	

MULTIPLE AZs (Three Data Centers):

Availability Zone: us-east-1a └ Public Subnet 1 └ Private Subnet 1	
Availability Zone: us-east-1b └ Public Subnet 2 └ Private Subnet 2	
Availability Zone: us-east-1c └ Public Subnet 3 └ Private Subnet 3	
Benefit: If one AZ fails → Others keep running	
Result: High availability	

Production systems use multiple AZs for:

- **High availability** - If one AZ fails, others continue
- **Load distribution** - Spread traffic across AZs
- **Disaster recovery** - Data replicated across AZs

This is why we create subnets in 3 AZs!

Exercise 2: Public vs Private Subnets

Scenario: You need to create subnets for your VPC.

Question: What's the difference between public and private subnets?

Think about:

- Which resources need internet access?
- Which resources should be hidden from the internet?
- How does traffic flow?

Your answer: _____

Answer Below 

Answer:**Public Subnets:****PUBLIC SUBNET**

- | Has route to Internet Gateway
- | Resources can reach internet directly
- | Internet can reach resources (if allowed)

Use for:

- Load Balancers (need internet access)
- NAT Gateways (provide internet to private)
- Bastion hosts (SSH access)

Private Subnets:**PRIVATE SUBNET**

- | NO direct route to Internet Gateway
- | Resources CANNOT reach the internet directly
- | Internet CANNOT reach resources

Use for:

- Application servers (backend Pods)
- Databases (RDS)
- Cache (ElastiCache)
- Kubernetes worker nodes

Security: Hidden from the internet (more secure!)

Traffic Flow:

INTERNET → Public Subnet → Private Subnet
(Load Balancer) (Application)

Application → Private Subnet → NAT Gateway → Internet
(Needs updates) (Public Subnet) (Download packages)

Why this matters:

- **Public subnets** = Resources that need internet (Load Balancers)
- **Private subnets** = Resources that should be hidden (Databases, Apps)
- **Security** = Private subnets can't be reached from the internet directly

This is production-grade networking!

Exercise 3: CIDR Blocks

Scenario: You need to choose IP address ranges for your VPC and subnets.

Question: Which CIDR blocks should you use?

VPC: 10.0.0.0/16 (65,536 IPs)

Subnets (need 6 total - 3 public, 3 private):

- Public Subnet 1 (us-east-1a):?
- Public Subnet 2 (us-east-1b):?
- Public Subnet 3 (us-east-1c):?
- Private Subnet 1 (us-east-1a):?
- Private Subnet 2 (us-east-1b):?
- Private Subnet 3 (us-east-1c):?

Think about:

- Each subnet needs enough IPs (typically /24 = 256 IPs)
- Subnets must not overlap
- Should be organized logically

Your answer: _____

Answer Below 

Answer:

VPC: 10.0.0.0/16 (65,536 IPs total)

Public Subnets:

- └ Public 1 (us-east-1a): 10.0.1.0/24 (256 IPs)
- └ Public 2 (us-east-1b): 10.0.2.0/24 (256 IPs)
- └ Public 3 (us-east-1c): 10.0.3.0/24 (256 IPs)

Private Subnets:

- └ Private 1 (us-east-1a): 10.0.11.0/24 (256 IPs)
- └ Private 2 (us-east-1b): 10.0.12.0/24 (256 IPs)
- └ Private 3 (us-east-1c): 10.0.13.0/24 (256 IPs)

Why this organization:

- **10.0.1-3.x** = Public subnets (easy to remember)
- **10.0.11-13.x** = Private subnets (separate range)
- **/24** = 256 IPs per subnet (enough for most use cases)
- **No overlap** = Each subnet has a unique IP range

CIDR Block Explanation:

10.0.0.0/16 means:

- └ Network: 10.0.0.0
- └ Subnet mask: /16 = 255.255.0.0
- └ IP range: 10.0.0.0 - 10.0.255.255
- └ Total IPs: 65,536

10.0.1.0/24 means:

- └ Network: 10.0.1.0
- └ Subnet mask: /24 = 255.255.255.0
- └ IP range: 10.0.1.0 - 10.0.1.255
- └ Total IPs: 256

This is the standard pattern for production VPCs!

Design Checkpoint

Before moving on, make sure you can answer these in plain English:

Question	Your Answer Should Be Like...
What is a VPC?	"My private network in AWS, like a home network but in the cloud"
Why multiple AZs?	"High availability, if one data center fails, others keep running."
Public vs Private subnet?	"Public = internet access, Private = hidden from internet"
What is a CIDR block?	"IP address range (e.g., 10.0.0.0/16 = 65,536 IPs)"

Can't answer one? Go back and re-read that exercise.

All clear? Great! Let's build the VPC!

Phase 2: Implementation

What We're Building

Before this task:

- No VPC (using default AWS network)
- No organized networking
- No foundation for production infrastructure

After this task:

- VPC with a proper CIDR block
- 3 Public subnets (across 3 AZs)
- 3 Private subnets (across 3 AZs)
- Internet Gateway (for public access)
- Route tables (control traffic flow)
- Foundation ready for EKS, RDS, etc.

Step-by-Step Plan

We'll do this in 7 steps:

```
Step 0: Configure Remote State Backend
        "Production-ready state storage (S3 + DynamoDB)"

Step 1: Create VPC
        "Your private network foundation"

Step 2: Create Internet Gateway
        "Connect VPC to the internet."

Step 3: Create Public Subnets
        "Subnets with internet access (3 AZs)"

Step 4: Create Public Route Table
        "Route public traffic to Internet Gateway"

Step 5: Create Private Subnets
        "Subnets without direct internet (3 AZs)"

Step 6: Verify Network Setup
        "Confirm everything is connected."
```

Note: This task uses the `provider.tf` and `variables.tf` files created in To. If you haven't completed To, do that first.

Note: Ensure `.gitignore` is configured (created in To). The `.gitignore` file prevents committing sensitive files like `terraform.tfvars` and state files.

Step 0: Configure Remote State Backend (Production Setup)

What we're doing: Setting up S3 bucket and DynamoDB table for remote Terraform state storage.

Why this matters:

- **Team collaboration:** Multiple people can work on same infrastructure
- **State locking:** Prevents concurrent modifications (via DynamoDB)
- **Backup & recovery:** State stored in S3, won't be lost
- **Production standard:** All professional teams use remote state
- **CI/CD ready:** Required for automated deployments

Local state (what we're avoiding):

`terraform.tfstate (on your laptop)`

- └ Lost if machine crashes
- └ Can't collaborate with team
- └ No locking → state conflicts
- └ Not production-ready

Remote state (what we're setting up):

S3 + DynamoDB

- └ Backed up automatically
- └ Team accessible
- └ State locking prevents conflicts
- └ Production-ready

Bootstrap: Create State Backend Resources

Why This Step Exists (The "Chicken and Egg" Problem)

The Problem:

- Terraform needs a place to store its state (what infrastructure exists)
- We want to use S3 (AWS storage) for this
- But we're using Terraform to create infrastructure... including the S3 bucket for state!
- This creates a circular dependency: "We need Terraform to create the bucket, but we need the bucket to run Terraform"

The Solution: Create the S3 bucket and DynamoDB table FIRST using AWS CLI (not Terraform), then configure Terraform to use them.

Think of it like:

- Building a house (Terraform infrastructure)
- But you need a toolbox (S3 bucket) to store your tools (Terraform state)
- So you buy the toolbox first (AWS CLI), then use it to build the house

Prerequisites Check

Before starting, verify you have AWS CLI configured:

```
# Check if AWS CLI is installed
aws --version

# Verify you're authenticated (should show your AWS account info)
aws sts get-caller-identity
```

Expected output:

```
{
  "UserId": "AIDAU3SLL26TCRTYJDFE2",
  "Account": "334091769766",
  "Arn": "arn:aws:iam::334091769766:user/yourname"
}
```

If you see an error, configure AWS CLI first:

```
aws configure
```

Step 0.1: Create a Unique Bucket Name

Why: S3 bucket names must be globally unique across ALL AWS accounts worldwide. Think of it like a domain name - only one person can have `example.com`.

What to do:

1. Open a text editor or note-taking app
2. Create a unique bucket name using this pattern:

```
h-game-tfstate-YOURNAME-TIMESTAMP
```

- Replace **YOURNAME** with your name, initials, or username
- Replace **TIMESTAMP** with current Unix timestamp (seconds since 1970)

How to get the timestamp:

```
date +%s
```

Example:

- Your name: **zee**
- Timestamp: **1706544000**
- Bucket name: **h-game-tfstate-zee-1706544000**

IMPORTANT: Write this bucket name down! You'll need it in Step 0.4.

Save it somewhere safe:

- Create a file: **echo "h-game-tfstate-zee-1706544000" > .state-bucket-name**
- Or write it in a text file
- Or copy it to your clipboard

Step 0.2: Create S3 Bucket for State Storage

Why S3?

- Stores Terraform state file (knows what infrastructure exists)
- Versioned (can see history of changes)
- Encrypted (protects sensitive data in state)
- Accessible by your team (if you share AWS credentials)

Command 1: Create the bucket

```
aws s3 mb s3://YOUR-BUCKET-NAME --region us-east-1
```

Replace **YOUR-BUCKET-NAME** with the name from Step 0.1

Example:

```
aws s3 mb s3://h-game-tfstate-zee-1706544000 --region us-east-1
```

Expected output:

```
make_bucket: h-game-tfstate-zee-1706544000
```

If you see an error:

- **BucketAlreadyExists**: Someone else has that name. Go back to Step 0.1 and add more characters.
- **AccessDenied**: Check your AWS credentials with `aws sts get-caller-identity`

Command 2: Enable versioning

Why: Keeps history of state changes. If something goes wrong, you can restore a previous version.

```
aws s3api put-bucket-versioning \
  --bucket YOUR-BUCKET-NAME \
  --versioning-configuration Status=Enabled
```

Expected output: (none - success is silent)

Verify it worked:

```
aws s3api get-bucket-versioning --bucket YOUR-BUCKET-NAME
```

Expected output:

```
{
  "Status": "Enabled"
}
```

Command 3: Enable encryption

Why: State files contain sensitive data (database passwords, API keys). Encryption protects them at rest.

```
aws s3api put-bucket-encryption \
  --bucket YOUR-BUCKET-NAME \
  --server-side-encryption-configuration '{
    "Rules": [{
      "ApplyServerSideEncryptionByDefault": {
        "SSEAlgorithm": "AES256"
      }
    }]
  }'
```

Expected output: (none - success is silent)

Verify it worked:

```
aws s3api get-bucket-encryption --bucket YOUR-BUCKET-NAME
```

Expected output:

```
{
  "ServerSideEncryptionConfiguration": {
    "Rules": [
      {
        "ApplyServerSideEncryptionByDefault": {
```

```

    "SSEAlgorithm": "AES256"
  }
}
]
}
}

```

Command 4: Block public access

Why: State files are sensitive. Never allow public access - only you (and your team) should see them.

```

aws s3api put-public-access-block \
  --bucket YOUR-BUCKET-NAME \
  --public-access-block-configuration \

"BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,Re
strictPublicBuckets=true"

```

Expected output: (none - success is silent)

Verify it worked:

```

aws s3api get-public-access-block --bucket YOUR-BUCKET-NAME

```

Expected output:

```

{
  "PublicAccessBlockConfiguration": {
    "BlockPublicAcls": true,
    "IgnorePublicAcls": true,
    "BlockPublicPolicy": true,
    "RestrictPublicBuckets": true
  }
}

```

Final verification:

```
# List your buckets (should see your new bucket)
aws s3 ls

# Check bucket details
aws s3 ls s3://YOUR-BUCKET-NAME
```

Expected: Your bucket name appears in the list, and the bucket is empty (no files yet).

Step 0.3: Create DynamoDB Table for State Locking**Why DynamoDB?**

- Prevents two people from running **terraform apply** at the same time
- Prevents corruption of state file
- Think of it like a "Do Not Disturb" sign on a hotel room door

What happens:

- Person A runs **terraform apply** → DynamoDB creates a lock
- Person B tries **terraform apply** → Sees the lock, waits
- Person A finishes → Lock is released
- Person B can now proceed

Command: Create the table

```
aws dynamodb create-table \
  --table-name h-game-terraform-state-lock \
  --attribute-definitions AttributeName=LockID,AttributeType=S \
  --key-schema AttributeName=LockID,KeyType=HASH \
  --billing-mode PAY_PER_REQUEST \
  --region us-east-1
```


Expected output:

```
{
  "TableDescription": {
    "TableName": "h-game-terraform-state-lock",
    "TableStatus": "CREATING",
    ...
  }
}
```

Wait for table to be ready:

```
aws dynamodb wait table-exists \
  --table-name h-game-terraform-state-lock \
  --region us-east-1
```

This command waits until the table is ready (may take 10-30 seconds)

Expected output: (none - command completes when table is ready)

Verify it worked:

```
aws dynamodb describe-table \
  --table-name h-game-terraform-state-lock \
  --query 'Table.TableStatus'
```

Expected output:

```
"ACTIVE"
```

If you see an error:

- **ResourceInUseException**: Table already exists (that's okay - skip this step)
- **AccessDenied**: Check your AWS credentials

Step 0.4: Configure Terraform Backend

Why: This tells Terraform WHERE to store its state file (the S3 bucket we just created).

What to do:

1. Navigate to your Terraform directory:

```
cd ~/projects/h-game-terraform
# OR
cd "/Users/mac/Desktop/devops playbook/h-game-terraform"
```

2. Create a new file called **backend.tf**:

```
touch backend.tf
```

3. Open **backend.tf** in your text editor and add this content:

```
# =====
# TERRAFORM REMOTE STATE BACKEND
# =====
# Stores state in S3 (backed up, team accessible)
# Uses DynamoDB for locking (prevents concurrent modifications)

terraform {
  backend "s3" {
    bucket      = "YOUR-BUCKET-NAME-HERE" # ⚠️ REPLACE THIS!
    key         = "terraform.tfstate"
    region      = "us-east-1"
    dynamodb_table = "h-game-terraform-state-lock"
    encrypt     = true
  }
}
```

4. **IMPORTANT:** Replace **YOUR-BUCKET-NAME-HERE** with your actual bucket name from Step 0.1

Example (if your bucket is **h-game-tfstate-zee-1706544000):**

```

terraform {
  backend "s3" {
    bucket      = "h-game-tfstate-zee-1706544000" # Your actual
bucket name
    key         = "terraform.tfstate"
    region      = "us-east-1"
    dynamodb_table = "h-game-terraform-state-lock"
    encrypt     = true
  }
}

```

5. Save the file

Understanding each field:

- **bucket**: Your S3 bucket name (where state is stored)
- **key**: Filename for state file in S3 (like a file path)
- **region**: AWS region (must match where you created the bucket)
- **dynamodb_table**: Table name for locking (prevents conflicts)
- **encrypt**: Enable encryption (security best practice)

Step 0.5: Initialize Terraform with Remote Backend

Why: This connects Terraform to your S3 bucket and DynamoDB table.

Command:

```
terraform init
```

What you'll see:

```
Initializing the backend...
```

```
Successfully configured the backend "s3"! The following attributes
are copied over from your backend configuration:
```

- bucket
- dynamodb_table
- encrypt
- key

```
- region

Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws...
...

Terraform has been successfully initialized!
```

If you see an error:

- **Error: Failed to get existing workspaces:** Check your bucket name in `backend.tf` is correct
- **Error: AccessDenied:** Check your AWS credentials
- **Error: NoSuchBucket:** The bucket doesn't exist - go back to Step 0.2

Verification: Confirm Everything Works

Check 1: Verify backend is configured

```
# Check if backend config exists
cat .terraform/terraform.tfstate
```

Expected: You should see JSON with `"backend": { "type": "s3", ... }` containing your bucket name.

Check 2: Verify S3 bucket has state file (after first terraform apply)

```
# List files in your bucket
aws s3 ls s3://YOUR-BUCKET-NAME/
```

Expected (before first apply): Empty (no files yet) **Expected (after first apply):** You'll see `terraform.tfstate`

Check 3: Verify DynamoDB table is active

```
aws dynamodb describe-table \
  --table-name h-game-terraform-state-lock \
```

```
--query 'Table.TableStatus'
```

Expected output:

```
"ACTIVE"
```

Checkpoint: Remote State Ready

You should have:

- S3 bucket created (encrypted, versioned, private)
- DynamoDB table created (for state locking)
- `backend.tf` file configured with your bucket name
- Terraform initialized with remote backend

Cost estimate:

- S3: ~\$0.023 per GB/month (state files are tiny, ~few KB)
- DynamoDB: Pay-per-request (free tier: 25 GB storage, 25 read/write units)
- **Total: ~\$0.50/month or less** (practically free for learning)

Important Notes for Beginners

1. Don't delete the S3 bucket!

- It contains your infrastructure state
- If deleted, Terraform won't know what infrastructure exists
- You'd have to manually import everything (very painful)

2. Save your bucket name

- Write it down somewhere safe
- You'll need it if you recreate infrastructure
- Consider adding it to a README or notes file

3. State locking works automatically

- If you run `terraform apply` twice, the second one waits
- DynamoDB prevents state corruption
- No manual steps needed

4. Team collaboration

- If teammates have AWS access, they can use the same state
- Everyone sees the same infrastructure state
- No need to share state files manually

What Happens Next?

Now that remote state is configured:

- Every **terraform apply** saves state to S3
- State changes are versioned (can see history)
- Multiple people can work safely (DynamoDB locking)
- State is backed up automatically (S3 versioning)

Next step: Proceed to Step 1: Create VPC (or your next task)

Troubleshooting

Problem: "Bucket name already exists"

- Solution: Go back to Step 0.1, create a more unique name (add more characters)

Problem: "AccessDenied" errors

- Solution: Run **aws sts get-caller-identity** to verify credentials
- Check your IAM user has S3 and DynamoDB permissions

Problem: "NoSuchBucket" when running terraform init

- Solution: Verify bucket name in **backend.tf** matches the bucket you created
- Check bucket exists: **aws s3 ls**

Problem: "Table already exists"

- **Solution:** That's fine! The table already exists from a previous attempt. **Skip Step 0.3.**

Remote state configured! You're ready to proceed.

Step 1: Create VPC ([all code files in repo](#))

PS: Two ways to run specific resources:

Option 1: Use **-target** (recommended)

```
# Run only VPC resources
terraform plan -target=aws_vpc.main
terraform apply -target=aws_vpc.main
```

This creates only the VPC, ignoring everything else.

Option 2: Comment out resources (simple but manual) - Add # in front of resources you don't want in other .tf files, then run terraform apply normally.

Example for your setup:

```
terraform apply -target=aws_vpc.main
terraform apply -target=aws_vpc.main
```

Note: There's a state lock from a previous operation. Wait a moment, then try again, or if it's stuck, you can force unlock (not recommended unless necessary):

What we're doing: Creating your private network in AWS.

Create the VPC configuration:

```
# Navigate to Terraform directory
cd ~/projects/h-game-terraform
```

Create variables file first

IDE Option: Instead of using cat, create variables.tf in your IDE and paste the code below (see IDE-EDITOR-GUIDE.md)

```
cat > variables.tf << 'EOF'
# =====
# TERRAFORM VARIABLES
# =====
# Variables make your code reusable and configurable
# Values are set in terraform.tfvars (not committed to Git)
```

```

variable "aws_region" {
  description = "AWS region for resources"
  type        = string
  default     = "us-east-1"
}

variable "vpc_cidr" {
  description = "CIDR block for VPC"
  type        = string
  default     = "10.0.0.0/16"
}

variable "environment" {
  description = "Environment name (e.g., production, staging, dev)"
  type        = string
  default     = "production"

  validation {
    condition     = contains(["dev", "development", "staging", "production", "prod"],
var.environment)
    error_message = "Environment must be dev, development, staging, production, or
prod."
  }
}

variable "project_name" {
  description = "Project name for resource naming"
  type        = string
  default     = "h-game"
}

variable "availability_zones" {
  description = "List of availability zones for subnets"
  type        = list(string)
  default     = ["us-east-1a", "us-east-1b", "us-east-1c"]
}

variable "public_subnet_cidrs" {
  description = "CIDR blocks for public subnets (one per availability zone)"
  type        = list(string)
  default     = ["10.0.1.0/24", "10.0.2.0/24", "10.0.3.0/24"]
}

```



```

variable "private_subnet_cidrs" {
  description = "CIDR blocks for private subnets (one per availability zone)"
  type        = list(string)
  default     = ["10.0.11.0/24", "10.0.12.0/24", "10.0.13.0/24"]
}

variable "internet_route_cidr" {
  description = "CIDR block for internet route (typically 0.0.0.0/0 for all traffic)"
  type        = string
  default     = "0.0.0.0/0"
}
EOF

```

Create **terraform.tfvars** (your actual values - DO NOT COMMIT TO GIT!)

```

cat > terraform.tfvars << 'EOF'
# =====
# TERRAFORM VARIABLE VALUES
# =====
# ⚠ IMPORTANT: Add terraform.tfvars to .gitignore!
# This file contains your actual values and should NOT be committed to Git

aws_region      = "us-east-1"
vpc_cidr        = "10.0.0.0/16"
environment     = "production"
project_name    = "h-game"

availability_zones = ["us-east-1a", "us-east-1b", "us-east-1c"]
public_subnet_cidrs = ["10.0.1.0/24", "10.0.2.0/24", "10.0.3.0/24"]
private_subnet_cidrs = ["10.0.11.0/24", "10.0.12.0/24", "10.0.13.0/24"]
internet_route_cidr = "0.0.0.0/0" # Standard CIDR for "all internet traffic"
EOF

```

Create **.gitignore** to prevent committing sensitive files

```
cat > .gitignore << 'EOF'
# =====
# TERRAFORM SENSITIVE FILES
# =====
# ⚠ NEVER commit these files - they contain sensitive data!

# Variable files with actual values
terraform.tfvars
*.tfvars
!terraform.tfvars.example # Allow example file

# State files (contain passwords, endpoints, etc.)
terraform.tfstate
terraform.tfstate.backup
*.tfstate
*.tfstate.*

# Terraform directories
.terraform/

# Lock file - KEEP THIS (ensures version consistency)
# NOTE: .terraform.lock.hcl SHOULD be committed to Git
# It's already allowed (not ignored) - ensures everyone uses same provider
versions

# Crash Logs
crash.log
crash.*.log

# Override files (for local testing)
override.tf
override.tf.json
*_override.tf
*_override.tf.json

# CLI configuration
.terraformrc
terraform.rc

# IDE files
```

```
.vscode/
.idea/
*.swp
*.swo
*~

# OS files
.DS_Store
Thumbs.db
EOF
```

Create provider configuration

```
cat > provider.tf << 'EOF'
# =====
# TERRAFORM & PROVIDER CONFIGURATION
# =====

terraform {
  required_version = ">= 1.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = var.aws_region # Use variable instead of hardcoded

  # Default tags applied to all resources (unless overridden)
  default_tags {
    tags = {
      Project      = var.project_name
      Environment  = var.environment
      ManagedBy    = "Terraform"
    }
  }
}
```

```
EOF
```

Create locals file for common tags and naming

```
cat > locals.tf << 'EOF'
# =====
# TERRAFORM LOCALS
# =====
# Locals are computed values used throughout your configuration
# They reduce duplication and make code more maintainable

locals {
  # Common tags applied to all resources
  common_tags = {
    Project      = var.project_name
    Environment  = var.environment
    ManagedBy    = "Terraform"
  }

  # Name prefix for consistent naming
  name_prefix = "${var.project_name}-${var.environment}"
}
EOF
```

Create the VPC configuration:

IDE Option: Instead of using cat, create vpc.tf in your IDE and paste the code below (see IDE-EDITOR-GUIDE.md)

```
# Create VPC configuration
cat > vpc.tf << 'EOF'
# =====
# VPC (Virtual Private Cloud)
# =====
# Your private network in AWS
# Like your home network (192.168.x.x) but in the cloud

resource "aws_vpc" "main" {
  # CIDR block: IP address range for your network
  # Uses variable instead of hardcoded value
  cidr_block = var.vpc_cidr

  # Enable DNS hostnames (required for EKS)
  # Allows resources to have DNS names like: ip-10-0-1-5.ec2.internal
  enable_dns_hostnames = true

  # Enable DNS resolution (required for EKS)
  # Allows resources to resolve DNS names
  enable_dns_support = true

  # Tags: Labels for organization (uses locals for common tags)
  tags = merge(
    local.common_tags,
    {
      Name = "${local.name_prefix}-vpc"
      Task = "T1"
    }
  )
}

# Output: VPC ID (needed for other resources)
output "vpc_id" {
  description = "ID of the VPC"
  value       = aws_vpc.main.id
}

# Output: VPC CIDR block (for reference)
output "vpc_cidr" {
```

```

    description = "CIDR block of the VPC"
    value       = aws_vpc.main.cidr_block
  }
EOF

```

Initialize and plan:

```

# Initialize Terraform (if first time in this directory)
terraform init

# Preview what will be created
Terraform fmt
terraform plan

```

Expected output:

```

Plan: 1 to add, 0 to change, 0 to destroy.

# aws_vpc.main will be created
+ resource "aws_vpc" "main" {
  + cidr_block           = "10.0.0.0/16"
  + enable_dns_hostnames = true
  + enable_dns_support   = true
  + id                   = (known after apply)
  ...
}

```

Apply the configuration:

```

# Create the VPC
terraform apply
# Type 'yes' when prompted

```

Expected output:

```

aws_vpc.main: Creating...
aws_vpc.main: Creation complete after 2s [id=vpc-abc123xyz]

```

```
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

```
Outputs:
```

```
vpc_id = "vpc-abc123xyz"  
vpc_cidr = "10.0.0.0/16"
```

Verify in AWS Console:

1. Go to AWS Console → VPC → Your VPCs
2. You should see: **h-game-vpc** with CIDR **10.0.0.0/16**

Checkpoint - Did it work?

- VPC created successfully
- VPC ID shown in outputs
- VPC visible in AWS Console

VPC created!

Step 2: Create Internet Gateway

What we're doing: Creating a gateway that connects your VPC to the internet.

Why: Public subnets need this to access the internet.

Add Internet Gateway configuration:

```
# Add to vpc.tf (or create internet-gateway.tf)
cat >> vpc.tf << 'EOF'

# =====
# INTERNET GATEWAY
# =====
# Connects your VPC to the internet
# Like your home router's internet connection

resource "aws_internet_gateway" "main" {
  # Attach to the VPC we just created
  vpc_id = aws_vpc.main.id

  tags = merge(
    local.common_tags,
    {
      Name = "${local.name_prefix}-igw"
      Task = "T1"
    }
  )
}

# Output: Internet Gateway ID
output "internet_gateway_id" {
  description = "ID of the Internet Gateway"
  value       = aws_internet_gateway.main.id
}
EOF
```


Plan and apply:

```
terraform fmt

# Preview changes
terraform plan

# Should show: 1 to add (Internet Gateway)

# Apply changes
terraform apply

# Type 'yes' when prompted
```

Expected output:

```
aws_internet_gateway.main: Creating...
aws_internet_gateway.main: Creation complete after 2s [id=igw-abc123xyz]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
```

Checkpoint - Did it work?

- Internet Gateway created
- Attached to VPC
- Visible in AWS Console (VPC → Internet Gateways)

Internet Gateway created!

Step 3: Create Public Subnets

What we're doing: Creating public subnets in 3 availability zones for high availability.

Add public subnets configuration:

```
# Create subnets.tf
cat > subnets.tf << 'EOF'
# =====
# PUBLIC SUBNETS
# =====
# Subnets with direct internet access
# Use for: Load Balancers, NAT Gateways

# Public Subnet 1
resource "aws_subnet" "public_1" {
  # Belongs to our VPC
  vpc_id = aws_vpc.main.id

  # CIDR block from variable (10.0.1.0/24 = 256 IPs)
  cidr_block = var.public_subnet_cidrs[0]

  # Availability Zone from variable
  availability_zone = var.availability_zones[0]

  # Enable auto-assign public IP (for resources in this subnet)
  # Resources get public IP automatically (needed for internet access)
  map_public_ip_on_launch = true

  tags = merge(
    local.common_tags,
    {
      Name = "${local.name_prefix}-public-1"
      Type = "public"
      AZ   = var.availability_zones[0]
      Task = "T1"
    }
  )
}

# Public Subnet 2
resource "aws_subnet" "public_2" {
  vpc_id      = aws_vpc.main.id
  cidr_block  = var.public_subnet_cidrs[1] # Second CIDR from list
```

```

availability_zone = var.availability_zones[1]    # Second AZ from list

map_public_ip_on_launch = true

tags = merge(
  local.common_tags,
  {
    Name = "${local.name_prefix}-public-2"
    Type = "public"
    AZ   = var.availability_zones[1]
    Task = "T1"
  }
)
}

# Public Subnet 3
resource "aws_subnet" "public_3" {
  vpc_id          = aws_vpc.main.id
  cidr_block      = var.public_subnet_cidrs[2]  # Third CIDR from list
  availability_zone = var.availability_zones[2]  # Third AZ from list

  map_public_ip_on_launch = true

  tags = merge(
    local.common_tags,
    {
      Name = "${local.name_prefix}-public-3"
      Type = "public"
      AZ   = var.availability_zones[2]
      Task = "T1"
    }
  )
}

# Outputs: Subnet IDs (needed for other resources)
output "public_subnet_1_id" {
  description = "ID of public subnet 1"
  value       = aws_subnet.public_1.id
}

output "public_subnet_2_id" {
  description = "ID of public subnet 2"
  value       = aws_subnet.public_2.id
}

```

```
output "public_subnet_3_id" {  
  description = "ID of public subnet 3"  
  value       = aws_subnet.public_3.id  
}  
EOF
```

Plan and apply:

```
terraform fmt  
  
# Preview changes  
terraform plan  
  
# Should show: 3 to add (3 public subnets)  
  
# Apply changes  
terraform apply  
  
# Type 'yes' when prompted
```

Expected output:

```
aws_subnet.public_1: Creating...  
aws_subnet.public_2: Creating...  
aws_subnet.public_3: Creating...  
aws_subnet.public_1: Creation complete after 1s [id=subnet-abc123]  
aws_subnet.public_2: Creation complete after 1s [id=subnet-xyz789]  
aws_subnet.public_3: Creation complete after 1s [id=subnet-def456]  
  
Apply complete! Resources: 5 added, 0 changed, 0 destroyed.
```

Verify in AWS Console:

1. Go to AWS Console → VPC → Subnets
2. You should see 3 subnets:
 - `h-game-public-1` (10.0.1.0/24, us-east-1a)
 - `h-game-public-2` (10.0.2.0/24, us-east-1b)
 - `h-game-public-3` (10.0.3.0/24, us-east-1c)

Checkpoint - Did it work?

- 3 public subnets created
- Each in different AZ
- Different CIDR blocks (no overlap)
- Visible in AWS Console

Public subnets created!

Step 4: Create Public Route Table

What we're doing: Creating a route table that routes public subnet traffic to the Internet Gateway.

Why: Subnets need route tables to know where to send traffic.

Add route table configuration:

```
# Add to subnets.tf
cat >> subnets.tf << 'EOF'

# =====
# PUBLIC ROUTE TABLE
# =====
# Routes traffic from public subnets to Internet Gateway
# This is how public subnets get internet access

resource "aws_route_table" "public" {
  # Belongs to our VPC
  vpc_id = aws_vpc.main.id

  # Route: Send all internet traffic to Internet Gateway
  route {
    cidr_block = var.internet_route_cidr      # All traffic (from variable)
    gateway_id = aws_internet_gateway.main.id # Send to Internet Gateway
  }

  tags = merge(
    local.common_tags,
    {
      Name = "${local.name_prefix}-public-rt"
      Type = "public"
      Task = "T1"
    }
  )
}

# Associate public subnets with public route table
# This connects subnets to the route table

resource "aws_route_table_association" "public_1" {
  subnet_id      = aws_subnet.public_1.id
  route_table_id = aws_route_table.public.id
}
```

```

}

resource "aws_route_table_association" "public_2" {
  subnet_id      = aws_subnet.public_2.id
  route_table_id = aws_route_table.public.id
}

resource "aws_route_table_association" "public_3" {
  subnet_id      = aws_subnet.public_3.id
  route_table_id = aws_route_table.public.id
}
EOF

```

Plan and apply:

```

terraform fmt

# Preview changes
terraform plan

# Should show: 3 to add (3 public subnets)

# Apply changes
terraform apply

# Type 'yes' when prompted

```

Expected output:

```

aws_route_table.public: Creating...
aws_route_table_association.public_1: Creating...
aws_route_table_association.public_2: Creating...
aws_route_table_association.public_3: Creating...
aws_route_table.public: Creation complete after 1s [id=rtb-abc123]
aws_route_table_association.public_1: Creation complete after 1s [id=rta-xyz789]
aws_route_table_association.public_2: Creation complete after 1s [id=rta-def456]
aws_route_table_association.public_3: Creation complete after 1s [id=rta-ghi789]

Apply complete! Resources: 9 added, 0 changed, 0 destroyed.

```

What this does:

- Creates a route table with route to Internet Gateway
- Associate all 3 public subnets with this route table
- Now public subnets can reach the internet!

Checkpoint - Did it work?

- Route table created
- Route to Internet Gateway configured
- All 3 public subnets associated

Public routing configured!

Step 5: Create Private Subnets

What we're doing: Creating private subnets in 3 availability zones. These won't have direct internet access (more secure).

Add private subnets configuration:

```
# Add to subnets.tf
cat >> subnets.tf << 'EOF'

# =====
# PRIVATE SUBNETS
# =====
# Subnets WITHOUT direct internet access
# More secure - hidden from internet
# Use for: Application servers, Databases, Kubernetes nodes

# Private Subnet 1
resource "aws_subnet" "private_1" {
  vpc_id          = aws_vpc.main.id
  cidr_block      = var.private_subnet_cidrs[0] # First private CIDR from
list
  availability_zone = var.availability_zones[0]   # First AZ from list

  # NO map_public_ip_on_launch (private subnet!)
  # Resources won't get public IPs automatically

  tags = merge(
    local.common_tags,
    {
      Name = "${local.name_prefix}-private-1"
      Type = "private"
      AZ   = var.availability_zones[0]
      Task = "T1"
    }
  )
}

# Private Subnet 2
resource "aws_subnet" "private_2" {
  vpc_id          = aws_vpc.main.id
  cidr_block      = var.private_subnet_cidrs[1] # Second private CIDR from
list
  availability_zone = var.availability_zones[1]   # Second AZ from list
```

```

tags = merge(
  local.common_tags,
  {
    Name = "${local.name_prefix}-private-2"
    Type = "private"
    AZ   = var.availability_zones[1]
    Task = "T1"
  }
)
}

# Private Subnet 3
resource "aws_subnet" "private_3" {
  vpc_id      = aws_vpc.main.id
  cidr_block  = var.private_subnet_cidrs[2] # Third private CIDR from
list
  availability_zone = var.availability_zones[2] # Third AZ from list

  tags = merge(
    local.common_tags,
    {
      Name = "${local.name_prefix}-private-3"
      Type = "private"
      AZ   = var.availability_zones[2]
      Task = "T1"
    }
  )
}

# Outputs: Private subnet IDs
output "private_subnet_1_id" {
  description = "ID of private subnet 1"
  value       = aws_subnet.private_1.id
}

output "private_subnet_2_id" {
  description = "ID of private subnet 2"
  value       = aws_subnet.private_2.id
}

output "private_subnet_3_id" {
  description = "ID of private subnet 3"
  value       = aws_subnet.private_3.id
}

```

```
}
EOF
```

Note: Private subnets don't have a route to the Internet Gateway yet. We'll add NAT Gateway in T2 for private subnet internet access.

Plan and apply:

```
terraform fmt

# Preview changes
terraform plan

# Should show: 3 to add (3 public subnets)

# Apply changes
terraform apply

# Type 'yes' when prompted
```

Expected output:

```
aws_subnet.private_1: Creating...
aws_subnet.private_2: Creating...
aws_subnet.private_3: Creating...
aws_subnet.private_1: Creation complete after 1s [id=subnet-pvt1]
aws_subnet.private_2: Creation complete after 1s [id=subnet-pvt2]
aws_subnet.private_3: Creation complete after 1s [id=subnet-pvt3]

Apply complete! Resources: 12 added, 0 changed, 0 destroyed.
```

Checkpoint - Did it work?

- 3 private subnets created
- Each in a different AZ
- Different CIDR blocks (10.0.11-13.x)
- No public IP assignment (private!)

Private subnets created!

Step 6: Verify Network Setup

What we're doing: Verifying the complete network setup is correct.

View all resources:

```
# Show all Terraform resources
terraform show

# Or view specific outputs
terraform output
```

Verify in AWS Console:

1. VPC:

```
# List VPCs
aws ec2 describe-vpcs --filters "Name=tag:Name,Values=h-game-vpc"

# Should show: vpc-abc123xyz with CIDR 10.0.0.0/16
```

2. Subnets:

```
# List all subnets in VPC
VPC_ID=$(terraform output -raw vpc_id)
aws ec2 describe-subnets --filters "Name=vpc-id,Values=$VPC_ID"

# Should show: 6 subnets (3 public, 3 private)
```

3. Internet Gateway:

```
# List Internet Gateways
aws ec2 describe-internet-gateways --filters
"Name=tag:Name,Values=h-game-igw"

# Should show: igw-abc123 attached to VPC
```

4. Route Tables:

```
# List route tables
aws ec2 describe-route-tables --filters "Name=vpc-id,Values=$VPC_ID"

# Should show:
# - Default route table (created automatically)
# - Public route table (with route to Internet Gateway)
```

Visual verification in AWS Console:

1. VPC Dashboard:

- Go to AWS Console → VPC → Your VPCs
- Click on **h-game-vpc**
- See: 6 subnets, 1 Internet Gateway

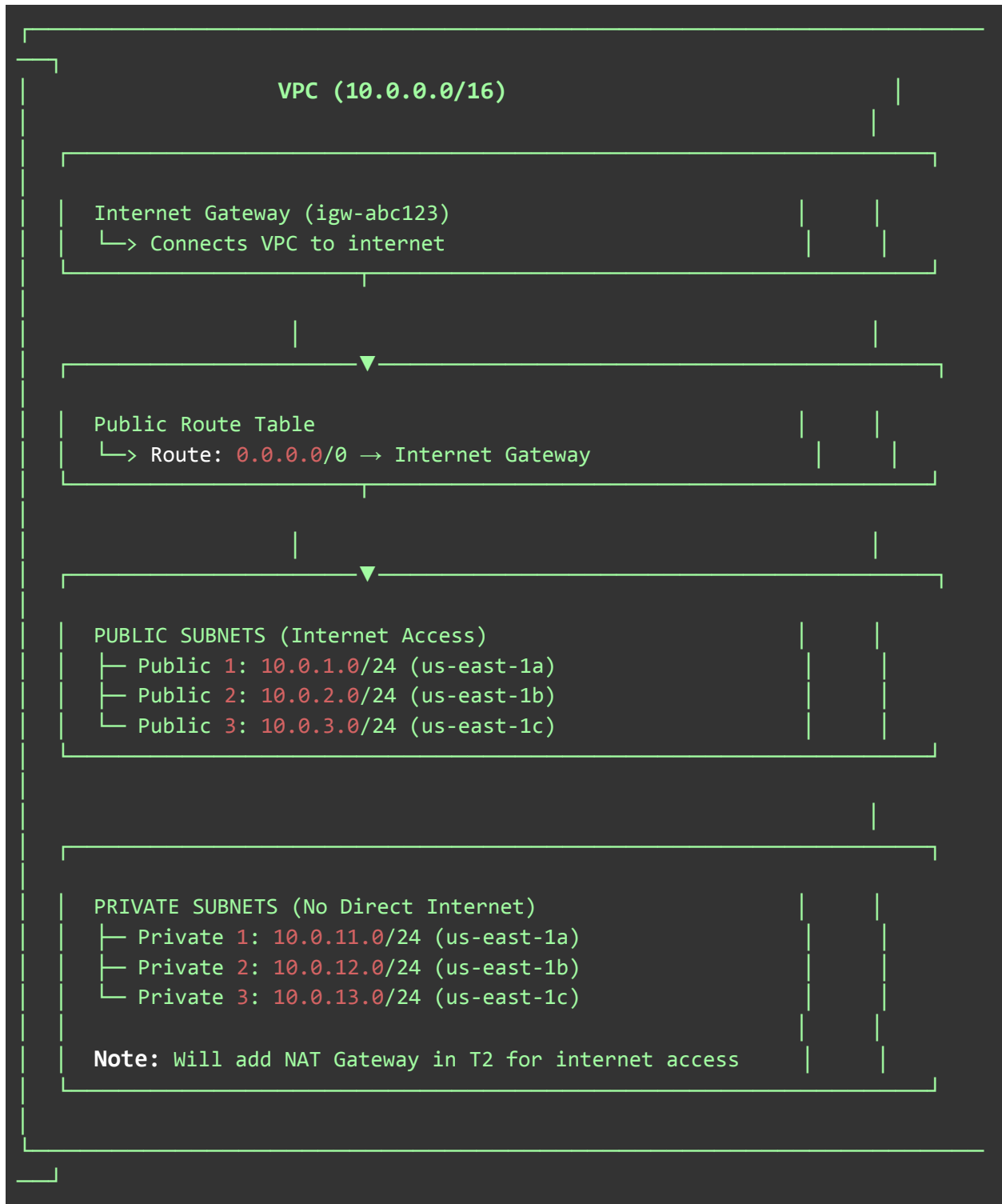
2. Subnets:

- Go to VPC → Subnets
- Filter by VPC: **h-game-vpc**
- See: 3 public (10.0.1-3.x), 3 private (10.0.11-13.x)

3. Route Tables:

- Go to VPC → Route Tables
- Find **h-game-public-rt**
- See: Route **0.0.0.0/0** → **igw-abc123**

Complete Network Architecture:



Checkpoint - Did it work?

- VPC created (10.0.0.0/16)
- Internet Gateway attached
- 3 public subnets (across 3 AZs)
- 3 private subnets (across 3 AZs)
- Public route table configured
- All resources are visible in the AWS Console

Network foundation complete!

Phase 3: Validation & Consolidation (30 min)

Quiz: VPC Understanding

Answer these questions to verify your understanding:

Question 1: What is a VPC?

Answer:

- **VPC** = Virtual Private Cloud
- Your own private network in AWS
- Like your home network (192.168.x.x), but in the cloud
- You control IP ranges, subnets, and routing
- Isolated from other AWS customers

Question 2: Why do we need public and private subnets?

Answer:

We use **public and private subnets** to control what is exposed to the internet and what stays protected.

- A **public subnet** can talk to the internet, so it's used for things like load balancers that must receive user traffic.
- A **private subnet** cannot be reached directly from the internet, so it's where apps, servers, and databases live safely. Private resources can still go out to the internet when needed, but outsiders can't reach them directly.

Simple rule: public faces users, private protects your systems.

Question 3: Why create subnets in 3 availability zones?

Answer:

- **High availability:** If one data center fails, others keep running
- **Load distribution:** Spread resources across AZs
- **Disaster recovery:** Data replicated across AZs
- **Production standard:** All production systems use multiple AZs

Key point: Multiple AZs = Resilience against data center failures.

Question 4: How does traffic flow from the public subnet to the internet?

Answer:

1. Resource in public subnet sends traffic
2. Route table checks the destination
3. Route **0.0.0.0/0** matches (all internet traffic)
4. Route sends to Internet Gateway
5. Internet Gateway forwards to Internet
6. Response comes back through the Internet Gateway
7. Route table routes to the resource

Flow: Resource → Route Table → Internet Gateway → Internet

Key point: Route tables control where traffic goes.

Consolidation Exercise: Draw Your Network

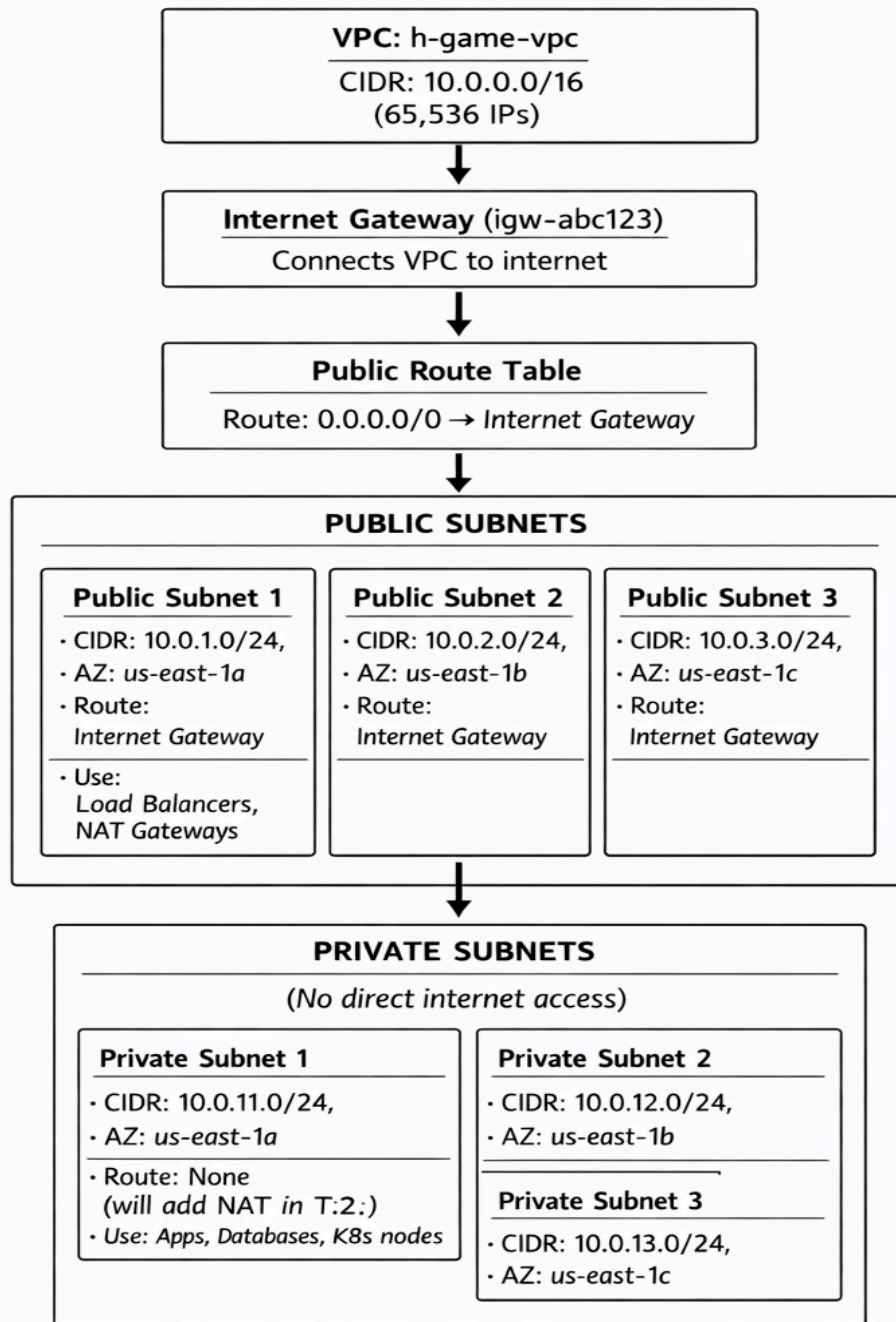
Your final challenge: Draw your complete VPC architecture from memory.

Include:

1. VPC with CIDR block
2. Internet Gateway
3. Public subnets (3, across 3 AZs)
4. Private subnets (3, across 3 AZs)
5. Route table (public)
6. How traffic flows

Take 5 minutes to draw this!

Reference: Complete VPC Architecture Below



TRAFFIC FLOW:

Internet → Internet Gateway → Public Route Table → Public Subnet → Resource

Resource → Public Subnet → Public Route Table → Internet Gateway → Internet

Checklist: Did You Complete Everything?

Before moving on, check these boxes:

Understanding Phase

- ☐ I understand what a VPC is (private network in AWS)
- ☐ I know why we use multiple AZs (high availability)
- ☐ I understand public vs private subnets
- ☐ I can explain CIDR blocks

Implementation Phase

- ☐ VPC created (10.0.0.0/16)
- ☐ Internet Gateway created and attached
- ☐ 3 public subnets created (across 3 AZs)
- ☐ Public route table created and configured
- ☐ 3 private subnets created (across 3 AZs)
- ☐ All resources verified in AWS Console

Verification Phase

- ☐ Can explain VPC architecture
- ☐ Can explain public vs private subnets
- ☐ Can explain route tables
- ☐ Can draw network architecture

Git

- ☐ Committed `vpc.tf` and `subnets.tf`
- ☐ Added `.gitignore` (exclude `terraform.tfstate`)

Key Takeaways

The Big Picture

What you learned in this task:

VPC FOUNDATION COMPLETE

- VPC (private network)
- Internet Gateway (internet access)
- Public subnets (3 AZs)
- Private subnets (3 AZs)
- Route tables (traffic control)

Network foundation is ready for infrastructure!

Critical Concepts

1. VPC:

- Your private network in AWS
- Like a home network, but in the cloud
- You control IP ranges, subnets, and routing

2. Subnets:

- Network segments within VPC
- Public = internet access
- Private = hidden from the internet
- Multiple AZs = high availability

3. Internet Gateway:

- Connects VPC to the internet
- Required for public subnets
- Like the router's internet connection

4. Route Tables:

- Control where traffic goes
- Public route table → Internet Gateway
- Private route table → (will add NAT in T2)

5. Network Architecture:

- VPC contains everything
- Subnets organize resources
- Route tables control traffic
- Multiple AZs provide resilience

What's Next?

Congratulations! You've built your network foundation!

You now have:

- Complete VPC with proper CIDR blocks
- Public and private subnets across 3 AZs
- Internet Gateway for public access
- Route tables configured
- Foundation ready for EKS, RDS, etc.

Next: TASK T2 - Security Groups & NAT

In T2, you'll learn:

- How to create security groups (firewall rules)
- How to add a NAT Gateway (for private subnet internet access)
- How to secure your network
- How all components work together

But most importantly: You now understand how networking works in AWS. This foundation will support all your infrastructure!

Proceed to TASK T2: Security Groups & NAT

TASK T2: Security Groups & NAT Gateway

Time: 3 hours | **Priority:** High | **Ticket:** DEVOPS-5102 | **Cost:** ~\$10 (NAT Gateway)

Mission Brief

Add security and private networking to your VPC. Create security groups (firewall rules) to control traffic, and add a NAT Gateway so private subnets can access the internet securely.

Outcomes:

- Understand security groups (firewall rules)
- Create security groups for different components
- Add NAT Gateway for private subnet internet access
- Understand how security fits into the complete system
- Configure private route tables

Prerequisites: Complete Task T1 (VPC Foundation)

Setup Note: Security groups and NAT Gateway will be created in your VPC (from T1). See Section 5 Overview for general setup details.

Cost Note: NAT Gateway costs ~\$0.045/hour (\$32/month if running 24/7). We'll use a single NAT Gateway (not one per AZ) to save costs. See Section 5 Overview for cost-saving strategies.

The One Thing You Must Know

Security groups are simply firewall rules that decide who is allowed to talk to what in your cloud.

Without them, your servers and databases are wide open, which is unsafe and unprofessional.

With security groups, you clearly say things like “only the backend can talk to the database” and nothing else gets through.

A NAT Gateway lets private resources reach the internet without being exposed, like using a one way door.

Once you understand this, security becomes simple: allow only what you intentionally choose.

WITHOUT SECURITY GROUPS:

- "All traffic allowed by default"
- "Database accessible from anywhere"
- "No control over who can access what"
- "Security nightmare"

WITH SECURITY GROUPS:

- "Only allowed traffic gets through."
- "Database only accessible from backend"
- "Complete control over access"
- "Production-grade security"

That's it. That's security groups.

Everything else (ingress, egress, rules) is just details. The core concept is:

"Only allow traffic you explicitly permit."

The Mental Model Shift:

BEFORE T2: "Resources can talk to each other freely"
"Private subnets can't reach the internet."
"No firewall rules"

AFTER T2: "Security groups control all traffic"
"Private subnets use NAT for the Internet."
"Least-privilege access (only what's needed)."

This is the difference between "default AWS" and "secure infrastructure."

Now let's see how security works...

Phase 1: Understanding Security (20 min)

What are Security Groups?

Think of security groups as the **security system of a well run building**.

By default, no one can walk in anywhere; every door is locked until a rule says otherwise.

Employees can enter the front door, IT staff can enter the server room, and only a few people can access sensitive offices.

In AWS, security groups work the same way by controlling which services are allowed to talk to each other and on which ports.

A database doesn't accept traffic from the internet, only from the backend that needs it. This approach keeps everything organized, intentional, and safe because **nothing is accessible unless you explicitly allow it**.

Key insight: Security groups = Firewall rules that control traffic to/from resources.

What is NAT Gateway?

The Private Subnet Internet Problem:

PRIVATE SUBNET RESOURCE:

Backend Pod (in private subnet)	
└ Needs to: Download npm packages	
└ Needs to: Call external APIs	
└ Problem: No direct internet access	
└ Solution: ???	

A NAT Gateway solves a simple but important problem: **private resources need the internet, but the internet should never reach them**.

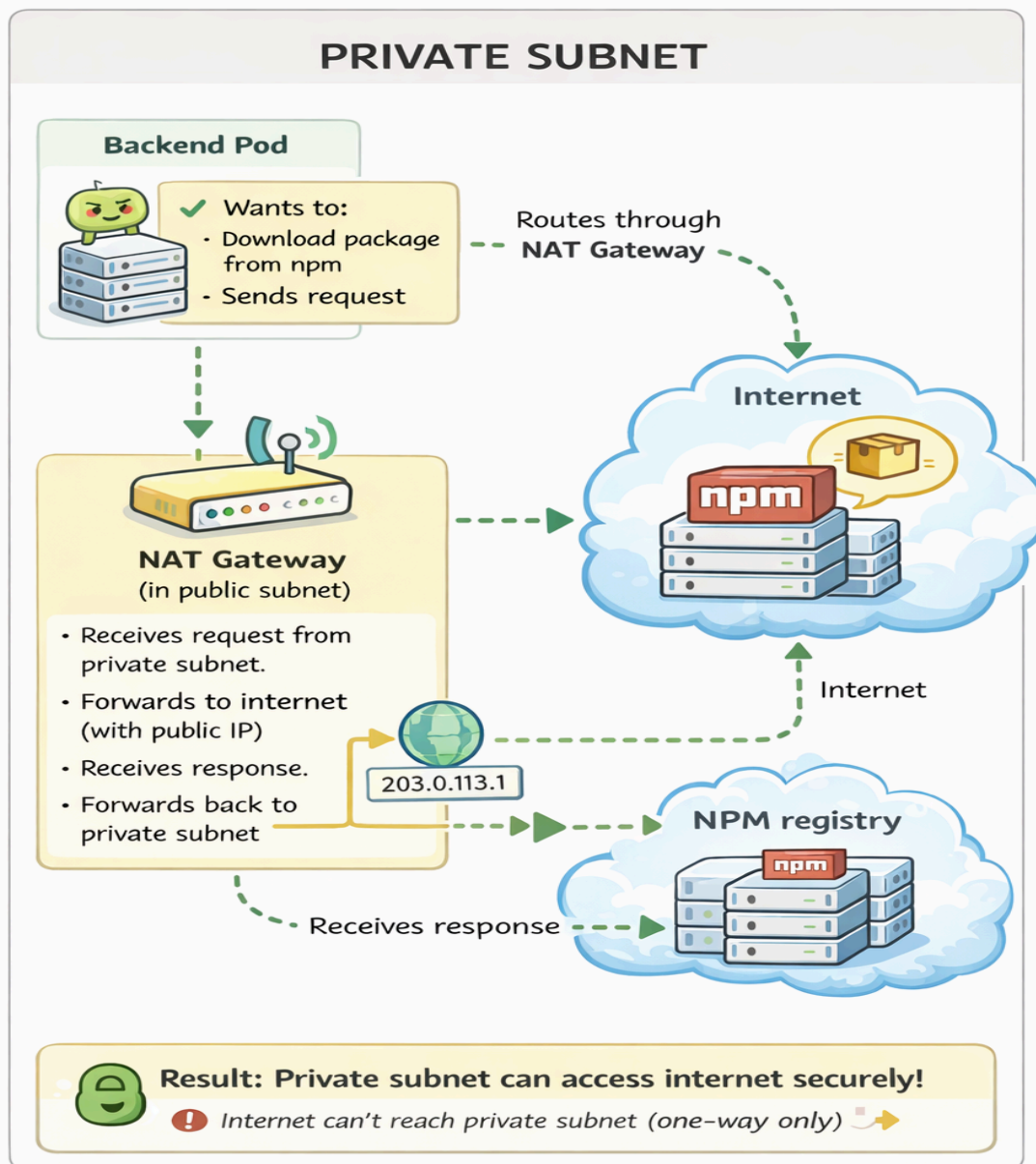
Imagine a backend service running in a private subnet that needs to download updates or call an external API; it can't do that directly because private subnets have no internet access.

The NAT Gateway **sits in a public subnet** and **acts like a trusted middleman**.

When the backend sends a request out, the NAT Gateway forwards it to the internet using its own public address and then returns the response.

From the outside world, nothing can start a connection back to the private resource, which keeps it hidden and secure while still fully functional.

Key insight: NAT Gateway = One way internet access for private subnets (outbound only, no inbound).



Exercise 1: Security Group Rules

Scenario: You need to secure your database.

Question: Which security group rules should you create?

Database needs:

- Accept connections from backend (port 5432)
- Reject all other traffic

Your task: Design the security group rules.

Ingress (Inbound) Rules:

```
Rule 1: Allow port 5432 from ???  
Rule 2: Deny all other traffic (default)
```

Egress (Outbound) Rules:

```
Rule 1: Allow all traffic (default for responses)
```

Your answer: Rule 1 should allow from _____

Answer Below 

Answer: Allow port 5432 from the backend security group

Security Group Configuration:

```
# Database Security Group
resource "aws_security_group" "database" {
  name          = "h-game-database-sg"
  description   = "Security group for RDS PostgreSQL database"
  vpc_id        = aws_vpc.main.id

  # Ingress Rule 1: Allow PostgreSQL (port 5432) from backend
  ingress {
    description      = "PostgreSQL from backend"
    from_port        = 5432                                # PostgreSQL port
    to_port          = 5432
    protocol          = "tcp"
    security_groups   = [aws_security_group.backend.id]     # Only from backend!
    # NOT from_port = 0.0.0.0/0 (that would allow anyone!)
  }

  # Ingress Rule 2: Deny all other traffic (default)
  # (No other ingress rules = deny all)

  # Egress: Allow all (database needs to respond)
  egress {
    from_port        = 0
    to_port          = 0
    protocol          = "-1" # All protocols
    cidr_blocks       = ["0.0.0.0/0"]
  }

  tags = {
    Name = "h-game-database-sg"
  }
}
```

Why this works:

- **Only backend can connect** (security_groups reference)
- **Port 5432 only** (PostgreSQL port)
- **No other traffic allowed** (default deny)
- **Least-privilege access** (only what's needed). **This is production grade security!**

Exercise 2: Why NAT Gateway?

Scenario: Your backend Pod (in private subnet) needs to download npm packages.

Question: How should it access the internet?

Option A: Put backend in public subnet

```
Backend in public subnet → Direct internet access
```

Option B: Use NAT Gateway

```
Backend in private subnet → NAT Gateway → Internet
```

Your answer: Which approach is better? _____

Answer Below 

Answer: Option B (NAT Gateway)**Why Option A is wrong:****Backend in public subnet:**

- └ Exposed to the internet (security risk)
- └ Gets public IP (can be attacked)
- └ Harder to secure
- └ Not production best practice

Why Option B is correct:**Backend in private subnet + NAT Gateway:**

- └ Hidden from internet (no public IP)
- └ Can access internet (via NAT)
- └ One-way access (outbound only)
- └ Internet can't reach backend directly
- └ Production best practice

Security principle:

- **Public subnets** = Resources that need to be reachable (Load Balancers)
- **Private subnets** = Resources that should be hidden (Apps, Databases)
- **NAT Gateway** = Allows private resources to reach the internet without exposing them

This is defense in depth!

Design Checkpoint

Before moving on, make sure you can answer these in plain English:

Question	Your Answer Should Be Like...
What is a security group?	"Firewall rules that control traffic to/from resources"
Why use security groups?	"Control who can access what (least-privilege security)"
What is NAT Gateway?	"Allows private subnets to access the internet without exposing them."
Why NAT Gateway?	"Private resources need internet, but shouldn't be exposed."

Can't answer one? Go back and re-read that exercise.

All clear? Great! Let's add security!

Phase 2: Implementation (2.5 hours)

What We're Building

Before this task:

- VPC with subnets (from T1)
- No security groups (default AWS rules)
- Private subnets can't access the internet

After this task:

- Security groups for all components
- NAT Gateway for private subnet internet access
- Private route tables configured
- Production grade security

Step-by-Step Plan

We'll do this in 5 steps:

Step 0: Configure Variables

"Set up variables for sec group and NAT"

Step 1: Create Security Groups

"Firewall rules for all components"

Step 2: Create NAT Gateway

"Internet access for private subnets"

Step 3: Create Private Route Table

"Route private traffic through NAT"

Step 4: Associate Private Subnets

"Connect private subnets to NAT"

Step 5: Verify Security Setup

"Confirm everything is secure."

Step 0: Configure Variables

Why we're doing this: Variables separate configuration from code, making infrastructure reusable and keeping sensitive data secure.

Add security and NAT variables (if not already in variables.tf):

```
# Add to variables.tf
# IDE Option: Instead of using cat, add these to your variables.tf file in your
# IDE

cat >> variables.tf << 'EOF'

# =====
# SECURITY & NAT VARIABLES
# =====

variable "enable_nat_gateway" {
  description = "Enable NAT Gateway for private subnet internet access"
  type        = bool
  default     = true
}

variable "nat_gateway_count" {
  description = "Number of NAT Gateways (1 for cost savings, 3 for high
availability)"
  type        = number
  default     = 1 # Single NAT saves ~$22/month vs 3 NATs
}

variable "backend_port" {
  description = "Backend application port"
  type        = number
  default     = 3001
}

variable "postgres_port" {
  description = "PostgreSQL database port"
  type        = number
  default     = 5432
}

variable "redis_port" {
  description = "Redis cache port"
```

```

    type      = number
    default   = 6379
  }

  variable "http_port" {
    description = "HTTP port"
    type        = number
    default     = 80
  }

  variable "https_port" {
    description = "HTTPS port"
    type        = number
    default     = 443
  }
EOF

```

Add to terraform.tfvars

```

cat >> terraform.tfvars << 'EOF'

# Security & NAT Configuration
enable_nat_gateway = true
nat_gateway_count  = 1 # Use 1 for Learning (saves cost), 3 for production
                      (high availability)

# Port Configuration (standard ports - rarely change)
backend_port       = 3001
postgres_port      = 5432
redis_port         = 6379
http_port          = 80
https_port         = 443
EOF

```

Variables configured!

Note: Ensure `.gitignore` is configured (created in T1). If you need to create it:

Create **.gitignore** (if not already done in T1)

```
cat > .gitignore << 'EOF'
# =====
# TERRAFORM SENSITIVE FILES
# =====
# ⚠ NEVER commit these files - they contain sensitive data!

# Variable files with actual values
terraform.tfvars
*.tfvars
!terraform.tfvars.example # Allow example file

# State files (contain passwords, endpoints, etc.)
terraform.tfstate
terraform.tfstate.backup
*.tfstate
*.tfstate.*

# Terraform directories
.terraform/

# Lock file - KEEP THIS (ensures version consistency)
# NOTE: .terraform.lock.hcl SHOULD be committed to Git
# It's already allowed (not ignored) - ensures everyone uses same provider
versions

# Crash Logs
crash.log
crash.*.log

# Override files (for local testing)
override.tf
override.tf.json
*_override.tf
*_override.tf.json

# CLI configuration
.terraformrc
terraform.rc

# IDE files
.vscode/
.idea/
```

```
*.swp  
*.sw0  
*~  
  
# OS files  
.DS_Store  
Thumbs.db  
EOF
```

Step 1: Create Security Groups

What we're doing: Creating firewall rules for all components (frontend, backend, database, cache).

Note: This task uses the `provider.tf` and `variables.tf` files created in T1. If you haven't completed **T1**, do that first.

Create security groups configuration:

```
# Create security-groups.tf
# IDE Option: Instead of using cat, create security-groups.tf in your IDE and paste the
# code below (see IDE-EDITOR-GUIDE.md)

cat > security-groups.tf << 'EOF'
# =====
# SECURITY GROUPS
# =====
# Firewall rules that control traffic
# Default: Deny all (nothing gets through)
# Rules: Allow specific traffic only

# =====
# ALB (Application Load Balancer) Security Group
# =====
# Allows internet traffic to reach Load Balancer
resource "aws_security_group" "alb" {
  name          = "${local.name_prefix}-alb-sg"
  description   = "Security group for Application Load Balancer"
  vpc_id        = aws_vpc.main.id

  # Ingress Rule: Allow HTTP from internet
  ingress {
    description = "HTTP from internet"
    from_port   = var.http_port           # HTTP port (from variable)
    to_port     = var.http_port
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]           # From anywhere (internet)
  }

  # Ingress Rule: Allow HTTPS from internet
  ingress {
    description = "HTTPS from internet"
    from_port   = var.https_port          # HTTPS port (from variable)
```

```

    to_port      = var.https_port
    protocol     = "tcp"
    cidr_blocks  = ["0.0.0.0/0"]          # From anywhere (internet)
  }

  # Egress: Allow all (ALB needs to forward to backend)
  egress {
    from_port    = 0
    to_port      = 0
    protocol     = "-1"                  # All protocols
    cidr_blocks  = ["0.0.0.0/0"]
  }

  tags = merge(
    local.common_tags,
    {
      Name = "${local.name_prefix}-alb-sg"
      Task = "T2"
    }
  )
}

# =====
# Backend Security Group
# =====
# Allows ALB to reach backend, backend to reach database/cache
resource "aws_security_group" "backend" {
  name          = "${local.name_prefix}-backend-sg"
  description   = "Security group for backend Pods"
  vpc_id        = aws_vpc.main.id

  # Ingress Rule: Allow HTTP from ALB only
  ingress {
    description     = "HTTP from ALB"
    from_port       = var.backend_port      # Backend port (from variable)
    to_port         = var.backend_port
    protocol        = "tcp"
    security_groups = [aws_security_group.alb.id] # Only from ALB!
  }

  # Egress: Allow all (backend needs to reach database, cache, internet)
  egress {
    from_port    = 0
    to_port      = 0
  }
}

```

```

    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
}

tags = merge(
    local.common_tags,
    {
        Name = "${local.name_prefix}-backend-sg"
        Task = "T2"
    }
)
}

# =====
# Database Security Group
# =====
# Allows backend to reach database, nothing else
resource "aws_security_group" "database" {
    name          = "${local.name_prefix}-database-sg"
    description   = "Security group for RDS PostgreSQL"
    vpc_id        = aws_vpc.main.id

    # Ingress Rule: Allow PostgreSQL from backend only
    ingress {
        description      = "PostgreSQL from backend"
        from_port         = var.postgres_port    # PostgreSQL port (from variable)
        to_port           = var.postgres_port
        protocol           = "tcp"
        security_groups    = [aws_security_group.backend.id] # Only from backend!
    }

    # Egress: Allow all (database needs to respond)
    egress {
        from_port = 0
        to_port   = 0
        protocol  = "-1"
        cidr_blocks = ["0.0.0.0/0"]
    }

    tags = merge(
        local.common_tags,
        {
            Name = "${local.name_prefix}-database-sg"
            Task = "T2"
        }
    )
}

```

```

    }
  )
}

# =====
# Cache Security Group
# =====
# Allows backend to reach cache, nothing else
resource "aws_security_group" "cache" {
  name          = "${local.name_prefix}-cache-sg"
  description   = "Security group for ElastiCache Redis"
  vpc_id        = aws_vpc.main.id

  # Ingress Rule: Allow Redis from backend only
  ingress {
    description      = "Redis from backend"
    from_port        = var.redis_port      # Redis port (from variable)
    to_port          = var.redis_port
    protocol         = "tcp"
    security_groups  = [aws_security_group.backend.id] # Only from backend!
  }

  # Egress: Allow all (cache needs to respond)
  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = merge(
    local.common_tags,
    {
      Name = "${local.name_prefix}-cache-sg"
      Task = "T2"
    }
  )
}

# =====
# EKS Cluster Security Group
# =====
# Allows cluster communication
resource "aws_security_group" "eks_cluster" {

```



```

name      = "${local.name_prefix}-eks-cluster-sg"
description = "Security group for EKS cluster"
vpc_id    = aws_vpc.main.id

# Ingress: Allow cluster communication (will be configured by EKS)
# EKS will add its own rules automatically

# Egress: Allow all (cluster needs to communicate)
egress {
  from_port = 0
  to_port   = 0
  protocol  = "-1"
  cidr_blocks = ["0.0.0.0/0"]
}

tags = merge(
  local.common_tags,
  {
    Name = "${local.name_prefix}-eks-cluster-sg"
    Task = "T2"
  }
)
}

# =====
# EKS Node Group Security Group
# =====
# Allows worker nodes to communicate
resource "aws_security_group" "eks_nodes" {
  name      = "${local.name_prefix}-eks-nodes-sg"
  description = "Security group for EKS worker nodes"
  vpc_id    = aws_vpc.main.id

# Ingress: Allow from cluster and other nodes
  ingress {
    description = "From cluster"
    from_port   = 1025                # Kubernetes ports
    to_port     = 65535
    protocol    = "tcp"
    security_groups = [aws_security_group.eks_cluster.id]
  }

  ingress {
    description = "From nodes"
  }
}

```

```

    from_port    = 1025
    to_port      = 65535
    protocol     = "tcp"
    self         = true                                # Allow from other nodes in same group
}

# Egress: Allow all (nodes need to pull images, communicate)
egress {
    from_port    = 0
    to_port      = 0
    protocol     = "-1"
    cidr_blocks  = ["0.0.0.0/0"]
}

tags = merge(
    local.common_tags,
    {
        Name = "${local.name_prefix}-eks-nodes-sg"
        Task = "T2"
    }
)
}

# Outputs: Security Group IDs (needed for other resources)
output "alb_security_group_id" {
    description = "ID of ALB security group"
    value       = aws_security_group.alb.id
}

output "backend_security_group_id" {
    description = "ID of backend security group"
    value       = aws_security_group.backend.id
}

output "database_security_group_id" {
    description = "ID of database security group"
    value       = aws_security_group.database.id
}

output "cache_security_group_id" {
    description = "ID of cache security group"
    value       = aws_security_group.cache.id
}
EOF

```

Format and validate code:

```
# Format Terraform code (fixes indentation, spacing)
terraform fmt
# Automatically formats your code:
# - Fixes indentation (2 spaces)
# - Aligns equals signs
# - Sorts attributes
# - Makes code readable

# Validate Terraform syntax and configuration
terraform validate
# Checks for errors:
# - Syntax errors (missing brackets, quotes)
# - Invalid attribute names
# - Type mismatches
# - Missing required attributes
# Should show: "Success! The configuration is valid."
```

Plan and apply:

```
# Preview changes
terraform plan
# Review the plan carefully:
# - Check what will be created (+)
# - Check what will be changed (~)
# - Check what will be destroyed (-)
# Always review before apply!

# Should show: 6 to add (6 security groups)

# Apply changes
terraform apply

# Type 'yes' when prompted
```

Expected output:

```
aws_security_group.alb: Creating...
aws_security_group.backend: Creating...
aws_security_group.database: Creating...
aws_security_group.cache: Creating...
aws_security_group.eks_cluster: Creating...
aws_security_group.eks_nodes: Creating...
aws_security_group.alb: Creation complete after 2s [id=sg-abc123]
aws_security_group.backend: Creation complete after 2s [id=sg-xyz789]
aws_security_group.database: Creation complete after 2s [id=sg-def456]
aws_security_group.cache: Creation complete after 2s [id=sg-ghi789]
aws_security_group.eks_cluster: Creation complete after 2s [id=sg-jkl012]
aws_security_group.eks_nodes: Creation complete after 2s [id=sg-mno345]

Apply complete! Resources: 15 added, 0 changed, 0 destroyed.
```

Verify in AWS Console:

1. Go to AWS Console → VPC → Security Groups
2. Filter by VPC: **h-game-vpc**
3. You should see 6 security groups:
 - **h-game-alb-sg**
 - **h-game-backend-sg**
 - **h-game-database-sg**
 - **h-game-cache-sg**
 - **h-game-eks-cluster-sg**
 - **h-game-eks-nodes-sg**

Checkpoint - Did it work?

- ☐ 6 security groups created
- ☐ Each has appropriate rules
- ☐ Rules reference other security groups (not IPs)
- ☐ Visible in AWS Console

Security groups created!

This step is about **putting security boundaries around every part of your system** so nothing is exposed by accident.

First, a security group is created for the load balancer, allowing traffic from the internet so users can reach the app. **The backend** gets its own security group, but it only accepts traffic from the load balancer, not from random sources.

Next, the database is locked down so only the backend can talk to it, keeping sensitive data off the internet.

The cache follows the same rule, listening only to the backend for fast internal communication.

Separate security groups are then created for the Kubernetes control plane and worker nodes so they can communicate safely with each other.

All outgoing traffic is allowed where needed, so services can respond or fetch updates.

Terraform creates all these rules in one go and keeps track of them so they can be reused and audited later.

By the end, every component has a clear “who can talk to me” rule, which is the foundation of secure, production-ready infrastructure.

Step 2: Create NAT Gateway

What we're doing: Creating a NAT Gateway so private subnets can access the internet.

Why: Private subnets need internet access (download packages, call APIs) but shouldn't be exposed.

Create NAT Gateway configuration:

```
# Create nat-gateway.tf
cat > nat-gateway.tf << 'EOF'
# =====
# NAT GATEWAY
# =====
# Allows private subnets to access internet
# Without exposing private resources to internet

# Elastic IP for NAT Gateway
# NAT Gateway needs a static public IP
resource "aws_eip" "nat" {
  # EIP must be in VPC (not EC2-Classic)
  domain = "vpc"

  tags = merge(
    local.common_tags,
    {
      Name = "${local.name_prefix}-nat-eip"
      Task = "T2"
    }
  )
}

# Ensure Internet Gateway exists first
depends_on = [aws_internet_gateway.main]
}

# NAT Gateway
# Placed in public subnet (needs internet access)
resource "aws_nat_gateway" "main" {
  # Allocate Elastic IP (static public IP)
  allocation_id = aws_eip.nat.id

  # Place in public subnet (needs internet access)
  subnet_id = aws_subnet.public_1.id # Using first public subnet
```

```

tags = merge(
  local.common_tags,
  {
    Name = "${local.name_prefix}-nat-gateway"
    Task = "T2"
  }
)

# Ensure EIP is created first
depends_on = [aws_eip.nat]
}

# Output: NAT Gateway ID
output "nat_gateway_id" {
  description = "ID of the NAT Gateway"
  value       = aws_nat_gateway.main.id
}

# Output: NAT Gateway public IP
output "nat_gateway_public_ip" {
  description = "Public IP of the NAT Gateway"
  value       = aws_eip.nat.public_ip
}
EOF

```

Plan and apply:

```

# Preview changes
terraform plan

# Should show: 2 to add (EIP + NAT Gateway)
# Cost: ~$0.045/hour for NAT Gateway

# Apply changes
terraform apply

# Type 'yes' when prompted

```

Expected output:

```
aws_eip.nat: Creating...
aws_eip.nat: Creation complete after 2s [id=eipalloc-abc123]
aws_nat_gateway.main: Creating...
aws_nat_gateway.main: Still creating... [10s elapsed]
aws_nat_gateway.main: Still creating... [20s elapsed]
aws_nat_gateway.main: Creation complete after 30s [id=nat-abc123xyz]

Apply complete! Resources: 17 added, 0 changed, 0 destroyed.
```

Note: NAT Gateway takes 2-5 minutes to create. This is normal!

Verify in AWS Console:

1. Go to AWS Console → VPC → NAT Gateways
2. You should see: **h-game-nat-gateway** in public subnet 1
3. Status: Available
4. Public IP: Shows the Elastic IP

Checkpoint - Did it work?

- ☐ Elastic IP created
- ☐ NAT Gateway created (takes 2-5 minutes)
- ☐ NAT Gateway in public subnet
- ☐ Status: Available

NAT Gateway created!

Step 3: Create Private Route Table ([all code files in repo](#))

What we're doing: Creating a route table that routes private subnet traffic through the NAT Gateway.

Why: Private subnets need a route to reach the internet via the NAT Gateway.

Add private route table configuration:

```
# Add to subnets.tf
cat >> subnets.tf << 'EOF'

# =====
# PRIVATE ROUTE TABLE
# =====
# Routes traffic from private subnets through NAT Gateway
# This is how private subnets get internet access

resource "aws_route_table" "private" {
  vpc_id = aws_vpc.main.id

  # Route: Send all internet traffic to NAT Gateway
  route {
    cidr_block      = var.internet_route_cidr # ALL internet traffic (from
variable)
    nat_gateway_id = aws_nat_gateway.main.id # Send to NAT Gateway
  }

  tags = merge(
    local.common_tags,
    {
      Name = "${local.name_prefix}-private-rt"
      Type = "private"
      Task = "T2"
    }
  )
}

# Associate private subnets with private route table
resource "aws_route_table_association" "private_1" {
  subnet_id      = aws_subnet.private_1.id
  route_table_id = aws_route_table.private.id
}
```

```
resource "aws_route_table_association" "private_2" {
  subnet_id      = aws_subnet.private_2.id
  route_table_id = aws_route_table.private.id
}

resource "aws_route_table_association" "private_3" {
  subnet_id      = aws_subnet.private_3.id
  route_table_id = aws_route_table.private.id
}
EOF
```

Plan and apply:

```
# Preview changes
terraform plan

# Should show: 4 to add (1 route table + 3 associations)

# Apply changes
terraform apply

# Type 'yes' when prompted
```

Expected output:

```
aws_route_table.private: Creating...
aws_route_table_association.private_1: Creating...
aws_route_table_association.private_2: Creating...
aws_route_table_association.private_3: Creating...
aws_route_table.private: Creation complete after 1s [id=rtb-private]
aws_route_table_association.private_1: Creation complete after 1s [id=rta-pvt1]
aws_route_table_association.private_2: Creation complete after 1s [id=rta-pvt2]
aws_route_table_association.private_3: Creation complete after 1s [id=rta-pvt3]

Apply complete! Resources: 21 added, 0 changed, 0 destroyed.
```

What this does:

- Creates a private route table with a route to NAT Gateway
- Associates all 3 private subnets with this route table
- Now private subnets can reach internet via NAT Gateway!

Why we need a private route table

Private subnets need internet access (e.g., to pull Docker images or download packages), but they can't use the Internet Gateway directly for security. **The NAT Gateway provides that internet access**, but subnets need a route table to know to use it. **Associating the private subnets with this route table** tells their traffic to go through the NAT Gateway instead of trying to reach the internet directly.

Analogy: Think of the route table as a sign at a junction: "To reach the internet, go through the NAT Gateway (not the Internet Gateway)." Without this association, private subnets don't know which path to take and can't reach the internet.

Checkpoint - Did it work?

- ☐ Private route table created
- ☐ Route to NAT Gateway configured
- ☐ All 3 private subnets are associated

Private routing configured!

Production Optimization Note: VPC Endpoints

Current setup: Nodes in private subnets access AWS APIs (ECR, CloudWatch) via NAT Gateway.

Optimization: VPC Endpoints allow private access to AWS services without NAT Gateway:

- **More secure:** Traffic stays within AWS network
- **Potentially cheaper:** No NAT Gateway data transfer costs
- **Faster:** Direct connection to AWS services

When to use:

- High data transfer to AWS services (ECR, S3, CloudWatch)
- Security requirements (no internet gateway needed)
- Cost optimization (can reduce NAT Gateway usage)

Cost: ~\$7/month per endpoint (vs NAT Gateway data transfer costs)

Example services: ECR, S3, CloudWatch Logs, Systems Manager

For learning, NAT Gateway is simpler and sufficient. **For production:** Consider VPC Endpoints for cost/security optimization.

Step 4: Verify Security Setup

What we're doing: Verifying the complete security and networking setup.

View all resources:

```
# Show all outputs
terraform output

# Should show:
# - VPC ID
# - Subnet IDs (public and private)
# - Security Group IDs
# - NAT Gateway ID
```

Verify security groups:

```
# List all security groups
aws ec2 describe-security-groups \
  --filters "Name=vpc-id,Values=$(terraform output -raw vpc_id)" \
  --query 'SecurityGroups[*].[GroupName,GroupId]' \
  --output table

# Should show: 6 security groups
```

Verify NAT Gateway:

```
# Describe NAT Gateway
aws ec2 describe-nat-gateways \
  --filter "Name=tag:Name,Values=h-game-nat-gateway" \
  --query 'NatGateways[*].[NatGatewayId,State,SubnetId]' \
  --output table

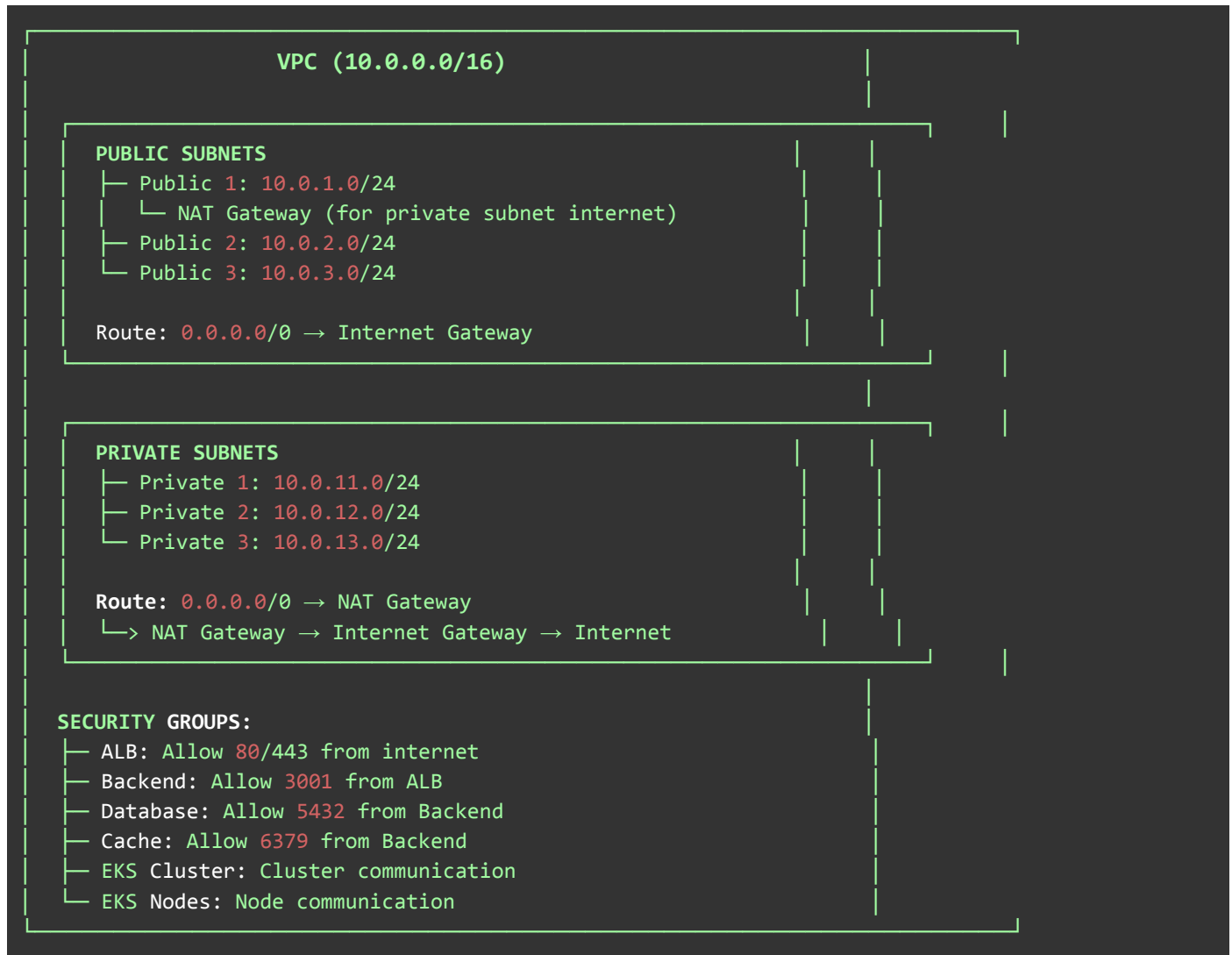
# Should show: nat-abc123, available, subnet-public-1
```

Verify route tables:

```
# List route tables
VPC_ID=$(terraform output -raw vpc_id)
aws ec2 describe-route-tables \
  --filters "Name=vpc-id,Values=$VPC_ID" \
  --query
'RouteTables[*].[RouteTableId,Tags[?Key==`Name`].Value|[0],Routes[*].
[DestinationCidrBlock,GatewayId,NatGatewayId]]' \
  --output table

# Should show:
# - Public route table: 0.0.0.0/0 → Internet Gateway
# - Private route table: 0.0.0.0/0 → NAT Gateway
```

Complete Security Architecture:



Checkpoint - Did it work?

- ☐ All security groups created
- ☐ NAT Gateway created and available
- ☐ Private route table configured
- ☐ All private subnets can reach the internet via NAT
- ☐ Security groups have least-privilege rules

Security and NAT complete!

Phase 3: Validation & Consolidation (10 min)

Quiz: Security Understanding

Answer these questions to verify your understanding:

Question 1: What is a security group?

Answer:

- **Security Group** = Firewall rules for AWS resources
- Controls inbound (ingress) and outbound (egress) traffic
- Default: Deny all (nothing gets through)
- Rules: Allow specific traffic only
- Stateful: If you allow inbound, outbound responses are automatically allowed

Key point: Security groups = Firewall that controls who can access what.

Question 2: Why use security groups instead of allowing all traffic?

Answer:

- **Security:** Only allow traffic you need (least-privilege)
- **Attack surface reduction:** Fewer open ports = fewer attack vectors
- **Compliance:** Required for production systems
- **Best practice:** Defense in depth

Example:

- **Allow all:** Database accessible from anywhere (security risk!)
- **Security group:** Database only accessible from backend (secure!)

Key point: Security groups enforce least-privilege access.

Question 3: What is NAT Gateway and why do we need it?

Answer:

- **NAT Gateway** = Network Address Translation gateway
- Allows private subnets to access the internet
- One-way access: Outbound only (private → internet)
- Internet cannot reach private resources directly
- More secure than putting resources in public subnets

Why needed:

- Private subnets need internet (download packages, call APIs)
- But shouldn't be exposed to internet (security)
- NAT Gateway provides secure internet access

Key point: NAT Gateway = Secure internet access for private resources.

Question 4: How does traffic flow from the private subnet to the internet?

Answer:

1. Resource in private subnet sends traffic
2. Route table checks destination (0.0.0.0/0 = internet)
3. Route sends to NAT Gateway
4. NAT Gateway forwards to Internet Gateway (with public IP)
5. Internet Gateway forwards to Internet
6. Response comes back through Internet Gateway → NAT Gateway → Private subnet

Flow: Private Resource → Route Table → NAT Gateway → Internet Gateway → Internet

Key point: Route tables control where traffic goes.

Checklist: Did You Complete Everything?

Before moving on, check these boxes:

Understanding Phase

- ☐ I understand what security groups are (firewall rules)
- ☐ I understand why NAT Gateway is needed
- ☐ I understand public vs private routing
- ☐ I completed all design exercises

Implementation Phase

- ☐ 6 security groups created (ALB, backend, database, cache, EKS cluster, EKS nodes)
- ☐ NAT Gateway created (in public subnet)
- ☐ Private route table created
- ☐ All private subnets associated with the private route table
- ☐ All resources verified

Verification Phase

- ☐ Can explain security groups
- ☐ Can explain NAT Gateway
- ☐ Can explain traffic flow
- ☐ Can draw a complete security architecture

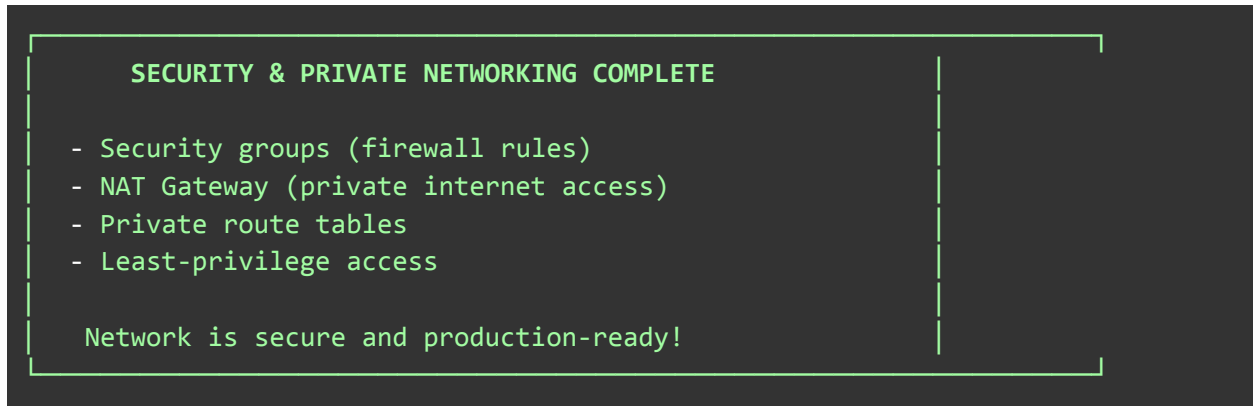
Git

- ☐ Committed **security-groups.tf** and **nat-gateway.tf**
- ☐ Updated **subnets.tf** with private route table

Key Takeaways

The Big Picture

What you learned in this task:



Critical Concepts

1. Security Groups:

- Firewall rules for AWS resources
- Default: Deny all
- Rules: Allow specific traffic only
- Least-privilege access

2. NAT Gateway:

- Allows private subnets to access the internet
- One-way access (outbound only)
- More secure than public subnets
- Required for private resources that need the internet

3. Route Tables:

- Control where traffic goes
- Public route table → Internet Gateway
- Private route table → NAT Gateway

4. Security Architecture:

- ALB: Accessible from the internet

- Backend: Only from ALB
- Database: Only from Backend
- Cache: Only from Backend
- Defense in depth!

What's Next?

Congratulations! You've secured your network!

You now have:

- ☐ Complete security groups for all components
- ☐ NAT Gateway for private subnet internet access
- ☐ Private route tables configured
- ☐ Production-grade security

Next: TASK T3 - EKS Cluster (Control Plane)

In T3, you'll learn:

- How to create an EKS cluster (managed Kubernetes)
- How EKS replaces MicroK8s
- How to configure cluster networking
- How cluster connects to your VPC

Most importantly, you now understand how security works in AWS. This foundation will protect all your infrastructure!

Proceed to TASK T3: EKS Cluster, Control Plane

TASK T3: EKS Cluster - Control Plane

Time: 4 hours | **Priority:** High | **Ticket:** DEVOPS-5103 | **Cost:** \$73/month (EKS control plane)

Mission Brief

Create an EKS (Elastic Kubernetes Service) cluster - AWS's managed Kubernetes. This replaces MicroK8s with a production-grade, managed Kubernetes control plane. The cluster will run in your VPC with proper networking and security.

Outcomes:

- Understand EKS vs self-managed Kubernetes
- Create EKS cluster with Terraform
- Configure cluster networking (VPC, subnets, security groups)
- Understand IAM roles for EKS
- Connect kubectl to EKS cluster

Prerequisites: Complete Task T2 (Security Groups & NAT)

Setup Note: EKS cluster will be created in your VPC (from T1). See Section 5 Overview for general setup details.

Cost Note: EKS control plane costs \$0.10/hour (\$73/month). This is the managed control plane (API server, etcd, controller manager). Worker nodes cost extra (we'll add in T4). See Section 5 Overview for cost-saving strategies.

IDE Note: This task involves creating Terraform files. Instead of using `cat > file.tf << 'EOF'` commands, you can use your IDE (VS Code, Cursor, etc.) to create and edit files. See [IDE-EDITOR-GUIDE.md](#) for details. Both methods work, use whichever you prefer!

The One Thing That Is Important

EKS is simply **Kubernetes without the hardest maintenance work.**

With self-managed Kubernetes, you are responsible for the control plane, including upgrades, backups, scaling, and fixing failures.

If something breaks, it's entirely on you to troubleshoot and recover.

With EKS, AWS runs and maintains the Kubernetes control plane for you.

You don't manage the API server or etcd, AWS does that reliably in the background.

Your job is to run worker nodes and deploy applications where your Pods live.

This shifts your focus from keeping Kubernetes alive to building and running software.

That's the real difference between self managed Kubernetes and managed Kubernetes.

EKS = Managed Kubernetes. AWS runs the control plane, you run the worker nodes. Same Kubernetes, less operational burden.

SELF-MANAGED KUBERNETES (MicroK8s):

- **"You manage:** API server, etcd, controller manager"
- **"You handle:** Upgrades, backups, scaling"
- **"You fix:** When control plane breaks"
- **"Operational burden:** High"

MANAGED KUBERNETES (EKS):

- **"AWS manages:** API server, etcd, controller manager"
- **"AWS handles:** Upgrades, backups, scaling"
- **"AWS fixes:** When control plane breaks"
- **"You manage:** Worker nodes (Pods run here)"
- **"Operational burden:** Low"

That's it. That's EKS.

Everything else (networking, IAM, node groups) is just configuration. The core concept is:

"AWS runs Kubernetes control plane, you run applications on worker nodes."

The Mental Model Shift:

BEFORE T3:

- "Kubernetes = I install and manage everything"
- "MicroK8s on EC2 = I manage control plane"
- "If control plane breaks, I fix it"

AFTER T3:

- "Kubernetes = AWS manages control plane"
- "EKS = AWS handles control plane operations"
- "If control plane breaks, AWS fixes it"
- "I focus on applications, not infrastructure"

This is the difference between "self-managed" and "managed Kubernetes."

Now let's see how EKS works...

Phase 1: Understanding EKS (30 min)

What is EKS?

EKS = Elastic Kubernetes Service

The Managed Service Analogy

SELF-MANAGED (MicroK8s):

You manage everything:

- Install Kubernetes
- Configure API server
- Manage etcd (cluster database)
- Handle upgrades
- Fix when it breaks
- Operational burden: HIGH

Time spent: 30% on apps, 70% on infrastructure

MANAGED (EKS):

AWS manages control plane:

- AWS runs API server
- AWS manages etcd
- AWS handles upgrades
- AWS fixes when it breaks
- Operational burden: LOW

You manage:

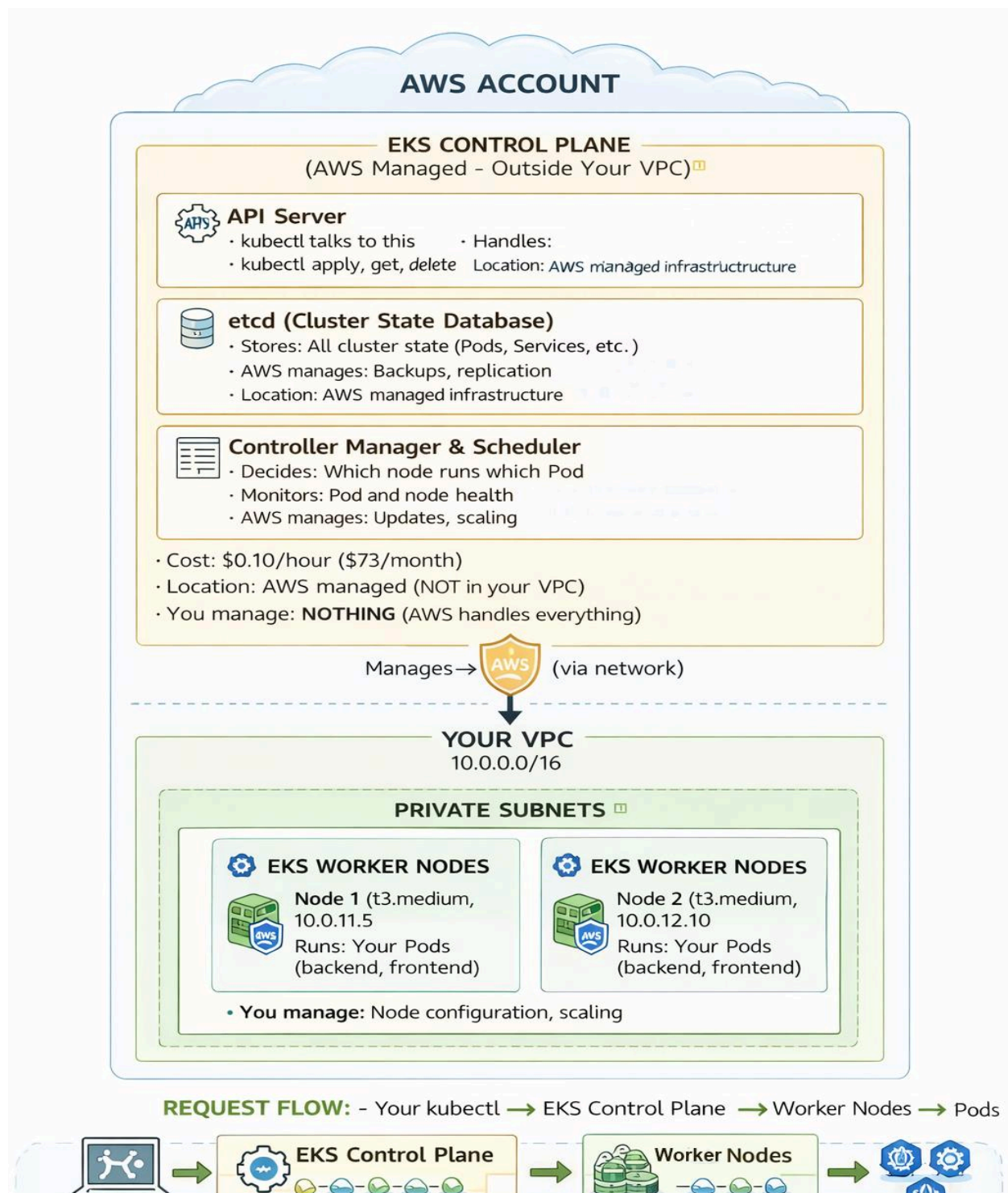
- Worker nodes (where Pods run)
- Applications (Deployments, Services)
- Focus on: Business logic, not infrastructure

Time spent: 80% on apps, 20% on infrastructure

Key insight: EKS = AWS runs Kubernetes control plane, you run applications.

EKS Architecture

Visual Architecture Diagram:



Key points:

- **Control plane** = AWS managed (outside your VPC, in AWS infrastructure)
- **Worker nodes** = You manage (in your VPC private subnets, EC2 instances)
- **Separation:** Control plane and nodes are separate (different infrastructure)
- **Connection:** Control plane manages nodes via network (not physically connected)
- **Same Kubernetes** = Same kubectl commands, same YAML
- **Less operational burden** = AWS handles control plane operations

EKS is split into two clear parts: the control plane and the worker nodes.

The **control plane** is fully managed by AWS and lives outside your VPC, meaning AWS runs the API server, etcd, the scheduler, and all upgrades and backups.

This is the part your `kubectl` talks to when you run commands like apply, get, or delete.

You don't install, patch, or fix anything in the control plane, AWS handles all of it for a small monthly cost.

Inside your own VPC, in private subnets, are the worker nodes, which are regular EC2 instances you manage.

These **worker nodes** are where your Pods actually run and where your applications live.

The **control plane** communicates with the worker nodes over the network to decide where Pods should run and to monitor health.

Even though the control plane and nodes live in different places, they work together as one Kubernetes cluster.

From your perspective, it's still standard Kubernetes with the same YAML and kubectl commands, just with far less operational effort.

Exercise 1: Why EKS Instead of MicroK8s?

Scenario: You need to choose between MicroK8s (self-managed) and EKS (managed).

Question: When should you use each?

Option A: MicroK8s (Self-Managed)

- You install and manage Kubernetes
- Full control over control plane
- Lower cost (no EKS fee)
- More operational work

Option B: EKS (Managed)

- AWS manages control plane
- Less control over control plane
- Higher cost (\$73/month)
- Less operational work

Your answer: Use EKS when _____

Answer Below 

Answer: Use EKS for production, MicroK8s for learning

When to use MicroK8s:

- Learning Kubernetes (K1-K9)
- Local development
- Cost-sensitive projects
- Full control needed
- Small scale (< 10 nodes)

When to use EKS:

- Production workloads
- Need high availability
- Want managed upgrades
- Compliance requirements
- Large scale (> 10 nodes)
- Team doesn't want to manage control plane

For this course:

- **K1-K9:** MicroK8s (learning, low cost)
- **T3-T10:** EKS (production, real-world experience)

This gives you experience with both!

Exercise 2: EKS Networking

Scenario: You're creating an EKS cluster.

Question: Which subnets should the cluster use?

Options:

- A) Public subnets only
- B) Private subnets only
- C) Both public and private subnets

Think about:

- Where should worker nodes run? (Public or private?)
- What about security?
- What about internet access?

Your answer: _____

Answer Below 

Answer: C) Both public and private subnets (but nodes in private)

EKS Cluster Configuration:

```
resource "aws_eks_cluster" "main" {
  vpc_config {
    # Cluster needs subnets in multiple AZs
    subnet_ids = [
      aws_subnet.private_1.id, # Private subnet 1
      aws_subnet.private_2.id, # Private subnet 2
      aws_subnet.private_3.id, # Private subnet 3
      # Can also include public subnets (for Load Balancers)
    ]

    # Endpoint access
    endpoint_private_access = true # Allow private access
    endpoint_public_access  = true # Allow public access (for
    kubectl)
  }
}
```

Why this configuration:

- Private subnets = Where worker nodes run (secure, hidden)
- Public subnets = Can include for Load Balancers
- Private access = Cluster accessible from VPC
- Public access = kubectl can connect from internet

Worker nodes should be in private subnets:

- More secure (hidden from internet)
- Access internet via NAT Gateway (from T2)
- Production best practice

This is the standard EKS setup!

Design Checkpoint

Before moving on, make sure you can answer these in plain English:

Question	Your Answer Should Be Like...
What is EKS?	"AWS managed Kubernetes - AWS runs control plane"
Why use EKS?	"Less operational burden, AWS handles control plane"
Where do worker nodes run?	"In private subnets (secure, hidden from internet)"
What does EKS cost?	"\$0.10/hour for control plane, nodes cost extra"

Can't answer one? Go back and re-read that exercise.

All clear? Great! Let's create the EKS cluster!

Phase 2: Implementation (3 hours)

What We're Building

Before this task:

- VPC with networking (from T1)
- Security groups (from T2)
- No Kubernetes cluster

After this task:

- EKS cluster (managed control plane)
- Cluster configured with VPC networking
- IAM roles for cluster
- kubectl configured to connect to EKS
- Ready for worker nodes (T4)

Step-by-Step Plan

We'll do this in 5 steps:

Step 0: Configure Variables

"Set Up variables for EKS Cluster"

Step 1: Create IAM Roles for EKS

"Permissions for cluster to manage AWS resources"

Step 2: Create EKS Cluster

"Managed Kubernetes control plane"

Step 3: Configure kubectl

"Connect to EKS cluster"

Step 4: Verify Cluster

"Confirm cluster is working"

Step 5: Test Cluster Connection

"Run kubectl commands"

Note: This task uses the `provider.tf` and `variables.tf` files created in T1. If you haven't completed T1, do that first.

Step 0: Configure Variables

Why we're doing this: Variables separate configuration from code, making infrastructure reusable and keeping sensitive data secure.

Add EKS cluster variables (if not already in `variables.tf`):

Add to `variables.tf`

```
# IDE Option: Instead of using cat, add these to your variables.tf file in your IDE

cat >> variables.tf << 'EOF'

# =====
# EKS CLUSTER VARIABLES
# =====

variable "eks_cluster_name" {
  description = "EKS cluster name"
  type        = string
  default     = "h-game-prod"
}

variable "kubernetes_version" {
  description = "Kubernetes version for EKS cluster"
  type        = string
  default     = "1.28"
}

variable "enable_cluster_logging" {
  description = "Enable EKS cluster logging to CloudWatch"
  type        = bool
  default     = true
}

variable "cluster_log_types" {
  description = "EKS cluster log types to enable"
  type        = list(string)
  default     = ["api", "audit", "authenticator", "controllerManager", "scheduler"]
}
EOF
```

Add to terraform.tfvars

```
cat >> terraform.tfvars << 'EOF'

# EKS Cluster Configuration
eks_cluster_name      = "h-game-prod"
kubernetes_version    = "1.28"
enable_cluster_logging = true
cluster_log_types      = ["api", "audit", "authenticator",
"controllerManager", "scheduler"]
EOF
```

Variables configured!

Step 1: Create IAM Roles for EKS

What we're doing: Creating IAM roles that give EKS permission to manage AWS resources (like creating Load Balancers, attaching ENIs).

Why: EKS needs permissions to:

- Create Load Balancers (for Services)
- Attach network interfaces (for Pods)
- Access ECR (to pull container images)
- Manage worker nodes

IAM Roles Explained (Beginner-Friendly):

IAM roles are like **permission badges** that AWS services wear to know what they're allowed to do.

- The **EKS** cluster has its own role so it can create load balancers and manage networking on your behalf.
- The worker nodes have a different role that lets them pull container images and send logs to AWS.

Each role has a specific job, and nothing gets extra permissions by accident.

These roles must exist before the EKS cluster starts, otherwise AWS doesn't know what permissions to give it.

That's why we create IAM roles first and then attach them when creating the cluster.

Once everything is connected, Kubernetes can safely use AWS services and your applications run smoothly.

Create IAM roles configuration:

```

# Create iam.tf
# IDE Option: Instead of using cat, create iam.tf in your IDE and paste the
code below (see IDE-EDITOR-GUIDE.md)

cat > iam.tf << 'EOF'
# =====
# IAM ROLES FOR EKS
# =====
# EKS needs permissions to manage AWS resources
# This is how EKS creates Load Balancers, attaches network interfaces, etc.

# =====
# EKS Cluster Role
# =====
# Role that EKS cluster assumes
# Allows cluster to manage AWS resources

resource "aws_iam_role" "eks_cluster" {
  name = "${local.name_prefix}-eks-cluster-role"

  # Trust policy: Allow EKS service to assume this role
  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Action = "sts:AssumeRole"
        Effect = "Allow"
        Principal = {
          Service = "eks.amazonaws.com" # EKS service can assume this role
        }
      }
    ]
  })

  tags = merge(
    local.common_tags,
    {
      Name = "${local.name_prefix}-eks-cluster-role"
      Task = "T3"
    }
  )
}

```

```

# Attach AWS managed policy: EKS Cluster Policy
# This gives cluster permissions to manage AWS resources
resource "aws_iam_role_policy_attachment" "eks_cluster_policy" {
  policy_arn = "arn:aws:iam::aws:policy/AmazonEKSClusterPolicy"
  role       = aws_iam_role.eks_cluster.name
}

# =====
# EKS Node Group Role
# =====
# Role that worker nodes assume
# Allows nodes to pull images, write logs, etc.

resource "aws_iam_role" "eks_nodes" {
  name = "${local.name_prefix}-eks-nodes-role"

  # Trust policy: Allow EC2 service to assume this role
  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Action = "sts:AssumeRole"
        Effect = "Allow"
        Principal = {
          Service = "ec2.amazonaws.com" # EC2 instances (worker nodes) can
assume this role
        }
      }
    ]
  })

  tags = merge(
    local.common_tags,
    {
      Name = "${local.name_prefix}-eks-nodes-role"
      Task = "T3"
    }
  )
}

# Attach AWS managed policies for worker nodes
resource "aws_iam_role_policy_attachment" "eks_worker_node_policy" {
  policy_arn = "arn:aws:iam::aws:policy/AmazonEKSWorkerNodePolicy"

```

```

    role      = aws_iam_role.eks_nodes.name
  }

resource "aws_iam_role_policy_attachment" "eks_cni_policy" {
  policy_arn = "arn:aws:iam::aws:policy/AmazonEKS_CNI_Policy"
  role       = aws_iam_role.eks_nodes.name
}

resource "aws_iam_role_policy_attachment" "eks_container_registry_policy" {
  policy_arn = "arn:aws:iam::aws:policy/AmazonEC2ContainerRegistryReadOnly"
  role       = aws_iam_role.eks_nodes.name
}

# Outputs: Role ARNs (needed for EKS cluster)
output "eks_cluster_role_arn" {
  description = "ARN of EKS cluster IAM role"
  value       = aws_iam_role.eks_cluster.arn
}

output "eks_nodes_role_arn" {
  description = "ARN of EKS nodes IAM role"
  value       = aws_iam_role.eks_nodes.arn
}
EOF

```

Plan and apply:

```
# Preview changes
terraform plan
# Review the plan carefully:
# - Check what will be created (+)
# - Check what will be changed (~)
# - Check what will be destroyed (-)
# Always review before apply!

# Should show: 2 to add (2 IAM roles) + 4 policy attachments

# Apply changes
terraform apply

# Type 'yes' when prompted
```

Expected output:

```
aws_iam_role.eks_cluster: Creating...
aws_iam_role.eks_nodes: Creating...
aws_iam_role_policy_attachment.eks_cluster_policy: Creating...
aws_iam_role_policy_attachment.eks_worker_node_policy: Creating...
aws_iam_role_policy_attachment.eks_cni_policy: Creating...
aws_iam_role_policy_attachment.eks_container_registry_policy: Creating...
aws_iam_role.eks_cluster: Creation complete after 2s [id=h-game-eks-cluster-role]
aws_iam_role.eks_nodes: Creation complete after 2s [id=h-game-eks-nodes-role]
...

Apply complete! Resources: 23 added, 0 changed, 0 destroyed.
```

Checkpoint - Did it work?

- ☐ IAM roles created
- ☐ Policies attached
- ☐ Role ARNs in outputs

IAM roles ready!

Step 2: Create EKS Cluster

What we're doing: Creating the EKS cluster (managed Kubernetes control plane).

Create EKS cluster configuration:

```
# Create eks-cluster.tf
# Option 1: Using command-line (shown below)
# Option 2: Using IDE - Create eks-cluster.tf in your IDE and paste the code
# (see IDE-EDITOR-GUIDE.md)

cat > eks-cluster.tf << 'EOF'
# =====
# EKS CLUSTER
# =====
# AWS managed Kubernetes control plane
# AWS runs: API server, etcd, controller manager
# You run: Worker nodes (in T4)

resource "aws_eks_cluster" "main" {
  name      = var.eks_cluster_name      # Cluster name (from variable)
  role_arn  = aws_iam_role.eks_cluster.arn # IAM role for cluster
  version   = var.kubernetes_version    # Kubernetes version (from variable)

  # VPC Configuration
  vpc_config {
    # Subnets where cluster will create resources
    # Use private subnets (more secure)
    subnet_ids = [
      aws_subnet.private_1.id,
      aws_subnet.private_2.id,
      aws_subnet.private_3.id,
    ]

    # Endpoint access configuration
    endpoint_private_access = true # Allow access from VPC
    endpoint_public_access  = true # Allow access from internet (for kubectl)

    # Security groups for cluster
    # EKS will create additional security groups automatically
    security_group_ids = [aws_security_group.eks_cluster.id]
  }

  # Enable Logging (optional but recommended)
```



```

    enabled_cluster_log_types = var.enable_cluster_logging ?
var.cluster_log_types : [] # From variables

# Ensure IAM roles are created before cluster
depends_on = [
    aws_iam_role_policy_attachment.eks_cluster_policy,
]

tags = merge(
    local.common_tags,
    {
        Name = "${local.name_prefix}-eks-cluster"
        Task = "T3"
    }
)
}

# Output: Cluster name
output "eks_cluster_name" {
    description = "Name of the EKS cluster"
    value       = aws_eks_cluster.main.name
}

# Output: Cluster endpoint (for kubectl)
output "eks_cluster_endpoint" {
    description = "Endpoint for EKS cluster"
    value       = aws_eks_cluster.main.endpoint
}

# Output: Cluster version
output "eks_cluster_version" {
    description = "Kubernetes version of the cluster"
    value       = aws_eks_cluster.main.version
}
EOF

```

Plan and apply:

```
# Preview changes
terraform plan

# Should show: 1 to add (EKS cluster)
# Note: This will take 10-15 minutes to create!

# Apply changes
terraform apply

# Type 'yes' when prompted
# Wait 10-15 minutes for cluster to be created
```

Expected output:

```
aws_eks_cluster.main: Creating...
aws_eks_cluster.main: Still creating... [10s elapsed]
aws_eks_cluster.main: Still creating... [1m0s elapsed]
aws_eks_cluster.main: Still creating... [2m0s elapsed]
...
aws_eks_cluster.main: Still creating... [10m0s elapsed]
aws_eks_cluster.main: Creation complete after 12m30s [id=h-game-prod]

Apply complete! Resources: 24 added, 0 changed, 0 destroyed.
```

Outputs:

```
eks_cluster_name      = "h-game-prod"
eks_cluster_endpoint  = "https://abc123xyz.gr7.us-east-1.eks.amazonaws.com"
eks_cluster_version   = "1.28"
```

Note: EKS cluster creation takes 10-15 minutes. This is normal! AWS is provisioning the managed control plane.

Verify in AWS Console:

1. Go to AWS Console → EKS → Clusters
2. You should see: **h-game-prod**
3. Status: Active
4. Version: 1.28

Checkpoint - Did it work?

- EKS cluster created (took 10-15 minutes)
- Cluster status: Active
- Endpoint shown in outputs
- Visible in AWS Console

EKS cluster created!

Step 3: Configure kubectl

What we're doing: Configuring kubectl to connect to your EKS cluster.

Install/Update AWS CLI and kubectl:

```
# Check AWS CLI version (need 2.7.0+)
aws --version

# Check kubectl version
kubectl version --client

# If not installed or outdated, install/update:
# macOS: brew install awscli kubectl
# Linux: Follow AWS/kubectl installation guides
```

Update kubeconfig:

```
# Add EKS cluster to kubeconfig
aws eks update-kubeconfig --region us-east-1 --name h-game-prod

# This command:
# 1. Gets cluster endpoint from AWS
# 2. Gets cluster certificate
# 3. Adds cluster to ~/.kube/config
# 4. Sets current context to h-game-prod
```

Expected output:

```
Added new context
arn:aws:eks:us-east-1:123456789012:cluster/h-game-prod to
/Users/yourname/.kube/config
```

Verify kubectl connection:

```
# Test connection to cluster
kubectl cluster-info

# Expected output:
# Kubernetes control plane is running at
https://abc123xyz.gr7.us-east-1.eks.amazonaws.com
# CoreDNS is running at
https://abc123xyz.gr7.us-east-1.eks.amazonaws.com/api/v1/namespaces/k
ube-system/services/kube-dns:dns/proxy
```

Check cluster nodes (will be empty until T4):

```
# List nodes (should be empty - no worker nodes yet)
kubectl get nodes

# Expected: No resources found (no worker nodes yet)
# We'll add worker nodes in T4!
```

Checkpoint - Did it work?

- kubectl configured
- `kubectl cluster-info` shows cluster endpoint
- Can connect to cluster
- No nodes yet (normal - we'll add in T4)

kubectl configured!

Step 4: Verify Cluster

What we're doing: Verifying the EKS cluster is fully operational.

Check cluster status:

```
# Describe cluster
aws eks describe-cluster --name h-game-prod --region us-east-1
# Or
aws eks describe-cluster --name h-game-prod --region us-east-1
--query
'cluster.{Status:status,Version:version,Endpoint:endpoint,VpcConfig:v
pcConfig}' --output json

# Should show:
# - Status: ACTIVE
# - Version: 1.29
# - Endpoint: https://...
# - VPC config: Subnets, security groups
```

Check cluster components:

```
# Check system Pods (should exist even without worker nodes)
kubectl get pods -n kube-system

# Expected: Some system Pods (CoreDNS, etc.)
# Note: Some Pods won't run until we add worker nodes (T4)

Check cluster resources:
# List all resources in cluster
kubectl get all --all-namespaces

# Should show: System resources (CoreDNS, etc.)
```

Verify in AWS Console:

1. Go to AWS Console → EKS → Clusters → h-game-prod
2. Check:
 - **Status:** Active

- **Version:** 1.29
- **Networking:** Shows your VPC and subnets
- **Logging:** Enabled

Please always verify codes in repo match with the ones in the book:
([all code files in repo](#))

Checkpoint - Did it work?

- Cluster status: Active
- kubectl can connect
- System Pods running (some)
- Cluster visible in AWS Console

Cluster verified!

Step 5: Test Cluster Connection

What we're doing: Running a test to confirm everything works.

Create a test namespace:

```
# Create test namespace
kubectl create namespace test

# Verify it was created
kubectl get namespaces

# Should show: test namespace
```

Create a test Pod:

```
# Create test Pod
kubectl run test-pod --image=nginx:alpine --namespace=test

# Note: This will be Pending (no worker nodes yet!)
# We'll add worker nodes in T4, then Pods will run
```

Check Pod status:

```
# Check Pod status
kubectl get pods -n test

# Expected: test-pod status = Pending
# Reason: No worker nodes to run on
# This is normal! We'll add nodes in T4.
```

Delete test resources:

```
# Clean up test resources
kubectl delete namespace test

# Verify deleted
kubectl get namespaces
# test namespace should be gone
```

Checkpoint - Did it work?

- Can create namespaces
- Can create Pods (they'll be Pending until T4)
- kubectl commands work
- Cluster is operational

Cluster connection tested!

Phase 3: Validation & Consolidation (30 min)

Quiz: EKS Understanding

Answer these questions to verify your understanding:

Question 1: What is EKS?

Answer:

- **EKS** = Elastic Kubernetes Service
- AWS managed Kubernetes
- AWS runs: Control plane (API server, etcd, controllers)
- You run: Worker nodes (where Pods run)
- Same Kubernetes: Same kubectl, same YAML

Key point: EKS = Managed Kubernetes control plane.

Question 2: Why use EKS instead of self-managed Kubernetes?

Answer:

- **Less operational burden:** AWS manages control plane
- **Automatic upgrades:** AWS handles Kubernetes version updates
- **High availability:** AWS ensures control plane is always available
- **Backups:** AWS handles etcd backups
- **Focus on apps:** You focus on applications, not infrastructure

Trade-off: Higher cost (\$73/month) but less operational work.

Key point: EKS = Less operational work, more focus on applications.

Question 3: Where does the EKS control plane run?

Answer:

- **Control plane:** AWS managed (not in your VPC)
- **Worker nodes:** Your VPC private subnets (you manage)
- **Connection:** Control plane manages worker nodes via network

Key point: Control plane = AWS managed, Nodes = Your VPC.

Question 4: Why do worker nodes need to be in private subnets?

Answer:

- **Security:** Hidden from internet (no public IPs)
- **Best practice:** Production systems should be in private subnets
- **Internet access:** Via NAT Gateway (from T2)
- **Defense in depth:** Multiple layers of security

Key point: Private subnets = More secure, production standard.

Checklist: Did You Complete Everything?

Before moving on, check these boxes:

Understanding Phase

- ☐ I understand what EKS is (managed Kubernetes)
- ☐ I understand why use EKS (less operational burden)
- ☐ I understand EKS architecture (control plane vs nodes)
- ☐ I completed all design exercises

Implementation Phase

- ☐ IAM roles created (cluster and nodes)
- ☐ EKS cluster created (took 10-15 minutes)
- ☐ kubectl configured to connect to cluster
- ☐ Cluster verified (status: Active)
- ☐ Test connection successful

Verification Phase

- ☐ Can explain EKS vs self-managed
- ☐ Can explain EKS architecture
- ☐ Can connect to cluster with kubectl
- ☐ Understand cluster is ready for worker nodes

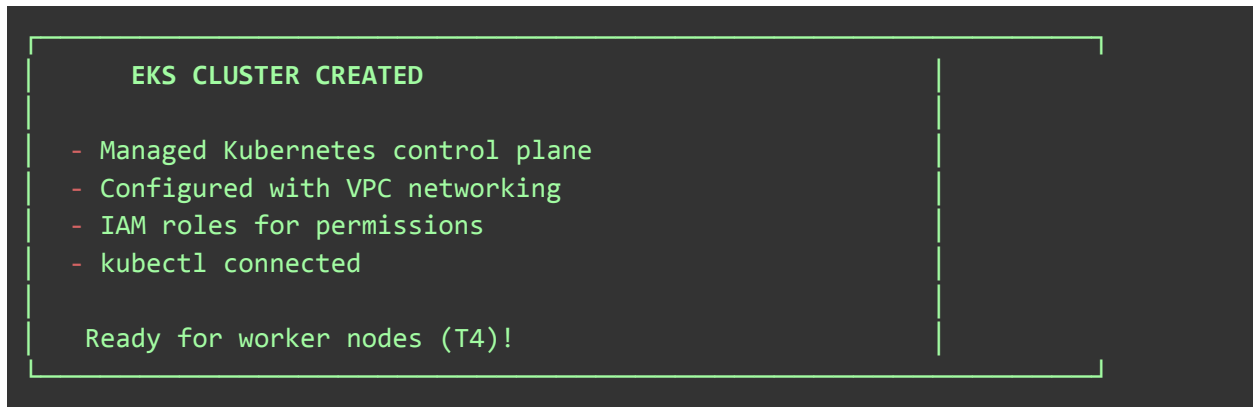
Git

- ☐ Committed `iam.tf` and `eks-cluster.tf`
- ☐ Documented cluster configuration

Key Takeaways

The Big Picture

What you learned in this task:



Critical Concepts

1. EKS:

- AWS managed Kubernetes
- AWS runs control plane
- You run worker nodes
- Same Kubernetes (kubectl, YAML)

2. IAM Roles:

- **Cluster role:** Permissions for EKS to manage AWS resources
- **Node role:** Permissions for worker nodes
- Required for EKS to work

3. Cluster Configuration:

- **VPC:** Your private network
- **Subnets:** Private subnets (secure)
- **Security groups:** Control traffic
- **Endpoints:** Private and public access

4. Connection:

- **kubectl:** Configured via `aws eks update-kubeconfig`

- **Same commands:** Works like MicroK8s
- **Ready for:** Worker nodes and applications

What's Next?

Congratulations! You've created your EKS cluster!

You now have:

- ☐ EKS cluster (managed Kubernetes control plane)
- ☐ Cluster configured with your VPC
- ☐ kubectl connected to cluster
- ☐ Ready for worker nodes

Next: TASK T4 - EKS Node Group (Worker Nodes)

In T4, you'll learn:

- How to create EKS node groups (worker nodes)
- How to configure node groups with your VPC
- How to deploy H-Game to EKS
- How applications run on EKS

But most importantly: You now understand managed Kubernetes. The control plane is ready, now let's add the worker nodes!

Proceed to TASK T4: EKS Node Group - Worker Nodes

TASK T4: EKS Node Group - Worker Nodes

Time: 3 hours | **Priority:** High | **Ticket:** DEVOPS-5104 | **Cost:** ~\$60/month (2-4 worker nodes)

Mission Brief

Add worker nodes to your EKS cluster. These are the EC2 instances where your Pods will run. Deploy H-Game to EKS and verify everything works the same as MicroK8s.

Outcomes:

- Create EKS node group (worker nodes)
- Configure nodes with proper networking and security
- Deploy H-Game to EKS (same YAML from K1-K9!)
- Verify applications run on EKS
- Understand how EKS nodes work

Prerequisites: Complete Task T3 (EKS Cluster)

Setup Note: EKS worker nodes will be EC2 instances in your VPC private subnets. See [Section 5 Overview](#) for general setup details.

Cost Note: Worker nodes cost ~\$0.0416/hour per t3.medium (\$30/month each). With 2-4 nodes, expect ~\$60-120/month. We'll use Spot instances to save 50%+. See [Section 5 Overview](#) for cost-saving strategies.

IDE Note: This task involves creating Terraform files. Instead of using `cat > file.tf << 'EOF'` commands, you can use your IDE (VS Code, Cursor, etc.) to create and edit files. See [IDE-EDITOR-GUIDE.md](#) for details. Both methods work, use whichever you prefer!

The One Thing You Must Note

EKS nodes are simply **EC2 servers that run your Kubernetes Pods**.

With MicroK8s, everything runs on one machine, so if it fails, the whole system goes down.

With EKS, your Pods are spread across multiple EC2 instances, making the system more reliable.

If one node fails, Kubernetes moves the Pods to healthy nodes automatically.

The key idea is simple: **same Kubernetes, but AWS manages it and runs it on multiple servers for you**.

MICROK8S (K1-K9):

- "One EC2 instance"
- "Kubernetes installed on it"
- "All Pods run on same server"
- "If server fails, everything down"

EKS (T4):

- "Multiple EC2 instances (nodes)"
- "AWS manages Kubernetes control plane"
- "Pods distributed across nodes"
- "If one node fails, Pods move to others"

That's it. That's EKS nodes.

Everything else (node groups, AMI, scaling) is just configuration. The core concept is:

"EC2 instances that run your Pods, managed by EKS control plane."

The Mental Model Shift:

BEFORE T4:

- "EKS cluster exists but has no nodes"
- "Pods can't run (no place to run)"
- "Cluster is empty"

AFTER T4:

- "EKS cluster has worker nodes"
- "Pods can run on nodes"
- "Applications deploy to EKS"
- "Same kubectl, same YAML, different infrastructure"

This is the difference between "empty cluster" and "running applications."

Now let's see how nodes work...

Phase 1: Understanding EKS Nodes (20 min)

What are EKS Worker Nodes?

The Restaurant Kitchen Analogy

Think of **EKS** like a restaurant kitchen.

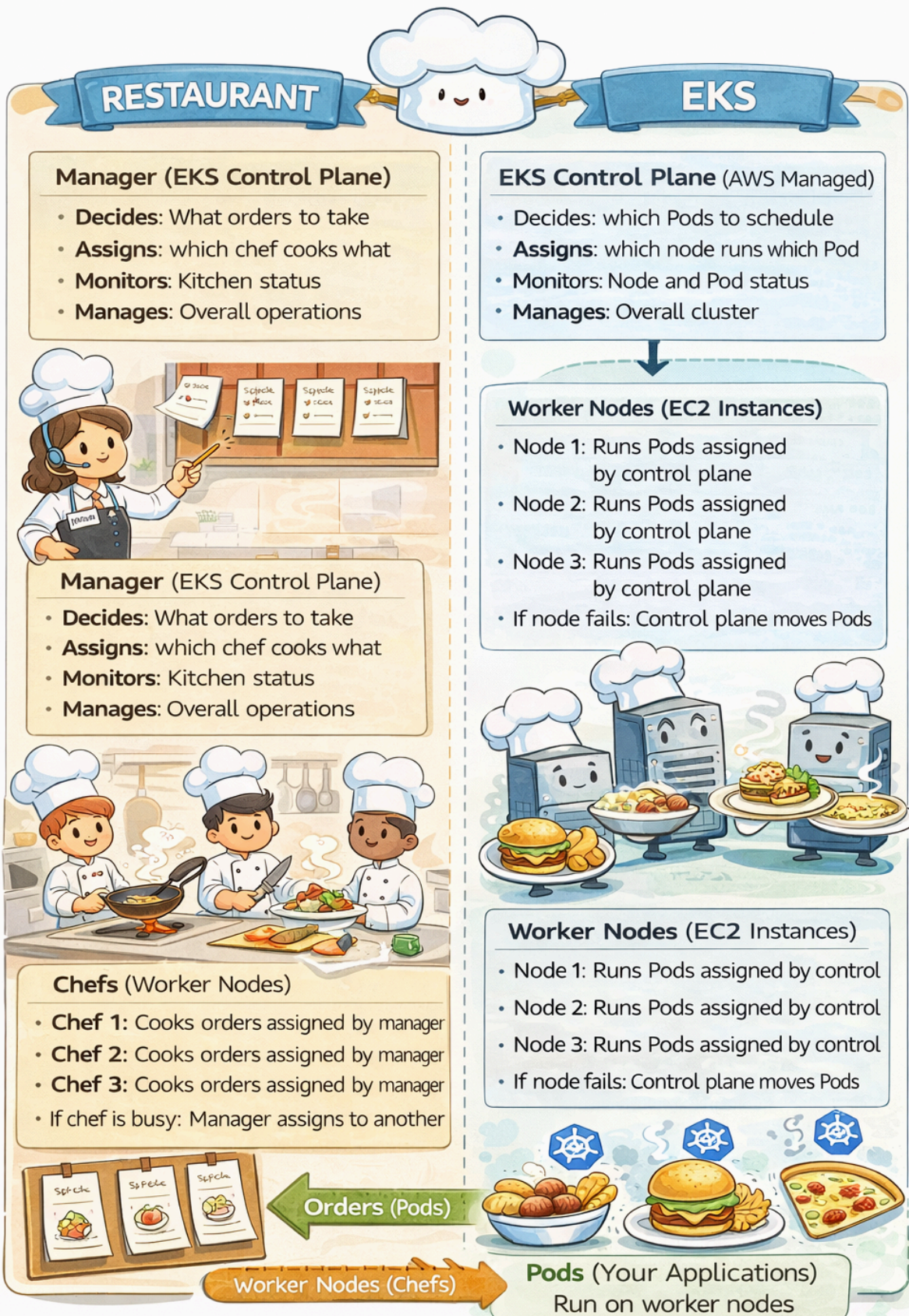
The EKS control plane is the manager that decides what work needs to be done and who should do it.

Worker nodes are the chefs, they are the actual machines that do the work by running your Pods.

Pods never run on the manager; they always run on the worker nodes.

If one worker node fails, the manager simply moves the work to another node so things keep running.

Key insight: Worker nodes = EC2 instances that run your Pods, managed by EKS control plane.



Exercise 1: Why Multiple Nodes?

Scenario: You need to decide how many worker nodes to create.

Question: Should you create:

- A) 1 node (single instance)
- B) 2-4 nodes (multiple instances)

Think about:

- What happens if one node fails?
- How does this affect high availability?
- What do production systems do?

Your answer: _____

Answer Below 

Answer: B) 2-4 nodes

Why multiple nodes:

Imagine running your entire app on one computer, if that computer crashes, everything goes offline.

That's what happens with a single Kubernetes node: all Pods live on one server, so one failure brings the whole system down.

With multiple nodes, Pods are spread across different servers instead of being stacked on one.

If one node fails, Kubernetes automatically moves the Pods to the remaining healthy nodes.

This keeps the app running without users noticing a problem.

Multiple nodes also help share the workload so no single server gets overloaded.

That's why real production systems always use more than one node.

For learning, two or three nodes are enough, but production setups usually start at three or more.

Production systems use multiple nodes for:

- **High availability** - If one node fails, others continue
- **Load distribution** - Spread Pods across nodes
- **Resource efficiency** - Better resource utilization

Exercise 2: Node Group Configuration

Scenario: You're creating an EKS node group.

Question: What configuration should nodes have?

Think about:

- Instance type? (t3.micro, t3.small, t3.medium)
- Which subnets? (Public or private?)
- How many nodes? (Min, max, desired)
- Spot instances? (Save 50% cost)

Your answer: _____

Answer Below 

Answer:**Instance Type:**

- **t3.medium** (2 vCPU, 4GB RAM) - Good balance for this learning
- **t3.small** (2 vCPU, 2GB RAM) - Too small for multiple Pods
- **t3.large** (2 vCPU, 8GB RAM) - More expensive, overkill for this learning

Subnets:

- **Private subnets** - More secure, production standard
- **Public subnets (BAD)**- Exposes nodes to internet (security risk)

Node Count:

- **Min:** 2 nodes (high availability)
- **Desired:** 2-3 nodes (balance cost and availability)
- **Max:** 4 nodes (can scale if needed)

Spot Instances:

- **Yes** - Save 50% cost
- **Risk:** Can be interrupted by AWS (2 minute warning)
- **Handling:** Kubernetes automatically reschedules Pods to other nodes
- **Result:** Zero downtime (Pods move, application continues)
- Acceptable for this learning!

Internet Access:

- Nodes need internet to pull images from ECR
- **Handled by:** NAT Gateway (from T2)
- Nodes in private subnets → NAT Gateway → Internet
- Already configured in T2 (private route table)

This is the standard configuration!

Design Checkpoint

Before moving on, make sure you can answer these in plain English:

Question	Your Answer Should Be Like...
What are EKS worker nodes?	"EC2 instances that run Pods, managed by EKS control plane"
Why multiple nodes?	"High availability, if one fails, others continue"
Where should nodes run?	"Private subnets (secure, hidden from internet)"
What instance type?	"t3.medium (good balance of cost and performance)"

Can't answer one? Go back and re-read that exercise.

All clear? Great! Let's create the node group!

Phase 2: Implementation (2.5 hours)

What We're Building

Before this task:

- EKS cluster (control plane only, no nodes)
- No place for Pods to run
- Cluster is empty

After this task:

- EKS node group (2-4 worker nodes)
- Nodes in private subnets
- H-Game deployed to EKS
- Applications running on EKS

Step-by-Step Plan

We'll do this in 5 steps:

Step 1: Create Node Group
"EC2 instances that run Pods"

Step 2: Wait for Nodes to Join
"Nodes register with cluster"

Step 3: Verify Nodes
"Confirm nodes are ready"

Step 4: Deploy H-Game to EKS
"Same YAML from K1-K9!"

Step 5: Verify Applications
"Confirm everything works"

Note: This task uses the `provider.tf` and `variables.tf` files created in T1. If you haven't completed T1, do that first.

Note: Ensure `.gitignore` is configured (created in T1). The `.gitignore` file prevents committing sensitive files like `terraform.tfvars` and state files.

Step 1: Create Node Group

What we're doing: Creating EC2 instances that will run your Pods.

Add EKS node variables first (if not already in [variables.tf](#)):

```
# Add to variables.tf
# IDE Option: Instead of using cat, add these to your variables.tf
# file in your IDE

cat >> variables.tf << 'EOF'

# =====
# EKS NODE VARIABLES
# =====

variable "eks_node_instance_type" {
  description = "EC2 instance type for EKS worker nodes"
  type        = string
  default     = "t3.medium"

  validation {
    condition = can(regex("^t3\\.(micro|small|medium|large)$",
var.eks_node_instance_type))
    error_message = "Instance type must be t3.micro, t3.small,
t3.medium, or t3.large."
  }
}

variable "eks_node_desired_size" {
  description = "Desired number of worker nodes"
  type        = number
  default     = 2
}
```

```

    validation {
        condition      = var.eks_node_desired_size >= 1 &&
var.eks_node_desired_size <= 10
        error_message = "Desired size must be between 1 and 10 nodes."
    }
}

variable "eks_node_min_size" {
    description = "Minimum number of worker nodes"
    type        = number
    default     = 2

    validation {
        condition      = var.eks_node_min_size >= 1
        error_message = "Minimum size must be at least 1 node."
    }
}

variable "eks_node_max_size" {
    description = "Maximum number of worker nodes"
    type        = number
    default     = 4

    validation {
        condition      = var.eks_node_max_size >= var.eks_node_min_size &&
var.eks_node_max_size <= 20
        error_message = "Maximum size must be >= min_size and <= 20
nodes."
    }
}

variable "node_group_name" {
    description = "EKS node group name"
    type        = string
    default     = "h-game-nodes"
}

variable "node_capacity_type" {

```

```

description = "EKS node capacity type: ON_DEMAND or SPOT"
type        = string
default     = "SPOT"

validation {
  condition = contains(["ON_DEMAND", "SPOT"],
var.node_capacity_type)
  error_message = "Capacity type must be either ON_DEMAND or SPOT."
}
}
EOF

```

Add to terraform.tfvars

```

cat >> terraform.tfvars << 'EOF'

# EKS Node Configuration
eks_node_instance_type = "t3.medium"
eks_node_desired_size  = 2
eks_node_min_size      = 2
eks_node_max_size      = 4
node_group_name        = "h-game-nodes"
node_capacity_type     = "SPOT" # Use SPOT for cost savings,
ON_DEMAND for production
EOF

```

Create node group configuration:

Note: these codes can be found in the repo, just incase incase you run into issues, you can crosscheck with code files in repo.

```

# Create eks-nodes.tf
# IDE Option: Instead of using cat, create eks-nodes.tf in your IDE and paste the
code below (see IDE-EDITOR-GUIDE.md)

cat > eks-nodes.tf << 'EOF'

```

```

# =====
# EKS NODE GROUP
# =====
# EC2 instances that run your Pods
# Managed by EKS control plane

resource "aws_eks_node_group" "main" {
  cluster_name      = aws_eks_cluster.main.name # Which cluster
  node_group_name   = var.node_group_name       # Node group name (from variable)
  node_role_arn     = aws_iam_role.eks_nodes.arn # IAM role for nodes
  subnet_ids       = [
    aws_subnet.private_1.id, # Private subnet 1
    aws_subnet.private_2.id, # Private subnet 2
    aws_subnet.private_3.id, # Private subnet 3
  ]

  # Instance configuration
  instance_types = [var.eks_node_instance_type] # From variable (default:
t3.medium)

  # Scaling configuration
  scaling_config {
    desired_size = var.eks_node_desired_size # From variable
    min_size     = var.eks_node_min_size     # From variable
    max_size     = var.eks_node_max_size     # From variable
  }

  # Update configuration
  update_config {
    max_unavailable = 1 # Update 1 node at a time (zero downtime)
  }

  # Use Spot instances (save 50% cost)
  capacity_type = var.node_capacity_type # From variable (SPOT or ON_DEMAND)

  # ⚠ Spot Instance Note:
  # Spot instances can be interrupted by AWS with 2-minute warning
  # When interrupted:

```

```

# - Kubernetes detects node is gone
# - Pods are automatically rescheduled to other nodes
# - Cluster autoscaler replaces the node if needed
# - Your application continues running (zero downtime!)
#
# For learning: This is acceptable (saves 50% cost)
# For production: Consider On-Demand for critical workloads
#                  Or use Spot with proper PodDisruptionBudgets

# AMI (Amazon Machine Image) - EKS optimized
# EKS automatically uses the correct AMI for your Kubernetes version

# 💡 Internet Access Note:
# Nodes are in private subnets (from T1) and need internet access to:
# - Pull container images from ECR (from T8)
# - Download Kubernetes components
# - Access AWS APIs
#
# This is handled by NAT Gateway (from T2) which routes private subnet
# traffic to internet. Nodes automatically use NAT Gateway via private
# route table configured in T2.

# Remote access (SSH) - Optional, for debugging
# remote_access {
#   ec2_ssh_key = "your-key-pair-name" # Uncomment if you want SSH access
# }

# Labels and taints (for advanced scheduling)
# We'll keep it simple for now

# Ensure cluster and IAM roles exist first
# Why: Node group needs the cluster to exist, and needs IAM roles to be fully
configured
# Terraform will wait for these resources before creating the node group
# Created in T3: Cluster (Step 2), IAM roles (Step 1)
depends_on = [
  aws_eks_cluster.main, # Cluster must exist first (created in T3 Step 2)
  aws_iam_role_policy_attachment.eks_worker_node_policy, # IAM role policies

```

```

from T3 Step 1
    aws_iam_role_policy_attachment.eks_cni_policy, # Network permissions
    aws_iam_role_policy_attachment.eks_container_registry_policy, # ECR access
permissions
]

tags = merge(
    local.common_tags,
    {
        Name = "${local.name_prefix}-eks-nodes"
        Task = "T4"
    }
)
}

# Output: Node group name
output "eks_node_group_name" {
    description = "Name of the EKS node group"
    value       = aws_eks_node_group.main.node_group_name
}
EOF

```

Format and validate code:

```

# Format Terraform code (fixes indentation, spacing)
terraform fmt

# Validate Terraform syntax and configuration
terraform validate

# Should show: "Success! The configuration is valid."

```

Plan and apply:

```

# Preview changes

```

```
terraform plan
```

```
# Should show: 1 to add (node group)
```

```
# Note: This will take 5-10 minutes to create!
```

```
# Apply changes
```

```
terraform apply
```

```
# Type 'yes' when prompted
```

```
# Wait 5-10 minutes for nodes to be created and join cluster
```

Expected output:

```
aws_eks_node_group.main: Creating...
```

```
aws_eks_node_group.main: Still creating... [10s elapsed]
```

```
aws_eks_node_group.main: Still creating... [1m0s elapsed]
```

```
...
```

```
aws_eks_node_group.main: Still creating... [5m0s elapsed]
```

```
aws_eks_node_group.main: Creation complete after 6m30s
```

```
[id=h-game-prod:h-game-nodes]
```

```
Apply complete! Resources: 25 added, 0 changed, 0 destroyed.
```

Note: Node group creation takes 5-10 minutes. Nodes need to:

1. Launch EC2 instances
2. Install Kubernetes components
3. Join the cluster
4. Become ready

Checkpoint - Did it work?

- Node group created
- Nodes launching (takes 5-10 minutes)
- Visible in AWS Console

Node group created!

Step 2: Wait for Nodes to Join

What we're doing: Waiting for nodes to finish launching and join the cluster.

Check node status:

```
# Check if nodes are joining
kubectl get nodes

# Initially, you might see:
# NAME                                STATUS    ROLES    AGE   VERSION
# ip-10-0-11-5.ec2.internal           NotReady <none>   30s   v1.28.0
# ip-10-0-12-10.ec2.internal          NotReady <none>   30s   v1.28.0

# Wait 2-3 minutes, then check again:
kubectl get nodes

# Should show:
# NAME                                STATUS    ROLES    AGE   VERSION
# ip-10-0-11-5.ec2.internal           Ready     <none>    3m    v1.28.0
# ip-10-0-12-10.ec2.internal          Ready     <none>    3m    v1.28.0
```

Watch nodes join (optional):

```
# Watch nodes in real-time
watch -n 5 kubectl get nodes

# Press Ctrl+C to stop watching
```

Expected timeline:

- 0-2 minutes: Nodes launching (NotReady)
- 2-5 minutes: Nodes installing Kubernetes components
- 5-10 minutes: Nodes Ready

Checkpoint - Did it work?

- ☐ Nodes visible in `kubectl get nodes`
- ☐ Status: Ready (after 5-10 minutes)
- ☐ 2 nodes ready (or desired count) **Nodes joined cluster!**

Step 3: Verify Nodes

What we're doing: Verifying nodes are fully operational.

Check node details:

```
# Describe nodes
kubectl describe nodes

# Should show:
# - Node IPs (10.0.11.x, 10.0.12.x - private subnet IPs)
# - Capacity (CPU, memory)
# - Conditions: Ready, NetworkReady, etc.
# - System Pods running (kube-proxy, aws-node, etc.)
```

Check system Pods:

```
# List all Pods in kube-system
kubectl get pods -n kube-system

# Should show:
# - aws-node-xxx (CNI plugin for networking)
# - kube-proxy-xxx (network proxy)
# - coredns-xxx (DNS)
# All should be Running
```

Install Metric Server:

```
kubectl apply -f
https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml
```

Check node resources:

```
# Check node resource usage
kubectl top nodes
```

```
# Should show:
# NAME                                CPU(cores)  MEMORY(bytes)
# ip-10-0-11-5.ec2.internal           50m         500Mi
# ip-10-0-12-10.ec2.internal          45m         480Mi
```

Verify in AWS Console:

1. Go to AWS Console → EC2 → Instances
2. **Filter by:** Tag Name = `h-game-eks-nodes`
3. **You should see:** 2 EC2 instances (t3.medium)
4. **Status:** Running
5. **Subnet:** Private subnets (10.0.11.x or 10.0.12.x)

Verify internet access (for image pulling):

```
# Check that nodes can reach internet via NAT Gateway
# This is required for pulling images from ECR

# Verify NAT Gateway is working:
terraform output nat_gateway_id

# Nodes in private subnets automatically route through NAT Gateway
# (configured in T2 with private route table)
#
# This enables:
# - Pulling container images from ECR (from T8)
# - Downloading Kubernetes components during node setup
# - Accessing AWS APIs (ECR, CloudWatch, etc.)
#
# Flow: Node → Private Route Table → NAT Gateway → Internet
# Gateway → Internet
```

Checkpoint - Did it work?

- ☐ Nodes are Ready
- ☐ System Pods running
- ☐ Nodes visible in AWS Console
- ☐ Nodes in private subnets
- ☐ Internet access via NAT Gateway (for image pulling from ECR)

Nodes verified!

Step 4: Deploy H-Game to EKS

What we're doing: Deploying H-Game to EKS using the same YAML from K1-K9!

The magic: Your Kubernetes manifests from K1-K9 work on EKS without changes!

CRITICAL: Image Registry Change Required (If you have your image on dockerhub, please skip this)

Your K1-K9 YAML uses:

```
image: localhost:32000/h-game-backend:v1
```

This won't work on EKS because:

- EKS nodes can't access your local registry (**localhost:32000** only works on your machine)
- EKS nodes are in AWS (different network from your local machine)
- Local registry is not accessible from AWS

What will happen:

```
kubect1 get pods -n h-game  
# Status: ImagePullBackOff  
# Error: Failed to pull image "localhost:32000/h-game-backend:v1"
```

Solutions:

Option 1: Use Public Images (Temporary - For Testing Infrastructure)

```
# For now, use public images to test infrastructure  
# Update your Deployment YAML:  
image: nginx:alpine # Works on EKS (public image)  
  
# This Lets you verify:  
# Nodes are working  
# Pods can be scheduled  
# Infrastructure is correct
```

```
# ⚠ Application won't work (wrong image)
```

Option 2: Use Docker Hub (If You Pushed Images There)

```
# If you pushed images to Docker Hub in K1-K9:  
image: yourusername/h-game-backend:v1 # Docker Hub image  
  
# This works on EKS (Docker Hub is public)
```

Option 3: Wait for T8 - ECR

```
# In T8, we'll:  
# 1. Create ECR repositories (private AWS registry)  
# 2. Push images to ECR  
# 3. Update YAML to use ECR images  
# 4. Deploy to EKS  
  
# For now: Just verify infrastructure works  
# We'll fix images in T8!
```

For this task (T4):

- **Focus on:** Verifying nodes work
- **Focus on:** Understanding EKS infrastructure
- ⚠ **Don't worry:** If Pods fail with ImagePullBackOff, that's expected
- **Next step:** T8 will fix image registry

Prerequisites

- EKS cluster created and nodes running
- kubectl configured: `aws eks update-kubeconfig --name h-game-prod --region us-east-1`
- All YAML files in `/Users/mac/Desktop/devops playbook/k8s/`
- EBS CSI driver addon installed (required for PostgreSQL persistent storage)
- Metrics Server installed (for HPA and `kubectl top`)

Verify Prerequisites

```
# Check cluster access
kubectl get nodes

# Check EBS CSI driver (should show Running pods)
kubectl get pods -n kube-system | grep ebs

# Check Metrics Server (should show Running pod)
kubectl get pods -n kube-system | grep metrics-server
```

If EBS CSI driver is not installed:

- The EBS CSI driver IAM role should be created in Terraform (`iam.tf`)
- Install addon: `aws eks create-addon --cluster-name h-game-prod --addon-name aws-ebs-csi-driver --service-account-role-arn <role-arn> --region us-east-1`
- See "EBS CSI Driver Setup" section below

If Metrics Server is not installed:

```
kubectl apply -f
https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml
```

Step 1: Create Namespace

```
# Create namespace (if not already created)
kubectl create namespace h-game
```

Step 2: Deploy Configuration (ConfigMap)

```
# Navigate to your Kubernetes manifests
cd "/Users/mac/Desktop/devops playbook/k8s"

# Deploy ConfigMap (non-sensitive configuration)
kubectl apply -f app-config.yaml

# Verify
kubectl get configmap -n h-game
# Should show: app-config
```

Step 3: Deploy Database (PostgreSQL)

Deploy in order:

1. Secret (database credentials)
2. Service + StatefulSet (database server)

```
# Deploy PostgreSQL Secret (sensitive credentials)
kubectl apply -f postgres-secret.yaml

# Deploy PostgreSQL Service + StatefulSet (single file contains both)
kubectl apply -f postgres-statefulset.yaml

# Check Pods
kubectl get pods -n h-game -l app=postgres

# Should show: 1 postgres Pod
# Status: Running (on EKS nodes!)
# Note: PostgreSQL image is public (postgres:15-alpine), so it works!

# Verify Service
```



```
kubectl get svc postgres-service -n h-game
# Should show: postgres-service (ClusterIP, port 5432)
```

Expected output:

NAME	READY	STATUS	RESTARTS	AGE
postgres-0	1/1	Running	0	2m

Note: If PostgreSQL pod stays in **Pending** status:

- Check EBS CSI driver: `kubectl get pods -n kube-system | grep ebs`
- Check PVC: `kubectl describe pvc postgres-storage-postgres-0 -n h-game`
- Ensure storage class is gp2 (EKS default), not `microk8s-hostpath`

Step 4: Deploy Cache (Redis)

```
# Deploy Redis (single file contains Deployment + Service)
kubectl apply -f redis-deployment.yaml

# Check Pods
kubectl get pods -n h-game -l app=redis

# Should show: 1 redis Pod
# Status: Running (on EKS nodes!)
# Note: Redis image is public (redis:7), so it works!

# Verify Service
kubectl get svc redis -n h-game
# Should show: redis (ClusterIP, port 6379)
```

Expected output:

NAME	READY	STATUS	RESTARTS	AGE
redis-xxxxxxxx-xxxxx	1/1	Running	0	1m

Step 5: Deploy Backend

Important: Backend depends on PostgreSQL and Redis, so deploy them first!

```
# Deploy backend Deployment
kubectl apply -f backend-deployment.yaml

# Deploy backend Service
kubectl apply -f backend-service.yaml

# Deploy HPA (Horizontal Pod Autoscaler)
kubectl apply -f backend-hpa.yaml

# Check Pods
kubectl get pods -n h-game -l app=backend

# Initially: Pods may be in CrashLoopBackOff (waiting for PostgreSQL)
# After PostgreSQL is Running: Pods should recover automatically
# Expected: 3 backend Pods Running

# Check HPA
kubectl get hpa -n h-game
# Should show: backend-hpa monitoring CPU and Memory
```

Expected output (with Docker Hub images):

NAME	READY	STATUS	RESTARTS	AGE
backend-649ccb9484-xxxxx	1/1	Running	0	2m
backend-649ccb9484-xxxxx	1/1	Running	0	2m
backend-649ccb9484-xxxxx	1/1	Running	0	2m

Verify Service:

```
kubectl get svc backend-service -n h-game
# Should show: backend-service (ClusterIP, port 3001)
```

Troubleshooting Backend:

- If pods are in **CrashLoopBackOff**: Check logs for database connection errors
- Wait for PostgreSQL to be fully Running before backend will work
- Backend will automatically retry and recover once PostgreSQL is ready

Step 6: Deploy Frontend

```
# Deploy frontend (single file contains Deployment + Service)
kubectl apply -f frontend-deployment.yaml

# Check Pods
kubectl get pods -n h-game -l app=frontend

# Expected scenarios:
# If using Docker Hub images (veeno/humor-game-frontend:latest): Pods
Running
# ⚠ If using localhost:32000: Pods ImagePullBackOff (expected!)
# → This is normal! We'll fix in T8 (ECR)
```

Expected output (with Docker Hub images):

NAME	READY	STATUS	RESTARTS	AGE
frontend-xxxxxxxx-xxxxx	1/1	Running	0	1m
frontend-xxxxxxxx-xxxxx	1/1	Running	0	1m

Verify Service:

```
kubectl get svc frontend-service -n h-game
# Should show: frontend-service (ClusterIP, port 80)
```

Step 7: Verify All Resources

Check All Pods

```
# List all Pods
kubectl get pods -n h-game

# Should show:
# - 3 backend Pods (Running)
# - 2 frontend Pods (Running)
# - 1 postgres Pod (Running)
# - 1 redis Pod (Running)
```

Expected output:

NAME	READY	STATUS	RESTARTS	AGE
backend-649ccb9484-xxxxx	1/1	Running	0	5m
backend-649ccb9484-xxxxx	1/1	Running	0	5m
backend-649ccb9484-xxxxx	1/1	Running	0	5m
frontend-xxxxxxxx-xxxxx	1/1	Running	0	3m
frontend-xxxxxxxx-xxxxx	1/1	Running	0	3m
postgres-0	1/1	Running	0	8m
redis-xxxxxxxx-xxxxx	1/1	Running	0	6m

Check Pod Distribution

```
# See which node each Pod is on
kubectl get pods -n h-game -o wide

# Should show:
# - Pods distributed across nodes
# - Node IPs (10.0.12.x, 10.0.13.x - private subnets)
```

Check All Services

```
# List Services
```

```
kubectl get svc -n h-game

# Should show:
# - frontend-service (ClusterIP, port 80)
# - backend-service (ClusterIP, port 3001)
# - postgres-service (ClusterIP, port 5432)
# - redis (ClusterIP, port 6379)
```

Expected output:

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
AGE				
backend-service	ClusterIP	172.20.170.230	<none>	3001/TCP
5m				
frontend-service	ClusterIP	172.20.xxx.xxx	<none>	80/TCP
3m				
postgres-service	ClusterIP	None	<none>	5432/TCP
8m				
redis	ClusterIP	172.20.xxx.xxx	<none>	6379/TCP
6m				

Step 8: Test Applications

Test Backend Health Endpoint

```
# Port forward to backend
kubectl port-forward service/backend-service 3001:3001 -n h-game

# In another terminal:
curl http://localhost:3001/health

# Should return: JSON health status
```

Expected response:

```
{
  "status": "healthy",
  "timestamp": "2026-01-31T22:00:00.000Z",
  "services": {
    "database": "connected",
    "redis": "connected",
    "api": "running"
  },
  "version": "1.0.0",
  "environment": "production"
}
```

Test Frontend**Frontend Configuration:**

- Frontend nginx proxies `/api/*` requests to `backend-service:3001`
- Frontend uses `/api` as the API base URL
- Environment variables are set in the deployment

Port-Forward Testing:

```
# Port forward to frontend (runs in foreground - keep terminal open)
kubectl port-forward service/frontend-service 8080:80 -n h-game

# In browser: http://localhost:8080
# The frontend should load and be able to connect to the backend API
```

Verify Frontend is Working:

```
# Test frontend health endpoint
curl http://localhost:8080/health
# Should return: healthy

# Test frontend API proxy (via nginx)
curl http://localhost:8080/api/health
```

```
# Should return: Backend health JSON response

# Check frontend logs
kubectl logs -n h-game -l app=frontend --tail=20
```

If Frontend Doesn't Load in Browser:

1. Check port-forward is running - Keep the terminal with `kubectl port-forward` open
2. Check browser console - Open DevTools (F12) and check for errors
3. Verify backend is accessible - Test `/api/health` endpoint
4. Check frontend pods - `kubectl get pods -n h-game -l app=frontend`

Troubleshooting Frontend:

```
# Test from inside frontend pod
kubectl exec -n h-game $(kubectl get pods -n h-game -l app=frontend
-o jsonpath='{.items[0].metadata.name}') -- curl -s
http://localhost/api/health

# Test backend connectivity from frontend pod
kubectl exec -n h-game $(kubectl get pods -n h-game -l app=frontend
-o jsonpath='{.items[0].metadata.name}') -- curl -s
http://backend-service:3001/health
```

For Production Access:

- Use ALB (Application Load Balancer) - see T5
- Configure Ingress with proper backend URL
- Set up DNS with Route53 - see T6

EBS CSI Driver Setup (If Not Already Done)

PostgreSQL requires persistent storage. EKS uses EBS CSI driver for this.

1. Create IAM Role (in Terraform)

The IAM role should already be in `h-game-terraform/iam.tf`:

```
# EBS CSI Driver IAM Role (for service account - IRSA)
resource "aws_iam_role" "eks_ebs_csi_driver" {
  name = "${local.name_prefix}-eks-ebs-csi-driver-role"
  # ... trust policy for OIDC ...
}

resource "aws_iam_role_policy_attachment" "eks_ebs_csi_driver_policy"
{
  policy_arn =
  "arn:aws:iam::aws:policy/service-role/AmazonEBSCSIDriverPolicy"
  role       = aws_iam_role.eks_ebs_csi_driver.name
}
```

Apply with Terraform:

```
cd h-game-terraform
terraform apply -target=aws_iam_role.eks_ebs_csi_driver
                -target=aws_iam_role_policy_attachment.eks_ebs_csi_driver_policy
```

2. Create OIDC Provider (One-time)

```
CLUSTER_OIDC_ISSUER=$(aws eks describe-cluster --name h-game-prod
--region us-east-1 --query 'cluster.identity.oidc.issuer' --output
text | sed 's|https://||')
aws iam create-open-id-connect-provider \
  --url "https://$CLUSTER_OIDC_ISSUER" \
  --client-id-list "sts.amazonaws.com" \
  --thumbprint-list "9e99a48a9960b14926bb7f3b02e22da2b0ab7280" \
  --region us-east-1
```

3. Install EBS CSI Driver Addon

```
# Get role ARN from Terraform output
ROLE_ARN=$(cd h-game-terraform && terraform output -raw
eks_nodes_role_arn | sed 's|nodes|ebs-csi-driver|')

# Install addon
```



```
aws eks create-addon \
  --cluster-name h-game-prod \
  --addon-name aws-ebs-csi-driver \
  --service-account-role-arn "$ROLE_ARN" \
  --region us-east-1

# Wait for addon to be ACTIVE (takes 1-2 minutes)
aws eks describe-addon --cluster-name h-game-prod --addon-name
aws-ebs-csi-driver --region us-east-1 --query 'addon.status'
```

4. Verify EBS CSI Driver

```
# Check pods
kubectl get pods -n kube-system | grep ebs

# Should show:
# ebs-csi-controller-xxx    6/6    Running
# ebs-csi-node-xxx         3/3    Running
```

5. Update PostgreSQL Storage Class

Ensure `postgres-statefulset.yaml` uses `gp2` (EKS default), not `microk8s-hostpath`:

```
storageClassName: gp2 # EKS default storage class (AWS EBS)
```

Troubleshooting

Backend Pods Not Starting

```
# Check Pod logs
kubectl logs -n h-game -l app=backend --tail=50

# Common issues:
# - "ENOTFOUND postgres-service" → PostgreSQL not deployed yet
# - "ENOTFOUND redis" → Redis not deployed yet
# - "configmap app-config not found" → ConfigMap not deployed
# - "secret postgres-secret not found" → Secret not deployed
```

Check Pod Status

```
# Describe a failing Pod
kubectl describe pod -n h-game <pod-name>

# Look for:
# - Events section (shows errors)
# - Container status
# - Volume mounts
```

Verify Dependencies

```
# Check if all dependencies exist
kubectl get configmap,secret,svc -n h-game

# Should show:
# - app-config (ConfigMap)
# - postgres-secret (Secret)
# - postgres-service, redis, backend-service, frontend-service
(Services)
```

PostgreSQL Pod Stuck in Pending

```
# Check PVC status
kubectl get pvc -n h-game

# If PVC is Pending:
kubectl describe pvc postgres-storage-postgres-0 -n h-game

# Check EBS CSI driver
kubectl get pods -n kube-system | grep ebs

# If EBS CSI driver pods are CrashLoopBackOff:
kubectl logs -n kube-system -l app=ebs-csi-controller --tail=50

# Common issue: OIDC provider not created or IAM role not configured
```

Frontend Not Accessible via Port-Forward

Common Issues:

1. Port-forward not running - Keep the terminal with `kubectl port-forward` open
2. Wrong port - Use `8080:80` (local:service)
3. Browser cache - Clear cache or use incognito mode
4. Backend not accessible - Frontend nginx proxies `/api/*` to backend-service

Debug Steps:

```
# 1. Verify port-forward is active
# Terminal should show: "Forwarding from 127.0.0.1:8080 -> 80"

# 2. Test frontend health
curl http://localhost:8080/health
# Should return: healthy

# 3. Test API proxy
curl http://localhost:8080/api/health
# Should return: Backend health JSON
```

```
# 4. Check frontend pods
kubectl get pods -n h-game -l app=frontend
# Should show: 2/2 Running

# 5. Check frontend logs
kubectl logs -n h-game -l app=frontend --tail=50

# 6. Check browser console (F12)
# Look for JavaScript errors or network failures
```

Frontend Configuration:

- Frontend deployment includes `API_BASE_URL=/api` environment variable
- Frontend nginx is configured to proxy `/api/*` to `backend-service:3001`
- Both services are in the same namespace (`h-game`)

If Still Not Working:

- Verify backend is running: `kubectl get pods -n h-game -l app=backend`
- Test backend directly: `kubectl port-forward service/backend-service 3001:3001 -n h-game` then `curl http://localhost:3001/health`
- For production: Use ALB + Ingress (see T5)

Understanding What's Happening

Infrastructure Verification

- Nodes are working: Pods are scheduled and running
- Networking works: Services provide DNS names (`postgres-service`, `redis`, etc.)
- Storage works: PostgreSQL has persistent storage
- Public images work: `postgres:15-alpine`, `redis:7` pull successfully

Image Registry Status

- **PostgreSQL:** Using public image (`postgres:15-alpine`) → Works!
- **Redis:** Using public image (`redis:7`) → Works!
- **Backend:** Using Docker Hub (`veeno/humor-game-backend:v1`) → Should work!

- **Frontend:** Using Docker Hub (veeno/humor-game-frontend:latest) → Should work!

Note: If backend/frontend use `localhost:32000`, they will fail with `ImagePullBackOff`. This is expected and will be fixed in T8 when we set up ECR.

Key Insight

The infrastructure is working! The same YAML files from K1-K9 work on EKS without changes. The only difference is the image registry (Docker Hub vs localhost vs ECR).

Checkpoint - Did it work?

- ☐ All Pods deployed
- ☐ Pods running on EKS nodes
- ☐ Services created
- ☐ Pods distributed across nodes
- ☐ PostgreSQL with persistent storage (EBS volume)
- ☐ Backend connected to database and cache
- ☐ HPA monitoring and scaling backend pods
- ☐ Backend health endpoint responding
- ☐ Frontend accessible via port-forward (limited functionality)
- ☐ Same YAML works on EKS!

H-Game deployed to EKS!

Note on Frontend Access:

- Port-forwarding works for both backend and frontend
- Frontend nginx proxies `/api/*` requests to `backend-service:3001` automatically
- Frontend deployment includes `API_BASE_URL=/api` environment variable
- For production access, set up ALB (Application Load Balancer) - see T5

Next Steps

1. **T8 - ECR:** Set up AWS ECR for private image registry
2. **T5 - ALB:** Set up Application Load Balancer for external access
3. **T6 - Route53:** Configure DNS for your domain
4. **Monitoring:** Set up CloudWatch, Prometheus, Grafana
5. **CI/CD:** Automate deployments with GitHub Actions or GitLab CI

Verify in AWS Console:

1. Go to AWS Console → EC2 → Instances
2. Filter by: Tag **Name** = `h-game-eks-nodes`
3. Click on a node → See running Pods in details

Phase 3: Validation & Consolidation (10 min)

Quiz: EKS Nodes Understanding

Answer these questions to verify your understanding:

Question 1: What are EKS worker nodes?

Answer:

- **Worker nodes** = EC2 instances that run your Pods
- Managed by EKS control plane
- Located in your VPC private subnets
- **Same Kubernetes:** Pods, Deployments, Services work the same

Key point: Nodes = EC2 instances where Pods run.

Question 2: Why use multiple nodes?

Answer:

- **High availability:** If one node fails, Pods move to others
- **Load distribution:** Spread Pods across nodes
- **Resource efficiency:** Better utilization
- **Production standard:** All production systems use multiple nodes

Key point: Multiple nodes = Resilience and availability.

Question 3: Do your K1-K9 Kubernetes manifests work on EKS?

Answer:

- **Yes!** Same Kubernetes, same YAML
- **kubectl apply** works the same
- **Pods, Deployments, Services** work the same
- **Only difference:** Image registry (localhost:32000 won't work, need ECR - T8)

Key point: Kubernetes is Kubernetes - manifests are portable!

Checklist: Did You Complete Everything?

Before moving on, check these boxes:

Understanding Phase

- ☐ I understand what EKS nodes are (EC2 instances that run Pods)
- ☐ I understand why multiple nodes (high availability)
- ☐ I understand node configuration
- ☐ I completed all design exercises

Implementation Phase

- ☐ Node group created
- ☐ Nodes joined cluster (Ready status)
- ☐ System Pods running
- ☐ H-Game deployed to EKS
- ☐ All Pods running on nodes

Verification Phase

- ☐ Can explain EKS nodes
- ☐ Can verify nodes are ready
- ☐ Can deploy applications to EKS
- ☐ Understand nodes vs control plane

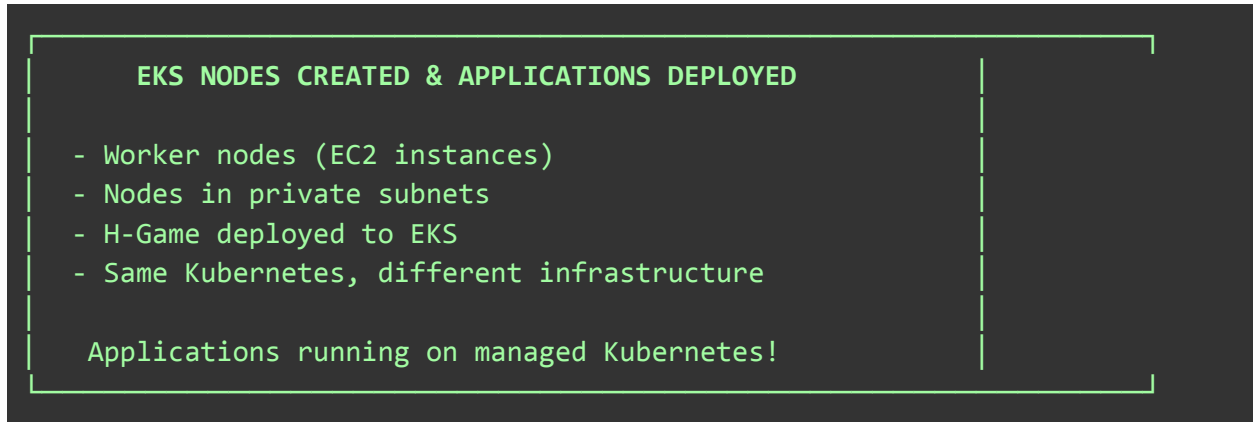
Git

- ☐ Committed `eks-nodes.tf`
- ☐ Documented node group configuration

Key Takeaways

The Big Picture

What you learned in this task:



Critical Concepts

1. EKS Nodes:

- EC2 instances that run Pods
- Managed by EKS control plane
- Located in private subnets
- Multiple nodes for high availability

2. Node Groups:

- Group of nodes with same configuration
- Auto-scaling (min, desired, max)
- Spot instances save 50% cost
- Update strategy (zero downtime)

3. Application Deployment:

- Same Kubernetes YAML works
- Same kubectl commands
- Pods run on worker nodes
- Services work the same

4. Infrastructure Difference:

- **MicroK8s:** One EC2, you manage control plane
- **EKS:** Multiple EC2 nodes, AWS manages control plane
- Same applications, better infrastructure

What's Next?

Congratulations! You've deployed applications to EKS!

You now have:

- ☐ EKS cluster with worker nodes
- ☐ H-Game running on EKS
- ☐ Production-grade Kubernetes infrastructure
- ☐ Same applications, better infrastructure

Next: TASK T5 - RDS PostgreSQL Migration

In T5, you'll learn:

- How to create RDS PostgreSQL (managed database)
- How to migrate data from MicroK8s PostgreSQL
- How to connect backend to RDS
- How managed databases work

But most importantly: You now understand how EKS works. Your applications are running on production-grade infrastructure!

Proceed to TASK T5: RDS PostgreSQL Migration

TASK T5: RDS PostgreSQL Migration

Time: 5 hours | **Priority:** High | **Ticket:** DEVOPS-5105 | **Cost:** ~\$15/month (db.t3.micro)

Mission Brief

Migrate from self managed PostgreSQL (StatefulSet) to AWS RDS (managed database). RDS handles backups, failover, updates, and scaling automatically. This is a critical production upgrade.

Outcomes:

- Understand RDS vs self-managed databases
- Create RDS PostgreSQL instance with Terraform
- Migrate data from MicroK8s PostgreSQL to RDS
- Update backend to connect to RDS
- Delete PostgreSQL StatefulSet (no longer needed)
- Understand managed database benefits

Prerequisites: Complete Task T4 (EKS Node Group)

Setup Note: RDS will be created in your VPC private subnets. We'll migrate data from your existing PostgreSQL StatefulSet (from K2), then delete the StatefulSet. See [Section 5 Overview](#) for general setup details.

Cost Note: RDS db.t3.micro costs ~\$15/month. Single-AZ for learning (saves \$15/month vs Multi-AZ). Backups are free (7-day retention). See [Section 5 Overview](#) for cost-saving strategies.

IDE Note: This task involves creating Terraform files. Instead of using `cat > file.tf << 'EOF'` commands, you can use your IDE (VS Code, Cursor, etc.) to create and edit files. See [IDE-EDITOR-GUIDE.md](#) for details. Both methods work - use whichever you prefer!

The One Thing That Makes This Click

- RDS is a **managed database**, which means AWS takes care of running the database for you.
When you run PostgreSQL yourself, you're responsible for backups, updates, monitoring, and fixing things when they break.
That can be stressful and time-consuming, especially when something goes wrong in production.
With RDS, AWS automatically handles backups, software updates, and recovery if the database fails.
If there's an issue, AWS replaces or fixes the database without you jumping in at 2 a.m.
- **You don't stop thinking about databases** completely, but you stop worrying about the hard operational work.
This lets you focus on building and improving your application instead of babysitting the database.
In simple terms, **AWS runs the database, and you run the app.**

The Mental Model Shift:

BEFORE T5:

- "PostgreSQL = I manage StatefulSet"
- "I handle backups (pg_dump)"
- "I handle updates (manual)"
- "If database breaks, I fix it"

AFTER T5:

- "PostgreSQL = AWS manages RDS"
- "AWS handles backups (automatic)"
- "AWS handles updates (automatic)"
- "If database breaks, AWS fixes it"
- "I focus on application logic"

This is the difference between "self-managed" and "managed database."

Now let's see how RDS works...

Phase 1: Understanding RDS (30 min)

What is RDS?

RDS = Relational Database Service

The Database Management Analogy

- **RDS is AWS's** way of running a database **for you**, so you don't have to manage it yourself.
If you run a database on your own server, you're responsible for installing it, backing it up, updating it, and fixing it when it breaks.
That takes a lot of time and distracts you from building your actual application.
- **With RDS**, AWS handles backups, updates, monitoring, and recovery automatically.
You still control your data, tables, and queries, but not the heavy operational work.
If something goes wrong, AWS fixes the database instead of you waking up at night to debug it.
This lets you spend most of your time improving the app instead of babysitting the database.

SELF-MANAGED DATABASE (PostgreSQL StatefulSet):

You manage everything:

- └─ Install PostgreSQL
- └─ Configure database
- └─ Set up backups (pg_dump scripts)
- └─ Handle updates (manual)
- └─ Monitor performance
- └─ Fix when it breaks
- └─ **Operational burden: HIGH**

Time spent: 40% on database, 60% on application

MANAGED DATABASE (RDS):**AWS manages database:**

- └ AWS runs PostgreSQL
- └ AWS handles backups (automatic, 7-day retention)
- └ AWS handles updates (automatic, scheduled)
- └ AWS monitors performance
- └ AWS fixes when it breaks
- └ **Operational burden:** LOW

You manage:

- └ Database schema (your tables)
- └ Application queries
- └ Focus on: Business logic, not database ops

Time spent: 10% on database, 90% on application

Key insight: RDS = AWS manages database operations, you manage your data.

RDS vs StatefulSet

Comparison:

Aspect	PostgreSQL StatefulSet (K2)	RDS PostgreSQL (T5)
Management	You manage everything	AWS manages operations
Backups	Manual (pg_dump scripts)	Automatic (7-day retention)
Updates	Manual (you upgrade)	Automatic (scheduled)
Failover	None (single Pod)	Automatic (Multi-AZ option)
Scaling	Manual (edit StatefulSet)	Automatic (modify instance)
Monitoring	You set up	Built-in CloudWatch
Cost	EC2 instance cost	Database instance cost
Operational Work	High	Low

When to use each:

- **StatefulSet:** Learning, development, full control needed
- **RDS:** Production, want managed operations, compliance requirements

Exercise 1: Why RDS Instead of StatefulSet?

Scenario: You need to choose between StatefulSet and RDS for production.

Question: What are the key benefits of RDS?

Think about:

- Backups
- Failover
- Updates
- Operational burden

Your answer: _____

Answer Below 

Answer:**Key Benefits of RDS:****1. Automatic Backups:****StatefulSet:**

- |— You write: `pg_dump` scripts
- |— You schedule: Cron jobs
- |— You manage: Backup storage
- |— If you forget → No backup!

RDS:

- |— AWS handles: Automatic daily backups
- |— AWS manages: 7-day retention (configurable)
- |— AWS stores: Backups in S3
- |— Always backed up!

2. Automatic Failover (Multi-AZ):**StatefulSet:**

- |— Single Pod
- |— If Pod fails → Database down
- |— Manual recovery required

RDS Multi-AZ:

- |— Primary database (us-east-1a)
- |— Standby replica (us-east-1b)
- |— If primary fails → Automatic failover to standby
- |— Zero downtime!

3. Automatic Updates:**StatefulSet:**

- |— You research: PostgreSQL updates
- |— You plan: Maintenance window
- |— You execute: Manual upgrade
- |— Risk of errors

RDS:

- └─ **AWS** notifies: Available updates
- └─ **You** schedule: Maintenance window
- └─ **AWS** executes: Automatic upgrade
- └─ Less risk!

4. Point-in-Time Recovery:**StatefulSet:**

- └─ **Restore** from: Last backup
- └─ If backup is old → Data loss
- └─ Limited recovery options

RDS:

- └─ **Restore** to: Any point in last 7 days
- └─ **Example:** "Restore to 2 hours ago"
- └─ Precise recovery!

This is why production uses RDS!

Exercise 2: RDS Configuration

Scenario: You're creating an RDS instance.

Question: What configuration should you use?

Think about:

- Instance type? (db.t3.micro for learning)
- Multi-AZ? (High availability, but +\$15/month)
- Subnets? (Private subnets for security)
- Security groups? (Only backend can access)

Your answer: _____

Answer Below 

Answer:**Instance Type:**

- **db.t3.micro** - 2 vCPU, 1GB RAM, \$15/month
- Good for learning and small workloads
- **Production:** db.t3.small or larger

Multi-AZ:

- **Single-AZ for learning** - Saves \$15/month
- **Multi-AZ for production** - Automatic failover, zero downtime

Subnets:

- **Private subnets** - More secure, hidden from internet
- **DB Subnet Group** - RDS needs subnets in multiple AZs

Security Groups:

- **Only backend can access** - Port 5432 from backend security group
- **No public access** - Database should be private

Backups:

- **7-day retention** - Free, sufficient for learning
- **Automated backups** - Enabled by default

This is the standard RDS configuration!

Design Checkpoint

Before moving on, make sure you can answer these in plain English:

Question	Your Answer Should Be Like...
What is RDS?	"AWS managed database - AWS handles operations"
Why use RDS?	"Automatic backups, failover, updates - less operational work"
Where should RDS run?	"Private subnets (secure, hidden from internet)"
Who can access RDS?	"Only backend (via security groups)"

Can't answer one? Go back and re read that exercise.

All clear? Great! Let's create RDS!

Phase 2: Implementation (4 hours)

What We're Building

Before this task:

- PostgreSQL StatefulSet (self-managed, from K2)
- Manual backups
- No automatic failover
- You manage everything

After this task:

- RDS PostgreSQL (managed by AWS)
- Automatic backups (7-day retention)
- Point-in-time recovery
- Backend connected to RDS
- StatefulSet deleted (migrated!)

Step-by-Step Plan

We'll do this in 7 steps:

- Step 1:** Create DB Subnet Group
"RDS needs subnets in multiple AZs"
- Step 2:** Create RDS Parameter Group
"Database configuration"
- Step 3:** Create RDS Instance
"Managed PostgreSQL database"
- Step 4:** Wait for RDS to be Available
"Takes 5-10 minutes"
- Step 5:** Migrate Data from StatefulSet
"Export from old, import to new"
- Step 6:** Update Backend to Use RDS
"Change connection string"
- Step 7:** Delete PostgreSQL StatefulSet
"Clean up old database"

Note: This task uses the `provider.tf` and `variables.tf` files created in T1. If you haven't completed T1, do that first.

Step 1: Create DB Subnet Group

What we're doing: Creating a subnet group that tells RDS which subnets to use.

Why: RDS needs subnets in multiple AZs for high availability.

Note: If this code doesn't work in your environment, check:

- Terraform version compatibility
- AWS provider version
- Your VPC CIDR block conflicts
- IAM permissions for VPC creation

Add RDS variables first (if not already in [variables.tf](#)):

```
# Add to variables.tf
# IDE Option: Instead of using cat, add these to your variables.tf file in your
# IDE

cat >> variables.tf << 'EOF'

# =====
# RDS VARIABLES
# =====

variable "rds_db_name" {
  description = "RDS database name"
  type        = string
  default     = "humor_memory_game"
  sensitive   = false
}

variable "rds_username" {
  description = "RDS master username"
  type        = string
  default     = "gameuser"
  sensitive   = false
}

variable "rds_password" {
  description = "RDS master password"
  type        = string
```



```

sensitive    = true # Marks as sensitive (won't show in Logs)
# ⚠️ Set this in terraform.tfvars, NOT here!

validation {
  condition = (
    length(var.rds_password) >= 16 &&
    can(regex("[A-Z]", var.rds_password)) &&
    can(regex("[a-z]", var.rds_password)) &&
    can(regex("[0-9]", var.rds_password))
  )
  error_message = <<-EOT
    Password must be at least 16 characters and contain:
    - At least one uppercase letter (A-Z)
    - At least one lowercase letter (a-z)
    - At least one number (0-9)
    Example: MySecurePassword123456
  EOT
}

variable "rds_instance_class" {
  description = "RDS instance type"
  type        = string
  default     = "db.t3.micro"

  validation {
    condition = can(regex("^db\\.t[34]\\.(micro|small|medium|large)$",
var.rds_instance_class))
    error_message = "Instance class must be db.t3 or db.t4 family (micro,
small, medium, large)."
  }
}

variable "rds_db_identifier" {
  description = "RDS instance identifier (unique name)"
  type        = string
  default     = "h-game-postgres"
}

variable "rds_port" {
  description = "RDS PostgreSQL port"
  type        = number
  default     = 5432
}

```

EOF

Add RDS values to terraform.tfvars

```
cat >> terraform.tfvars << 'EOF'

# RDS Configuration
rds_db_name      = "humor_memory_game"
rds_username     = "gameuser"
rds_password     = "devops123" # ⚠ Change this to a secure password!
rds_instance_class = "db.t3.micro"
rds_db_identifier = "h-game-postgres"
rds_port        = 5432
EOF
```

⚠ **IMPORTANT:** Ensure .gitignore includes terraform.tfvars (created in T1)

```
# This prevents committing passwords to Git!
# If you don't have .gitignore yet, create it:
cat > .gitignore << 'EOF'
# =====
# TERRAFORM SENSITIVE FILES
# =====
# ⚠ NEVER commit these files - they contain sensitive data!

# Variable files with actual values
terraform.tfvars
*.tfvars
!terraform.tfvars.example # Allow example file

# State files (contain passwords, endpoints, etc.)
terraform.tfstate
terraform.tfstate.backup
*.tfstate
*.tfstate.*

# Terraform directories
.terraform/

# Lock file - KEEP THIS (ensures version consistency)
# NOTE: .terraform.lock.hcl SHOULD be committed to Git
# It's already allowed (not ignored) - ensures everyone uses same provider
versions
```

```
# Crash Logs
crash.log
crash.*.log

# Override files (for local testing)
override.tf
override.tf.json
*_override.tf
*_override.tf.json

# CLI configuration
.terraformrc
terraform.rc

# IDE files
.vscode/
.idea/
*.swp
*.swo
*~

# OS files
.DS_Store
Thumbs.db
EOF
```

Create DB subnet group configuration:

```

# Create rds.tf
# IDE Option: Instead of using cat, create rds.tf in your IDE and paste the code below (see
# IDE-EDITOR-GUIDE.md)

cat > rds.tf << 'EOF'
# =====
# RDS POSTGRESQL (Managed Database)
# =====
# AWS managed PostgreSQL
# Replaces: PostgreSQL StatefulSet (from K2)

# =====
# DB SUBNET GROUP
# =====
# Tells RDS which subnets to use
# RDS needs subnets in multiple AZs

resource "aws_db_subnet_group" "main" {
  name          = "${local.name_prefix}-db-subnet-group"
  subnet_ids = [
    aws_subnet.private_1.id, # Private subnet 1 (us-east-1a)
    aws_subnet.private_2.id, # Private subnet 2 (us-east-1b)
    aws_subnet.private_3.id, # Private subnet 3 (us-east-1c)
  ]

  tags = merge(
    local.common_tags,
    {
      Name = "${local.name_prefix}-db-subnet-group"
      Task = "T5"
    }
  )
}

# Output: Subnet group name
output "db_subnet_group_name" {
  description = "Name of the DB subnet group"
  value       = aws_db_subnet_group.main.name
}
EOF

```

Plan and apply:

```
# Preview changes
terraform plan

# Should show: 1 to add (DB subnet group)

# Apply changes
terraform apply

# Type 'yes' when prompt
```

Expected output:

```
aws_db_subnet_group.main: Creating...
aws_db_subnet_group.main: Creation complete after 2s
[id=h-game-db-subnet-group]

Apply complete! Resources: 26 added, 0 changed, 0 destroyed.
```

Checkpoint - Did it work?

- ☐ DB subnet group created
- ☐ Includes all 3 private subnets

DB subnet group ready!

Step 2: Create RDS Parameter Group

What we're doing: Creating a parameter group for database configuration.

Why: Allows you to customize PostgreSQL settings.

A parameter group is where you **control how the database behaves**, not the data itself.

It lets you customize PostgreSQL settings like connection limits or timeouts instead of using AWS defaults.

The database reads these settings when it starts and runs according to them.

If you don't change anything, the default values still work perfectly fine.

Think of a parameter group as the **settings panel** for your database engine.

Add parameter group configuration:

```
# Add to rds.tf
cat >> rds.tf << 'EOF'

# =====
# DB PARAMETER GROUP
# =====
# Custom database configuration
# Allows you to tune PostgreSQL settings

resource "aws_db_parameter_group" "postgres" {
  family = "postgres15" # PostgreSQL 15 family
  name    = "${local.name_prefix}-postgres-params"

  # Example parameters (optional - defaults work fine)
  # parameter {
  #   name = "max_connections"
  #   value = "100"
  # }

  tags = merge(
    local.common_tags,
    {
      Name = "${local.name_prefix}-postgres-params"
      Task = "T5"
    }
  )
}
```

```
}  
EOF
```

Plan and apply:

```
# Preview changes  
terraform plan  
  
# Should show: 1 to add (parameter group)  
  
# Apply changes  
terraform apply  
  
# Type 'yes' when prompted
```

Checkpoint - Did it work?

☐ Parameter group created

Parameter group ready!

Step 3: Create RDS Instance

What we're doing: Creating the RDS PostgreSQL instance.

Add RDS instance configuration:

```
# Add to rds.tf
cat >> rds.tf << 'EOF'

# =====
# RDS POSTGRESQL INSTANCE
# =====
# Managed PostgreSQL database
# AWS handles: Backups, updates, monitoring

resource "aws_db_instance" "postgres" {
  # Basic configuration
  identifier      = var.rds_db_identifier      # Database identifier (from
variable)
  engine          = "postgres"                 # Database engine
  engine_version  = "15.4"                     # PostgreSQL version
  instance_class  = var.rds_instance_class     # Instance size (from variable)

  # Storage configuration
  allocated_storage    = 20                    # Initial storage (GB)
  max_allocated_storage = 100                  # Auto-scaling max (GB)
  storage_type         = "gp3"                 # Storage type (General Purpose SSD)
  storage_encrypted    = true                  # Encrypt storage at rest

  # Database configuration
  db_name      = var.rds_db_name               # Database name (from variable)
  username     = var.rds_username              # Master username (from variable)
  password     = var.rds_password              # Master password (from variable -
set in terraform.tfvars)
  port         = var.rds_port                  # PostgreSQL port (from variable)
  # ⚠ Password is stored in terraform.tfvars (not committed to Git)
  # ⚠ In production, use AWS Secrets Manager (we'll cover in DevSecOps)

  # =====
  # LIFECYCLE RULES
  # =====
  lifecycle {
    # Prevent accidental deletion
    prevent_destroy = false # Set to true for production (prevents accidental
```

```

deletion)
    # To delete: Remove this line, then destroy

    # Ignore password changes (don't update DB on every apply)
    ignore_changes = [
        password, # Don't update password on every apply (only when explicitly
changed)
    ]

    # Note: create_before_destroy cannot be used with RDS
}

# Lifecycle Rules Explained:
# - prevent_destroy = false: Currently false for learning (can destroy easily)
#   Production: Set to true to prevent accidental deletion
#   To delete: Remove this line, then destroy
# - ignore_changes = [password]: Don't update database password on every apply
#   Prevents unnecessary downtime
#   Change password manually if needed

# Network configuration
db_subnet_group_name = aws_db_subnet_group.main.name # Which subnets to use
vpc_security_group_ids = [aws_security_group.database.id] # Security group (only
backend can access)
publicly_accessible = false # No public access (private subnet)

# Parameter group
parameter_group_name = aws_db_parameter_group.postgres.name

# Dependencies: Ensure VPC networking is ready before creating RDS
depends_on = [
    aws_db_subnet_group.main,
    aws_db_parameter_group.postgres,
    aws_security_group.database,
]

# Backup configuration
backup_retention_period = 7 # Keep backups for 7 days (free)
backup_window = "03:00-04:00" # Backup window (UTC)
maintenance_window = "mon:04:00-mon:05:00" # Maintenance window (UTC)

# Multi-AZ (for high availability)
# Skip for learning (saves $15/month)
# multi_az = true # Uncomment for production

```

```

# Deletion protection (prevent accidental deletion)
deletion_protection = false # Set to true for production

# Skip final snapshot for Learning (saves time on destroy)
skip_final_snapshot = true # Set to false for production

# Tags
tags = merge(
  local.common_tags,
  {
    Name = var.rds_db_identifier
    Task = "T5"
  }
)

# Ensure subnet group and security group exist first
depends_on = [
  aws_db_subnet_group.main,
  aws_security_group.database,
]
}

# Output: RDS endpoint (connection string)
output "rds_endpoint" {
  description = "RDS instance endpoint"
  value       = aws_db_instance.postgres.endpoint
}

# Output: RDS address (hostname for connection)
output "rds_address" {
  description = "RDS instance address"
  value       = aws_db_instance.postgres.address
}

# Output: RDS port
output "rds_port" {
  description = "RDS instance port"
  value       = var.rds_port # From variable
}
EOF

```

Format and validate code:

```
# Format Terraform code (fixes indentation, spacing)
terraform fmt
# Automatically formats your code:
# - Fixes indentation (2 spaces)
# - Aligns equals signs
# - Sorts attributes
# - Makes code readable

# Validate Terraform syntax and configuration
terraform validate
# Checks for errors:
# - Syntax errors (missing brackets, quotes)
# - Invalid attribute names
# - Type mismatches
# - Missing required attributes
# Should show: "Success! The configuration is valid."
```

Plan and apply:

```
# Preview changes
terraform plan

# Should show: 1 to add (RDS instance)
# Note: This will take 5-10 minutes to create!

# Apply changes
terraform apply

# Type 'yes' when prompted
# Wait 5-10 minutes for RDS to be created
```

Expected output:

```
aws_db_instance.postgres: Creating...
aws_db_instance.postgres: Still creating... [10s elapsed]
```

```
aws_db_instance.postgres: Still creating... [1m0s elapsed]
...
aws_db_instance.postgres: Still creating... [5m0s elapsed]
aws_db_instance.postgres: Creation complete after 8m30s
[id=h-game-postgres]

Apply complete! Resources: 27 added, 0 changed, 0 destroyed.

Outputs:

rds_endpoint = "h-game-postgres.abc123xyz.us-east-1.rds.amazonaws.com:5432"
rds_address  = "h-game-postgres.abc123xyz.us-east-1.rds.amazonaws.com"
rds_port     = 5432
```

Note: RDS creation takes 5-10 minutes. AWS is:

1. Launching database instance
2. Installing PostgreSQL
3. Creating database
4. Configuring networking
5. Running health checks

Verify in AWS Console:

1. Go to AWS Console → RDS → Databases
2. You should see: `h-game-postgres`
3. Status: Available (takes 5-10 minutes)
4. Endpoint: Shows connection string

Checkpoint - Did it work?

- ☐ RDS instance created (took 5-10 minutes)
- ☐ Status: Available
- ☐ Endpoint shown in outputs
- ☐ Visible in AWS Console

RDS instance created!

What You're Learning

You're migrating your database from a Kubernetes StatefulSet (running inside your cluster) to AWS RDS (a managed database service). This is a **real-world production practice**, most companies use managed databases like RDS instead of running databases in Kubernetes.

Don't worry if this feels complex, we'll go step by step, and you can always ask for help!

Step 4: Wait for RDS to be Available

What's Happening?

After creating the RDS instance in Step 3, AWS needs time to:

- Launch the database server
- Install PostgreSQL
- Configure networking
- Run health checks

This takes 5-10 minutes - it's normal! Grab a coffee, while AWS does the work.

Why This Step Matters

You can't use RDS until it's fully ready. Trying to connect too early will fail, and that's okay - just wait a bit longer!

Step 4.1: Check RDS Status

Let's see if RDS is ready yet:

```
# Navigate to your Terraform directory
cd h-game-terraform

# Check RDS status
aws rds describe-db-instances \
  --db-instance-identifier h-game-postgres \
  --query
'DBInstances[0].[DBInstanceStatus,Endpoint.Address,Endpoint.Port]' \
```

```
--output table \
--region us-east-1
```

What to expect:

Status	What It Means	What to Do
creating	Still setting up (normal!)	Wait 5-10 minutes, then check again
available	Ready to use!	Continue to Step 4.2
backing-up	Creating initial backup	Wait a few more minutes
modifying	Being updated	Wait until it's available

If you see **creating**:

- This is **completely normal**!
- RDS takes 5-10 minutes to create
- Check again in 2-3 minutes: `aws rds describe-db-instances --db-instance-identifier h-game-postgres --query 'DBInstances[0].DBInstanceStatus' --output text --region us-east-1`

If you see **available**:

- **Great!** RDS is ready!
- Note the **Endpoint.Address** - you'll need this later
- It looks like: `h-game-postgres.abc123xyz.us-east-1.rds.amazonaws.com`

Step 4.2: Test Connection from EKS

Why test? This confirms that:

1. RDS is accessible from your Kubernetes cluster
2. Network security groups are configured correctly
3. You can connect before migrating data

Don't worry if this seems complex, we're just running a temporary test pod that connects to RDS and then disappears.

```
# Get RDS endpoint from Terraform
cd h-game-terraform
RDS_ENDPOINT=$(terraform output -raw rds_address)
echo "Testing connection to: $RDS_ENDPOINT"

# Create a temporary test pod (it will delete itself when done)
kubectl run postgres-test-$(date +%s) \
  --image=postgres:15-alpine \
  --rm -it --restart=Never \
  -n h-game \
  --env="PGPASSWORD=$(kubectl get secret postgres-secret -n h-game -o \
jsonpath='{.data.postgres-password}' | base64 -d)" \
  --env="PGUSER=$(kubectl get secret postgres-secret -n h-game -o \
jsonpath='{.data.postgres-user}' | base64 -d)" \
  --env="PGDATABASE=$(kubectl get secret postgres-secret -n h-game -o \
jsonpath='{.data.postgres-db}' | base64 -d)" \
  -- psql -h "$RDS_ENDPOINT" -U "$PGUSER" -d "$PGDATABASE" -c "SELECT \
version();"

```

What this command does:

1. Creates a temporary pod with PostgreSQL client tools
2. Gets database credentials from your Kubernetes secret
3. Connects to RDS and runs a simple query
4. Deletes itself when done (`--rm` flag)

Expected output:

```
PostgreSQL 15.15 on x86_64-pc-linux-gnu...
```

If it works: Great! RDS is accessible from your cluster.

If it fails:

- **"Connection timeout"** → Security groups might not be configured yet. Check Step 3 security groups.

- **"Password authentication failed"** → Secret might have wrong password. Check `k8s/UPDATE_RDS_SECRET.md`.
- **"No route to host"** → Network issue. Verify RDS is in the same VPC as your EKS cluster.

Don't panic! These errors are fixable. Check the troubleshooting section at the end.

Checkpoint: Step 4 Complete

You're done with Step 4 when:

- ☐ RDS status shows **available**
- ☐ You have the RDS endpoint address
- ☐ (Optional) Connection test succeeded

If something didn't work:

- RDS still **creating**? Wait 5-10 minutes - this is normal!
- Connection test failed? Check troubleshooting section below.
- Not sure? That's okay! Ask for help.

Ready for Step 5? Great! Let's migrate your data.

Step 5: Migrate Data from StatefulSet

What's Happening?

You're moving your existing database data (if you have any) from the old PostgreSQL StatefulSet to the new RDS instance.

Important: If you don't have any data yet, that's perfectly fine! You can skip the export step and just initialize the schema on RDS.

Step 5.1: Check If You Have Data

First, let's see if you have any data to migrate:

```
migrate:
# Check if PostgreSQL StatefulSet still exists
kubectl get statefulset postgres -n h-game 2>/dev/null

# If it exists, check if there's data
kubectl exec postgres-0 -n h-game -- psql -U gameuser -d
humor_memory_game \
  -c "SELECT COUNT(*) as total_games FROM games;" \
  -c "SELECT COUNT(*) as total_users FROM users;" 2>/dev/null || echo
"No data or StatefulSet doesn"
```

What to do:

- **If you see numbers > 0:** You have data! Continue to Step 5.2.
- **If you see "No resources found" or "0 rows":** No data to migrate! Skip to Step 5.3 (schema initialization).

Step 5.2: Export Data from StatefulSet (If You Have Data)

What we're doing: Creating a backup file of your current database.

```
# Make sure you're in the right directory
cd /Users/mac/Desktop/devops\ playbook

# Export data from StatefulSet
```

```
kubectl exec postgres-0 -n h-game -- \
  pg_dump -U gameuser humor_memory_game > migration-backup.sql

# Check if the file was created
ls -lh migration-backup.sql

# Verify it's not empty (should show file size > 0)
wc -l migration-backup.sql
```

What this does:

- `pg_dump` creates a SQL file with all your data
- Saves it as `migration-backup.sql` in your current directory
- This is your **safety backup** - keep it until migration is complete!

Expected output:

- File created: `migration-backup.sql`
- File size: Usually a few KB to MB (depends on your data)

If it fails:

- **"No such file or directory"** → StatefulSet might be deleted. That's okay, you might not have data to migrate.
- **"Permission denied"** → Check that you have write permissions in the current directory.

Step 5.3: Initialize RDS Schema

What we're doing: Setting up the database structure (tables, etc.) on RDS. This uses the same SQL file you used for the StatefulSet.

Option A: Using Kubernetes Job (Recommended - Automated)

This is the **professional way** - it runs automatically and uses your existing ConfigMap:

```
# Apply the RDS initialization job
kubectl apply -f k8s/rds-init-job.yaml

# Watch the job run
kubectl get jobs -n h-game
```

```
kubectl logs -n h-game job/rds-init -f
```

What to expect:

- Job creates a pod
- Pod runs the SQL initialization script
- Pod completes and deletes itself
- Job shows **COMPLETIONS 1/1**

If it works: Schema is initialized! Continue to Step 5.4.

If it fails:

- Check logs: `kubectl logs -n h-game job/rds-init`
- Common issues:
 - **"Secret not found"** → Create the secret first (see [k8s/UPDATE_RDS_SECRET.md](#))
 - **"Connection refused"** → RDS might not be ready yet, or security groups need updating

Option B: Manual Initialization (If Job Fails)

If the Job doesn't work, you can do it manually:

```
# Get RDS endpoint
cd h-game-terraform
RDS_ENDPOINT=$(terraform output -raw rds_address)

# Get database credentials from secret
DB_USER=$(kubectl get secret postgres-secret -n h-game -o
jsonpath='{.data.postgres-user}' | base64 -d)
DB_PASSWORD=$(kubectl get secret postgres-secret -n h-game -o
jsonpath='{.data.postgres-password}' | base64 -d)
DB_NAME=$(kubectl get secret postgres-secret -n h-game -o
jsonpath='{.data.postgres-db}' | base64 -d)

# Run initialization from a temporary pod
kubectl run postgres-init-$(date +%s) \
  --image=postgres:15-alpine \
  --rm -i --restart=Never \
```

```

-n h-game \
--env="PGPASSWORD=$DB_PASSWORD" \
--env="PGUSER=$DB_USER" \
--env="PGDATABASE=$DB_NAME" \
--command -- sh -c "cat /tmp/init.sql | psql -h $RDS_ENDPOINT -U
\ $PGUSER -d \ $PGDATABASE"

# Note: This requires the init SQL to be in a ConfigMap
# If you don't have it, use the combined-init.sql file:
kubectl create configmap postgres-init-sql \
--from-file=combined-init.sql=database/combined-init.sql \
-n h-game --dry-run=client -o yaml | kubectl apply -f -

```

Step 5.4: Import Data to RDS (If You Have Data)

Only do this if you exported data in Step 5.2!

```

# Get RDS endpoint
cd h-game-terraform
RDS_ENDPOINT=$(terraform output -raw rds_address)

# Get database credentials
DB_USER=$(kubectl get secret postgres-secret -n h-game -o
jsonpath='{.data.postgres-user}' | base64 -d)
DB_PASSWORD=$(kubectl get secret postgres-secret -n h-game -o
jsonpath='{.data.postgres-password}' | base64 -d)
DB_NAME=$(kubectl get secret postgres-secret -n h-game -o
jsonpath='{.data.postgres-db}' | base64 -d)

# Import data using a temporary pod
cat migration-backup.sql | kubectl run postgres-import-$(date +%s) \
--image=postgres:15-alpine \
--rm -i --restart=Never \
-n h-game \
--env="PGPASSWORD=$DB_PASSWORD" \
--env="PGUSER=$DB_USER" \
--env="PGDATABASE=$DB_NAME" \
--command -- psql -h "$RDS_ENDPOINT" -U "$PGUSER" -d "$PGDATABASE"

```

What this does:

- Reads your backup file
- Sends it to a temporary pod
- Pod imports the data into RDS
- Pod deletes itself when done

Expected output:

- Various **CREATE TABLE**, **INSERT**, etc. statements
- No errors (or only minor warnings)

If it fails:

- **"Relation already exists"** → Schema was already initialized. That's okay!
- **"Connection timeout"** → Check security groups and RDS status.

Step 5.5: Verify Data Migration

Let's confirm everything worked:

```
# Get RDS endpoint
cd h-game-terraform
RDS_ENDPOINT=$(terraform output -raw rds_address)

# Verify data in RDS
kubect1 run postgres-verify-$(date +%s) \
  --image=postgres:15-alpine \
  --rm -it --restart=Never \
  -n h-game \
  --env="PGPASSWORD=$(kubect1 get secret postgres-secret -n h-game -o \
  jsonpath='{.data.postgres-password}' | base64 -d)" \
  --env="PGUSER=$(kubect1 get secret postgres-secret -n h-game -o \
  jsonpath='{.data.postgres-user}' | base64 -d)" \
  --env="PGDATABASE=$(kubect1 get secret postgres-secret -n h-game -o \
  jsonpath='{.data.postgres-db}' | base64 -d)" \
  -- psql -h "$RDS_ENDPOINT" -U "$PGUSER" -d "$PGDATABASE" \
  -c "SELECT COUNT(*) as total_games FROM games;" \
  -c "SELECT COUNT(*) as total_users FROM users;" \
  -c "SELECT 'RDS migration successful!' as status;"
```

Expected output:

- Row counts (might be 0 if you didn't have data - that's fine!)
- "RDS migration successful!" message

Checkpoint: Step 5 Complete**You're done with Step 5 when:**

- ☐ RDS schema is initialized (tables exist)
- ☐ If you had data) Data is imported to RDS
- ☐ Verification query shows expected results

If something didn't work:

- No data to migrate? That's perfectly fine! Just initialize the schema.
- Import failed? Check the error message - it's usually a connection or permission issue.
- Not sure? That's okay! The important thing is that RDS has the schema (tables). Data can be added later.

Ready for Step 6? Great! Let's connect your backend to RDS.

Step 6: Update Backend to Use RDS**What's Happening?**

You're changing your backend application to connect to RDS instead of the old PostgreSQL StatefulSet. This is like changing the address your app uses to find the database.

Important: Your backend is currently pointing to `postgres-service` (the old StatefulSet). We need to change it to the RDS endpoint.

Step 6.1: Safety First, Verify RDS is Working

Before making any changes, let's make sure RDS is ready!

```
# Get RDS endpoint
cd h-game-terraform
RDS_ENDPOINT=$(terraform output -raw rds_address)
```

```

echo "RDS Endpoint: $RDS_ENDPOINT"

# Test connection one more time
kubectl run postgres-final-test-$(date +%s) \
  --image=postgres:15-alpine \
  --rm -it --restart=Never \
  -n h-game \
  --env="PGPASSWORD=$(kubectl get secret postgres-secret -n h-game -o
jsonpath='{.data.postgres-password}' | base64 -d)" \
  --env="PGUSER=$(kubectl get secret postgres-secret -n h-game -o
jsonpath='{.data.postgres-user}' | base64 -d)" \
  --env="PGDATABASE=$(kubectl get secret postgres-secret -n h-game -o
jsonpath='{.data.postgres-db}' | base64 -d)" \
  -- psql -h "$RDS_ENDPOINT" -U "$PGUSER" -d "$PGDATABASE" \
  -c "SELECT 'RDS is ready!' as status, version();"

```

If this works: RDS is ready! Continue to Step 6.2.

If this fails: Don't update the backend yet! Fix the RDS connection first (check troubleshooting).

Step 6.2: Get RDS Connection Details

You'll need these values to update your backend:

```

# Get RDS endpoint and port
cd h-game-terraform
RDS_ENDPOINT=$(terraform output -raw rds_address)
RDS_PORT=$(terraform output -raw rds_port)

echo "📝 Write these down - you'll need them:"
echo "RDS Endpoint: $RDS_ENDPOINT"
echo "RDS Port: $RDS_PORT"
echo ""
echo "Database credentials are in Kubernetes Secret: postgres-secret"

```

Save these values - you'll use them in the next step!

Step 6.3: Update Backend Deployment

What we're changing:

- **DB_HOST**: From `postgres-service` → To RDS endpoint
- Everything else stays the same (port, credentials, etc.)

Option A: Edit the YAML File (Recommended - Most Control)

1. Open the file:

```
# Open in your editor
code k8s/backend-deployment.yaml
# Or use: nano, vim, or your preferred editor
```

2. Find the database environment variables (around line 100-120):

```
# BEFORE (what you have now):
- name: DB_HOST
  value: "postgres-service" # ← Old StatefulSet service
```

3. Change it to:

```
# AFTER (what you need):
- name: DB_HOST
  value: "h-game-postgres.cyb44k06izcw.us-east-1.rds.amazonaws.com"
# ← RDS endpoint
# Replace with YOUR RDS endpoint from Step 6.2!
```

4. Save the file

Apply the changes:

```
kubectl apply -f k8s/backend-deployment.yaml
```

What happens:

- Kubernetes sees the change
- Creates new pods with the updated configuration
- Old pods are gradually replaced (rolling update)
- Takes 1-2 minutes

Option B: Quick Update with kubectl (Faster, Less Control)

If you prefer a quick command:

```
# Get RDS endpoint
cd h-game-terraform
RDS_ENDPOINT=$(terraform output -raw rds_address)

# Update backend deployment
kubectl set env deployment/backend \
  DB_HOST="$RDS_ENDPOINT" \
  -n h-game

# This triggers a rolling update automatically
```

Both methods work! Choose whichever you're more comfortable with.

Step 6.4: Watch the Update

Let's see the backend pods restart with the new configuration:

```
# Watch pods restart
kubectl get pods -n h-game -l app=backend -w
```

What to expect:

1. Old pods show **Terminating**
2. New pods show **ContainerCreating** → **Running**
3. All pods eventually show **Running** (takes 1-2 minutes)

Press Ctrl+C to stop watching when all pods are **Running**.

Step 6.5: Verify Backend Connects to RDS

Check the logs to see if the backend connected successfully:

```
# Check backend logs
kubectl logs -n h-game -l app=backend --tail=50

# Look for these messages:
# Good: "Database connected", "Connected to PostgreSQL", "Database
ready"
# Bad: "Connection refused", "ECONNREFUSED", "timeout"
```

If you see connection errors:

- **"Connection timeout"** → Security groups might not allow backend → RDS traffic
- **"Password authentication failed"** → Secret might have wrong password
- **"No route to host"** → Network issue, check VPC configuration

Don't panic! These are fixable. Check the troubleshooting section.

Step 6.6: Test Backend Health

Let's verify the backend is working with RDS:

```
# Port forward to backend (in one terminal)
kubectl port-forward service/backend-service 3001:3001 -n h-game

# In another terminal, test the health endpoint
curl http://localhost:3001/health

# Or test the API
curl http://localhost:3001/api/leaderboard
```

Expected response:

```
{
  "status": "healthy",
```

```
"services": {  
  "database": "connected", // ← This confirms RDS connection!  
  "redis": "connected",  
  "api": "running"  
}  
}
```

If it works: Success! Your backend is now using RDS!

If it doesn't work:

- Check backend logs (Step 6.5)
- Verify RDS is accessible (Step 6.1)
- Check that the secret has correct credentials

Checkpoint: Step 6 Complete

You're done with Step 6 when:

- Backend deployment updated with RDS endpoint
- Backend pods restarted and running
- Backend logs show "Database connected" or similar
- Health check shows "database": "connected"

If something didn't work:

- Pods not starting? Check logs: `kubectl logs -n h-game -l app=backend`
- Connection errors? Verify RDS endpoint and security groups.
- Health check failing? Check backend logs for specific errors.

Ready for Step 7? Great! Let's clean up the old StatefulSet.

Step 7: Delete PostgreSQL StatefulSet

What's Happening?

Now that your backend is using RDS, you don't need the old PostgreSQL StatefulSet anymore. We'll delete it to free up resources.

Important: Only do this **after** you've verified that:

1. RDS is working (Step 4)
2. Data is migrated (Step 5)
3. Backend is using RDS (Step 6)

Don't worry - if something goes wrong, you can always recreate the StatefulSet. But let's be careful!

Step 7.1: Final Verification - RDS is Working

Before deleting anything, let's make absolutely sure RDS works:

```
# Get RDS endpoint
cd h-game-terraform
RDS_ENDPOINT=$(terraform output -raw rds_address)

# Test RDS connection
kubectl run postgres-final-verify-$(date +%s) \
  --image=postgres:15-alpine \
  --rm -it --restart=Never \
  -n h-game \
  --env="PGPASSWORD=$(kubectl get secret postgres-secret -n h-game -o
jsonpath='{.data.postgres-password}' | base64 -d)" \
  --env="PGUSER=$(kubectl get secret postgres-secret -n h-game -o
jsonpath='{.data.postgres-user}' | base64 -d)" \
  --env="PGDATABASE=$(kubectl get secret postgres-secret -n h-game -o
jsonpath='{.data.postgres-db}' | base64 -d)" \
  -- psql -h "$RDS_ENDPOINT" -U "$PGUSER" -d "$PGDATABASE" \
  -c "SELECT 'RDS is working perfectly!' as status;" \
  -c "SELECT COUNT(*) as total_tables FROM information_schema.tables
WHERE table_schema = 'public';"
```

Expected output:

- "RDS is working perfectly!"
- Table count > 0 (shows your tables exist)

If this works: RDS is confirmed working! Safe to delete StatefulSet.

If this fails: Don't delete the StatefulSet yet! Fix RDS first.

Step 7.2: Verify Backend is Using RDS

Double-check that your backend is actually using RDS, not the StatefulSet:

```
# Check what DB_HOST the backend pods are using
kubectl get deployment backend -n h-game -o
jsonpath='{.spec.template.spec.containers[0].env[?(@.name=="DB_HOST")
].value}' && echo

# Should show:
h-game-postgres.cyb44k06izcw.us-east-1.rds.amazonaws.com
# NOT: postgres-service
```

If it shows the RDS endpoint: Backend is using RDS! Safe to continue.

If it shows `postgres-service`: Backend is still using StatefulSet! Go back to Step 6 and update it first.

Step 7.3: Delete the StatefulSet

Now we can safely delete the old StatefulSet:

```
# Delete the StatefulSet
kubectl delete statefulset postgres -n h-game

# Verify it's gone
kubectl get statefulset -n h-game

# Should show: No resources found
```

What happens:

- Kubernetes stops the PostgreSQL pods
- Pods are terminated
- StatefulSet is removed
- **Note:** The PersistentVolumeClaim (PVC) with your data is still there (safety backup)

This is safe because:

- Your backend is already using RDS
- Your data is already in RDS (if you migrated it)
- The PVC still exists as a backup

Step 7.4: Verify StatefulSet is Deleted

```
# Check StatefulSets
kubectl get statefulset -n h-game

# Check Pods (should show no postgres pods)
kubectl get pods -n h-game -l app=postgres

# Should show: No resources found
```

If you see "No resources found": StatefulSet is deleted!

If you still see postgres pods:

- They might be terminating (wait 30 seconds)
- Or StatefulSet deletion failed (check with `kubectl get statefulset -n h-game`)

Step 7.5: (Optional) Delete the Service

The PostgreSQL Service is no longer needed:

```
# Delete the PostgreSQL service
kubectl delete service postgres-service -n h-game

# Verify it's gone
```

```
kubectl get service -n h-game | grep postgres  
  
# Should show: No resources found
```

This is safe - your backend is using RDS directly, not the service.

Step 7.6: (Optional) Delete the PVC - Only If You're Sure!

WARNING: This permanently deletes the old database data. Only do this if:

1. RDS has all your data
2. Backend is working with RDS
3. You've tested everything thoroughly

If you're not sure, keep the PVC! It doesn't cost anything and serves as a backup.

```
# List PVCs  
kubectl get pvc -n h-game  
  
# If you're 100% sure, delete it:  
kubectl delete pvc postgres-storage-postgres-0 -n h-game  
  
# Verify  
kubectl get pvc -n h-game  
# Should show: No resources found (or other PVCs, but not postgres)
```

Recommendation: Keep the PVC for a few days as a safety backup, then delete it when you're confident everything works.

Step 7.7: Final Verification - Everything Still Works

Let's make sure deleting the StatefulSet didn't break anything:

```
# 1. Check backend is still running  
kubectl get pods -n h-game -l app=backend  
  
# All should show: Running  
  
# 2. Test backend health
```



```

kubectl port-forward service/backend-service 3001:3001 -n h-game &
sleep 2
curl http://localhost:3001/health

# Should show: "database": "connected"

# 3. Test RDS directly
cd h-game-terraform
RDS_ENDPOINT=$(terraform output -raw rds_address)
kubectl run postgres-final-test-$(date +%s) \
  --image=postgres:15-alpine \
  --rm -it --restart=Never \
  -n h-game \
  --env="PGPASSWORD=$(kubectl get secret postgres-secret -n h-game -o
jsonpath='{.data.postgres-password}' | base64 -d)" \
  --env="PGUSER=$(kubectl get secret postgres-secret -n h-game -o
jsonpath='{.data.postgres-user}' | base64 -d)" \
  --env="PGDATABASE=$(kubectl get secret postgres-secret -n h-game -o
jsonpath='{.data.postgres-db}' | base64 -d)" \
  -- psql -h "$RDS_ENDPOINT" -U "$PGUSER" -d "$PGDATABASE" \
  -c "SELECT 'Migration complete! RDS is working!' as status;"

```

If everything works: Congratulations! Migration is complete!

Checkpoint: Step 7 Complete

You're done with Step 7 when:

- ☐ StatefulSet is deleted
- ☐ PostgreSQL Service is deleted (optional)
- ☐ Backend is still working with RDS
- ☐ Final verification tests pass

If something didn't work:

- Backend stopped working? Check logs: `kubectl logs -n h-game -l app=backend`
- Can't connect to RDS? Verify security groups and RDS status.
- Not sure? That's okay! The important thing is that RDS is working.

Migration Complete!

Congratulations! You've successfully migrated from a Kubernetes StatefulSet to AWS RDS. This is a **real production skill** that companies use every day!

What You've Learned:

- How to create and manage RDS instances
- How to migrate data between databases
- How to update applications to use new databases
- How to safely clean up old resources

What's Next?

- Monitor RDS performance in CloudWatch
- Set up automated backups
- Consider Multi-AZ for high availability (production)
- Explore AWS Secrets Manager for credential management

Great job!

Troubleshooting

Common Issues and Solutions

Issue: RDS Status Stuck on "creating"

Solution: This is normal! RDS takes 5-10 minutes. Wait and check again:

```
aws rds describe-db-instances --db-instance-identifier  
h-game-postgres --query 'DBInstances[0].DBInstanceStatus' --output  
text --region us-east-1
```

Issue: "Connection timeout" from EKS to RDS

Solution: Security groups might not be configured. Check:

1. RDS security group allows traffic from EKS nodes
2. RDS security group allows traffic from EKS cluster security group
3. Both are in the same VPC

Issue: "Password authentication failed"

Solution: Secret might have wrong password. Update it:

```
# See k8s/UPDATE_RDS_SECRET.md for instructions
```

Issue: Backend pods crashing after update

Solution: Check logs:

```
kubectl logs -n h-game -l app=backend --tail=100
```

Common causes:

- Wrong RDS endpoint
- Wrong credentials in secret
- Security group blocking traffic

Issue: "No route to host"

Solution: Network configuration issue. Verify:

1. RDS is in the same VPC as EKS
2. Security groups allow traffic
3. Route tables are configured correctly

Remember: Most issues are fixable! Don't give up.

Phase 3: Validation & Consolidation (30 min)

Quiz: RDS Understanding

Answer these questions to verify your understanding:

Question 1: What is RDS?

Answer:

- **RDS** = Relational Database Service
- AWS managed database
- **AWS handles:** Backups, updates, monitoring, failover
- **You manage:** Schema, queries, application logic
- **Supports:** PostgreSQL, MySQL, MariaDB, Oracle, SQL Server
- **Key point:** RDS = Managed database operations.

Question 2: What are the key benefits of RDS?

Answer:

- **Automatic backups:** 7-day retention (configurable)
- **Point-in-time recovery:** Restore to any point in last 7 days
- **Automatic updates:** AWS handles PostgreSQL version updates
- **Multi-AZ failover:** Automatic failover to standby (zero downtime)
- **Monitoring:** Built-in CloudWatch metrics
- **Less operational work:** Focus on application, not database
- **Key point:** RDS = Less operational burden, more reliability.

Question 3: How does RDS connect to your application?

Answer:

- **RDS endpoint:**
database-name.abc123xyz.us-east-1.rds.amazonaws.com
- **Connection:** Backend connects via endpoint (like connecting to any PostgreSQL)
- **Security:** Security group controls access (only backend can connect)

- Network: RDS in private subnet, backend in private subnet (secure)
- **Key point:** RDS = Standard PostgreSQL connection, just different hostname.

Checklist: Did You Complete Everything?

Before moving on, check these boxes:

Understanding Phase

- ☐ I understand what RDS is (managed database)
- ☐ I understand RDS benefits (backups, failover, updates)
- ☐ I understand migration process
- ☐ I completed all design exercises

Implementation Phase

- ☐ DB subnet group created
- ☐ Parameter group created
- ☐ RDS instance created (took 5-10 minutes)
- ☐ Data migrated from StatefulSet (or schema ready)
- ☐ Backend updated to use RDS
- ☐ Backend connects to RDS successfully
- ☐ StatefulSet deleted (migrated!)

Verification Phase

- ☐ Can explain RDS vs StatefulSet
- ☐ Can explain RDS benefits
- ☐ Can connect to RDS
- ☐ Understand managed database operations

Git

- ☐ Committed `rds.tf`
- ☐ Committed updated `backend-deployment-rds.yaml`
- ☐ Documented migration process

Key Takeaways

The Big Picture

What you learned in this task:

RDS MIGRATION COMPLETE

- RDS PostgreSQL (managed database)
- Automatic backups (7-day retention)
- Data migrated from StatefulSet
- Backend connected to RDS
- StatefulSet deleted (migrated!)

Database is now managed by AWS!

Critical Concepts

1. RDS:

- AWS managed database
- AWS handles operations (backups, updates, monitoring)
- You manage data (schema, queries)
- Less operational burden

2. Migration:

- Export from old database
- Import to RDS
- Update application connection
- Delete old database

3. Benefits:

- Automatic backups

- Point-in-time recovery
- Automatic updates
- Multi-AZ failover (optional)
- Built-in monitoring

4. Connection:

- RDS endpoint (hostname)
- Standard PostgreSQL connection
- Security groups control access
- Private subnets (secure)

What's Next?

Congratulations! You've migrated to managed database!

You now have:

- ☐ RDS PostgreSQL (managed by AWS)
- ☐ Automatic backups configured
- ☐ Backend connected to RDS
- ☐ Old StatefulSet removed

Next: TASK T6 ElastiCache Redis Migration

In T6, you'll learn:

- How to create ElastiCache Redis (managed cache)
- How to migrate from Redis Deployment to ElastiCache
- How to connect backend to ElastiCache
- How managed cache works

But most importantly: You now understand managed databases. Your database operations are automated!

Proceed to TASK T6: ElastiCache Redis Migration

TASK T6: ElastiCache Redis Migration

Time: 3 hours | **Priority:** High | **Ticket:** DEVOPS-5106 | **Cost:** ~\$13/month (cache.t3.micro)

Mission Brief

Migrate from self managed Redis (Deployment) to AWS ElastiCache (managed cache). ElastiCache handles backups, failover, and scaling automatically. This completes the managed services migration.

Outcomes:

- Understand ElastiCache vs self-managed Redis
- Create ElastiCache Redis cluster with Terraform
- Migrate cache configuration from Deployment to ElastiCache
- Update backend to connect to ElastiCache
- Delete Redis Deployment (no longer needed)
- Understand managed cache benefits

Prerequisites: Complete Task T5 (RDS PostgreSQL Migration)

Setup Note: ElastiCache will be created in your VPC private subnets. We'll migrate from your existing Redis Deployment (from K3), then delete the Deployment. See Section 5 Overview for general setup details.

Cost Note: ElastiCache cache.t3.micro costs ~\$13/month. Single node for learning (saves \$13/month vs Multi-AZ). Backups are optional (cost extra). See Section 5 Overview for cost-saving strategies.

The One Thing That Will Make This Click

ElastiCache is AWS's way of running a cache **for you**, so you don't have to manage Redis yourself.

When you run Redis on your own, you're responsible for installing it, updating it, monitoring it, and fixing it when it breaks.

That means more time spent on operations and less time improving your application.

With ElastiCache, AWS handles things like updates, monitoring, scaling, and failover automatically.

You only decide what data to cache and how your app should use it to be faster.

If something goes wrong with the cache, AWS fixes the infrastructure instead of you debugging Redis at midnight.

The mental shift is simple: **AWS runs the cache, you use it to make your app faster.**

Phase 1: Understanding ElastiCache (20 min)

What is ElastiCache?

ElastiCache = Managed Redis/Memcached

The Cache Management Analogy

SELF-MANAGED CACHE (Redis Deployment):

You manage everything:

- └─ Install Redis
- └─ Configure Redis
- └─ Handle updates (manual)
- └─ Monitor performance
- └─ Fix when it breaks
- └─ **Operational burden: HIGH**

Cache data: Ephemeral (lost if Pod crashes)

MANAGED CACHE (ElastiCache):

AWS manages cache:

- └─ AWS runs Redis
- └─ AWS handles updates (automatic)
- └─ AWS monitors performance
- └─ AWS fixes when it breaks
- └─ **Operational burden: LOW**

You manage:

- └─ Cache keys (what to cache)
- └─ Cache strategy (when to cache)
- └─ Focus on: Application performance

Cache data: Can be backed up (optional)

Key insight: ElastiCache = AWS manages cache operations, you use it for performance.

ElastiCache vs Redis Deployment

Comparison:

Aspect	Redis Deployment (K3)	ElastiCache Redis (T6)
Management	You manage everything	AWS manages operations
Backups	None (ephemeral)	Optional (snapshots)
Updates	Manual (you upgrade)	Automatic (scheduled)
Failover	None (single Pod)	Automatic (Multi-AZ option)
Scaling	Manual (edit Deployment)	Automatic (modify cluster)
Monitoring	You set up	Built-in CloudWatch
Cost	EC2 instance cost	Cache instance cost
Operational Work	High	Low

When to use each:

- **Redis Deployment:** Learning, development, ephemeral cache
- **ElastiCache:** Production, want managed operations, need persistence

Exercise 1: Why ElastiCache Instead of Deployment?

Scenario: You need to choose between Redis Deployment and ElastiCache for production.

Question: What are the key benefits of ElastiCache?

Think about:

- Cache persistence
- Failover
- Updates
- Operational burden

Your answer: _____

Answer Below 

Answer:

Key Benefits of ElastiCache:

Key Benefits of ElastiCache:

1. Managed Operations:

Redis Deployment:

- You manage: Redis installation, configuration
- You handle: Updates, monitoring
- Operational work: High

ElastiCache:

- AWS manages: Redis installation, configuration
- AWS handles: Updates, monitoring
- Operational work: Low

2. Automatic Failover (Multi-AZ):

Redis Deployment:

- Single Pod
- If Pod fails → Cache lost, need to rebuild
- Manual recovery

ElastiCache Multi-AZ:

- Primary cache (us-east-1a)
- Replica cache (us-east-1b)
- If primary fails → Automatic failover to replica
- Zero downtime, cache preserved

3. Optional Persistence:

Redis Deployment:

- Ephemeral (data lost if Pod crashes)
- Cache rebuilds from database

ElastiCache:

- └ Can enable snapshots (backup cache data)
- └ Can restore from snapshot
- └ Faster cache recovery

4. Better Performance:

Redis Deployment:

- └ Runs on shared node (with other Pods)
- └ Resource contention possible
- └ Variable performance

ElastiCache:

- └ Dedicated instance (only Redis)
- └ No resource contention
- └ Consistent performance

This is why production uses ElastiCache!

Exercise 2: ElastiCache Configuration

Scenario: You're creating an ElastiCache cluster.

Question: What configuration should you use?

Think about:

- Node type? (cache.t3.micro for learning)
- Multi-AZ? (High availability, but +\$13/month)
- Subnets? (Private subnets for security)
- Security groups? (Only backend can access)

Your answer: _____

Answer Below 

Answer:

Node Type:

- **cache.t3.micro** - 0.5GB RAM, \$13/month
- Good for learning and small workloads
- **Production:** cache.t3.small or larger

Multi-AZ:

- **Single node for learning** - Saves \$13/month
- **Multi-AZ for production** - Automatic failover, zero downtime

Subnets:

- **Private subnets** - More secure, hidden from internet
- **Cache Subnet Group** - ElastiCache needs subnets in multiple AZs

Security Groups:

- **Only backend can access** - Port 6379 from backend security group
- **No public access** - Cache should be private

Engine:

- **Redis 7.0** - Latest stable version
- **Cluster mode disabled** - Simpler for learning

This is the standard ElastiCache configuration!

Design Checkpoint

Before moving on, make sure you can answer these in plain English:

Question	Your Answer Should Be Like...
What is ElastiCache?	"AWS managed cache - AWS handles operations"
Why use ElastiCache?	"Automatic failover, updates, less operational work"
Where should ElastiCache run?	"Private subnets (secure, hidden from internet)"
Who can access ElastiCache?	"Only backend (via security groups)"

Can't answer one? Go back and re-read that exercise.

All clear? Great! Let's create ElastiCache!

Phase 2: Implementation (2.5 hours)

What We're Building

Before this task:

- Redis Deployment (self-managed, from K3)
- Ephemeral cache (data lost if Pod crashes)
- Manual management

After this task:

- ElastiCache Redis (managed by AWS)
- Optional persistence (snapshots)
- Backend connected to ElastiCache
- Deployment deleted (migrated!)

Step-by-Step Plan

We'll do this in 5 steps:

Step 1: Create Cache Subnet Group
"ElastiCache needs subnets in multiple AZs"

Step 2: Create ElastiCache Cluster
"Managed Redis cache"

Step 3: Wait for ElastiCache to be Available
"Takes 5-10 minutes"

Step 4: Update Backend to Use ElastiCache
"Change connection string"

Step 5: Delete Redis Deployment
"Clean up old cache"

Note: This task uses the `provider.tf` and `variables.tf` files created in T1. If you haven't completed T1, do that first.

Note: Ensure `.gitignore` is configured (created in T1). The `.gitignore` file prevents committing sensitive files like `terraform.tfvars` and state files.

Step 1: Create Cache Subnet Group

What we're doing: Creating a subnet group that tells ElastiCache which subnets to use.

Why: ElastiCache needs subnets in multiple AZs for high availability.

Add ElastiCache variables first (if not already in variables.tf):

```
# Add to variables.tf
# IDE Option: Instead of using cat, add these to your variables.tf file in
your IDE

cat >> variables.tf << 'EOF'

# =====
# ELASTICACHE VARIABLES
# =====

variable "elasticache_node_type" {
  description = "ElastiCache node type"
  type        = string
  default     = "cache.t3.micro"

  validation {
    condition = can(regex("^cache\\.t3\\.?(micro|small|medium)$",
var.elasticache_node_type))
    error_message = "ElastiCache node type must be cache.t3.micro,
cache.t3.small, or cache.t3.medium."
  }
}

variable "elasticache_num_cache_nodes" {
  description = "Number of cache nodes"
  type        = number
  default     = 1
}

variable "redis_port" {
  description = "Redis cache port"
  type        = number
  default     = 6379
}
```

```

variable "redis_cluster_id" {
  description = "ElastiCache cluster identifier"
  type        = string
  default     = "h-game-redis"
}
EOF

```

Add to terraform.tfvars

```

cat >> terraform.tfvars << 'EOF'

# ElastiCache Configuration
elasticache_node_type      = "cache.t3.micro"
elasticache_num_cache_nodes = 1
redis_port                 = 6379
redis_cluster_id          = "h-game-redis"
EOF

```

Create cache subnet group configuration:

```

# Create elasticache.tf
# IDE Option: Instead of using cat, create elasticache.tf in your IDE and
# paste the code below (see IDE-EDITOR-GUIDE.md)

cat > elasticache.tf << 'EOF'
# =====
# ELASTICACHE REDIS (Managed Cache)
# =====
# AWS managed Redis
# Replaces: Redis Deployment (from K3)

# =====
# CACHE SUBNET GROUP
# =====
# Tells ElastiCache which subnets to use
# ElastiCache needs subnets in multiple AZs

resource "aws_elasticache_subnet_group" "main" {
  name      = "${local.name_prefix}-cache-subnet-group"

```

```

subnet_ids = [
    aws_subnet.private_1.id, # Private subnet 1 (us-east-1a)
    aws_subnet.private_2.id, # Private subnet 2 (us-east-1b)
    aws_subnet.private_3.id, # Private subnet 3 (us-east-1c)
]

tags = merge(
    local.common_tags,
    {
        Name = "${local.name_prefix}-cache-subnet-group"
        Task = "T6"
    }
)
}

# Output: Subnet group name
output "cache_subnet_group_name" {
    description = "Name of the cache subnet group"
    value       = aws_elasticache_subnet_group.main.name
}
EOF

```

Plan and apply:

```

# Preview changes
terraform plan

# Should show: 1 to add (cache subnet group)

# Apply changes
terraform apply

# Type 'yes' when prompted

```

Expected output:

```
aws_elasticache_subnet_group.main: Creating...  
aws_elasticache_subnet_group.main: Creation complete after 2s  
[id=h-game-cache-subnet-group]  
  
Apply complete! Resources: 28 added, 0 changed, 0 destroyed.
```

Checkpoint - Did it work?

- Cache subnet group created
- Includes all 3 private subnets

Cache subnet group ready!

Step 2: Create ElastiCache Cluster

What we're doing: Creating the ElastiCache Redis cluster.

Add ElastiCache cluster configuration:

```
# Add to elasticache.tf
cat >> elasticache.tf << 'EOF'

# =====
# ELASTICACHE REDIS CLUSTER
# =====
# Managed Redis cache
# AWS handles: Backups, updates, monitoring

resource "aws_elasticache_replication_group" "redis" {
  # Basic configuration
  replication_group_id      = var.redis_cluster_id # Cluster identifier (from
variable)
  description               = "${var.project_name} Redis cache"
  engine                   = "redis"              # Cache engine
  engine_version            = "7.0"               # Redis version

  # Node configuration
  node_type                 = var.elasticache_node_type # From variable
  port                      = var.redis_port           # Redis port (from
variable)

  # Cluster configuration
  num_cache_clusters        = 1 # Single node for learning (saves cost)
  # For production: Use num_cache_clusters = 2+ for Multi-AZ

  # Network configuration
  subnet_group_name         = aws_elasticache_subnet_group.main.name # Which
subnets
  security_group_ids         = [aws_security_group.cache.id]          # Security
group (only backend can access)

  # Parameter group (optional - defaults work fine)
  # parameter_group_name = "default.redis7"

  # Snapshot configuration (optional - for persistence)
  snapshot_retention_limit   = 0 # 0 = no snapshots (ephemeral, saves cost)
```

```

# For production: Set to 1-5 days for cache backups

# Automatic failover (requires Multi-AZ)
automatic_failover_enabled = false # Requires num_cache_clusters >= 2
# For production: Set to true with Multi-AZ

# Maintenance window
maintenance_window          = "mon:05:00-mon:06:00" # UTC

# Tags
tags = merge(
  local.common_tags,
  {
    Name = var.redis_cluster_id
    Task = "T6"
  }
)

# Ensure subnet group and security group exist first
depends_on = [
  aws_elasticache_subnet_group.main,
  aws_security_group.cache,
]
}

# Output: ElastiCache endpoint (connection string)
output "elasticache_endpoint" {
  description = "ElastiCache Redis endpoint"
  value       = aws_elasticache_replication_group.redis.primary_endpoint_address
}

# Output: ElastiCache port
output "elasticache_port" {
  description = "ElastiCache Redis port"
  value       = aws_elasticache_replication_group.redis.port
}
EOF

```


Format and validate code:

```
# Format Terraform code (fixes indentation, spacing)
terraform fmt
# Automatically formats your code:
# - Fixes indentation (2 spaces)
# - Aligns equals signs
# - Sorts attributes
# - Makes code readable

# Validate Terraform syntax and configuration
terraform validate
# Checks for errors:
# - Syntax errors (missing brackets, quotes)
# - Invalid attribute names
# - Type mismatches
# - Missing required attributes
# Should show: "Success! The configuration is valid."
```

Plan and apply:

```
# Preview changes
terraform plan
# Review the plan carefully:
# - Check what will be created (+)
# - Check what will be changed (~)
# - Check what will be destroyed (-)
# Always review before apply!
```

Should show: 1 to add (ElastiCache cluster)

Note: This will take 5-10 minutes to create!

Apply changes

```
terraform apply

# Type 'yes' when prompted
# Wait 5-10 minutes for ElastiCache to be created
```

Expected output:

```
aws_elasticache_replication_group.redis: Creating...
aws_elasticache_replication_group.redis: Still creating... [10s elapsed]
aws_elasticache_replication_group.redis: Still creating... [1m0s elapsed]
... aws_elasticache_replication_group.redis: Still creating... [5m0s
elapsed] aws_elasticache_replication_group.redis: Creation complete after
7m30s [id=h-game-redis]
Apply complete! Resources: 29 added, 0 changed, 0 destroyed.
```

Outputs:

```
elasticache_endpoint = "h-game-redis.abc123.cache.amazonaws.com"
elasticache_port = 6379
```

Note: ElastiCache creation takes 5-10 minutes. AWS is:

1. Launching cache instance
2. Installing Redis
3. Configuring networking
4. Running health checks

Verify in AWS Console:

1. Go to AWS Console → ElastiCache → Redis clusters
2. You should see: **h-game-redis**
3. **Status:** Available (takes 5-10 minutes)
4. **Endpoint:** Shows connection string

Checkpoint: Did it work?

- ☐ ElastiCache cluster created (took 5-10 minutes)
- ☐ **Status:** Available
- ☐ Endpoint shown in outputs
- ☐ Visible in AWS Console

ElastiCache cluster created!

Step 3: Wait for ElastiCache to be Available

What we're doing: Waiting for ElastiCache to finish initializing.

Check ElastiCache status:

```
# Describe ElastiCache cluster
aws elasticache describe-replication-groups \
  --replication-group-id h-game-redis \
  --query
'ReplicationGroups[0].[Status,PrimaryEndpoint.Address,PrimaryEndpoint
.Port]' \
  --output table

# Should show:
# Status: available
# Address: h-game-redis.abc123.cache.amazonaws.com
# Port: 6379
```

Test connection (from EKS cluster):

```
# Create a test Pod in EKS cluster
kubectl run redis-test --image=redis:7-alpine --rm -i --restart=Never
--namespace=h-game -- redis-cli -h
h-game-redis.cxedij.ng.0001.use1.cache.amazonaws.com -p 6379 PING

# Should return: PONG
# This confirms ElastiCache is accessible from EKS cluster!
```

Checkpoint: Did it work?

- ☐ ElastiCache status: available
- ☐ Can connect from EKS cluster
- ☐ Cache is ready

ElastiCache ready!

Step 4: Update Backend to Use ElastiCache

What we're doing: Updating your existing `backend-deployment.yaml` (already updated in T5 for RDS) to connect to ElastiCache instead of the Redis Deployment.

Important: You're modifying the same file you updated in T5. This file now connects to both RDS (from T5) and ElastiCache (from T6).

 **CRITICAL:** Before making changes, verify ElastiCache is accessible!

BEFORE making changes - Verify ElastiCache is accessible:

```
# 1. Get ElastiCache endpoint from Terraform
cd terraform/
ELASTICACHE_ENDPOINT=$(terraform output -raw elasticache_endpoint)
echo "ElastiCache Endpoint: $ELASTICACHE_ENDPOINT"

# 2. Verify ElastiCache is accessible from EKS cluster
kubectl run redis-test \
  --image=redis:7-alpine \
  --rm -it --restart=Never \
  -- redis-cli -h $ELASTICACHE_ENDPOINT PING

# Should return: PONG (connection successful!)
```

Get ElastiCache connection details:

```
# Get ElastiCache endpoint
ELASTICACHE_ENDPOINT=$(terraform output -raw elasticache_endpoint)
ELASTICACHE_PORT=$(terraform output -raw elasticache_port)

echo "ElastiCache Endpoint: $ELASTICACHE_ENDPOINT"
echo "ElastiCache Port: $ELASTICACHE_PORT"

# Should show:
# ElastiCache Endpoint: h-game-redis.abc123.cache.amazonaws.com
# ElastiCache Port: 6379
```

Update your existing backend-deployment.yaml:

 **Migration Note:** You're updating the same file from T5. Here's what changes:

BEFORE (K3 version - using Kubernetes Service):

```
# backend-deployment.yaml (from K3)
env:
  # ... (DB_HOST, DB_PORT, etc. from T5 - unchanged)

  - name: REDIS_HOST
    value: "redis-service"           # ← Kubernetes Service DNS
  - name: REDIS_PORT
    value: "6379"
```

AFTER (T6 version - using ElastiCache endpoint):

```
# backend-deployment.yaml (updated for ElastiCache)
env:
  # ... (DB_HOST, DB_PORT, etc. from T5 - unchanged)

  - name: REDIS_HOST
    value: "h-game-redis.abc123.cache.amazonaws.com" # ←
ElastiCache endpoint
  - name: REDIS_PORT
    value: "6379"
```

How to update:

```
# First, get the ElastiCache endpoint from Terraform
cd terraform/
ELASTICACHE_ENDPOINT=$(terraform output -raw elasticache_endpoint)
echo "ElastiCache Endpoint: $ELASTICACHE_ENDPOINT"
# Should show: h-game-redis.abc123.cache.amazonaws.com
```

```
# Option 1: Edit existing file directly (RECOMMENDED)
```

```
# Open your k8s/backend-deployment.yaml file and change:
#   value: "redis-service"
# To:
#   value: "$ELASTICACHE_ENDPOINT" # Use the ElastiCache endpoint
# from terraform output
# Or use the actual endpoint value:
#   value: "h-game-redis.abc123.cache.amazonaws.com"
```

```
# Option 2: Use kubectl patch (quick update)
kubectl set env deployment/backend \
  REDIS_HOST=$ELASTICACHE_ENDPOINT \
  -n h-game
```

```
# Option 3: Create updated file (if you prefer)
cat > backend-deployment-elasticache.yaml << 'EOF'
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
  namespace: h-game
spec:
  replicas: 3
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      containers:
        - name: backend
          image: localhost:32000/h-game-backend:v2 # Will update to ECR in T8
          # image: yourusername/h-game-backend:v2 # EC2/Docker Hub

      env:
        # Database connection (from T5 - RDS)
        - name: DB_HOST
          value: "h-game-postgres.abc123xyz.us-east-1.rds.amazonaws.com"
```

```

- name: DB_PORT
  value: "5432"

- name: DB_NAME
  value: "humor_memory_game"

- name: DB_USER
  value: "gameuser"

- name: DB_PASSWORD
  value: "devops123"

# Redis connection - UPDATED FOR ELASTICACHE
- name: REDIS_HOST
  value: "h-game-redis.abc123.cache.amazonaws.com" # ElastiCache
endpoint
  # Get from: terraform output elasticache_endpoint

- name: REDIS_PORT
  value: "6379" # Redis port (same as before)

# ... (rest of configuration from K3/K8)
EOF

```

Apply updated Deployment:

```

# Apply updated Deployment
kubectl apply -f backend-deployment-elasticache.yaml

# Watch Pods restart
kubectl get pods -n h-game -l app=backend -w

# Pods will restart with new ElastiCache connection

```

Verify backend connects to ElastiCache:

```

# Check backend Logs
kubectl logs -l app=backend -n h-game --tail=50

# Should show:
# - "Redis connected" or similar
# - No connection errors

```


Test backend cache:

```
# Port forward to backend  
kubectl port-forward service/backend-service 3001:3001 -n h-game  
  
# In another terminal:  
# Test cache (should be faster on second request)  
curl http://localhost:3001/api/leaderboard  
  
# First request: Hits database, caches result  
# Second request: Served from cache (faster!)
```

Checkpoint: Did it work?

- ☐ Backend Deployment updated
- ☐ Pods restarted
- ☐ Backend connects to ElastiCache
- ☐ Cache working (faster responses)

Backend connected to ElastiCache!

Step 5: Delete Redis Deployment

What we're doing: Deleting the old Redis Deployment since we've migrated to ElastiCache.

 **Important:** Only do this after verifying ElastiCache works!

Verify ElastiCache is working:

```
# Test cache connection
kubectl run redis-test \
  --image=redis:7-alpine \
  --rm -it --restart=Never \
  -- redis-cli -h $(terraform output -raw elasticache_endpoint) PING

# Should return: PONG
```

Delete Redis Deployment:

```
# Delete Redis Deployment
kubectl delete deployment redis -n h-game

# Verify Deployment is gone
kubectl get deployment -n h-game

# Should show: No redis deployment

# Delete Redis Service (if exists)
kubectl delete service redis-service -n h-game

# Verify Service is gone
kubectl get service -n h-game

# Should show: No redis-service
```

Verify Redis/ElastiCache is Working

Step 1: Get ElastiCache Endpoint

```
cd h-game-terraform
terraform output -raw elasticache_endpoint
```

Expected Output:

```
h-game-redis.cxedij.ng.0001.use1.cache.amazonaws.com
```

Step 2: Test Connection from EKS Cluster

```
kubectl run redis-test \
  --image=redis:7-alpine \
  --rm -i --restart=Never \
  --namespace=h-game \
  -- redis-cli -h
h-game-redis.cxedij.ng.0001.use1.cache.amazonaws.com -p 6379 PING
```

Expected Output:

```
PONG
pod "redis-test" deleted from h-game namespace
```

If you see PONG, connection works!

Step 3: Test Redis Operations (SET/GET)

```
kubectl run redis-ops-test \
  --image=redis:7-alpine \
  --rm -i --restart=Never \
  --namespace=h-game \
  -- sh -c "redis-cli -h
h-game-redis.cxedij.ng.0001.use1.cache.amazonaws.com -p 6379 SET
```

```
test_key 'Hello Redis' && redis-cli -h  
h-game-redis.cxedij.ng.0001.use1.cache.amazonaws.com -p 6379 GET  
test_key && redis-cli -h  
h-game-redis.cxedij.ng.0001.use1.cache.amazonaws.com -p 6379 DEL  
test_key"
```

Expected Output:

```
OK  
Hello Redis  
1  
pod "redis-ops-test" deleted from h-game namespace
```

If you see OK, value, and 1, Redis operations work!

Step 4: Check Backend Health Endpoint

```
kubect1 port-forward -n h-game svc/backend-service 3001:3001 &  
sleep 2  
curl http://localhost:3001/health  
kill %1
```

Expected Output:

```
{  
  "status": "healthy",  
  "services": {  
    "database": "connected",  
    "redis": "connected",    ←  Redis is connected!  
    "api": "running"  
  }  
}
```

If "redis": "connected", backend is using Redis!

Step 5: Check Backend Logs for Redis Connection

```
kubectl logs -n h-game -l app=backend --tail=50 | grep -i redis
```

Expected Output:

```
Connecting to Redis...  
Redis: Connected and ready!  
Redis connected successfully!
```

If you see "Connected and ready!", backend connected successfully!

Step 6: Test Cache Functionality

```
kubectl port-forward -n h-game svc/backend-service 3001:3001 &  
sleep 2  
  
# First request (no cache)  
curl -s http://localhost:3001/api/leaderboard | jq -r  
' .metadata.cached // "not cached" '  
  
# Second request (should be cached)  
curl -s http://localhost:3001/api/leaderboard | jq -r  
' .metadata.cached // "not cached" '  
  
kill %1
```

Expected Output:

```
not cached    ← First request  
true          ← Second request (cached!)
```

If second request shows "true", caching is working!

Quick One-Liner Test

```
kubectl run redis-quick-test --image=redis:7-alpine --rm -i  
--restart=Never --namespace=h-game -- redis-cli -h $(cd  
h-game-terraform && terraform output -raw elasticache_endpoint) -p  
6379 PING
```

Expected Output:

```
PONG  
pod "redis-quick-test" deleted from h-game namespace
```

PONG = Redis is working!

Troubleshooting

If PING fails:

- Check security groups allow EKS nodes to access ElastiCache
- Verify ElastiCache status: `aws elasticache describe-replication-groups --replication-group-id h-game-redis`
- Check backend logs: `kubectl logs -n h-game -l app=backend | grep -i redis`

If backend shows "redis": "disconnected":

- Check backend environment variables: `kubectl exec -n h-game <pod-name> -- env | grep REDIS`
- Verify REDIS_HOST matches ElastiCache endpoint
- Check backend logs for connection errors

All tests passing? Redis/ElastiCache is fully operational!

Checkpoint: Did it work?

- ☐ Redis Deployment deleted
- ☐ ElastiCache is working
- ☐ Backend connected to ElastiCache
- ☐ Old cache cleaned up

Migration complete!

Phase 3: Validation & Consolidation (10 min)

Quiz: ElastiCache Understanding

Answer these questions to verify your understanding:

Question 1: What is ElastiCache?

Answer:

- **ElastiCache** = AWS managed cache service
- **Supports:** Redis and Memcached
- **AWS handles:** Backups, updates, monitoring, failover
- **You manage:** Cache keys, cache strategy
- **Focus on:** Application performance, not cache operations

Key point: ElastiCache = Managed cache operations.

Question 2: What are the key benefits of ElastiCache?

Answer:

- **Automatic failover:** Multi-AZ option for zero downtime
- **Automatic updates:** AWS handles Redis version updates
- **Optional persistence:** Snapshots for cache backups
- **Better performance:** Dedicated instances, no resource contention
- **Less operational work:** Focus on application, not cache

Key point: ElastiCache = Less operational burden, better performance.

Question 3: How does ElastiCache connect to your application?

Answer:

- **ElastiCache endpoint:** cluster-name.abc123.cache.amazonaws.com
- **Connection:** Backend connects via endpoint (like connecting to any Redis)
- **Security:** Security group controls access (only backend can connect)
- **Network:** ElastiCache in private subnet, backend in private subnet (secure)

Key point: ElastiCache = Standard Redis connection, just different hostname.

Checklist: Did You Complete Everything?

Before moving on, check these boxes:

Understanding Phase

- ☐ I understand what ElastiCache is (managed cache)
- ☐ I understand ElastiCache benefits (failover, updates, performance)
- ☐ I understand migration process
- ☐ I completed all design exercises

Implementation Phase

- ☐ Cache subnet group created
- ☐ ElastiCache cluster created (took 5-10 minutes)
- ☐ Backend updated to use ElastiCache
- ☐ Backend connects to ElastiCache successfully
- ☐ Redis Deployment deleted (migrated!)

Verification Phase

- ☐ Can explain ElastiCache vs Deployment
- ☐ Can explain ElastiCache benefits
- ☐ Can connect to ElastiCache
- ☐ Understand managed cache operations

Git

- ☐ Committed `elasticsearch.tf`
- ☐ Committed updated `backend-deployment-elasticsearch.yaml`
- ☐ Documented migration process

Key Takeaways

The Big Picture

What you learned in this task:

ELASTICACHE MIGRATION COMPLETE	
-	ElastiCache Redis (managed cache)
-	Optional persistence (snapshots)
-	Backend connected to ElastiCache
-	Redis Deployment deleted (migrated!)
Cache is now managed by AWS!	

Critical Concepts

1. ElastiCache:

- AWS managed cache
- AWS handles operations (backups, updates, monitoring)
- You manage cache strategy (keys, TTL)
- Less operational burden

2. Migration:

- Update application connection
- Connect to ElastiCache endpoint
- Delete old Deployment
- Same Redis protocol, different hostname

3. Benefits:

- Automatic failover (Multi-AZ option)
- Automatic updates

- Optional persistence
- Better performance (dedicated instances)

4. Connection:

- ElastiCache endpoint (hostname)
- Standard Redis connection
- Security groups control access
- Private subnets (secure)

What's Next?

Congratulations! You've migrated to managed cache!

You now have:

- ☐ ElastiCache Redis (managed by AWS)
- ☐ Backend connected to ElastiCache
- ☐ Old Redis Deployment removed
- ☐ Complete managed services stack (EKS, RDS, ElastiCache)

Next: TASK T7 - Application Load Balancer

In T7, you'll learn:

- ☐ How to create an ALB (production-grade ingress)
- ☐ How to replace Ingress with ALB
- ☐ How to configure health checks
- ☐ How to expose your application publicly

But most importantly: You now understand managed cache. Your cache operations are automated!

Proceed to TASK T7: Application Load Balancer

TASK T7: Application Load Balancer

Time: 5 hours | **Priority:** High | **Ticket:** DEVOPS-5107 | **Cost:** ~\$18/month (ALB)

Mission Brief

Replace Kubernetes Ingress with AWS Application Load Balancer (ALB). ALB provides production grade traffic routing, SSL/TLS termination, health checks, and auto-scaling. This is the final piece for production-ready external access.

Outcomes:

- Understand ALB vs Kubernetes Ingress
- Create ALB with Terraform
- Configure target groups (backend Pods)
- Configure health checks
- Expose H-Game via ALB
- Delete Kubernetes Ingress (replaced by ALB)
- Understand production ingress patterns

Prerequisites: Complete Task T6 (ElastiCache Migration)

Setup Note: ALB will be created in your VPC public subnets. We'll replace your existing Kubernetes Ingress (from K5) with ALB, then delete the Ingress. See Section 5 Overview for general setup details.

Cost Note: ALB costs ~\$0.0225/hour (\$16/month) + data transfer. For learning, expect ~\$18/month. See Section 5 Overview for cost-saving strategies.

What Makes This Click

ALB = Production-grade ingress. AWS handles SSL/TLS, health checks, routing. You get enterprise-grade load balancing without managing infrastructure.

Before (Kubernetes Ingress):

Traffic enters your app through an ingress controller running inside the Kubernetes cluster. You are responsible for managing SSL certificates, configuring health checks,

and making sure it scales correctly. It works, but it's tightly coupled to the cluster and is best suited for development, testing, or internal services.

After (Application Load Balancer):

Traffic first hits an AWS-managed Application Load Balancer that sits outside the cluster. AWS handles SSL/TLS, health checks, traffic routing, and scaling automatically. The load balancer is built for internet-facing, high-traffic applications and keeps working even if parts of the cluster change.

Bottom line:

Ingress is useful for learning and small setups, but ALB is designed for real production workloads where reliability, security, and scaling matter.

Phase 1: Understanding ALB (30 min)

What is an Application Load Balancer?

An Application Load Balancer (ALB) is an AWS managed **Layer 7 load balancer** that handles web traffic for your application.

If you think of a restaurant, Kubernetes Ingress is like a host working inside the kitchen, it gets the job done but is limited by the space and resources of the kitchen itself. You also have to manage things like certificates and health checks on your own, which is why it's better suited for development or small setups.

An ALB, on the other hand, is like a professional host at the entrance of a busy restaurant. It operates outside the kitchen, scales automatically as more customers arrive, and handles important tasks like SSL/TLS, health checks, and routing for you. It can direct traffic based on URLs or domains and is built to handle real world, high traffic scenarios.

The key idea is simple: **ALB is a production ready, AWS managed load balancer that safely and efficiently routes traffic to your Kubernetes Pods.**

ALB vs Kubernetes Ingress

Comparison:

Aspect	Kubernetes Ingress (K5)	Application Load Balancer (T7)
Location	In cluster (Pod)	Outside cluster (AWS managed)
SSL/TLS	You manage (cert-manager)	AWS handles (ACM)
Health Checks	You configure	AWS handles automatically
Scaling	Limited to cluster	AWS scales automatically
Cost	Included in cluster	~\$18/month
Use Case	Development, internal	Production, public-facing

When to use each:

- **Ingress:** Development, internal tools, cost-sensitive
- **ALB:** Production, public-facing, need enterprise features

Exercise 1: Why ALB Instead of Ingress?

Scenario: You need to choose between Ingress and ALB for production.

Question: What are the key benefits of ALB?

Think about:

- SSL/TLS management
- Health checks
- Scaling
- Operational burden

Your answer: _____

Answer Below 

Answer:

Key Benefits of ALB:

1. AWS Managed:

Ingress:

- └ You manage: Ingress controller Pod
- └ You handle: Updates, monitoring
- └ Operational work: Medium

ALB:

- └ AWS manages: Load balancer infrastructure
- └ AWS handles: Updates, monitoring, scaling
- └ Operational work: Low

2. Automatic SSL/TLS:

Ingress:

- └ You install: cert-manager
- └ You configure: Let's Encrypt certificates
- └ You manage: Certificate renewal
- └ More setup work

ALB:

- └ AWS Certificate Manager (ACM)
- └ Automatic certificate provisioning
- └ Automatic renewal
- └ Less setup work

3. Better Health Checks:

Ingress:

- └ Basic health checks
- └ Limited configuration
- └ Good enough for development

ALB:

- └ Advanced health checks
- └ Configurable thresholds
- └ Automatic target removal (unhealthy)
- └ Production-grade

4. Automatic Scaling:

Ingress:

- | Limited to cluster resources
- | Manual scaling
- | Can become bottleneck

ALB:

- | AWS scales automatically
- | Handles any traffic volume
- | Never a bottleneck

This is why production uses ALB!

Exercise 2: ALB Architecture

Scenario: You're designing ALB architecture.

Question: How should ALB route traffic to your Pods?

Think about:

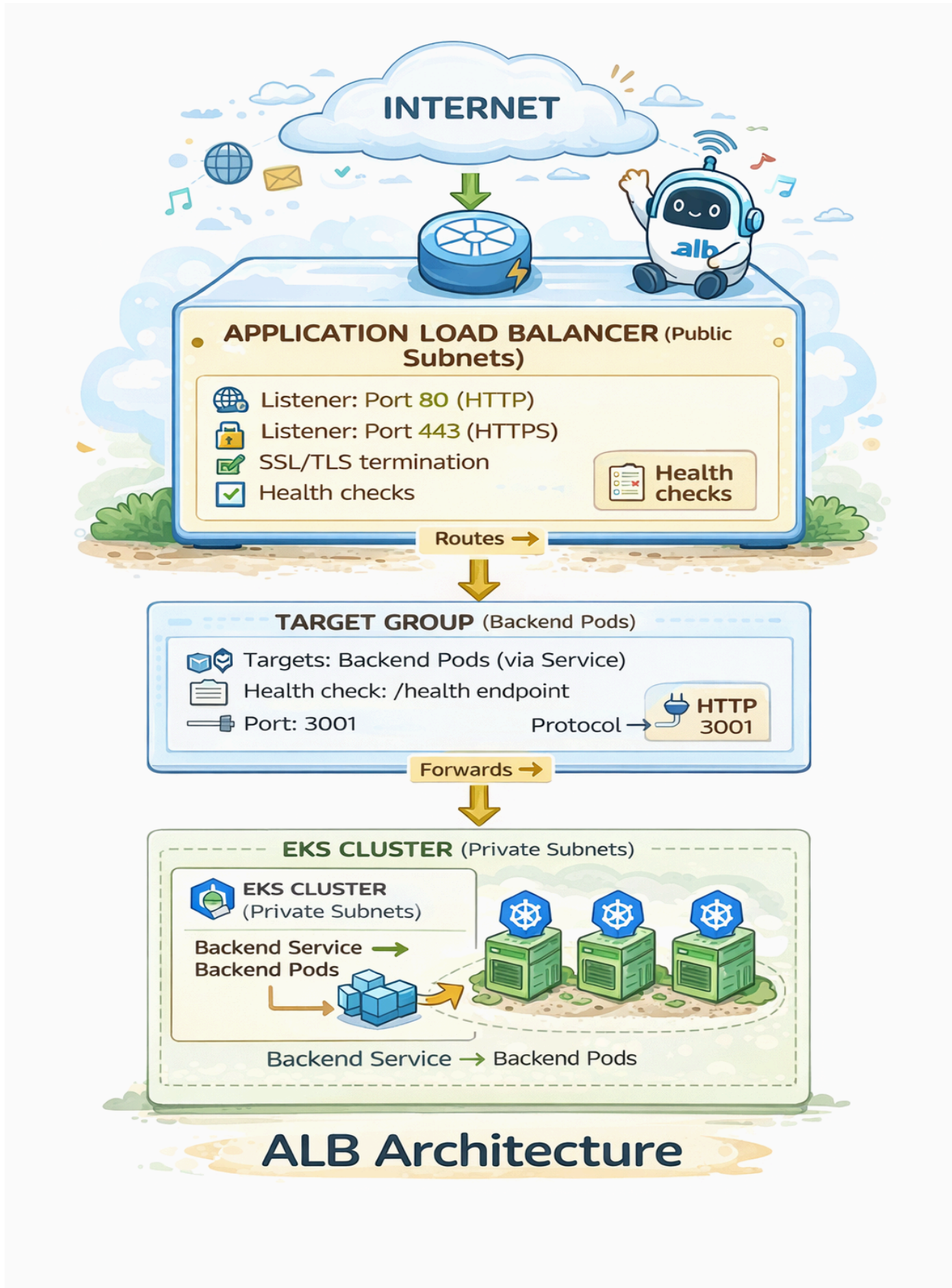
- ALB location (public subnets)
- Target groups (backend Pods)
- Health checks
- Routing rules

Your answer: _____

Answer Below 

Answer:

ALB Architecture:



Key Components:

- **ALB:** In public subnets (internet-facing)
- **Target Group:** Points to backend Service
- **Health Checks:** Monitor Pod health
- **Routing:** ALB → Target Group → Service → Pods

This is the standard ALB setup!

Design Checkpoint

Before moving on, make sure you can answer these in plain English:

Question	Your Answer Should Be Like...
What is ALB?	"AWS managed load balancer - routes traffic to Pods"
Why use ALB?	"Production-grade, AWS handles SSL/health checks"
Where should ALB run?	"Public subnets (internet-facing)"
How does ALB route traffic?	"ALB → Target Group → Service → Pods"

Can't answer one? Go back and re-read that exercise.

All clear? Great! Let's create the ALB!

Phase 2: Implementation (4 hours)

What We're Building

Before this task:

- Kubernetes Ingress (development-grade, from K5)
- Limited to cluster resources
- You manage SSL certificates

After this task:

- Application Load Balancer (production-grade)
- AWS managed SSL/TLS
- Automatic health checks
- Public access via ALB URL

Step-by-Step Plan

We'll do this in 8 steps:

Step 0: Configure Variables

Step 1: Ensure ALB Security Groups exists

Step 2: Configure Security Groups for Pod Access

Step 3: Verify Service Types

Step 4: Create Ingress Resource

Step 5: Wait for ALB Provisioning

Step 6: Verify Target Registration

Step 7: Test Your Application

Note: This task uses the `provider.tf` and `variables.tf` files created in T1. If you haven't completed T1, do that first.

Note: Ensure `.gitignore` is configured (created in T1). The `.gitignore` file prevents committing sensitive files like `terraform.tfvars` and state files.

Exposing Your Application with ALB via Kubernetes Ingress

Difficulty: Beginner to Intermediate

Time: ~45 minutes

Prerequisites: EKS cluster, backend/frontend deployments running

What You'll Build

Internet → ALB (Ingress-managed) → Target Groups → Pod IPs → Your Application

Simple explanation:

- **Ingress** = Kubernetes resource that defines routing rules
- **ALB** = AWS Load Balancer (created automatically by Ingress Controller)
- **Target Groups** = Lists of Pod IPs (created automatically)
- **Path-based routing** = `/api` → Backend, `/` → Frontend

Prerequisites Checklist

Before starting, verify you have:

- EKS cluster created and running
- Backend pods running (`kubectl get pods -n h-game -l app=backend`)
- Frontend pods running (`kubectl get pods -n h-game -l app=frontend`)
- Backend and frontend services created
- Database migrated to RDS (tables exist)
- Redis/ElastiCache accessible
- OIDC provider created in IAM
- AWS Load Balancer Controller installed

Quick Prerequisites Setup

If any items above are missing, follow these steps:

1. Verify EKS Cluster

```
cd "/Users/mac/Desktop/devops playbook/h-game-terraform"
terraform output eks_cluster_name
# Should show: h-game-prod
```

2. Create OIDC Provider (Required for Controller)

Check if it exists:

```
aws eks describe-cluster --name h-game-prod --query
"cluster.identity.oidc.issuer" --output text
# Should return: https://oidc.eks.us-east-1.amazonaws.com/id/...

# Check if provider exists in IAM
aws iam list-open-id-connect-providers | grep -i eks
```

If missing, add to **h-game-terraform/iam.tf**:

```
# =====
# EKS OIDC PROVIDER
# =====
# Required for IAM Roles for Service Accounts (IRSA)

data "tls_certificate" "eks" {
  url = aws_eks_cluster.main.identity[0].oidc[0].issuer
}

resource "aws_iam_openid_connect_provider" "eks" {
  client_id_list  = ["sts.amazonaws.com"]
  thumbprint_list =
[data.tls_certificate.eks.certificates[0].sha1_fingerprint]
  url             = aws_eks_cluster.main.identity[0].oidc[0].issuer
}
```



```

tags = merge(
  local.common_tags,
  {
    Name = "${local.name_prefix}-eks-oidc-provider"
    Task = "T7"
  }
)
}

```

Add TLS provider to [h-game-terraform/provider.tf](#):

```

required_providers {
  aws = {
    source  = "hashicorp/aws"
    version = "~> 6.0"
  }
  tls = {
    source  = "hashicorp/tls"
    version = "~> 4.0"
  }
}

```

Apply:

```

cd "/Users/mac/Desktop/devops playbook/h-game-terraform"
terraform init
terraform apply -target=aws_iam_openid_connect_provider.eks
                 -target=data.tls_certificate.eks

```

3. Install AWS Load Balancer Controller

Check if installed:

```

kubectl get deployment -n kube-system aws-load-balancer-controller

```

If not installed:

```
# Add Helm repo
helm repo add eks https://aws.github.io/eks-charts
helm repo update

# Get cluster name
cd "/Users/mac/Desktop/devops playbook/h-game-terraform"
CLUSTER_NAME=$(terraform output -raw eks_cluster_name)

# Install controller
helm install aws-load-balancer-controller
eks/aws-load-balancer-controller \
  -n kube-system \
  --set clusterName=$CLUSTER_NAME \
  --set serviceAccount.create=false \
  --set serviceAccount.name=aws-load-balancer-controller

# Wait for pods
sleep 30
kubectl get pods -n kube-system -l
app.kubernetes.io/name=aws-load-balancer-controller
```

Verify IAM role is attached:

```
kubectl get sa aws-load-balancer-controller -n kube-system \
  -o jsonpath='{.metadata.annotations.eks\.amazonaws\.com/role-arn}'
# Should show: arn:aws:iam::...:role/...
```

4. Run Database Migration (If Not Done)

Check if tables exist:

```
# Test via backend pod
kubectl exec -n h-game $(kubectl get pods -n h-game -l app=backend -o
jsonpath='{.items[0].metadata.name}') -- \
  node -e "const {Pool}=require('pg'); const p=new
Pool({host:process.env.DB_HOST,user:process.env.DB_USER,password:proc
ess.env.DB_PASSWORD,database:process.env.DB_NAME}); p.query('SELECT 1
FROM users LIMIT 1').then(()=>console.log('✅ Tables
```

```
exist')).catch(e=>console.log('❌ Tables missing:',e.message));"
```

If tables are missing, run migration:

```
# Apply migration ConfigMap and Job
kubectl apply -f k8s/postgres-migration-configmap.yaml
kubectl apply -f k8s/postgres-migration-job.yaml

# Watch migration
kubectl logs -f job/postgres-migration -n h-game

# Verify success
kubectl get job postgres-migration -n h-game
# Should show: COMPLETIONS 1/1
```

5. Verify Services Are Running

```
# Check pods
kubectl get pods -n h-game

# Check services
kubectl get svc -n h-game

# Test backend health
kubectl port-forward -n h-game svc/backend-service 3001:3001 &
curl http://localhost:3001/health
pkill -f "kubectl port-forward"

# Test frontend health
kubectl port-forward -n h-game svc/frontend-service 8080:80 &
curl http://localhost:8080/health
pkill -f "kubectl port-forward"
```

Step 0: Configure Variables

Why: Variables separate configuration from code, making infrastructure reusable.

Add ALB variables to `h-game-terraform/variables.tf`:

```
# =====
# ALB VARIABLES
# =====

variable "alb_name" {
  description = "Name of the Application Load Balancer"
  type        = string
  default     = "h-game-alb"
}

variable "backend_port" {
  description = "Backend application port"
  type        = number
  default     = 3001
}

variable "alb_health_check_path" {
  description = "Health check path for ALB target group"
  type        = string
  default     = "/health"

  validation {
    condition     = can(regex("^/.*$", var.alb_health_check_path))
    error_message = "Health check path must start with / (e.g., /health)."
  }
}

variable "alb_health_check_timeout" {
  description = "Health check timeout in seconds"
  type        = number
  default     = 5
}
```

```

variable "alb_health_check_interval" {
  description = "Health check interval in seconds"
  type        = number
  default     = 30
}

variable "alb_health_check_healthy_threshold" {
  description = "Number of consecutive successful health checks
required to mark target as healthy"
  type        = number
  default     = 2
}

variable "alb_health_check_unhealthy_threshold" {
  description = "Number of consecutive failed health checks required
to mark target as unhealthy"
  type        = number
  default     = 2
}

variable "alb_deregistration_delay" {
  description = "Time in seconds to wait before removing unhealthy
target"
  type        = number
  default     = 30
}

variable "alb_enable_deletion_protection" {
  description = "Enable ALB deletion protection"
  type        = bool
  default     = false # Set to true for production
}

```

Add to `h-game-terraform/terraform.tfvars`:

```

# ALB Configuration
alb_name           = "h-game-alb"

```

```
backend_port          = 3001
alb_health_check_path = "/health"
alb_health_check_timeout = 5
alb_health_check_interval = 30
alb_health_check_healthy_threshold = 2
alb_health_check_unhealthy_threshold = 2
alb_deregistration_delay = 30
alb_enable_deletion_protection = false
```

Apply variables:

```
cd "/Users/mac/Desktop/devops playbook/h-game-terraform"
terraform fmt
terraform validate
```

Checkpoint: Variables configured!

Step 1: Ensure ALB Security Group Exists

Why: The Ingress controller needs a security group to attach to the ALB it creates.

Check if it exists in `h-game-terraform/security-groups.tf`:

```
resource "aws_security_group" "alb" {
  name          = "${local.name_prefix}-alb-sg"
  description   = "Security group for Application Load Balancer"
  vpc_id        = aws_vpc.main.id

  ingress {
    description = "HTTP from internet"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    description = "HTTPS from internet"
```

```

    from_port    = 443
    to_port      = 443
    protocol     = "tcp"
    cidr_blocks  = ["0.0.0.0/0"]
  }

  egress {
    from_port    = 0
    to_port      = 0
    protocol     = "-1"
    cidr_blocks  = ["0.0.0.0/0"]
  }
}

```

If missing, add it. Then apply:

```

cd "/Users/mac/Desktop/devops playbook/h-game-terraform"
terraform apply -target=aws_security_group.alb

```

Get the security group ID:

```

cd "/Users/mac/Desktop/devops playbook/h-game-terraform"
terraform output -raw alb_security_group_id
# Save this value - you'll need it in Step 2

```

Checkpoint: ALB security group ready!

Step 2: Configure Security Groups for Pod Access

Why: With IP target type, ALB routes directly to Pod IPs. We need to allow ALB → Pods communication.

Add to `h-game-terraform/security-groups.tf`:

```

# =====
# EKS-Managed Security Group Rules
# =====

```

```

# EKS automatically creates a security group for the cluster
# These rules allow ALB to reach pods on the EKS-managed security
group

data "aws_security_group" "eks_cluster_managed" {
  filter {
    name   = "tag:aws:eks:cluster-name"
    values = [aws_eks_cluster.main.name]
  }
  filter {
    name   = "tag:Name"
    values = ["eks-cluster-sg-${aws_eks_cluster.main.name}-*"]
  }
}

# Allow ALB to reach backend pods (port 3001)
resource "aws_security_group_rule" "alb_to_eks_managed_nodes_backend"
{
  description          = "Allow ALB to reach backend pods on
EKS-managed node security group"
  type                 = "ingress"
  from_port            = var.backend_port # 3001
  to_port              = var.backend_port
  protocol              = "tcp"
  source_security_group_id = aws_security_group.alb.id
  security_group_id      =
data.aws_security_group.eks_cluster_managed.id

  lifecycle {
    create_before_destroy = true
  }
}

# Allow ALB to reach frontend pods (port 80)
resource "aws_security_group_rule"
"alb_to_eks_managed_nodes_frontend" {
  description          = "Allow ALB to reach frontend pods on
EKS-managed node security group"
  type                 = "ingress"

```



```

    from_port      = 80
    to_port        = 80
    protocol       = "tcp"
    source_security_group_id = aws_security_group.alb.id
    security_group_id      =
data.aws_security_group.eks_cluster_managed.id

    lifecycle {
      create_before_destroy = true
    }
  }
}

```

Apply:

```

cd "/Users/mac/Desktop/devops_playbook/h-game-terraform"
terraform apply -target=data.aws_security_group.eks_cluster_managed \
  -target=aws_security_group_rule.alb_to_eks_managed_nodes_backend \
  -target=aws_security_group_rule.alb_to_eks_managed_nodes_frontend

```

Checkpoint: Security groups configured!

Step 3: Verify Service Types

Why: Ingress works with ClusterIP services (default). Verify your services are correct.

Check backend service (k8s/backend-service.yaml):

```

apiVersion: v1
kind: Service
metadata:
  name: backend-service
  namespace: h-game
spec:
  type: ClusterIP # Correct for Ingress
  selector:
    app: backend
  ports:
    - port: 3001

```

```
targetPort: 3001
```

Check frontend service (k8s/frontend-service.yaml):

```
apiVersion: v1
kind: Service
metadata:
  name: frontend-service
  namespace: h-game
spec:
  type: ClusterIP # Correct for Ingress
  selector:
    app: frontend
  ports:
    - port: 80
      targetPort: 80 # Frontend runs on port 80
```

Apply if needed:

```
kubectl apply -f k8s/backend-service.yaml
kubectl apply -f k8s/frontend-service.yaml
```

Checkpoint: Services configured correctly!

Step 4: Create Ingress Resource

Why: Ingress is the Kubernetes-native way to expose services. It automatically creates and manages the ALB.

Get Terraform outputs:

```
cd "/Users/mac/Desktop/devops playbook/h-game-terraform"

# Get ALB security group ID
ALB_SG=$(terraform output -raw alb_security_group_id)
echo "ALB Security Group: $ALB_SG"
```

```
# Get public subnet IDs
PUBLIC_SUBNETS=$(terraform output -json public_subnet_ids | jq -r
'join(",")')
echo "Public Subnets: $PUBLIC_SUBNETS"
```

Create **k8s/ingress.yaml**:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: h-game
  namespace: h-game
  annotations:
    # ALB Configuration
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/target-type: ip
    alb.ingress.kubernetes.io/security-groups: REPLACE_WITH_ALB_SG
    alb.ingress.kubernetes.io/subnets: REPLACE_WITH_PUBLIC_SUBNETS

    # Health Check Configuration
    alb.ingress.kubernetes.io/healthcheck-interval-seconds: '15'
    alb.ingress.kubernetes.io/healthcheck-timeout-seconds: '10'
    alb.ingress.kubernetes.io/healthcheck-path: /health
    alb.ingress.kubernetes.io/healthy-threshold-count: '2'
    alb.ingress.kubernetes.io/unhealthy-threshold-count: '3'

    # ALB Name
    alb.ingress.kubernetes.io/load-balancer-name: h-game-alb-ingress

spec:
  ingressClassName: alb
  rules:
    - http:
        paths:
          # Backend API routes (specific paths first - order matters!)
          - path: /api
            pathType: Prefix
```

```

    backend:
      service:
        name: backend-service
        port:
          number: 3001

# Health check endpoint
- path: /health
  pathType: Exact
  backend:
    service:
      name: backend-service
      port:
        number: 3001

# Frontend routes (catch-all, comes last)
- path: /
  pathType: Prefix
  backend:
    service:
      name: frontend-service
      port:
        number: 80

```

Replace placeholders:

```

cd "/Users/mac/Desktop/devops playbook/h-game-terraform"
ALB_SG=$(terraform output -raw alb_security_group_id)
PUBLIC_SUBNETS=$(terraform output -json public_subnet_ids | jq -r
'join(",")')

cd ../k8s

# Replace placeholders (macOS syntax)
sed -i '' "s|REPLACE_WITH_ALB_SG|$ALB_SG|g" ingress.yaml
sed -i '' "s|REPLACE_WITH_PUBLIC_SUBNETS|$PUBLIC_SUBNETS|g"
ingress.yaml

```

```
# Verify replacements
grep -E "security-groups|subnets" ingress.yaml
```

Apply Ingress:

```
kubectl apply -f k8s/ingress.yaml
```

Watch Ingress creation:

```
# Watch for ADDRESS to appear (takes 2-3 minutes)
kubectl get ingress h-game -n h-game -w

# Press Ctrl+C when you see the ADDRESS column populated
```

Expected output:

NAME	CLASS	HOSTS	ADDRESS
h-game	alb	*	
h-game-alb-ingress-XXXXX			us-east-1.elb.amazonaws.com
			80
			2m

Checkpoint: Ingress created! ALB is being provisioned.

Step 5: Wait for ALB Provisioning

Why: AWS needs 2-3 minutes to create the ALB infrastructure.

Monitor progress:

```
# Check Ingress status
kubectl get ingress h-game -n h-game

# Check controller logs
kubectl logs -n kube-system -l
app.kubernetes.io/name=aws-load-balancer-controller --tail=20

# Wait for ADDRESS
```

```

echo "Waiting for ALB to be created..."
while [ -z "$(kubectl get ingress h-game -n h-game -o
jsonpath='{.status.loadBalancer.ingress[0].hostname}')" ]; do
    echo "Still waiting... (checking every 10 seconds)"
    sleep 10
done

ALB_URL=$(kubectl get ingress h-game -n h-game -o
jsonpath='{.status.loadBalancer.ingress[0].hostname}')
echo "✅ ALB created: http://$ALB_URL"

```

Expected wait time: 2-3 minutes

Checkpoint: ALB is provisioning!

Step 6: Verify Target Registration

Why: Ensure Pod IPs are registered and healthy in the target groups.

Wait for targets to register:

```

# Wait 60 seconds for targets to register
echo "Waiting 60 seconds for targets to register..."
sleep 60

# Get target group ARNs created by Ingress
aws elbv2 describe-target-groups \
    --query 'TargetGroups[?contains(TargetGroupName,
`k8s-hgame`)].{Name:TargetGroupName,ARN:TargetGroupArn}' \
    --output table

```

Check target health:

```

# Get backend target group ARN
BACKEND_TG=$(aws elbv2 describe-target-groups \
    --query 'TargetGroups[?contains(TargetGroupName,
`backend`)].TargetGroupArn' \
    --output text | head -1)

```

```
# Check backend targets
aws elbv2 describe-target-health --target-group-arn "$BACKEND_TG" \
  --output json | jq '.TargetHealthDescriptions[] | {Target:
.Target.Id, State: .TargetHealth.State}'

# Get frontend target group ARN
FRONTEND_TG=$(aws elbv2 describe-target-groups \
  --query 'TargetGroups[?contains(TargetGroupName,
`frontend`)].TargetGroupArn' \
  --output text | head -1)

# Check frontend targets
aws elbv2 describe-target-health --target-group-arn "$FRONTEND_TG" \
  --output json | jq '.TargetHealthDescriptions[] | {Target:
.Target.Id, State: .TargetHealth.State}'
```

Expected output:

```
{
  "Target": "10.0.11.129",
  "State": "healthy"
}
```

If targets show "initial" or "unhealthy":

- Wait another 60 seconds (health checks need time)
- Check security group rules (Step 2)
- Verify pods are running: `kubectl get pods -n h-game`

Checkpoint: Targets registered and healthy!

Step 7: Test Your Application

Why: Verify everything works end-to-end.

Get ALB URL:

```
ALB_URL=$(kubectl get ingress h-game -n h-game -o
```

```
jsonpath='{.status.loadBalancer.ingress[0].hostname}'  
echo "Your ALB URL: http://$ALB_URL"
```

Test endpoints:

```
# 1. Health check  
echo "=== Testing Health Endpoint ==="  
curl -s http://$ALB_URL/health | jq '.'  
  
# Expected:  
{"status":"healthy","services":{"database":"connected",...}}  
  
# 2. API welcome  
echo ""  
echo "=== Testing API Endpoint ==="  
curl -s http://$ALB_URL/api | jq '.message'  
  
# Expected: "Welcome to the Humor Memory Game API! 🎮😄"  
  
# 3. Leaderboard  
echo ""  
echo "=== Testing Leaderboard ==="  
curl -s http://$ALB_URL/api/leaderboard | jq '.success, (.leaderboard  
| length)'  
  
# Expected: true, 8  
  
# 4. Game start  
echo ""  
echo "=== Testing Game Start ==="  
curl -s -X POST http://$ALB_URL/api/game/start \  
  -H "Content-Type: application/json" \  
  -d '{"username":"testuser"}' | jq '.success, .message'  
  
# Expected: true, "🎯 Game started! Good luck, testuser! 🍀"  
  
# 5. Frontend (in browser)  
echo ""
```



```
echo "=== Frontend URL ==="  
echo "Open in browser: http://$ALB_URL"
```

Test in browser:

1. Open `http://$ALB_URL` in your browser
2. You should see the game UI
3. Try starting a new game
4. Verify it works without errors

Checkpoint: Application is accessible and working!

Step 8: Update Frontend API URL

Why: Frontend needs to know the ALB URL to make API calls.

Get ALB URL:

```
ALB_URL=$(kubectl get ingress h-game -n h-game -o  
jsonpath='{.status.loadBalancer.ingress[0].hostname}')
```

```
echo "ALB URL: http://$ALB_URL"
```

Update frontend deployment (`k8s/frontend-deployment.yaml`):

```
env:  
- name: API_BASE_URL  
  value: "http://$ALB_URL/api" # Replace $ALB_URL with actual value  
- name: NODE_ENV  
  value: "production"
```

Or use sed to replace:

```
cd "/Users/mac/Desktop/devops playbook/k8s"  
ALB_URL=$(kubectl get ingress h-game -n h-game -o  
jsonpath='{.status.loadBalancer.ingress[0].hostname}')
```

```
# Replace in frontend deployment (macOS syntax)  
sed -i '' "s|value: \".*api\"|value: \"http://$ALB_URL/api\"|g"
```

```
frontend-deployment.yaml

# Apply updated deployment
kubectl apply -f frontend-deployment.yaml

# Wait for rollout
kubectl rollout status deployment/frontend -n h-game
```

Checkpoint: Frontend configured with correct API URL!

Success! What You've Accomplished

Created Kubernetes Ingress resource
ALB automatically created and managed by Kubernetes
Target groups automatically created
Path-based routing configured (`/api` → Backend, `/` → Frontend)
Pod IPs automatically registered as targets
Security groups configured for ALB → Pods communication
Application accessible via public URL

Your application is now accessible at:

```
http://h-game-alb-ingress-XXXXX.us-east-1.elb.amazonaws.com
```

Troubleshooting

Issue 1: Ingress Has No ADDRESS

Symptom: `kubectl get ingress` shows empty ADDRESS column

Diagnosis:

```
# Check Ingress events
kubectl describe ingress h-game -n h-game | tail -20

# Check controller logs
```

```
kubectl logs -n kube-system -l  
app.kubernetes.io/name=aws-load-balancer-controller --tail=50
```

Common causes:

- Security group not found → Verify **ALB_SG** value
- Subnet not found → Verify **PUBLIC_SUBNETS** value
- IAM permissions missing → Check controller IAM role
- OIDC provider missing → Create OIDC provider (see Prerequisites)

Fix:

```
# Verify security group exists  
cd "/Users/mac/Desktop/devops playbook/h-game-terraform"  
terraform output alb_security_group_id  
  
# Verify subnets  
terraform output public_subnet_ids  
  
# Check controller IAM role  
kubectl get sa aws-load-balancer-controller -n kube-system \  
-o jsonpath='{.metadata.annotations.eks\.amazonaws\.com/role-arn}'
```

Issue 2: Targets Not Registering

Symptom: Target groups show no targets or targets are unhealthy

Diagnosis:

```
# Check target groups  
aws elbv2 describe-target-groups \  
--query 'TargetGroups[?contains(TargetGroupName,  
`k8s-hgame`)].TargetGroupName' \  
--output text  
  
# Check target health  
TG_ARN="<target-group-arn>"
```

```
aws elbv2 describe-target-health --target-group-arn "$TG_ARN"
```

Fixes:

A. Security group rules missing:

```
# Verify security group rules exist
cd "/Users/mac/Desktop/devops playbook/h-game-terraform"
terraform apply
-target=aws_security_group_rule.alb_to_eks_managed_nodes_backend \
-target=aws_security_group_rule.alb_to_eks_managed_nodes_frontend
```

B. Pods not running:

```
kubectl get pods -n h-game
kubectl describe pod <pod-name> -n h-game
```

C. Health check failing:

```
# Test health endpoint from pod
kubectl exec -n h-game <pod-name> -- curl -s
http://localhost:3001/health
```

Issue 3: Frontend Timeout (504 Gateway Timeout)

Symptom: Frontend shows timeout, backend works

Diagnosis:

Check frontend target health

```
FRONTEND_TG=$(aws elbv2 describe-target-groups \
  --query 'TargetGroups[?contains(TargetGroupName,
`frontend`)].TargetGroupArn' \
  --output text | head -1)

aws elbv2 describe-target-health --target-group-arn "$FRONTEND_TG"
```

Fix:

```
# Verify security group rule for port 80 exists
cd "/Users/mac/Desktop/devops playbook/h-game-terraform"
terraform apply
-target=aws_security_group_rule.alb_to_eks_managed_nodes_frontend

# Verify frontend pods are running
kubectl get pods -n h-game -l app=frontend

# Test frontend health endpoint
kubectl exec -n h-game $(kubectl get pods -n h-game -l app=frontend
-o jsonpath='{.items[0].metadata.name}') -- \
curl -s http://localhost:80/health
```

Issue 4: "relation users does not exist" (500 Error)

Symptom: Game start returns 500 error

Diagnosis:

```
# Check backend logs
kubectl logs -n h-game -l app=backend --tail=50 | grep -i
"relation\|users\|error"
```

Fix:

```
# Run database migration
kubectl apply -f k8s/postgres-migration-configmap.yaml
kubectl apply -f k8s/postgres-migration-job.yaml

# Watch migration
kubectl logs -f job/postgres-migration -n h-game

# Verify tables exist
kubectl exec -n h-game $(kubectl get pods -n h-game -l app=backend -o
jsonpath='{.items[0].metadata.name}') -- \
node -e "const {Pool}=require('pg'); const p=new
```

```
Pool({host:process.env.DB_HOST,user:process.env.DB_USER,password:process.env.DB_PASSWORD,database:process.env.DB_NAME}); p.query('SELECT COUNT(*) FROM users').then(r=>console.log('✅ Users table exists:',r.rows[0].count)).catch(e=>console.log('❌ Error:',e.message));"
```

Issue 5: Controller Not Working

Symptom: Controller pods crashing or not starting

Diagnosis:

```
# Check controller status
kubectl get pods -n kube-system -l
app.kubernetes.io/name=aws-load-balancer-controller

# Check logs
kubectl logs -n kube-system -l
app.kubernetes.io/name=aws-load-balancer-controller --tail=50
```

Common errors:

A. "No OpenIDConnect provider found":

```
# Create OIDC provider (see Prerequisites section)
cd "/Users/mac/Desktop/devops playbook/h-game-terraform"
terraform apply -target=aws_iam_openid_connect_provider.eks

# Restart controller
kubectl rollout restart deployment aws-load-balancer-controller -n
kube-system
```

B. "DescribeListenerAttributes permission denied":

```
# Add missing permission to IAM policy
# In iam.tf, add to aws_iam_policy.aws_load_balancer_controller:
"elasticloadbalancing:DescribeListenerAttributes",
```

```
# Apply
terraform apply -target=aws_iam_policy.aws_load_balancer_controller

# Restart controller
kubectl rollout restart deployment aws-load-balancer-controller -n
kube-system
```

Issue 6: Wrong Routing (404 on /api)

Symptom: `/api` endpoints return 404

Diagnosis:

```
# Check Ingress path order
kubectl get ingress h-game -n h-game -o yaml | grep -A 30 "rules:"
```

Fix: Path order matters! Specific paths (`/api`, `/health`) must come **before** catch-all (`/`).

Correct order:

```
paths:
- path: /api      # ✓ First (specific)
- path: /health  # ✓ Second (specific)
- path: /        # ✓ Last (catch-all)
```

Wrong order:

```
paths:
- path: /        # ✗ Wrong! Catches everything first
- path: /api     # ✗ Never reached
```

Key Concepts Explained

Ingress vs Manual ALB

Ingress (What we used):

- Kubernetes-native
- One YAML file
- Automatic ALB management
- Industry standard
- Easy path-based routing

Manual ALB (Alternative):

- Educational (understand components)
- More control
- More files to manage
- More steps

Path-Based Routing

How it works:

1. Request comes to ALB: `http://alb-url/api/leaderboard`
2. ALB checks paths in order:
 - `/api` matches → Route to backend-service:3001
 - Request forwarded to backend pod
3. Backend responds with leaderboard data

Path matching:

- **Prefix** = Matches path and all sub-paths
 - `/api` matches `/api`, `/api/leaderboard`, `/api/game/start`
- **Exact** = Matches only exact path
 - `/health` matches only `/health` (not `/health/check`)

IP Target Type

What it means:

- ALB routes directly to Pod IPs (e.g., `10.0.11.129`)

- Not node IPs (e.g., **10.0.11.50**)

Why it's better:

- Pod-level health checks
- Works when pods move between nodes
- Better for Kubernetes

Next Steps

1. Add HTTPS/SSL

Add ACM certificate annotation to Ingress:

```
annotations:
  alb.ingress.kubernetes.io/certificate-arn: arn:aws:acm:...
  alb.ingress.kubernetes.io/ssl-policy:
    ELBSecurityPolicy-TLS-1-2-2017-01
  alb.ingress.kubernetes.io/listen-ports: '[{"HTTP": 80}, {"HTTPS":
443}]'
  alb.ingress.kubernetes.io/ssl-redirect: '443'
```

2. Add Custom Domain

Create Route53 record:

```
# Get ALB DNS name
ALB_DNS=$(kubectl get ingress h-game -n h-game -o
jsonpath='{.status.loadBalancer.ingress[0].hostname}')
ALB_ZONE=$(aws elbv2 describe-load-balancers \
  --query "LoadBalancers[?DNSName=='$ALB_DNS'].CanonicalHostedZoneId"
  \
  --output text)

# Create Route53 record (replace with your domain)
aws route53 change-resource-record-sets \
  --hosted-zone-id YOUR_ZONE_ID \
  --change-batch '{
```

```

"Changes": [{
  "Action": "UPSERT",
  "ResourceRecordSet": {
    "Name": "game.yourdomain.com",
    "Type": "A",
    "AliasTarget": {
      "DNSName": "'$ALB_DNS'",
      "HostedZoneId": "'$ALB_ZONE'",
      "EvaluateTargetHealth": true
    }
  }
}]
}'

```

3. Enable Access Logs

Add annotation:

```

annotations:
  alb.ingress.kubernetes.io/load-balancer-attributes: |
    access_logs.s3.enabled=true,
    access_logs.s3.bucket=your-logs-bucket

```

Final Verification Checklist

- ☐ Ingress created and has ADDRESS
- ☐ ALB visible in AWS Console
- ☐ Target groups created (backend and frontend)
- ☐ Targets registered and healthy
- ☐ Health endpoint works: `curl http://$ALB_URL/health`
- ☐ API endpoint works: `curl http://$ALB_URL/api`
- ☐ Frontend loads in browser: `http://$ALB_URL`
- ☐ Game start works: No 500 errors
- ☐ Leaderboard loads: `curl http://$ALB_URL/api/leaderboard`

What You Learned

1. **Kubernetes Ingress** - Industry-standard way to expose services
2. **AWS Load Balancer Controller** - Automates ALB management
3. **Path-based routing** - Route different paths to different services
4. **IP target type** - Direct pod-level routing
5. **Security groups** - Network isolation for ALB → Pods
6. **Troubleshooting** - How to diagnose and fix common issues

Congratulations! You've successfully exposed your application using production-grade Kubernetes Ingress with automatic ALB management!

Route53 & ACM Setup Guide (Optional - Requires Domain Name)

Difficulty: Intermediate

Time: ~30 minutes

Prerequisites: Domain name registered, Ingress ALB already created

What You'll Build

```
Custom Domain (game.yourdomain.com) → Route53 → ALB → Your  
Application  
HTTPS (SSL/TLS) → Secure encrypted connections
```

Simple explanation:

- **Route53** = AWS DNS service (maps domain to ALB)
- **ACM** = AWS Certificate Manager (free SSL certificates)
- **HTTPS** = Secure HTTP (encrypted traffic)

Important Notes

1. **This is OPTIONAL** - You can skip this if you don't have a domain
2. **Domain Required** - You need a registered domain (e.g., yourdomain.com)
3. **Ingress First** - Complete Ingress setup first (ALB must exist)
4. **DNS Propagation** - Changes take 24-48 hours to propagate globally

Step 1: Add Variables to Terraform

Add to **h-game-terraform/variables.tf**:

```
# =====  
# ROUTE53 & ACM VARIABLES (OPTIONAL)  
# =====  
  
variable "domain_name" {  
    description = "Domain name for the application (e.g.,  
yourdomain.com). Leave empty to skip Route53 setup."  
    type        = string
```

```

    default      = "" # Empty = skip Route53 setup
}

variable "app_subdomain" {
    description = "Subdomain for the application (e.g., game for
game.yourdomain.com)"
    type        = string
    default     = "game"
}

variable "certificate_domain" {
    description = "Domain for ACM certificate (can be wildcard, e.g.,
*.yourdomain.com). Leave empty to skip ACM setup."
    type        = string
    default     = "" # Empty = skip ACM setup
}

```

Add to `h-game-terraform/terraform.tfvars`:

```

# Route53 & ACM Configuration (OPTIONAL - leave empty to skip)
domain_name      = "" # e.g., "yourdomain.com" or "" to
skip
app_subdomain    = "game" # e.g., "game" for
game.yourdomain.com
certificate_domain = "" # e.g., "*.yourdomain.com" or "" to
skip

```

If you have a domain, fill in the values:

```

domain_name      = "yourdomain.com"
app_subdomain    = "game" # Results in: game.yourdomain.com
certificate_domain = "*.yourdomain.com" # Wildcard covers all
subdomains

```

Step 2: Add ACM Provider (Required for Certificates)

Update `h-game-terraform/provider.tf`:

```
# Provider alias for us-east-1 (required for ACM certificates used by
ALB)
# ACM certificates for ALB must be in us-east-1 (AWS requirement)
provider "aws" {
  alias   = "us_east_1"
  region = "us-east-1"

  default_tags {
    tags = {
      Project      = var.project_name
      Environment  = var.environment
      ManagedBy    = "Terraform"
    }
  }
}
```

Step 3: Verify Ingress ALB Exists

Before setting up Route53, ensure your Ingress ALB is created:

```
# Get ALB DNS name from Ingress
kubectl get ingress h-game -n h-game -o
jsonpath='{.status.loadBalancer.ingress[0].hostname}'

# Should return: h-game-alb-ingress-XXXXX.us-east-1.elb.amazonaws.com
```

If empty, apply Ingress first:

```
kubectl apply -f k8s/ingress.yaml
# Wait 2-3 minutes for ALB to be created
```

Step 4: Apply Route53 Configuration

The Route53 configuration is already in **route53.tf** (created automatically).

If you have a domain, apply it:

```
cd "/Users/mac/Desktop/devops playbook/h-game-terraform"

# Preview changes
terraform plan -target=aws_route53_zone.main
               -target=aws_route53_record.alb

# Apply Route53
terraform apply -target=aws_route53_zone.main
               -target=aws_route53_record.alb
```

Expected output:

```
aws_route53_zone.main[0]: Creating...
aws_route53_zone.main[0]: Creation complete after 5s
aws_route53_record.alb[0]: Creating...
aws_route53_record.alb[0]: Creation complete after 2s
```

Get nameservers:

```
terraform output route53_nameservers
# Example: ["ns-123.awsdns-12.com", "ns-456.awsdns-45.net", ...]
```

Step 5: Update Domain Registrar

Update your domain registrar with Route53 nameservers:

1. **Go to your domain registrar** (GoDaddy, Namecheap, Google Domains, etc.)
2. **Find DNS/Nameserver settings**
3. **Replace nameservers** with Route53 nameservers from Step 4
4. **Save changes**

Example:

- Old nameservers: ns1.godaddy.com, ns2.godaddy.com
- New nameservers: ns-123.awsdns-12.com, ns-456.awsdns-45.net, ns-789.awsdns-78.org, ns-012.awsdns-01.co.uk

Wait 24-48 hours for DNS propagation.

Step 6: Apply ACM Certificate Configuration

The ACM configuration is already in **acm.tf** (created automatically).

If you have a domain and want HTTPS, apply it:

```
cd "/Users/mac/Desktop/devops playbook/h-game-terraform"

# Preview changes
terraform plan -target=aws_acm_certificate.main
               -target=aws_acm_certificate_validation.main

# Apply ACM (this may take 5-10 minutes for validation)
terraform apply -target=aws_acm_certificate.main
               -target=aws_route53_record.cert_validation
               -target=aws_acm_certificate_validation.main
```

Expected output:

```
aws_acm_certificate.main[0]: Creating...
aws_acm_certificate.main[0]: Creation complete after 2s
aws_route53_record.cert_validation["*.yourdomain.com"]: Creating...
aws_route53_record.cert_validation["*.yourdomain.com"]: Creation
complete after 2s
```



```
aws_acm_certificate_validation.main[0]: Creating...
aws_acm_certificate_validation.main[0]: Still creating... [30s
elapsed]
aws_acm_certificate_validation.main[0]: Still creating... [1m0s
elapsed]
aws_acm_certificate_validation.main[0]: Creation complete after 5m30s
```

Get certificate ARN:

```
terraform output certificate_arn
# Example:
arn:aws:acm:us-east-1:334091769766:certificate/12345678-1234-1234-123
4-123456789012
```

Step 7: Update Ingress for HTTPS

Get certificate ARN:

```
cd "/Users/mac/Desktop/devops playbook/h-game-terraform"
CERT_ARN=$(terraform output -raw certificate_arn)
echo "Certificate ARN: $CERT_ARN"
```

Update `k8s/ingress.yaml` with HTTPS annotations:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: h-game
  namespace: h-game
  annotations:
    # ... existing annotations ...

    # HTTPS Configuration (ADD THESE)
    alb.ingress.kubernetes.io/certificate-arn:
REPLACE_WITH_CERTIFICATE_ARN
    alb.ingress.kubernetes.io/ssl-policy:
ELBSecurityPolicy-TLS-1-2-2017-01
    alb.ingress.kubernetes.io/listen-ports: '[{"HTTP": 80}, {"HTTPS":
```

```
443}]]'
    alb.ingress.kubernetes.io/ssl-redirect: '443' # Redirect HTTP to
HTTPS
```

Replace certificate ARN:

```
cd "/Users/mac/Desktop/devops playbook/k8s"
CERT_ARN=$(cd ../h-game-terraform && terraform output -raw
certificate_arn)

# Replace placeholder (macOS syntax)
sed -i '' "s|REPLACE_WITH_CERTIFICATE_ARN|$CERT_ARN|g" ingress.yaml

# Verify replacement
grep certificate-arn ingress.yaml
```

Apply updated Ingress:

```
kubectl apply -f k8s/ingress.yaml

# Wait 2-3 minutes for ALB to update
kubectl get ingress h-game -n h-game -w
```

Step 8: Test Custom Domain & HTTPS

After DNS propagation (24-48 hours):

```
# Get custom domain URL
cd "/Users/mac/Desktop/devops playbook/h-game-terraform"
DOMAIN_URL=$(terraform output -raw custom_domain_url)
echo "Custom Domain: $DOMAIN_URL"

# Test HTTP (should redirect to HTTPS)
curl -I http://game.yourdomain.com/health
# Should show: 301 redirect to HTTPS

# Test HTTPS
```

```
curl https://game.yourdomain.com/health
# Should return: JSON health status (over HTTPS)
```

Test in browser:

1. Open <https://game.yourdomain.com>
2. You should see a padlock icon, indicating HTTPS
3. Verify the game works

Verification Checklist

- Route53 hosted zone created
- A record pointing to ALB created
- Nameservers updated at domain registrar
- DNS propagated (24-48 hours wait)
- ACM certificate requested
- Certificate validated (DNS records created)
- Certificate ARN obtained
- Ingress updated with HTTPS annotations
- HTTPS working: `curl https://game.yourdomain.com/health`
- HTTP redirect working: `curl -I http://game.yourdomain.com/health`

Troubleshooting

Issue: Route53 Zone Not Found

Symptom: Error: No matching Route53 zone found

Fix:

```
# Verify domain_name variable is set
cd "/Users/mac/Desktop/devops playbook/h-game-terraform"
terraform output domain_name

# If empty, set it in terraform.tfvars
```

Issue: ALB Not Found

Symptom: Error: No matching ALB found

Fix:

```
# Ensure Ingress ALB exists first
kubectl get ingress h-game -n h-game

# Verify ALB name matches
kubectl get ingress h-game -n h-game -o yaml | grep
load-balancer-name
# Should show: h-game-alb-ingress
```

Issue: Certificate Validation Failing

Symptom: Certificate stuck in "Pending validation"

Fix:

```
# Check validation DNS records
aws route53 list-resource-record-sets \
  --hosted-zone-id $(terraform output -raw route53_zone_id) \
  --query 'ResourceRecordSets[?Type==`CNAME`]'
```

```
# Verify DNS records exist
# Wait 5-10 minutes for validation
```

Issue: DNS Not Resolving

Symptom: `curl game.yourdomain.com` times out

Fix:

- Wait 24-48 hours for DNS propagation
- Verify nameservers updated at registrar
- Check Route53 record: `aws route53 list-resource-record-sets --hosted-zone-id <zone-id>`

Notes for Readers Without Domain

If you don't have a domain:

1. **Skip Route53 setup** - Leave `domain_name = ""` in `terraform.tfvars`
2. **Skip ACM setup** - Leave `certificate_domain = ""` in `terraform.tfvars`
3. **Use ALB DNS name** - Continue using `http://h-game-alb-ingress-XXXXX.us-east-1.elb.amazonaws.com`
4. **HTTP is fine for learning** - Production requires HTTPS, but HTTP works for testing

When you get a domain later:

1. Set `domain_name` and `certificate_domain` in `terraform.tfvars`
2. Run `terraform apply`
3. Update nameservers at registrar
4. Wait for DNS propagation
5. Update Ingress with HTTPS annotations

What You Learned

1. **Route53** - AWS DNS service for custom domains
2. **ACM** - Free SSL certificates for HTTPS
3. **DNS Propagation** - Time for DNS changes to spread globally
4. **HTTPS Configuration** - Secure encrypted connections
5. **Ingress HTTPS** - Adding SSL to Kubernetes Ingress

Congratulations! You've configured custom domain and HTTPS for your application!

Phase 3: Validation & Consolidation (30 min)

Quiz: ALB Understanding

Answer these questions to verify your understanding:

Question 1: What is an Application Load Balancer?

Answer:

- **ALB** = Application Load Balancer
- AWS managed Layer 7 load balancer
- Routes HTTP/HTTPS traffic to targets
- Handles SSL/TLS termination, health checks
- Scales automatically based on traffic

Key point: ALB = Production-grade load balancer managed by AWS.

Question 2: How does ALB route traffic to Pods?

Answer:

1. **Listener** receives traffic (port 80/443)
2. **ALB** routes to **Target Group**
3. **Target Group** forwards to registered targets (nodes/Pods)
4. **Targets** (nodes) forward to **Service** → **Pods**

Flow: Internet → ALB → Target Group → Nodes → Service → Pods

Key point: ALB uses target groups to route traffic to Pods.

Question 3: What are the benefits of ALB over Ingress?

Answer:

- **AWS managed:** Less operational work
- **Automatic scaling:** Handles any traffic volume
- **SSL/TLS:** AWS Certificate Manager integration
- **Advanced health checks:** Production-grade monitoring

- **Better performance:** Dedicated infrastructure

Key point: ALB = Production-grade, AWS managed ingress.

Checklist: Did You Complete Everything?

Before moving on, check these boxes:

Understanding Phase

- ☐ I understand what ALB is (AWS managed load balancer)
- ☐ I understand ALB benefits (production-grade, AWS managed)
- ☐ I understand ALB architecture
- ☐ I completed all design exercises

Implementation Phase

- ☐ Target group created
- ☐ ALB created (took 2-5 minutes)
- ☐ Listener configured
- ☐ Targets registered
- ☐ ALB accessible via DNS name
- ☐ Health checks working

Verification Phase

- ☐ Can explain ALB vs Ingress
- ☐ Can explain ALB architecture
- ☐ Can access application via ALB
- ☐ Understand production ingress patterns

Git

- ☐ Committed `alb.tf`
- ☐ Committed `backend-service-nodeport.yaml`
- ☐ Documented ALB configuration

Key Takeaways

The Big Picture

What you learned in this task:

In this task, you set up a **production-ready entry point** for your application. An Application Load Balancer was created to handle traffic from the internet. It routes requests to your backend using a target group and listener rules. Your app is now accessible through a public ALB URL. This is what real production ingress looks like.

Critical Concepts

1. ALB:

- AWS managed load balancer
- Production-grade ingress
- Automatic scaling
- SSL/TLS termination

2. Target Groups:

- Define which Pods receive traffic
- Health checks monitor Pod health
- Automatic target removal (unhealthy)

3. Listeners:

- Define how ALB handles traffic
- Port 80 (HTTP), Port 443 (HTTPS)
- Routing rules
- SSL/TLS termination (with ACM certificates)

4. Route53 (Optional):

- Maps custom domain to ALB
- Production requirement for custom domains
- Health check integration

5. ACM (Optional):

- Free SSL/TLS certificates
- Required for HTTPS
- Auto-renewal

4. Architecture:

- ALB in public subnets
- Routes to target group
- Target group → Nodes → Service → Pods

What's Next?

Congratulations! You've created production-grade ingress!

You now have:

- Application Load Balancer (AWS managed)
- Public access via ALB URL
- Health checks configured
- Production-ready ingress
- Route53 DNS (optional - custom domain)
- HTTPS with ACM (optional - SSL/TLS)

Next: TASK T8 - ECR Container Registry

In T8, you'll learn:

- How to create ECR repositories
- How to push images to ECR
- How to update EKS to pull from ECR
- How to replace localhost:32000 with ECR

But most importantly: You now understand production ingress. Your application is accessible via enterprise-grade load balancing!

Proceed to TASK T8: ECR Container Registry

TASK T8: ECR Container Registry

Time: 2 hours | **Priority:** High | **Ticket:** DEVOPS-5108 | **Cost:** ~\$1/month (storage)

Mission Brief

Create AWS ECR (Elastic Container Registry) repositories and migrate from `localhost:32000` to ECR. ECR provides private, secure container image storage integrated with AWS. This completes the container registry migration.

Outcomes:

- Understand ECR vs local registries
- Create ECR repositories with Terraform
- Push Docker images to ECR
- Update EKS Deployments to pull from ECR
- Replace `localhost:32000` with ECR
- Understand managed container registry benefits

Prerequisites: Complete Task T7 (Application Load Balancer)

Setup Note: ECR will be created in your AWS account. We'll push your existing images (from K1-K9, currently in `localhost:32000` or Docker Hub) to ECR and update Deployments. See Section 5 Overview for general setup details.

Cost Note: ECR costs ~\$0.10/GB/month for storage + data transfer. For learning, expect ~\$1/month. Images are stored in S3 (managed by AWS). See Section 5 Overview for cost-saving strategies.

The One Thing That Makes This Click

ECR is **AWS's private place to store your Docker images**, built for real deployments.

A local registry only works on your machine, so Kubernetes in the cloud can't reach it and it's not suitable for production.

With **ECR**, your images live inside AWS and EKS can pull them securely without exposing anything to the public.

Access is controlled using AWS permissions instead of hard coded passwords.

In simple terms, **ECR replaces local or public registries with a secure, production-ready image store for AWS.**

The Mental Model Shift:

BEFORE T8:

- "Images = localhost:32000 or Docker Hub"
- "EKS can't access localhost"
- "Docker Hub = public (security concern)"
- "Not integrated with AWS"

AFTER T8:

- "Images = ECR (private AWS registry)"
- "EKS can access ECR (same AWS account)"
- "Private and secure (IAM controlled)"
- "Integrated with AWS ecosystem"

This is the difference between "development registry" and "production registry."

Now let's see how ECR works...

Phase 1: Understanding ECR (15 min)

What is ECR?

ECR = Elastic Container Registry

ECR is **where your Docker images live so AWS can use them.**

A local registry like `localhost:32000` only exists on your laptop, so AWS and EKS cannot see or pull images from it.

ECR is a private container registry inside your AWS account, and EKS can securely pull images from it using AWS permissions.

This makes ECR suitable for production, while a local registry is only useful for learning and development.

In simple terms, **local registries are for your computer, ECR is for AWS.**

The Container Storage Analogy

LOCAL REGISTRY (`localhost:32000`):

Local Docker Registry

- Only on your machine
- EKS can't access it
- Not accessible from AWS
- Good for: Development only

ECR (AWS Managed):

AWS ECR (Private Registry)

- In your AWS account
- EKS can access it (same account)
- Private and secure (IAM controlled)
- Integrated with AWS
- Good for: Production

└

Key insight: ECR = Private container registry in AWS that EKS can access.

ECR vs Local Registry

Comparison:

Aspect	localhost:32000	ECR
Location	Local machine	AWS (cloud)
Access	Local only	AWS services (EKS)
Security	Local network	IAM controlled
Integration	None	Integrated with EKS
Cost	Free	~\$0.10/GB/month
Use Case	Development	Production

When to use each:

- **localhost:32000:** Local development, learning
- **ECR:** Production, EKS deployments, AWS integration

Exercise 1: Why ECR Instead of Local Registry?

Scenario: You need to choose between localhost:32000 and ECR for EKS.

Question: Why can't EKS use localhost:32000?

Think about:

- Network access
- Security
- Production requirements

Your answer: _____

Answer Below 

Answer:

Why EKS Can't Use localhost:32000:

1. Network Access:

localhost:32000:

- └ Only accessible from your local machine
- └ EKS nodes are in AWS (different network)
- └ EKS can't reach your local machine

ECR:

- └ In AWS (same network as EKS)
- └ EKS nodes can access ECR
- └ Integrated with AWS networking

2. Security:

localhost:32000:

- └ No authentication
- └ Anyone on local network can access
- └ Not secure for production

ECR:

- └ IAM authentication
- └ Only authorized services can access
- └ Secure for production

3. Production Requirements:

localhost:32000:

- └ Single point of failure (your machine)
- └ Not scalable
- └ Not production-ready

ECR:

- └ AWS managed (high availability)
- └ Scalable
- └ Production-ready

This is why production uses ECR!

Design Checkpoint

Before moving on, make sure you can answer these in plain English:

Question	Your Answer Should Be Like...
What is ECR?	"AWS private container registry - stores Docker images"
Why use ECR?	"EKS can access it, secure, integrated with AWS"
How does ECR work?	"Push images to ECR, EKS pulls from ECR"
What does ECR cost?	"~\$0.10/GB/month for storage"

Can't answer one? Go back and re-read that exercise.

All clear? Great! Let's create ECR repositories!

Phase 2: Implementation (1.5 hours)

What We're Building

Before this task:

- Images in localhost:32000 or Docker Hub
- EKS can't access localhost:32000
- Using Docker Hub (public, less secure)

After this task:

- ECR repositories created
- Images pushed to ECR
- EKS Deployments updated to pull from ECR
- Private, secure container registry

Step-by-Step Plan

We'll do this in 6 steps:

- Step 0:** Configure Variables
"Set up variables for ECR repositories"
- Step 1:** Create ECR Repositories
"Private registries for each image"
- Step 2:** Authenticate Docker to ECR
"Get permission to push images"
- Step 3:** Push Images to ECR
"Upload frontend, backend images"
- Step 4:** Update EKS Deployments
"Change image references to ECR"
- Step 5:** Verify Images Pull from ECR
"Confirm Pods use ECR images"

Note: This task uses the `provider.tf` and `variables.tf` files created in T1. If you haven't completed T1, do that first.

Note: Ensure `.gitignore` is configured (created in T1). The `.gitignore` file prevents committing sensitive files like `terraform.tfvars` and state files.

Step 0: Configure Variables

Why we're doing this: Variables separate configuration from code, making infrastructure reusable and keeping sensitive data secure.

Add ECR variables (if not already in variables.tf):

```
# Add to variables.tf
# IDE Option: Instead of using cat, add these to your variables.tf file in your
IDE

cat >> variables.tf << 'EOF'

# =====
# ECR VARIABLES
# =====

variable "ecr_backend_repository" {
  description = "ECR repository name for backend images"
  type        = string
  default     = "h-game-backend"
}

variable "ecr_frontend_repository" {
  description = "ECR repository name for frontend images"
  type        = string
  default     = "h-game-frontend"
}

variable "ecr_image_tag_mutability" {
  description = "Image tag mutability: MUTABLE or IMMUTABLE"
  type        = string
  default     = "MUTABLE"

  validation {
    condition     = contains(["MUTABLE", "IMMUTABLE"],
var.ecr_image_tag_mutability)
    error_message = "Image tag mutability must be either MUTABLE or IMMUTABLE."
  }
}

variable "ecr_scan_on_push" {
  description = "Enable image scanning on push (for security)"
```

```
type      = bool
default   = true
}
EOF
```

Add to terraform.tfvars

```
cat >> terraform.tfvars << 'EOF'

# ECR Configuration
ecr_backend_repository = "h-game-backend"
ecr_frontend_repository = "h-game-frontend"
ecr_image_tag_mutability = "Mutable" # Mutable allows overwriting
tags, IMMUTABLE prevents it
ecr_scan_on_push          = true      # Enable security scanning
EOF
```

Variables configured!

Step 1: Create ECR Repositories

What we're doing: Creating ECR repositories for frontend and backend images.

Create ECR repositories configuration:

```
# Create ecr.tf
cat > ecr.tf << 'EOF'
# =====
# ECR (Elastic Container Registry)
# =====
# Private container registry in AWS
# Replaces: localhost:32000 (from K1-K9)

# =====
# ECR REPOSITORY - Backend
# =====
resource "aws_ecr_repository" "backend" {
  name                       = var.ecr_backend_repository # From variable
  image_tag_mutability       = var.ecr_image_tag_mutability # From variable

  # Image scanning (optional - for security)
  image_scanning_configuration {
    scan_on_push = var.ecr_scan_on_push # From variable
  }

  # Encryption (optional - for security)
  encryption_configuration {
    encryption_type = "AES256" # AWS managed encryption
  }

  # Lifecycle policy (optional - clean up old images)
  # Automatically delete images older than 30 days
  lifecycle_policy {
    policy = jsonencode({
      rules = [
        {
          rulePriority = 1
          description  = "Keep last 10 images"
          selection = {
            tagStatus      = "any"
            countType      = "imageCountMoreThan"
            countNumber    = 10
          }
        }
      ]
    })
  }
}
```

```

        }
        action = {
            type = "expire"
        }
    }
}
])
})
}

tags = merge(
    local.common_tags,
    {
        Name = var.ecr_backend_repository
        Task = "T8"
    }
)
}

# =====
# ECR REPOSITORY - Frontend
# =====
resource "aws_ecr_repository" "frontend" {
    name                        = var.ecr_frontend_repository # From variable
    image_tag_mutability       = var.ecr_image_tag_mutability # From variable

    image_scanning_configuration {
        scan_on_push = var.ecr_scan_on_push # From variable
    }

    encryption_configuration {
        encryption_type = "AES256"
    }

    lifecycle_policy {
        policy = jsonencode({
            rules = [
                {
                    rulePriority = 1
                    description  = "Keep last 10 images"
                    selection = {
                        tagStatus      = "any"
                        countType      = "imageCountMoreThan"
                        countNumber    = 10
                    }
                }
            ]
        })
    }
}

```

```

        }
        action = {
            type = "expire"
        }
    }
}
])
})
}

tags = merge(
    local.common_tags,
    {
        Name = var.ecr_frontend_repository
        Task = "T8"
    }
)
}

# Outputs: Repository URLs (for pushing images)
output "ecr_backend_repository_url" {
    description = "URL of the backend ECR repository"
    value       = aws_ecr_repository.backend.repository_url
}

output "ecr_frontend_repository_url" {
    description = "URL of the frontend ECR repository"
    value       = aws_ecr_repository.frontend.repository_url
}

# Outputs: Repository ARNs (for IAM policies if needed)
output "ecr_backend_repository_arn" {
    description = "ARN of the backend ECR repository"
    value       = aws_ecr_repository.backend.arn
}

output "ecr_frontend_repository_arn" {
    description = "ARN of the frontend ECR repository"
    value       = aws_ecr_repository.frontend.arn
}
EOF

```

Format and validate code:

```
# Format Terraform code (fixes indentation, spacing)
terraform fmt
# Automatically formats your code:
# - Fixes indentation (2 spaces)
# - Aligns equals signs
# - Sorts attributes
# - Makes code readable

# Validate Terraform syntax and configuration
terraform validate
# Checks for errors:
# - Syntax errors (missing brackets, quotes)
# - Invalid attribute names
# - Type mismatches
# - Missing required attributes
# Should show: "Success! The configuration is valid."
```

Plan and apply:

```
# Preview changes
terraform plan
# Review the plan carefully:
# - Check what will be created (+)
# - Check what will be changed (~)
# - Check what will be destroyed (-)
# Always review before apply!
```

Should show: 2 to add (2 ECR repositories)

```
# Apply changes
terraform apply

#Type 'yes' when prompted
```


Expected output:

```
aws_ecr_repository.backend: Creating... aws_ecr_repository.frontend:
Creating... aws_ecr_repository.backend: Creation complete after 2s
[id=h-game-backend] aws_ecr_repository.frontend: Creation complete after 2s
[id=h-game-frontend]
Apply complete! Resources: 34 added, 0 changed, 0 destroyed.
```

Outputs:

```
ecr_backend_repository_url =
"123456789012.dkr.ecr.us-east-1.amazonaws.com/h-game-backend"
ecr_frontend_repository_url =
"123456789012.dkr.ecr.us-east-1.amazonaws.com/h-game-frontend"
```

Verify in AWS Console:

1. Go to AWS Console → ECR → Repositories
2. You should see: **h-game-backend** and **h-game-frontend**
3. Both are empty (no images yet)

Checkpoint - Did it work?

- ☐ 2 ECR repositories created
- ☐ Repository URLs shown in outputs
- ☐ Visible in AWS Console

ECR repositories ready!

Step 2: Authenticate Docker to ECR

What we're doing: Getting Docker permission to push images to ECR.

Why: ECR requires AWS authentication.

Get AWS account ID and region:

```
# Get AWS account ID
AWS_ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output
text)

# Get AWS region
AWS_REGION=$(aws configure get region)

echo "Account ID: $AWS_ACCOUNT_ID"
echo "Region: $AWS_REGION"
```

Authenticate Docker to ECR:

```
# Get ECR Login token and authenticate Docker
aws ecr get-login-password --region $AWS_REGION | \
  docker login --username AWS --password-stdin \
  $AWS_ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com

# Expected output:
# Login Succeeded
```

Checkpoint: Did it work?

- ☐ Docker authenticated to ECR
- ☐ Login successful

Docker authenticated!

Step 3: Push Images to ECR

What we're doing: Building and pushing frontend and backend images to ECR.

Get ECR repository URLs:

```
# Get repository URLs
BACKEND_REPO=$(terraform output -raw ecr_backend_repository_url)
FRONTEND_REPO=$(terraform output -raw ecr_frontend_repository_url)

echo "Backend repo: $BACKEND_REPO"
echo "Frontend repo: $FRONTEND_REPO"
```

Build and push backend image:

```
# Navigate to backend directory
cd ~/projects/H-Game/backend

# Build image (tag with ECR URL)
docker build -t $BACKEND_REPO:latest .

# Also tag as v1 (for versioning)
docker tag $BACKEND_REPO:latest $BACKEND_REPO:v1

# Push to ECR
docker push $BACKEND_REPO:latest
docker push $BACKEND_REPO:v1

# Expected output:
# The push refers to repository [...]
# latest: digest: sha256:abc123... size: 1234
```

Build and push frontend image:

```
# Navigate to frontend directory
cd ~/projects/H-Game/frontend

# Build image (tag with ECR URL)
```

```
docker build -t $FRONTEND_REPO:latest .

# Also tag as v1
docker tag $FRONTEND_REPO:latest $FRONTEND_REPO:v1

# Push to ECR
docker push $FRONTEND_REPO:latest
docker push $FRONTEND_REPO:v1

# Expected output:
# The push refers to repository [...]
# latest: digest: sha256:xyz789... size: 5678
```

Verify images in ECR:

```
# List images in backend repository
aws ecr list-images --repository-name h-game-backend

# List images in frontend repository
aws ecr list-images --repository-name h-game-frontend

# Should show: latest and v1 tags
```

Verify in AWS Console:

1. Go to AWS Console → ECR → Repositories → **h-game-backend**
2. You should see: Images with tags **latest** and **v1**
3. Same for **h-game-frontend**

Checkpoint: Did it work?

- ☐ Backend image pushed to ECR
- ☐ Frontend image pushed to ECR
- ☐ Images visible in AWS Console

Images pushed to ECR!

Step 4: Update EKS Deployments

What we're doing: Updating your existing `backend-deployment.yaml` and `frontend-deployment.yaml` (from K2 and K4) to pull images from ECR instead of `localhost:32000`.

Important: You're modifying the existing files from K2 and K4. Change the image references to use ECR.

Get ECR repository URLs:

```
# Get repository URLs
BACKEND_REPO=$(terraform output -raw ecr_backend_repository_url)
FRONTEND_REPO=$(terraform output -raw ecr_frontend_repository_url)

echo "Backend ECR: $BACKEND_REPO"
echo "Frontend ECR: $FRONTEND_REPO"

# Should show:
# Backend ECR:
123456789012.dkr.ecr.us-east-1.amazonaws.com/h-game-backend
# Frontend ECR:
123456789012.dkr.ecr.us-east-1.amazonaws.com/h-game-frontend
```

Update backend Deployment:

Migration Note: You're updating the file from K2 (already modified in T5 and T6). Here's what changes:

BEFORE (K2 version - using local registry):

```
# backend-deployment.yaml (from K2)
spec:
  containers:
    - name: backend
      image: localhost:32000/h-game-backend:v1 # ← Local registry
      (EKS can't access!)
```

AFTER (T8 version - using ECR):

```
# backend-deployment.yaml (updated for ECR)
spec:
  containers:
    - name: backend
      image:
123456789012.dkr.ecr.us-east-1.amazonaws.com/h-game-backend:latest #
← ECR
```

How to update:

```
# Option 1: Edit existing file directly (RECOMMENDED)
# Open your k8s/backend-deployment.yaml file and change:
#   image: localhost:32000/h-game-backend:v1
# To:
#   image: $BACKEND_REPO:latest
```

```
# Option 2: Use kubectl set image (quick update)
kubectl set image deployment/backend \
  backend=$BACKEND_REPO:latest \
  -n h-game

# Verify update
kubectl get deployment backend -n h-game -o yaml | grep image:

# Should show: ECR repository URL
```

Update frontend Deployment:

Migration Note: You're updating the file from K4. Here's what changes:

BEFORE (K4 version - using local registry):

```
# frontend-deployment.yaml (from K4)
spec:
  containers:
    - name: frontend
      image: localhost:32000/h-game-frontend:v1 # ← Local registry
      (EKS can't access!)
```

AFTER (T8 version - using ECR):

```
# frontend-deployment.yaml (updated for ECR)
spec:
  containers:
    - name: frontend
      image:
123456789012.dkr.ecr.us-east-1.amazonaws.com/h-game-frontend:latest
# ← ECR
```

How to update:

```
# Option 1: Edit existing file directly (RECOMMENDED)
# Open your k8s/frontend-deployment.yaml file and change:
#   image: localhost:32000/h-game-frontend:v1
# To:
#   image: $FRONTEND_REPO:latest
```

```
# Option 2: Use kubectl set image (quick update)
kubectl set image deployment/frontend \
  frontend=$FRONTEND_REPO:latest \
  -n h-game

# Verify update
kubectl get deployment frontend -n h-game -o yaml | grep image:

# Should show: ECR repository URL
```

Watch Pods restart:

```
# Watch Pods restart with new images
kubectl get pods -n h-game -w

# Pods will:
# 1. Pull new image from ECR
# 2. Restart with ECR image
# 3. Become Ready

# Wait for all Pods to become Ready
```

Test image pull from ECR:

```
# 1. Verify images are from ECR
kubectl describe pod <backend-pod-name> -n h-game | grep Image:
# Should show:
123456789012.dkr.ecr.us-east-1.amazonaws.com/h-game-backend

kubectl describe pod <frontend-pod-name> -n h-game | grep Image:
# Should show:
123456789012.dkr.ecr.us-east-1.amazonaws.com/h-game-frontend

# 2. Check image pull events
kubectl get events -n h-game --sort-by='.lastTimestamp' | grep Pulled
# Should show: Successfully pulled image from ECR

# 3. Verify Pods are running
kubectl get pods -n h-game
# Should show: All Pods Running and Ready

# 4. Test application (if ALB is configured)
curl http://<alb-dns>/health
# Should return: Application health status
```


Checkpoint: Did it work?

- ☐ Backend Deployment updated
- ☐ Frontend Deployment updated
- ☐ Pods restarting with ECR images
- ☐ Pods become Ready
- ☐ Images successfully pulled from ECR
- ☐ Application working with ECR images

Deployments updated!

Migration Complete: Your Kubernetes files now use:

- RDS for database (from T5)
- ElastiCache for cache (from T6)
- ECR for images (from T8)
- All managed AWS services!

Step 5: Verify Images Pull from ECR

What we're doing: Verifying Pods are using ECR images.

Check Pod images:

```
# Check backend Pod images
kubectl get pods -n h-game -l app=backend -o
jsonpath='{.items[*].spec.containers[*].image}'

# Should show: ECR repository URL (not localhost:32000)

# Check frontend Pod images
kubectl get pods -n h-game -l app=frontend -o
jsonpath='{.items[*].spec.containers[*].image}'

# Should show: ECR repository URL
```

Check Pod events (image pull):

```
# Check backend Pod events
kubectl describe pod -n h-game -l app=backend | grep -A 5 Events

# Should show: "Pulling image" from ECR

Test application:
# Port forward to backend
kubectl port-forward service/backend-service 3001:3001 -n h-game

# In another terminal:
curl http://localhost:3001/api/health

# Should return: Health status (using ECR image!)
```

Verify in AWS Console:

1. Go to AWS Console → ECR → Repositories → **h-game-backend**
2. Check: Image pull count (should increase as Pods pull)

Checkpoint - Did it work?

- ☐ Pods using ECR images
- ☐ Image pulls successful
- ☐ Application working with ECR images

ECR migration complete!

Phase 3: Validation & Consolidation (15 min)

Quiz: ECR Understanding

Answer these questions to verify your understanding:

Question 1: What is ECR?

Answer:

- **ECR** = Elastic Container Registry
- AWS managed private container registry
- Stores Docker images
- Integrated with EKS (same AWS account)
- IAM controlled access

Key point: ECR = Private container registry in AWS.

Question 2: Why use ECR instead of localhost:32000?

Answer:

- **EKS access:** EKS can access ECR (can't access localhost)
- **Security:** IAM controlled, private
- **Integration:** Integrated with AWS services
- **Production:** Production-ready, scalable

Key point: ECR = Accessible by EKS, secure, production-ready.

Question 3: How do you push images to ECR?

Answer:

1. **Authenticate:** `aws ecr get-login-password | docker login`
2. **Build:** `docker build -t $ECR_REPO:tag .`
3. **Push:** `docker push $ECR_REPO:tag`

Key point: Standard Docker commands, just different registry URL.

Checklist: Did You Complete Everything?

Before moving on, check these boxes:

Understanding Phase

- ☐ I understand what ECR is (AWS private container registry)
- ☐ I understand why use ECR (EKS access, security)
- ☐ I understand ECR workflow (push, pull)
- ☐ I completed all design exercises

Implementation Phase

- ☐ ECR repositories created (backend, frontend)
- ☐ Docker authenticated to ECR
- ☐ Images pushed to ECR
- ☐ Deployments updated to use ECR
- ☐ Pods using ECR images

Verification Phase

- ☐ Can explain ECR vs local registry
- ☐ Can push images to ECR
- ☐ Can update Deployments to use ECR
- ☐ Understand managed container registry

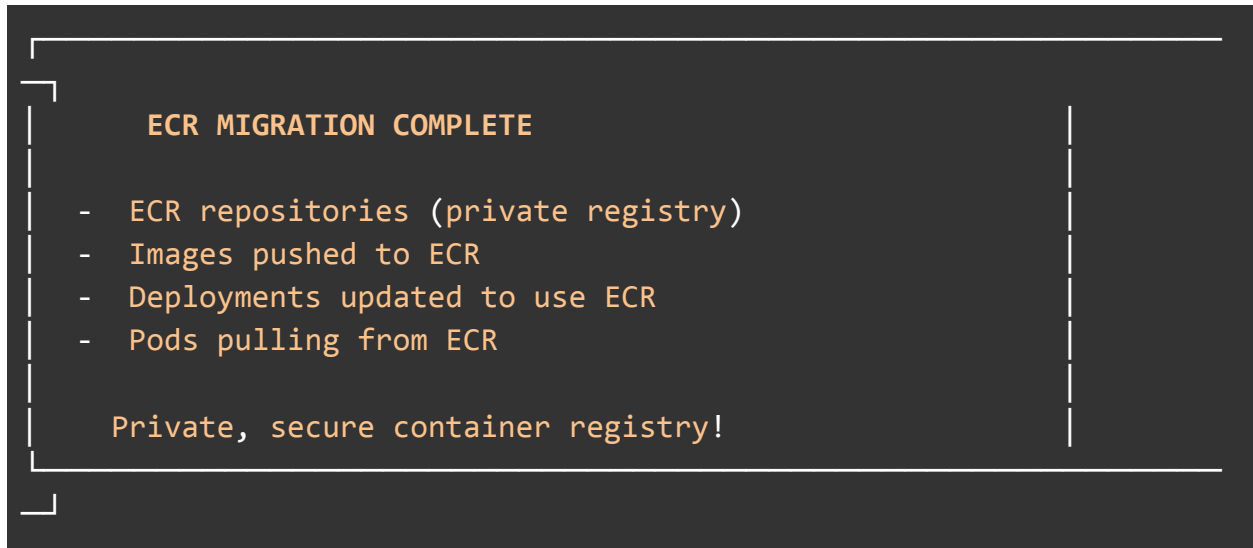
Git

- ☐ Committed `ecr.tf`
- ☐ Documented ECR migration process

Key Takeaways

The Big Picture

What you learned in this task:



Critical Concepts

1. ECR:

- AWS managed private container registry
- Stores Docker images
- Integrated with EKS
- IAM controlled access

2. Workflow:

- Authenticate Docker to ECR
- Build and tag images
- Push to ECR
- Update Deployments
- EKS pulls from ECR

3. Benefits:

- EKS can access (same AWS account)
- Private and secure

- Integrated with AWS
- Production-ready

4. Migration:

- Replace **localhost:32000** with ECR URLs
- Update Deployments
- Pods automatically pull new images

What's Next?

Congratulations! You've migrated to ECR!

You now have:

- ECR repositories (private registry)
- Images in ECR
- EKS pulling from ECR
- Production ready container registry

Next: TASK T9 Auto-Scaling & Production Hardening

In T9, you'll learn:

- How to configure auto scaling for EKS nodes
- How to set up cluster autoscaler
- How to harden infrastructure for production
- How to optimize costs

But most importantly: You now understand managed container registries. Your images are stored securely in AWS!

Proceed to TASK T9: Auto-Scaling & Production Hardening

TASK T9: Auto-Scaling & Production Hardening

Time: 4 hours | **Priority:** High | **Ticket:** DEVOPS-5109 | **Cost:** Variable (scales with usage)

Mission Brief

Configure auto-scaling for your EKS cluster and harden infrastructure for production. Add cluster autoscaler, configure resource limits, optimize costs, and implement production best practices.

Outcomes:

- Understand auto-scaling concepts
- Configure cluster autoscaler for EKS
- Set resource requests and limits
- Implement production hardening
- Optimize infrastructure costs
- Understand production best practices

Prerequisites: Complete Task T8 (ECR Container Registry)

Setup Note: Auto-scaling will work automatically in your EKS cluster. Infrastructure scales based on demand. See [Section 5 Overview](#) for general setup details.

Cost Note: Auto-scaling costs vary based on usage. Nodes scale up when needed (more cost) and down when not needed (less cost). This optimizes costs by only paying for what you use. See [Section 5 Overview](#) for cost-saving strategies.

The One Thing That Makes This Click

Auto-scaling means your infrastructure adjusts itself based on traffic, instead of staying the same size all the time.

With static infrastructure, you pay for servers that are always running, even when no one is using your app.

If traffic suddenly increases, those fixed servers can't keep up and the app becomes slow.

With auto scaling, the system adds more servers when traffic rises and removes them when traffic drops.

In simple terms, you only pay for what you actually need, and your app stays fast when usage changes.

That's it. That's auto-scaling.

Everything else (cluster autoscaler, HPA, resource limits) is just configuration. The core concept is:

"Infrastructure adapts to demand automatically."

The Mental Model Shift:

BEFORE T9: "Infrastructure = Fixed size"
"Always running same number of nodes"
"Pay for capacity, not usage"
"Manual scaling required"

AFTER T9: "Infrastructure = Dynamic size"
"Nodes scale based on demand"
"Pay for usage, not capacity"
"Automatic scaling"

This is the difference between "static" and "dynamic infrastructure."

Now let's see how auto-scaling works...

Phase 1: Understanding Auto-Scaling (30 min)

What is Auto-Scaling?

Auto-Scaling = Infrastructure that adapts to demand

The Restaurant Staffing Analogy

Think of static infrastructure like a restaurant that keeps the same number of staff all day. During busy hours, everyone is overwhelmed, and during quiet hours, you're paying people to stand around doing nothing.

Auto-scaling works like hiring staff only when customers show up. When traffic increases, more servers are added, and when things slow down, extra servers are removed.

This means your application stays fast during busy periods and cheaper during quiet ones.

In simple terms, **auto-scaling** lets your infrastructure adjust automatically to demand, so you only pay for what you actually use.

STATIC STAFFING:

```
Restaurant always has 10 waiters
├─ Lunch rush: 10 waiters (busy)
├─ Afternoon: 10 waiters (mostly idle)
├─ Dinner rush: 10 waiters (busy)
└─ Problem: Paying for idle waiters
```

AUTO-SCALING STAFFING:

```
Restaurant scales waiters based on customers
├─ Lunch rush: 15 waiters (scaled up)
├─ Afternoon: 5 waiters (scaled down)
├─ Dinner rush: 15 waiters (scaled up)
└─ Benefit: Pay only for waiters you need
```



Types of Auto-Scaling

1. Cluster Auto-Scaler (Node Level) - This Task:

Scales: Number of nodes (EC2 instances)

When: Pods can't be scheduled (not enough nodes)

How: Adds/removes nodes automatically

Example: 2 nodes → 4 nodes when traffic increases

2. Horizontal Pod Autoscaler (HPA) - Pod Level:

Scales: Number of Pods

When: CPU/memory usage exceeds threshold

How: Adds/removes Pod replicas automatically

Example: 3 Pods → 6 Pods when CPU > 70%

Note: Covered in K6 (Section 4) - we'll reference it but focus on Cluster Autoscaler here

3. Vertical Pod Autoscaler (VPA) - Resource Level:

Scales: CPU/memory requests per Pod

When: Pods need more/less resources

How: Adjusts resource requests automatically

Example: 200m CPU → 300m CPU when Pod needs more

Note: Advanced topic, not covered in this course

Auto-scaling in Kubernetes happens at different levels, depending on what needs to grow.

- The **Cluster Autoscaler** adds or removes entire nodes when there isn't enough space to run Pods, which is what you focused on in this task.
- The **Horizontal Pod Autoscaler** increases or decreases the number of Pods when usage like CPU gets high.
- The **Vertical Pod Autoscaler** adjusts how much CPU or memory a single Pod can

use when its needs change.

Together, these tools help Kubernetes stay fast and efficient as demand changes.

For this task: We'll focus on **Cluster Auto-Scaler** (node-level scaling, most important for cost optimization).

HPA (Pod-level scaling) Note:

- **HPA is covered in K6 (Section 4 - Kubernetes tasks)**
- If you completed K6, you already know how HPA works
- HPA scales Pods (application-level): 3 Pods → 6 Pods when CPU > 70%
- Cluster Autoscaler scales nodes (infrastructure-level): 2 nodes → 4 nodes when Pods can't be scheduled
- **They work together:** HPA scales Pods, Cluster Autoscaler scales nodes to run those Pods
- This task (T9) focuses on infrastructure-level scaling (nodes)

Exercise 1: Why Auto-Scaling?

Scenario: You have fixed infrastructure (4 nodes always running).

Question: What are the problems with fixed infrastructure?

Think about:

- Cost
- Resource utilization
- Traffic spikes
- Idle capacity

Your answer: _____

Answer Below 

Answer:

Problems with Fixed Infrastructure:

1. Cost Waste:

Fixed: 4 nodes always running

- └ Cost: \$120/month (4 × \$30/month)
- └ Usage: 50% average utilization
- └ Waste: \$60/month on unused capacity

Auto-scaling: 2-4 nodes based on demand

- └ Cost: \$60-120/month (varies)
- └ Usage: 80% average utilization
- └ Savings: \$30-60/month

2. Traffic Spikes:

Fixed: 4 nodes

- └ Normal traffic: 4 nodes (underutilized)
- └ Traffic spike: 4 nodes (overloaded, slow)
- └ Problem: Can't handle spikes

Auto-scaling: 2-8 nodes

- └ Normal traffic: 2 nodes (efficient)
- └ Traffic spike: 8 nodes (scales up, handles spike)
- └ Benefit: Handles any traffic

3. Idle Capacity:

Fixed: 4 nodes at night

- └ Traffic: Very low
- └ Nodes: Mostly idle
- └ Waste: Paying for unused capacity

Auto-scaling: 1-2 nodes at night

- └ Traffic: Very low
- └ Nodes: Right-sized
- └ Savings: Pay only for what you use

This is why production uses auto-scaling!

Design Checkpoint

Before moving on, make sure you can answer these in plain English:

Question	Your Answer Should Be Like...
What is auto-scaling?	"Infrastructure that grows/shrinks based on demand"
Why use auto-scaling?	"Pay for usage, not capacity - saves money"
What does cluster autoscaler do?	"Adds/removes nodes based on Pod scheduling needs"
How does auto-scaling save money?	"Scales down when not needed, only pay for what you use"

Can't answer one? Go back and re-read that exercise.

All clear? Great! Let's configure auto-scaling!

Phase 2: Implementation (3 hours)

What We're Building

Before this task:

- Fixed node count (2-4 nodes always running)
- Manual scaling required
- Paying for unused capacity
- Not optimized for production

After this task:

- Cluster autoscaler configured
- Nodes scale automatically (2-8 nodes)
- Resource requests/limits set
- Production hardening applied
- Cost optimized

Step-by-Step Plan

We'll do this in 11 steps:

- Step 0:** Configure Variables
"Set up variables for monitoring and alerts"
- Step 1:** Update Node Group for Auto-Scaling
"Configure min/max/desired node counts"
- Step 2:** Install Cluster Autoscaler
"Kubernetes component that scales nodes"
- Step 3:** Configure Resource Requests/Limits
"Tell Kubernetes how much resources Pods need"
- Step 4:** Test Auto-Scaling
"Trigger scaling and verify it works"
- Step 5:** Production Hardening
"Apply security and best practices"
- Step 6:** Cost Optimization
"Review and optimize costs"
- Step 6.5:** Enable CloudTrail (Audit Logging)
"Log all AWS API calls for security auditing"
- Step 6.6:** Enable VPC Flow Logs
"Capture network traffic for security monitoring"
- Step 6.7:** Configure AWS Secrets Manager
"Secure credential storage with encryption"
- Step 7:** Configure CloudWatch Monitoring
"Set up logs, metrics, and alarms"

Note: This task uses the [provider.tf](#) and [variables.tf](#) files created in T1. If you haven't completed T1, do that first.

Note: Ensure `.gitignore` is configured (created in T1). The `.gitignore` file prevents committing sensitive files like `terraform.tfvars` and state files.

Step 0: Configure Variables

Why we're doing this: Variables separate configuration from code, making infrastructure reusable and keeping sensitive data secure.

Note: This task uses variables from previous tasks:

- `var.project_name` (from T1)
- `var.rds_username` (from T5)
- `var.rds_db_name` (from T5)
- `var.postgres_port` (from T2)
- `var.eks_cluster_name` (from T3)

Add monitoring and alert variables (if not already in `variables.tf`):

```
# Add to variables.tf
# IDE Option: Instead of using cat, add these to your variables.tf file in
your IDE

cat >> variables.tf << 'EOF'

# =====
# MONITORING & ALERTS VARIABLES
# =====

variable "alert_email" {
    description = "Email address for infrastructure alerts (leave empty to
disable email alerts)"
    type        = string
    default     = ""
}
EOF

# Add to terraform.tfvars
cat >> terraform.tfvars << 'EOF'

# Monitoring & Alerts Configuration
alert_email = "your-email@example.com" # ⚠ Replace with your email, or
leave empty to disable
EOF
```

Variables configured!

Step 1: Update Node Group for Auto-Scaling

What we're doing: Updating the EKS node group to support auto-scaling.

Update node group configuration:

```
# Update eks-nodes.tf
# Modify the existing node group configuration

# Edit eks-nodes.tf
cat >> eks-nodes.tf << 'EOF'

# Update existing node group (or create new one with auto-scaling)
# The node group from T4 already has scaling_config, but let's verify
it's optimal

# Current configuration (from T4):
# scaling_config {
#   desired_size = 2
#   min_size     = 2
#   max_size     = 4
# }

# For auto-scaling, we want:
# - min_size: 2 (high availability)
# - max_size: 8 (handle traffic spikes)
# - desired_size: 2 (start small, scale up as needed)
EOF
```

Update Terraform configuration:

```
# Update eks-nodes.tf to increase max_size
# Edit the file manually or use sed

# Option 1: Manual edit (RECOMMENDED - more reliable)
# Open eks-nodes.tf and change:
#   max_size = 4
# To:
#   max_size = 8

# Option 2: Using sed (if you prefer)
# sed -i.bak 's/max_size      = 4/max_size      = 8/' eks-nodes.tf

# Verify change
grep max_size eks-nodes.tf

# Should show: max_size = 8
```

Format and validate code:

```
# Format Terraform code (fixes indentation, spacing)
terraform fmt
# Automatically formats your code:
# - Fixes indentation (2 spaces)
# - Aligns equals signs
# - Sorts attributes
# - Makes code readable

# Validate Terraform syntax and configuration
terraform validate
# Checks for errors:
# - Syntax errors (missing brackets, quotes)
# - Invalid attribute names
# - Type mismatches
# - Missing required attributes
# Should show: "Success! The configuration is valid."
```

Plan and apply:

```
# Preview changes
terraform plan
# Review the plan carefully:
# - Check what will be created (+)
# - Check what will be changed (~)
# - Check what will be destroyed (-)
# Always review before apply!
```

Should show: Node group will be updated (max_size: 4 → 8)

Apply changes

```
terraform apply
```

Type 'yes' when prompted

Expected output:

```
aws_eks_node_group.main: Modifying... aws_eks_node_group.main:
Modifications complete after 2m0s [id=h-game-prod:h-game-nodes]
Apply complete! Resources: 34 added, 0 changed, 1 modified.
```

Checkpoint: Did it work?

- ☐ Node group updated
- ☐ Max size increased to 8
- ☐ Ready for auto-scaling
- ☐ Node group configured for auto-scaling!

Step 2: Install Cluster Autoscaler

What we're doing: Installing cluster autoscaler, which automatically adds/removes nodes based on Pod scheduling needs.

Create IAM policy for cluster autoscaler:

```
# Create IAM policy for cluster autoscaler using Terraform
(idempotent)
cat >> cluster-autoscaler-iam.tf << 'EOF'
# =====
# CLUSTER AUTOSCALER IAM POLICY
# =====
# IAM policy for EKS Cluster Autoscaler
# Allows autoscaler to manage node groups

resource "aws_iam_policy" "cluster_autoscaler" {
  name          = "AmazonEKSClusterAutoscalerPolicy"
  description   = "Policy for EKS Cluster Autoscaler to manage node
groups"

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Effect = "Allow"
        Action = [
          "autoscaling:DescribeAutoScalingGroups",
          "autoscaling:DescribeAutoScalingInstances",
          "autoscaling:DescribeLaunchConfigurations",
          "autoscaling:DescribeScalingActivities",
          "autoscaling:DescribeTags",
          "ec2:DescribeInstanceTypes",
          "ec2:DescribeLaunchTemplateVersions"
        ]
        Resource = "*"
      },
      {
        Effect = "Allow"
        Action = [
```

```

        "autoscaling:SetDesiredCapacity",
        "autoscaling:TerminateInstanceInAutoScalingGroup",
        "ec2:DescribeImages",
        "ec2:GetInstanceTypesFromInstanceRequirements",
        "eks:DescribeNodegroup"
    ]
    Resource = "*"
}
]
})

tags = merge(
    local.common_tags,
    {
        Name = "${local.name_prefix}-cluster-autoscaler-policy"
        Task = "T9"
    }
)
}

# Attach policy to EKS node role (from T3)
resource "aws_iam_role_policy_attachment" "cluster_autoscaler" {
    role      = aws_iam_role.eks_nodes.name # From T3
    policy_arn = aws_iam_policy.cluster_autoscaler.arn
}
EOF

```

Apply IAM policy

```

terraform init    # Initialize if needed
terraform plan    # Preview changes
terraform apply   # Create policy (idempotent - safe to run multiple
times)

```


Production Note: IRSA (IAM Roles for Service Accounts)

Current approach (Node Role Attachment):

- Simpler to set up
- Works for learning
- Less secure (all Pods on nodes have these permissions)

Production approach (IRSA):

- More secure (only cluster autoscaler Pod has permissions)
- Follows least-privilege principle
- ⚠ More complex setup (requires OIDC provider, service account)

For learning: Node role attachment is fine.

For production: Consider IRSA for better security.

IRSA setup: Requires creating OIDC provider, service account with IAM role, and updating cluster autoscaler to use service account. We'll keep it simple for learning, but be aware of this for production.

Install cluster autoscaler:

```
# Add cluster autoscaler Helm repo
helm repo add autoscaler https://kubernetes.github.io/autoscaler
helm repo update

# Get values from Terraform outputs (or use terraform.tfvars values)
# Note: Helm uses shell variables, not Terraform variables directly
# We'll use terraform output to get the cluster name, or reference
from tfvars

# Option 1: Use terraform output (recommended)
CLUSTER_NAME=$(terraform output -raw eks_cluster_name 2>/dev/null ||
echo "h-game-prod")
AWS_REGION=$(terraform output -raw aws_region 2>/dev/null || echo
"us-east-1")

# Option 2: Or set manually from your terraform.tfvars values
# CLUSTER_NAME="h-game-prod"
# AWS_REGION="us-east-1"
```

```
# Install cluster autoscaler
helm install cluster-autoscaler \
  autoscaler/cluster-autoscaler \
  --namespace kube-system \
  --set autoDiscovery.clusterName=${CLUSTER_NAME} \
  --set aws.region=${AWS_REGION} \
  --set
extraArgs.scale-down-delay-after-add=${autoscaler_scale_down_delay:-1
0m} \
  --set
extraArgs.scale-down-unneeded-time=${autoscaler_scale_down_unneeded_t
ime:-10m} \
  --set
extraArgs.scale-down-utilization-threshold=${autoscaler_scale_down_ut
ilization_threshold:-0.5}

# Verify installation
kubectl get pods -n kube-system -l
"app.kubernetes.io/name=aws-cluster-autoscaler,app.kubernetes.io/inst
ance=cluster-autoscaler"

# Should show: cluster-autoscaler Pod running
```

Checkpoint - Did it work?

- ☐ IAM policy created and attached
- ☐ Cluster autoscaler installed
- ☐ Pod running in kube-system

Cluster autoscaler installed!

Step 3: Configure Resource Requests/Limits

What we're doing: Setting resource requests and limits for Pods so Kubernetes can make better scheduling decisions.

Why: Resource requests tell Kubernetes how much CPU/memory Pods need. This helps cluster autoscaler decide when to scale.

Update backend Deployment with resource requests:

```
# Get current backend Deployment
kubectl get deployment backend -n h-game -o yaml >
backend-deployment-resources.yaml
```

Add Resource Requests and Limits

```
# Edit to add resource requests and limits
cat >> backend-deployment-resources.yaml << 'EOF'
# Add to container spec:
# resources:
#   requests:
#     cpu: "200m"      # 0.2 CPU cores
#     memory: "256Mi"  # 256 MB RAM
#   limits:
#     cpu: "500m"      # 0.5 CPU cores max
#     memory: "512Mi"  # 512 MB RAM max
EOF
```

Or create updated Deployment

```
# Or create updated Deployment
cat > backend-deployment-with-resources.yaml << 'EOF'
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
  namespace: h-game
spec:
  replicas: 3
```

```

selector:
  matchLabels:
    app: backend
template:
  metadata:
    labels:
      app: backend
  spec:
    containers:
      - name: backend
        image:
123456789012.dkr.ecr.us-east-1.amazonaws.com/h-game-backend:latest
        # ... (existing env vars)

# Resource requests and limits
resources:
  requests:
    cpu: "200m"      # Minimum CPU needed
    memory: "256Mi"  # Minimum memory needed
  limits:
    cpu: "500m"      # Maximum CPU allowed
    memory: "512Mi"  # Maximum memory allowed
EOF

```

Apply updated Deployment

```

kubectl apply -f "/Users/mac/Desktop/devops
playbook/k8s/backend-deployment.yaml" && kubectl rollout status
deployment/backend -n h-game --timeout=120s

```

Update frontend Deployment:

```

# Similar update for frontend
kubectl set resources deployment frontend \
  --requests=cpu=100m,memory=128Mi \
  --limits=cpu=200m,memory=256Mi \
  -n h-game

```

Verify resource configuration:

```
# Check resource requests/limits
kubectl describe deployment backend -n h-game | grep -A 5 Resources

# Should show: Requests and Limits configured
```

Checkpoint: Did it work?

- ☐ Resource requests set for backend
- ☐ Resource limits set for backend
- ☐ Resource requests set for frontend
- ☐ Pods restarted with new resources

Resource configuration complete!

Step 4: Test Auto-Scaling

What we're doing: Testing that auto-scaling works by creating Pods that require more nodes.

Check current node count:

```
# Check current nodes
kubectl get nodes

# Should show: 2 nodes (desired_size)
```

Create test Deployment that requires scaling:

```
# Create test Deployment that needs more nodes
cat > test-scaling.yaml << 'EOF'
apiVersion: apps/v1
kind: Deployment
metadata:
  name: test-scaling
  namespace: h-game
spec:
  replicas: 10 # Many Pods to trigger scaling
  selector:
    matchLabels:
      app: test-scaling
  template:
    metadata:
      labels:
        app: test-scaling
    spec:
      containers:
      - name: test
        image: nginx:alpine
        resources:
          requests:
            cpu: "500m" # Each Pod needs 0.5 CPU
            memory: "256Mi"
EOF
```

Apply test Deployment

```
kubectl apply -f test-scaling.yaml

# Watch Pods (some will be Pending - need more nodes)
kubectl get pods -n h-game -l app=test-scaling -w

# Cluster autoscaler should detect Pending Pods
# And scale up nodes (takes 2-5 minutes)
```

Monitor cluster autoscaler logs:

```
# Check cluster autoscaler logs
kubectl logs -n kube-system -l app=cluster-autoscaler --tail=50

# Should show:
# - "Pod default/test-scaling-xxx is unschedulable"
# - "Scale-up: setting group size to 3"
# - "Successfully increased size"
```

Check nodes scaling up:

```
# Watch nodes
watch -n 5 kubectl get nodes

# Should see: Nodes increasing (2 → 3 → 4, etc.)
# Takes 2-5 minutes per node
```

Clean up test:

```
# Delete test Deployment
kubectl delete deployment test-scaling -n h-game

# Watch nodes scale down (after 10 minutes)
watch -n 30 kubectl get nodes

# Should see: Nodes decreasing (4 → 3 → 2)
# Cluster autoscaler scales down unused nodes
```

Checkpoint - Did it work?

- ☐ Test Deployment created
- ☐ Nodes scaled up (more nodes added)
- ☐ Pods scheduled on new nodes
- ☐ Nodes scaled down (after cleanup)

Auto-scaling working!

Step 5: Production Hardening

What we're doing: Applying production best practices and security hardening.

Enable EKS logging (if not already enabled):

```
# Check current Logging
aws eks describe-cluster --name h-game-prod --query
'logging.clusterLogging'

# Enable all log types
aws eks update-cluster-config \
  --name h-game-prod \
  --logging
'{"enableLogTypes":["api","audit","authenticator","controllerManager",
"scheduler"]}'
```

Enable RDS automated backups (verify):

```
# Check RDS backup configuration
aws rds describe-db-instances \
  --db-instance-identifier h-game-postgres \
  --query 'DBInstances[0].[BackupRetentionPeriod,AutomatedBackups]'

# Should show: 7 days retention, enabled
```

Review security groups:

```
# List all security groups
aws ec2 describe-security-groups \
  --filters "Name=vpc-id,Values=$(terraform output -raw vpc_id)" \
  --query 'SecurityGroups[*].[GroupName,GroupId,IpPermissions]' \
  --output table

# Verify: Only necessary ports open
# Verify: Security groups reference each other (not 0.0.0.0/0)
```

Enable ECR image scanning (verify):

```
# Check ECR scanning configuration
aws ecr describe-repositories --query
'repositories[*].[repositoryName,imageScanningConfiguration]'

# Should show: scanOnPush = true
```

Checkpoint - Did it work?

- ☐ EKS logging enabled
- ☐ RDS backups configured
- ☐ Security groups reviewed
- ☐ ECR scanning enabled

Production hardening applied!

Step 6.5: Enable CloudTrail (Audit Logging)

What we're doing: Enabling CloudTrail to log all AWS API calls for security auditing and compliance.

Why: Production requires audit logging to track who did what and when. CloudTrail logs all AWS API calls.

Create CloudTrail configuration:

```
# Add CloudTrail configuration
cat >> cloudtrail.tf << 'EOF'
# =====
# CLOUDTRAIL (AUDIT LOGGING)
# =====
# Logs all AWS API calls for security auditing
# Required for compliance and security investigations

# Data sources (required for S3 bucket naming)
data "aws_caller_identity" "current" {}
data "aws_region" "current" {}

# S3 bucket for CloudTrail Logs
# Note: Using timestamp() in bucket name means bucket name changes on every
# apply
# For production, use a fixed name or data source for account ID
resource "aws_s3_bucket" "cloudtrail_logs" {
  bucket =
    "${local.name_prefix}-cloudtrail-logs-${data.aws_caller_identity.current.account_id}"

  # Lifecycle rule: Prevent accidental deletion of audit logs
  lifecycle {
    prevent_destroy = true # Critical: Audit logs must never be deleted
  }

  # ⚠ Production Enhancement: MFA Delete Protection
  # MFA delete requires root account and can only be enabled via AWS Console or
  # CLI
  # To enable: aws s3api put-bucket-versioning --bucket <bucket-name>
  # --versioning-configuration Status=Enabled,MFADelete=Enabled --mfa <mfa-serial>
  # <mfa-token>
  # This adds an extra layer of protection against accidental deletion
```

```

tags = merge(
  local.common_tags,
  {
    Name = "${local.name_prefix}-cloudtrail-logs"
    Task = "T9"
  }
)
}

# S3 bucket versioning (for audit trail)
resource "aws_s3_bucket_versioning" "cloudtrail_logs" {
  bucket = aws_s3_bucket.cloudtrail_logs.id

  versioning_configuration {
    status = "Enabled"
  }
}

# S3 bucket encryption
resource "aws_s3_bucket_server_side_encryption_configuration" "cloudtrail_logs"
{
  bucket = aws_s3_bucket.cloudtrail_logs.id

  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm = "AES256"
    }
  }
}

# CloudTrail trail
resource "aws_cloudtrail" "main" {
  name           = "${local.name_prefix}-cloudtrail"
  s3_bucket_name = aws_s3_bucket.cloudtrail_logs.id

  # Log all API calls
  include_global_service_events = true # Log all AWS services

  # Enable log file validation (detects tampering)
  enable_log_file_validation = true

  # Enable log file integrity validation

```

```

enable_logging = true

# Event selector (log all events)
event_selector {
  read_write_type      = "All"
  include_management_events = true
  exclude_management_event_sources = []
}

tags = merge(
  local.common_tags,
  {
    Name = "${local.name_prefix}-cloudtrail"
    Task = "T9"
  }
)

depends_on = [aws_s3_bucket_policy.cloudtrail_logs]
}

# S3 bucket policy for CloudTrail
resource "aws_s3_bucket_policy" "cloudtrail_logs" {
  bucket = aws_s3_bucket.cloudtrail_logs.id

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Sid      = "AWSCloudTrailAclCheck"
        Effect   = "Allow"
        Principal = {
          Service = "cloudtrail.amazonaws.com"
        }
        Action   = "s3:GetBucketAcl"
        Resource = aws_s3_bucket.cloudtrail_logs.arn
      },
      {
        Sid      = "AWSCloudTrailWrite"
        Effect   = "Allow"
        Principal = {
          Service = "cloudtrail.amazonaws.com"
        }
        Action   = "s3:PutObject"
      }
    ]
  })
}

```

```

        Resource = "${aws_s3_bucket.cloudtrail_logs.arn}/*"
        Condition = {
            StringEquals = {
                "s3:x-amz-acl" = "bucket-owner-full-control"
            }
        }
    ]
})
}

# Output: CloudTrail ARN
output "cloudtrail_arn" {
    description = "ARN of the CloudTrail trail"
    value       = aws_cloudtrail.main.arn
}
EOF

```

Apply changes:

```

# Preview changes
terraform plan

# Should show:
# - S3 bucket for CloudTrail logs
# - S3 bucket versioning
# - S3 bucket encryption
# - CloudTrail trail
# - S3 bucket policy

# Apply changes
terraform apply

```

Verify CloudTrail:

```
# Check CloudTrail status
aws cloudtrail get-trail-status --name h-game-cloudtrail

# Should show: IsLogging = true

# View recent events (optional)
aws cloudtrail lookup-events --lookup-attributes
AttributeKey=EventName,AttributeValue=CreateVpc --max-results 5
```

Checkpoint - Did it work?

- ☐ CloudTrail enabled
- ☐ S3 bucket for logs created
- ☐ Logging active
- ☐ All API calls being logged

CloudTrail configured!

Production Note: CloudTrail is essential for security auditing and compliance. It logs all AWS API calls, allowing you to track who made changes and when. First 90 days are free, then ~\$2-5/month.

Step 6.6: Enable VPC Flow Logs

What we're doing: Enabling VPC Flow Logs to capture network traffic information for security monitoring.

Why: VPC Flow Logs help detect suspicious network activity, troubleshoot connectivity issues, and meet compliance requirements.

Create VPC Flow Logs configuration:

```
# Add VPC Flow Logs configuration
cat >> vpc-flow-logs.tf << 'EOF'
# =====
# VPC FLOW LOGS
# =====
# Captures network traffic information
# Required for security monitoring and troubleshooting

# CloudWatch Log Group for VPC Flow Logs
resource "aws_cloudwatch_log_group" "vpc_flow_logs" {
  name           = "/aws/vpc/flowlogs"
  retention_in_days = 7 # Keep logs for 7 days (free tier: 5GB/month)

  tags = {
    Name       = "h-game-vpc-flow-logs"
    Environment = "production"
    ManagedBy   = "Terraform"
    Task        = "T9"
  }
}

# IAM role for VPC Flow Logs
resource "aws_iam_role" "vpc_flow_logs" {
  name = "${local.name_prefix}-vpc-flow-logs-role"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Action = "sts:AssumeRole"
        Effect = "Allow"
        Principal = {
          Service = "vpc-flow-logs.amazonaws.com"
        }
      }
    ]
  })
}
```



```

    }
  }
]
})

tags = merge(
  local.common_tags,
  {
    Name = "${local.name_prefix}-vpc-flow-logs-role"
    Task = "T9"
  }
)
ManagedBy = "Terraform"
Task       = "T9"
}
}

```

IAM policy for VPC Flow Logs

```

resource "aws_iam_role_policy" "vpc_flow_logs" {
  name = "${local.name_prefix}-vpc-flow-logs-policy"
  role = aws_iam_role.vpc_flow_logs.id
}

```

```

policy = jsonencode({
  Version = "2012-10-17"
  Statement = [
    {
      Effect = "Allow"
      Action = [
        "logs:CreateLogGroup",
        "logs:CreateLogStream",
        "logs:PutLogEvents",
        "logs:DescribeLogGroups",
        "logs:DescribeLogStreams"
      ]
      Resource = "*"
    }
  ]
})
}

```

VPC Flow Log

```

resource "aws_flow_log" "main" {
  iam_role_arn = aws_iam_role.vpc_flow_logs.arn
}

```

```

log_destination = aws_cloudwatch_log_group.vpc_flow_logs.arn
traffic_type    = "ALL" # Log all traffic (ACCEPT, REJECT, ALL)
vpc_id          = aws_vpc.main.id

tags = merge(
  local.common_tags,
  {
    Name = "${local.name_prefix}-vpc-flow-logs"
    Task = "T9"
  }
)
}

# Output: Flow Log ID
output "vpc_flow_log_id" {
  description = "ID of the VPC Flow Log"
  value       = aws_flow_log.main.id
}
EOF

```

Apply changes:

```

# Preview changes
terraform plan

# Should show:
# - CloudWatch Log group
# - IAM role for VPC Flow Logs
# - IAM policy
# - VPC Flow Log

# Apply changes
terraform apply

```

Also apply SNS alerts in Repo. (read up on it)

Verify VPC Flow Logs:

```
# Check Flow Log status
aws ec2 describe-flow-logs --filter
"Name=resource-id,Values=$(terraform output -raw vpc_id)"

# Should show: FlowLogStatus = ACTIVE

# View logs in CloudWatch (optional)
aws logs tail /aws/vpc/flowlogs --follow
```

Checkpoint - Did it work?

- ☐ VPC Flow Logs enabled
- ☐ CloudWatch log group created
- ☐ Network traffic being logged
- ☐ Logs accessible in CloudWatch

VPC Flow Logs configured!

Production Note: VPC Flow Logs capture network traffic metadata (source IP, destination IP, ports, protocol). Essential for security monitoring, troubleshooting, and compliance. Costs ~\$0.50/GB of logs.

Step 6.7: Configure AWS Secrets Manager

What we're doing: Setting up AWS Secrets Manager to store sensitive credentials (RDS passwords, API keys) securely with encryption and automatic rotation.

Why: Production requires encrypted credential storage. Secrets Manager provides encryption at rest, automatic rotation, and IAM integration, better than Kubernetes Secrets (base64-encoded).

Note: Kubernetes Secrets (from K8) are base64-encoded, not encrypted. Secrets Manager provides **true** encryption and is the production standard.

Create Secrets Manager configuration:

```
# ⚠ STEP 1: Add random provider to versions.tf (if not already present)
# Check if versions.tf exists and has random provider
if ! grep -q "hashicorp/random" versions.tf 2>/dev/null; then
    # If provider.tf exists, add random provider there instead
    if [ -f provider.tf ]; then
        # Update provider.tf to include random provider
        # Edit provider.tf manually and add random provider to required_providers
        block
            echo "⚠ Please manually add random provider to your provider.tf or
versions.tf:"
            echo "  random = {"
            echo "    source = \"hashicorp/random\""
            echo "    version = \"~> 3.1\""
            echo "  }"
        else
            # Create versions.tf with random provider
            cat > versions.tf << 'VERSIONEOF'
terraform {
    required_version = ">= 1.0"

    required_providers {
        aws = {
            source = "hashicorp/aws"
            version = "~> 5.0"
        }
        random = {
            source = "hashicorp/random"
```

```

        version = "~> 3.1"
    }
}
}
VERSIONEOF
fi

# Initialize Terraform to download random provider
terraform init
fi

# ⚠ NOTE: Data sources (aws_region.current, aws_caller_identity.current) are
already defined in cloudtrail.tf (Step 6.5)
# No need to redefine them here - they're available to all resources in the same
Terraform workspace

# STEP 2: Create Secrets Manager configuration
cat >> secrets-manager.tf << 'EOF'
# =====
# AWS SECRETS MANAGER
# =====
# Secure credential storage with encryption
# Replaces: Kubernetes Secrets (base64-encoded)
# Better: Encrypted at rest, automatic rotation, IAM integration

# RDS Database Password Secret
resource "aws_secretsmanager_secret" "rds_password" {
    name          = "${var.project_name}/rds/password"
    description   = "RDS PostgreSQL database password"

    # Automatic rotation (optional - requires Lambda function)
    # rotation_rules {
    #     automatically_after_days = 30 # Rotate every 30 days
    # }

    tags = merge(
        local.common_tags,
        {
            Name = "${local.name_prefix}-rds-password"
            Task = "T9"
        }
    )
}

# RDS Database Password Secret Version

```

```

# ⚠️ Uses existing RDS password from T5 (var.rds_password)
# Note: If you want to generate a new password, you can use random_password
resource
resource "aws_secretsmanager_secret_version" "rds_password" {
  secret_id = aws_secretsmanager_secret.rds_password.id

  # Use existing RDS password from T5 (from terraform.tfvars)
  # If RDS doesn't exist yet, host will be empty (that's OK)
  secret_string = jsonencode({
    username = var.rds_username          # From variable (T5)
    password = var.rds_password         # From variable (T5) -
existing password
    engine   = "postgres"
    host     = try(aws_db_instance.postgres.address, "") # From T5 (will be
empty if RDS doesn't exist yet)
    port     = var.postgres_port        # From variable (T2)
    dbname   = var.rds_db_name          # From variable (T5)
  })

  # ⚠️ IMPORTANT: RDS Dependency Note
  # - RDS is created in T5 (RDS PostgreSQL Migration)
  # - If you haven't completed T5 yet, the host field will be empty (that's OK)
  # - After creating RDS in T5, run terraform apply again to update the secret
with the RDS endpoint
  # - The try() function handles missing RDS gracefully (no error if RDS doesn't
exist)

  lifecycle {
    ignore_changes = [secret_string] # Don't overwrite if manually changed
  }
}

# API Keys Secret (example - for external APIs)
resource "aws_secretsmanager_secret" "api_keys" {
  name          = "${var.project_name}/api/keys"
  description   = "External API keys for backend"

  tags = merge(
    local.common_tags,
    {
      Name = "${local.name_prefix}-api-keys"
      Task = "T9"
    }
  )
}

```

```

# API Keys Secret Version (example)
resource "aws_secretsmanager_secret_version" "api_keys" {
  secret_id = aws_secretsmanager_secret.api_keys.id

  secret_string = jsonencode({
    external_api_key = "your-api-key-here" # ⚠️ Replace with actual key
    payment_gateway_key = "your-payment-key" # ⚠️ Replace with actual key
  })

  lifecycle {
    ignore_changes = [secret_string] # Don't overwrite if manually changed
  }
}

# IAM Policy for EKS Pods to access Secrets Manager
# This allows backend Pods to retrieve secrets
resource "aws_iam_policy" "secrets_manager_access" {
  name          = "${local.name_prefix}-secrets-manager-access"
  description   = "Allow EKS Pods to read secrets from Secrets Manager"

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Effect = "Allow"
        Action = [
          "secretsmanager:GetSecretValue",
          "secretsmanager:DescribeSecret"
        ]
        Resource = [
          aws_secretsmanager_secret.rds_password.arn,
          aws_secretsmanager_secret.api_keys.arn
        ]
      },
      {
        Effect = "Allow"
        Action = [
          "kms:Decrypt"
        ]
        Resource = "*" # KMS key for Secrets Manager encryption
        Condition = {
          StringEquals = {
            "kms:ViaService" =
"secretsmanager.${data.aws_region.current.name}.amazonaws.com"

```

```

    }
  }
}
])
})

tags = merge(
  local.common_tags,
  {
    Name = "${local.name_prefix}-secrets-manager-policy"
    Task = "T9"
  }
)
}

# Attach policy to EKS node role (for Pods to access secrets)
# ⚠️ For production, use IRSA (IAM Roles for Service Accounts) instead
# This is a simplified approach for learning
# Note: aws_iam_role.eks_nodes is created in T3 (EKS Cluster task)
resource "aws_iam_role_policy_attachment" "secrets_manager_access" {
  role      = aws_iam_role.eks_nodes.name # From T3 (EKS node role)
  policy_arn = aws_iam_policy.secrets_manager_access.arn
}

# Output: Secret ARNs (for reference)
output "rds_password_secret_arn" {
  description = "ARN of the RDS password secret"
  value       = aws_secretsmanager_secret.rds_password.arn
  sensitive   = true
}

output "api_keys_secret_arn" {
  description = "ARN of the API keys secret"
  value       = aws_secretsmanager_secret.api_keys.arn
  sensitive   = true
}
EOF

```


Update RDS to use Secrets Manager password (optional):

If you want to update RDS to use the secret password:

```
# Get current RDS password from secret
aws secretsmanager get-secret-value \
  --secret-id h-game/rds/password \
  --query SecretString \
  --output text | jq -r .password

# Update RDS master password (if needed)
# ⚠ This will cause brief connection interruption
aws rds modify-db-instance \
  --db-instance-identifier h-game-postgres \
  --master-user-password $(aws secretsmanager get-secret-value \
  --secret-id h-game/rds/password --query SecretString --output text | \
  jq -r .password) \
  --apply-immediately
```

Apply changes:

```
# Preview changes
terraform plan

# Should show:
# - Secrets Manager secrets (RDS password, API keys)
# - Secret versions
# - Random password generator
# - IAM policy for Secrets Manager access
# - IAM role policy attachment

# Apply changes
terraform apply
```

Verify Secrets Manager:

```
# List secrets
aws secretsmanager list-secrets --query 'SecretList[*].[Name,ARN]'
--output table

# Get secret value (RDS password)
aws secretsmanager get-secret-value \
  --secret-id h-game/rds/password \
  --query SecretString \
  --output text

# Should show: JSON with username, password, host, port, dbname
```

Update backend to use Secrets Manager:

Option 1: Update backend Deployment to retrieve secrets:

Get secret ARN from Terraform output

```
terraform output rds_password_secret_arn
```

Create Kubernetes ConfigMap with secret ARN

```
cat > backend-secrets-manager.yaml << 'EOF'
apiVersion: v1
kind: ConfigMap
metadata:
  name: secrets-manager-config
  namespace: h-game
data:
  # Secret ARN (backend will retrieve values using AWS SDK)
  RDS_SECRET_ARN: "REPLACE_WITH_TERRAFORM_OUTPUT" # ⚠️ Replace with
actual ARN
  API_KEYS_SECRET_ARN: "REPLACE_WITH_TERRAFORM_OUTPUT" # ⚠️ Replace
with actual ARN
EOF
```

Apply ConfigMap

```
kubectl apply -f backend-secrets-manager.yaml

# Update backend Deployment to use Secrets Manager
# Backend code needs AWS SDK to retrieve secrets:
# Example Node.js code:
# const AWS = require('aws-sdk');
# const secretsManager = new AWS.SecretsManager();
# const secret = await secretsManager.getSecretValue({
#   SecretId: process.env.RDS_SECRET_ARN
# }).promise();
# const { password, host, port, dbname } =
  JSON.parse(secret.SecretString);
```

Option 2: Use External Secrets Operator (Advanced - Production):

For production, use External Secrets Operator to sync Secrets Manager → Kubernetes Secrets automatically:

```
# Install External Secrets Operator (optional - advanced)
# kubectl apply -f
https://external-secrets.io/operator-manifests.yaml

# Create ExternalSecret resource
# This automatically syncs Secrets Manager → Kubernetes Secret
# See: https://external-secrets.io/
```

Migration Note: If RDS already exists with a hardcoded password, you can:

1. Keep using the existing password (update secret manually)
2. Or update RDS to use the new secret password (causes brief connection interruption)

Checkpoint - Did it work?

- ☐ Secrets Manager secrets created
- ☐ RDS password stored securely
- ☐ IAM policy for access created
- ☐ Secrets accessible via AWS CLI
- ☐ Backend can retrieve secrets (with code update)

Secrets Manager configured!

Production Note: Secrets Manager provides:

- Encryption at rest (KMS encryption)
- Automatic rotation (with Lambda function)
- IAM integration (fine-grained access control)
- Audit logging (CloudTrail integration)
- Versioning (track secret changes)

Cost: ~\$0.40/secret/month + \$0.05/10,000 API calls

Migration from Kubernetes Secrets:

Current: Kubernetes Secrets (base64, from K8)

Production: Secrets Manager (encrypted, IAM-controlled)

Backend code needs AWS SDK to retrieve secrets

Step 6: Cost Optimization

What we're doing: Reviewing and optimizing infrastructure costs.

Check current costs:

```
# Get cost estimate (requires Cost Explorer API)
# Or check AWS Console → Cost Explorer

# Manual cost breakdown:
# EKS Control Plane: $73/month
# EKS Nodes (2-8): $60-240/month (varies with auto-scaling)
# RDS: $15/month
# ElastiCache: $13/month
# ALB: $18/month
# NAT Gateway: $11/month (single)
# ECR: $1/month
# Total: ~$191-371/month (varies with node scaling)
```

Cost optimization strategies:

```
# 1. Use Spot instances (already configured in T4)
# Saves: 50% on compute

# 2. Right-size instances
# Current: t3.medium (good balance)
# Could use: t3.small (saves 50%, less capacity)

# 3. Single NAT Gateway (already done)
# Saves: $22/month vs 3 NAT Gateways

# 4. Auto-scaling (configured)
# Saves: Scales down when not needed

# 5. Destroy when not using
# Saves: 100% when destroyed!
```

Create cost monitoring:

```
# Create SNS topic for cost alerts (optional)  
aws sns create-topic --name cost-alerts  
  
# this has been created in repo, you apply there  
  
# Set up CloudWatch billing alarm (optional)  
# AWS Console → CloudWatch → Alarms → Create alarm
```

Checkpoint - Did it work?

- ☐ Costs reviewed
- ☐ Optimization strategies identified
- ☐ Auto-scaling reduces costs
- ☐ Spot instances save 50%

Cost optimization complete!

Step 7: Configure CloudWatch Monitoring

What we're doing: Setting up CloudWatch logs, metrics, and alarms for your infrastructure, with email notifications so you know when something breaks.

Why: Production requires visibility into system health. Alarms without notifications are useless - you need to know when something goes wrong!

Enable EKS logging to CloudWatch:

```
# Add CloudWatch configuration
cat >> cloudwatch.tf << 'EOF'
# =====
# CLOUDWATCH MONITORING
# =====
# Logs, metrics, and alarms for infrastructure monitoring

# CloudWatch Log Group for EKS
resource "aws_cloudwatch_log_group" "eks_cluster" {
  name           = "/aws/eks/${var.eks_cluster_name}/cluster"
  retention_in_days = 7 # Keep logs for 7 days (free tier: 5GB/month)

  tags = merge(
    local.common_tags,
    {
      Name = "${local.name_prefix}-eks-logs"
      Task = "T9"
    }
  )
}

# CloudWatch Log Group for ALB Access Logs
resource "aws_cloudwatch_log_group" "alb_access" {
  name           = "/aws/alb/${var.project_name}/access"
  retention_in_days = 7

  tags = merge(
    local.common_tags,
    {
      Name = "h-game-alb-logs"
      Task = "T9"
    }
  )
}
```

```

}

# CloudWatch Metric Alarm: High CPU Usage
resource "aws_cloudwatch_metric_alarm" "high_cpu" {
  alarm_name          = "${local.name_prefix}-high-cpu"
  comparison_operator = "GreaterThanOrEqualToThreshold"
  evaluation_periods  = 2
  metric_name         = "CPUUtilization"
  namespace           = "AWS/EKS"
  period              = 300 # 5 minutes
  statistic           = "Average"
  threshold           = 80 # Alert if CPU > 80%
  alarm_description   = "Alert when CPU usage exceeds 80%"

  dimensions = {
    ClusterName = aws_eks_cluster.main.name
  }

  tags = merge(
    local.common_tags,
    {
      Name = "${local.name_prefix}-cpu-alarm"
      Task = "T9"
    }
  )
}

# CloudWatch Metric Alarm: High Memory Usage
resource "aws_cloudwatch_metric_alarm" "high_memory" {
  alarm_name          = "${local.name_prefix}-high-memory"
  comparison_operator = "GreaterThanOrEqualToThreshold"
  evaluation_periods  = 2
  metric_name         = "MemoryUtilization"
  namespace           = "AWS/EKS"
  period              = 300
  statistic           = "Average"
  threshold           = 80
  alarm_description   = "Alert when memory usage exceeds 80%"

  dimensions = {
    ClusterName = aws_eks_cluster.main.name
  }
}

```



```

tags = merge(
  local.common_tags,
  {
    Name = "${local.name_prefix}-memory-alarm"
    Task = "T9"
  }
)
}
EOF

```

Step 7.5: Enable Email Alerts

Problem: Your alarms trigger but nobody gets notified!

Solution: Add email notifications so you know when something breaks.

Note: The `alert_email` variable should already be configured in Step 0. If you haven't done Step 0 yet, go back and configure it first.

```

# Verify alert_email is set (from Step 0)
# If not set, add it to terraform.tfvars:
# alert_email = "your-email@example.com"

# Create SNS topic for alerts
cat >> sns-alerts.tf << 'EOF'
# =====
# SNS TOPIC FOR EMAIL ALERTS
# =====

resource "aws_sns_topic" "alerts" {
  name = "${local.name_prefix}-infrastructure-alerts"

  tags = merge(
    local.common_tags,
    {
      Name = "${local.name_prefix}-alerts"
      Task = "T9"
    }
  )
}

```

```

}

resource "aws_sns_topic_subscription" "email" {
  count      = var.alert_email != "" ? 1 : 0
  topic_arn = aws_sns_topic.alerts.arn
  protocol  = "email"
  endpoint  = var.alert_email
}
EOF

```

Update existing alarms to send notifications

Edit `cloudwatch.tf` and add this line to BOTH alarm resources:

```

# alarm_actions = var.alert_email != "" ? [aws_sns_topic.alerts.arn]
: []
# ok_actions     = var.alert_email != "" ? [aws_sns_topic.alerts.arn]
: []
#
# The updated alarms should look like this:

```

```

cat >> cloudwatch.tf << 'EOF'

# Update existing alarms to send email notifications
# (Add these lines to the existing alarm resources above)

# For high_cpu alarm, add:
# alarm_actions = var.alert_email != "" ? [aws_sns_topic.alerts.arn]
: []
# ok_actions     = var.alert_email != "" ? [aws_sns_topic.alerts.arn]
: []

# For high_memory alarm, add:
# alarm_actions = var.alert_email != "" ? [aws_sns_topic.alerts.arn]
: []
# ok_actions     = var.alert_email != "" ? [aws_sns_topic.alerts.arn]
: []

```

```
EOF
```

```
# Better: Manually edit cloudwatch.tf
# Find the two alarm resources (high_cpu and high_memory)
# Add these two lines to each alarm:
#   alarm_actions = var.alert_email != "" ?
[aws_sns_topic.alerts.arn] : []
#   ok_actions     = var.alert_email != "" ?
[aws_sns_topic.alerts.arn] : []
```

Apply changes:

```
# Preview changes
terraform plan

# Apply changes
terraform apply

# ⚠️ IMPORTANT: Check your email for SNS subscription confirmation
# Subject: "AWS Notification - Subscription Confirmation"
# Click "Confirm subscription" link in the email
# Without confirmation, you won't receive alerts!
```

Test it (optional):

```
# Trigger CPU alarm to test email notification
# Create a temporary high-CPU workload
kubectl run stress --image=containerstack/cpustress --limits=cpu=2 -n
h-game

# Wait 5-10 minutes for alarm to trigger
# You'll receive email: "ALARM: h-game-high-cpu"

# Clean up
kubectl delete pod stress -n h-game
```

Checkpoint: Did it work?

- ☐ CloudWatch log groups created
- ☐ Metric alarms configured (CPU, Memory)
- ☐ SNS topic created
- ☐ Email subscription confirmed (check email!)
- ☐ Alarms send email notifications
- ☐ Test alarm triggered email

Email alerts configured! You'll now be notified when things break.

Note: **Section 5** covers basic infrastructure monitoring (logs + alerts). Complete observability (application metrics, dashboards, distributed tracing) is covered in **Section 6: DevSecOps**.

Verify EKS cluster logging (already enabled in T3):

Note: EKS cluster logging was already enabled [in](#) T3 (Step 3). The CloudWatch [log](#) group created above will receive logs from the existing EKS cluster configuration.

```
# Verify EKS logging is enabled (from T3)
aws eks describe-cluster --name h-game-prod --query
'logging.clusterLogging'

# Should show: enabled_cluster_log_types = ["api", "audit",
"authenticator", "controllerManager", "scheduler"]

# CloudWatch log group (created above) will receive logs
# Logs are sent to: /aws/eks/h-game-prod/cluster
# No changes needed - logging already configured!
```

Enable ALB access logs:

Important: The ALB was created [in](#) T7. You need to update the existing ALB resource [in](#) your alb.tf file, not create a new one.

```
# Step 1: Create S3 bucket for ALB Logs (new resource)
```

```

cat >> alb-logs.tf << 'EOF'
# S3 bucket for ALB access logs
# Note: Using account ID ensures unique bucket name that doesn't
# change on every apply
resource "aws_s3_bucket" "alb_logs" {
    bucket =
"${local.name_prefix}-alb-logs-${data.aws_caller_identity.current.account_id}"

    # Lifecycle rule: Prevent accidental deletion of access logs
    lifecycle {
        prevent_destroy = true # Critical: Access logs must never be
deleted
    }

    tags = merge(
        local.common_tags,
        {
            Name = "h-game-alb-logs"
            Task = "T9"
        }
    )
}

# S3 bucket versioning (optional)
resource "aws_s3_bucket_versioning" "alb_logs" {
    bucket = aws_s3_bucket.alb_logs.id

    versioning_configuration {
        status = "Enabled"
    }
}
EOF

```

```

# Step 2: Update existing ALB in alb.tf to enable access logs
# Edit alb.tf and add this block to the existing aws_lb resource:
#
# resource "aws_lb" "main" {

```

```
# # ... existing configuration ...
#
# # ADD THIS BLOCK:
# access_logs {
#     bucket = aws_s3_bucket.alb_logs.id
#     enabled = true
# }
# }
```

```
# Apply changes
terraform apply
```

View logs and alarms:

```
# View EKS cluster logs
aws logs tail /aws/eks/h-game-prod/cluster --follow

# View CloudWatch alarms
aws cloudwatch describe-alarms --alarm-name-prefix h-game

# Check alarm state
aws cloudwatch describe-alarms --alarm-names h-game-high-cpu

# View in AWS Console
# AWS Console → CloudWatch → Alarms
# Filter by: h-game
# You'll see: h-game-high-cpu, h-game-high-memory
```

Phase 3: Validation & Consolidation (30 min)

Quiz: Auto-Scaling Understanding

Answer these questions to verify your understanding:

Question 1: What is auto-scaling?

Answer:

- **Auto scaling** = Infrastructure that adapts to demand automatically
- Scales up when demand increases
- Scales down when demand decreases
- Pay for usage, not capacity

Key point: Auto scaling = Dynamic infrastructure that adapts to demand.

Question 2: How does cluster autoscaler work?

Answer:

1. **Detects:** Pods that can't be scheduled (need more nodes)
2. **Scales up:** Adds nodes to cluster
3. **Schedules:** Pods on new nodes
4. **Scales down:** Removes unused nodes (after delay)

Key point: Cluster autoscaler = Automatically adds/removes nodes based on Pod scheduling needs.

Question 3: How does auto-scaling save money?

Answer:

- **Scales down:** Removes nodes when not needed (saves money)
- **Scales up:** Adds nodes only when needed (pay for usage)
- **Right-sizing:** Infrastructure matches demand
- **No waste:** Don't pay for idle capacity

Key point: Auto-scaling = Pay for usage, not capacity.

Checklist: Did You Complete Everything?

Before moving on, check these boxes:

Understanding Phase

- ☐ I understand what auto-scaling is (infrastructure adapts to demand)
- ☐ I understand cluster autoscaler (scales nodes)
- ☐ I understand resource requests/limits
- ☐ I completed all design exercises

Implementation Phase

- ☐ Node group updated for auto-scaling
- ☐ Cluster autoscaler installed
- ☐ Resource requests/limits configured
- ☐ Auto-scaling tested (scales up/down)
- ☐ Production hardening applied
- ☐ Cost optimization reviewed

Verification Phase

- ☐ Can explain auto-scaling
- ☐ Can explain cluster autoscaler
- ☐ Can test auto-scaling
- ☐ Understand production best practices

Git

- ☐ Committed updated `eks-nodes.tf`
- ☐ Documented auto-scaling configuration

Key Takeaways

The Big Picture

What you learned in this task:

AUTO-SCALING & HARDENING COMPLETE

- Cluster autoscaler (scales nodes)
- Resource requests/limits configured
- Production hardening applied
- Cost optimization reviewed

Production-ready, cost-optimized infrastructure!

Critical Concepts

1. Auto-Scaling:

- Infrastructure adapts to demand
- Scales up when needed
- Scales down when not needed
- Pay for usage, not capacity

2. Cluster Autoscaler:

- Kubernetes component
- Scales nodes based on Pod scheduling
- Adds nodes when Pods can't be scheduled
- Removes nodes when unused

3. Resource Management:

- Requests: Minimum resources needed
- Limits: Maximum resources allowed
- Helps Kubernetes make scheduling decisions

4. Production Hardening:

- Enable logging
- Configure backups
- Review security
- Optimize costs

What's Next?

Congratulations! You've hardened your infrastructure!

You now have:

- ☐ Auto-scaling configured
- ☐ Production hardening applied
- ☐ Cost optimization reviewed
- ☐ Production-ready infrastructure

Next: TASK T10 - Infrastructure Testing & Documentation

In T10, you'll learn:

- How to test infrastructure changes
- How to document your infrastructure
- How to create runbooks
- How to ensure reproducibility

But most importantly: You now understand production infrastructure. Your system is scalable, secure, and cost-optimized!

Proceed to TASK T10: Infrastructure Testing & Documentation

TASK T10: Infrastructure Testing & Documentation

Time: 3 hours | **Priority:** High | **Ticket:** DEVOPS-5110 | **Cost:** \$0 (testing only)

Mission Brief

Test your infrastructure, create comprehensive documentation, and ensure everything is reproducible. This is the capstone task that validates your complete Terraform infrastructure and prepares it for production use.

Outcomes:

- Test infrastructure changes safely
- Create complete documentation
- Generate architecture diagrams
- Create runbooks for common operations
- Ensure infrastructure is reproducible
- Validate production readiness

Prerequisites: Complete Task T9 (Auto-Scaling & Production Hardening)

Setup Note: All testing is non-destructive and validates your infrastructure. See Section 5 Overview for general setup details.

Cost Note: This task costs \$0. It's all about testing and documentation. No new resources created.

The One Thing That Makes This Click 💡

Documentation, think of it like an **insurance for your infrastructure**.

Without it, things may work, but only one person knows how, and problems turn into guesswork.

With documentation, every part of the system can be understood, rebuilt, and fixed by anyone on the team.

It preserves knowledge even when people leave and makes troubleshooting clear instead of stressful.

That's the difference between infrastructure that merely works and infrastructure that can be maintained long term.

Now let's document everything...

Phase 1: Understanding Documentation (20 min)

Why Documentation Matters

The Restaurant Recipe Analogy

Documentation is like a **written recipe in a restaurant**.

Without it, everything depends on what one person remembers, and if they leave, the knowledge is gone.

With documentation, anyone can follow the steps, learn faster, and get the same result every time.

This makes work easier to share, easier to repeat, and easier to maintain.

That's why documentation turns fragile knowledge into something reliable and reusable.

Key insight: Documentation = Preserved knowledge that enables reproducibility.

Types of Documentation

1. Architecture Documentation:

- What components exist
- How they connect
- Visual diagrams

2. Operational Documentation:

- How to deploy
- How to update
- How to troubleshoot

3. Code Documentation:

- Terraform code comments
- Variable descriptions
- Output explanations

4. Runbooks:

- Common operations
- Troubleshooting steps
- Emergency procedures

Design Checkpoint

Before moving on, make sure you can answer these in plain English:

Question	Your Answer Should Be Like...
Why document infrastructure?	"Preserve knowledge, enable reproducibility"
What should be documented?	"Architecture, deployment, operations, code"
Who is documentation for?	"Team members, future you, stakeholders"
How do you test infrastructure?	"terraform plan, validate, test changes"

Can't answer one? Go back and re-read that exercise.

All clear? Great! Let's create documentation!

Phase 2: Implementation (2.5 hours)

What We're Building

Before this task:

- Infrastructure works
- Limited documentation
- Knowledge in your head
- Hard to reproduce

After this task:

- Complete documentation
- Architecture diagrams
- Runbooks for operations
- Reproducible infrastructure
- Production-ready documentation

Step-by-Step Plan

We'll do this in 10 steps (Steps 7-10 are optional but recommended):

- Step 1:** Test Infrastructure
"Validate everything works"
- Step 2:** Create README
"Main documentation file"
- Step 3:** Create Architecture Diagram
"Visual representation of infrastructure"
- Step 4:** Create Runbooks
"Common operations guide"
- Step 5:** Document Terraform Code
"Code comments and descriptions"
- Step 6:** Configure Remote State Backend (Optional - For Production)
"Move state to S3 for production"
- Step 7:** Terraform Workspaces (Optional - For Multiple Environments)
"Manage dev/staging/prod with same code"
- Step 8:* Resource Import (Troubleshooting)
"Import existing resources into Terraform"
- Step 9:** Terraform Graph Visualization (Optional)
"Visualize resource dependencies"
- Step 10:** Pre-commit Hooks (Optional - For Code Quality)
"Automated code quality checks"

Step 1: Test Infrastructure

What we're doing: Running tests to validate infrastructure is working correctly.

Test 1: Terraform validation:

```
# Validate Terraform configuration  
terraform validate  
  
# Should return: Success! The configuration is valid.
```

Test 2: Terraform plan (dry run):

```
# Check for any unplanned changes  
terraform plan  
  
# Should show: No changes (infrastructure matches code)  
# If changes shown, review and apply if needed
```

Test 3: Verify all resources:

```
# List all Terraform resources  
terraform state list  
  
# Should show: All resources (VPC, subnets, EKS, RDS, etc.)  
  
# Verify resource counts match expectations  
terraform state list | wc -l  
  
# Should match: Expected number of resources
```

Test 4: Verify EKS cluster:

```
# Check cluster status  
kubectl cluster-info  
  
# Should show: Cluster is accessible  
  
# Check nodes
```



```
kubectl get nodes
```

```
# Should show: 2+ nodes in Ready state
```

Test 5: Verify applications:

```
# Check all Pods
```

```
kubectl get pods -n h-game --all-namespaces
```

```
# Should show: ALL Pods Running
```

```
# Test backend
```

```
kubectl port-forward service/backend-service 3001:3001 -n h-game &  
curl http://localhost:3001/api/health
```

```
# Should return: Health status
```

Test 6: Verify RDS:

```
# Check RDS status
```

```
aws rds describe-db-instances \  
  --db-instance-identifier h-game-postgres \  
  --query 'DBInstances[0].DBInstanceStatus'
```

```
# Should show: available
```

Test 7: Verify ElastiCache:

```
# Check ElastiCache status
```

```
aws elasticache describe-replication-groups \  
  --replication-group-id h-game-redis \  
  --query 'ReplicationGroups[0].Status'
```

```
# Should show: available
```

Test 8: Verify ALB:

```
# Check ALB status
ALB_ARN=$(terraform output -raw alb_arn)
aws elbv2 describe-load-balancers \
  --load-balancer-arns $ALB_ARN \
  --query 'LoadBalancers[0].State.Code'

# Should show: active

# Test ALB endpoint
ALB_URL=$(terraform output -raw alb_dns_name)
curl http://$ALB_URL/health

# Should return: Health status
```

Checkpoint - Did it work?

- ☐ All Terraform validation passed
- ☐ All resources verified
- ☐ EKS cluster working
- ☐ Applications running
- ☐ All services accessible

Infrastructure tested!

Step 2: Create README

What we're doing: Creating comprehensive README documentation.

Create README.md:

```
# H-Game -- Production Infrastructure
```

```
**Production-grade AWS infrastructure for the H-Game application, provisioned  
and managed with Terraform.**
```

```
This repository contains everything required to deploy, operate, and reproduce  
the H-Game infrastructure in a production environment.
```

```
---
```

```
## Table of Contents
```

1. Overview
2. Architecture
3. Prerequisites
4. Deployment
5. Configuration
6. Operations
7. Troubleshooting
8. Security Posture
9. Cost Management
10. Additional Resources

```
---
```

```
## 1. Overview
```

```
This infrastructure deploys a **highly available, scalable H-Game application**  
on AWS using managed services and Infrastructure as Code.
```

```
### Core Services
```

- ****Amazon EKS**** -- Managed Kubernetes cluster
- ****Amazon RDS (PostgreSQL)**** -- Persistent relational database
- ****Amazon ElastiCache (Redis)**** -- In-memory caching
- ****Application Load Balancer (ALB)**** -- Public ingress
- ****Amazon ECR**** -- Container registry

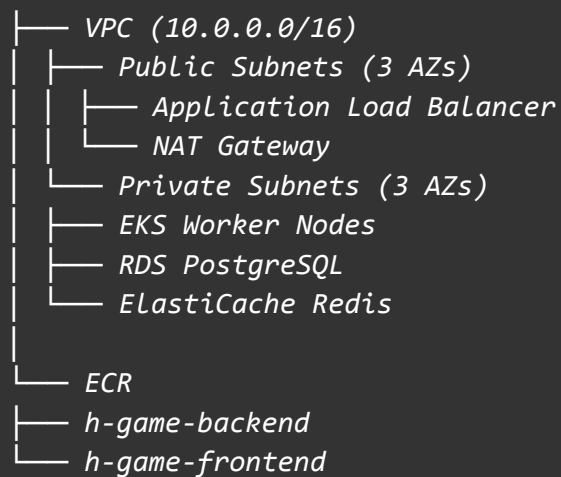
- ****Amazon VPC**** -- Isolated network with public and private subnets

****Estimated Cost:**** ~\$191-\$371 / month (depends on auto-scaling)

2. Architecture

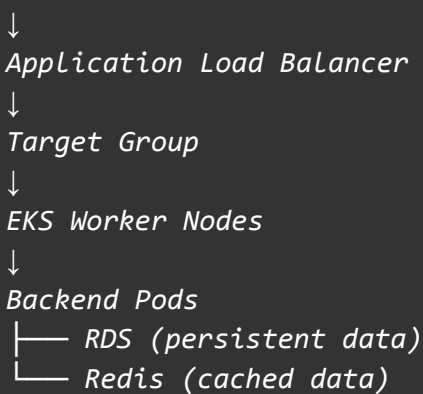
High-Level Components

AWS Account



Request Flow

Internet



Data Flow

1. Client sends request to ALB
2. ALB forwards traffic to EKS target group
3. Backend pods process requests
4. Data retrieved from RDS or Redis
5. Response returned to client

3. Prerequisites

Ensure the following tools are installed and configured:

- AWS CLI (authenticated)
- Terraform $\geq 1.6.0$
- kubectl
- Docker
- Helm (required for cluster autoscaler)

4. Deployment

Infrastructure Provisioning

```
```bash
git clone <your-repo>
cd terraform
```

```
terraform init
terraform plan
terraform apply
```

Post-Deployment

```
aws eks update-kubeconfig \
 --name h-game-prod \
 --region us-east-1
```

```
kubectl apply -f ../k8s/ -n h-game
kubectl get pods -n h-game
```

## 5. Configuration

### Variables

All configurable values are defined in `variables.tf`.

Key variables include:

`environment` -- dev / staging / prod

`region` -- AWS region

`node_instance_type` -- EKS node type

`node_min_size` -- Minimum nodes

`node_max_size` -- Maximum nodes

### Outputs

`terraform output`

Important outputs:

`vpc_id`

`eks_cluster_name`

`rds_endpoint`

`alb_dns_name`

## 6. Operations

### Common Tasks

Update Infrastructure

`terraform plan`

`terraform apply`

### Scale EKS Nodes

Update `eks-nodes.tf` scaling configuration

Re-apply Terraform

Update Application Image

```
kubectl set image deployment/backend \
 backend=$ECR_REPO:<new-tag> \
 -n h-game
```

### View Logs

```
kubectl logs -l app=backend -n h-game
```

### Runbooks

Detailed runbooks are located in `docs/runbooks/`:

Deployment

Troubleshooting

Backup & Restore

## 7. Troubleshooting

### *Pods Not Starting*

```
kubectl get pods -n h-game
kubectl describe pod <pod-name> -n h-game
kubectl logs <pod-name> -n h-game
```

### *Common causes:*

*Image pull failures*  
*Resource limits*  
*Node capacity*

### *RDS Connectivity Issues*

```
aws ec2 describe-security-groups --group-ids <sg-id>
```

```
kubectl run postgres-test \
 --image=postgres:15-alpine \
 --rm -it -- \
 psql -h $RDS_ENDPOINT
```

### *ALB Not Responding*

```
aws elbv2 describe-target-health \
 --target-group-arn $TG_ARN
```

```
aws elbv2 describe-load-balancers \
 --load-balancer-arns $ALB_ARN
```

## *8. Security Posture*

### *Implemented Controls*

#### *Infrastructure (Terraform)*

#### *Least-privilege security groups*

#### *Private subnets*

#### *HTTPS / TLS*

#### *Encryption at rest (RDS, ECR, S3)*

#### *CloudTrail auditing*

#### *VPC Flow Logs*

#### *Secrets Manager*

#### *ECR image scanning*

#### *IAM least privilege*

#### *Kubernetes*

#### *RBAC*

#### *Network Policies*

#### *Pod Security Standards*

## Secrets & ConfigMaps

### Recommended Enhancements

Service

Purpose

Priority

AWS WAF

Protect ALB

Medium

GuardDuty

Threat detection

Medium

Security Hub

Centralized security

Medium

VPC Endpoints

Cost & security

Low

## 9. Cost Management

### Monthly Cost Breakdown

Service

Cost

EKS Control Plane

~\$73

EKS Nodes

\$60-240

RDS

~\$15

ElastiCache

~\$13

ALB

~\$18

NAT Gateway

~\$11

ECR

~\$1

Total

~\$191-371

### Cost Optimization

Use Spot instances (up to 50% savings)

Auto-scaling down during low usage

Destroy infrastructure when idle



```
terraform destroy
⚠ Deletes all resources
```

#### 10. Additional Resources

[Terraform AWS Provider Documentation](#)

[Amazon EKS Documentation](#)

[Amazon RDS Documentation](#)

#### Contributors

Infrastructure designed and implemented using Terraform following DevOps best practices.

Last Updated: YYYY-MM-DD

### Checkpoint - Did it work?

- README.md created
- All sections included
- Clear and comprehensive

**README created!**

## Step 3: Create Architecture Diagram

**What we're doing:** Creating a visual architecture diagram.

**Create architecture diagram :**

```
Create docs/architecture.md
mkdir -p docs
cat > docs/architecture.md << 'EOF'
Below is a much cleaner, professional, and readable docs/architecture.md that:
Reads like real internal architecture documentation
Separates structure, flow, network, and security clearly
Keeps diagrams readable (no wall-of-text ASCII)
Is suitable for production handover, audits, and interviews
You can copy-paste this directly.
```

### # Infrastructure Architecture

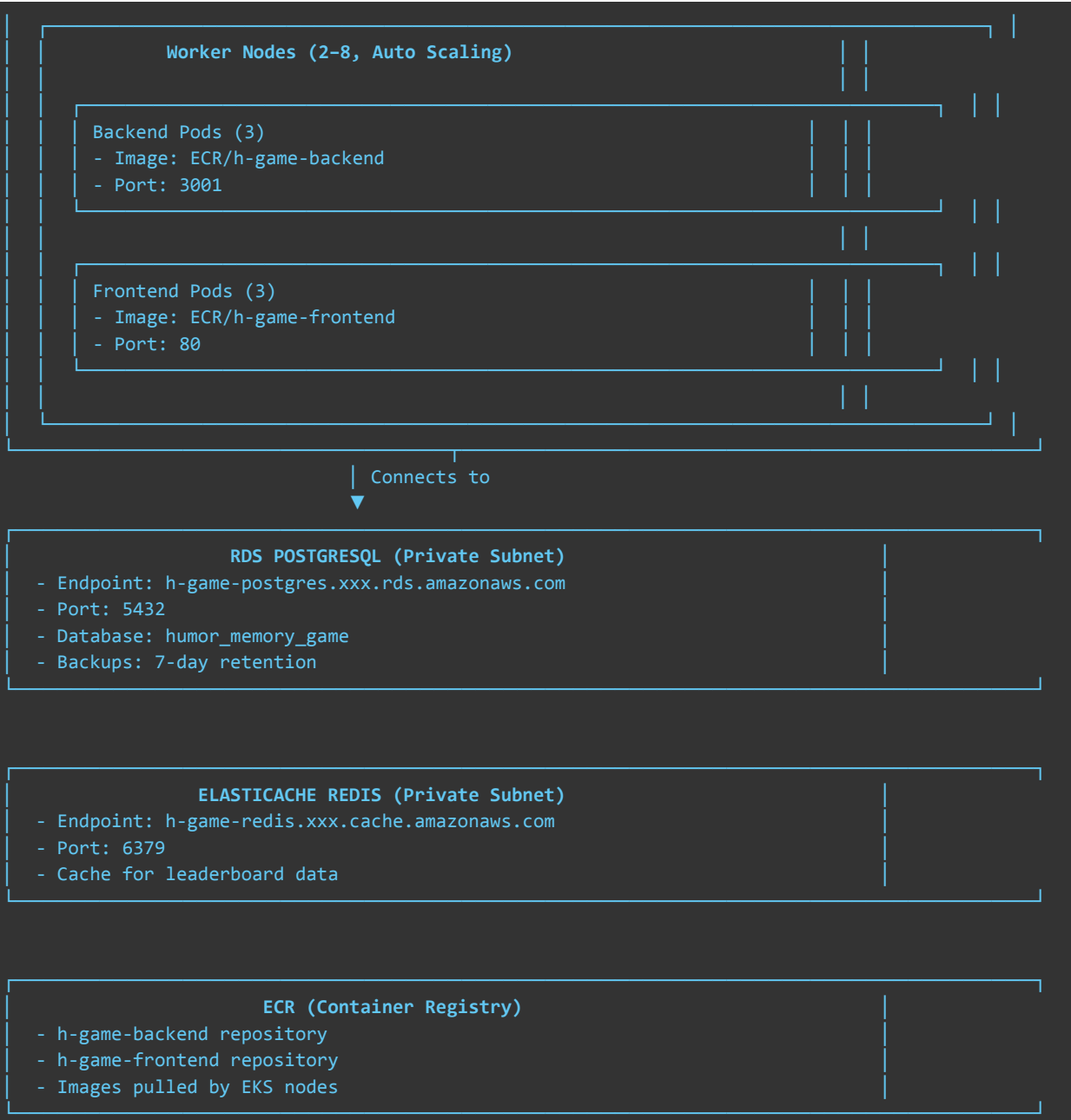
This document describes the **production architecture** for the H-Game platform.

It explains how traffic flows through the system, how components are isolated, and how security is enforced.

---

### ## 1. Architecture Overview





The H-Game platform runs on **AWS-managed services**, deployed via Terraform, with Kubernetes (EKS) as the compute layer.

### High-Level Design Goals

- High availability across multiple AZs
- Private-by-default networking
- Horizontal scalability
- Clear separation of concerns

- Minimal operational overhead

---

## ## 2. System Components

### ### External Access

- **\*\*Internet\*\***
- **\*\*Application Load Balancer (ALB)\*\***
  - Public entry point
  - Handles HTTP/HTTPS traffic
  - Terminates SSL/TLS
  - Performs health checks (`/health`)

---

### ### Compute Layer

- **\*\*Amazon EKS (Private Subnets)\*\***
  - Managed Kubernetes control plane
  - Worker nodes distributed across 3 AZs
  - Auto-scaling enabled (2-8 nodes)

### #### Workloads

- **\*\*Backend Pods\*\***
  - Image: `ECR/h-game-backend`
  - Port: `3001`
  - Handles API and business logic
- **\*\*Frontend Pods\*\***
  - Image: `ECR/h-game-frontend`
  - Port: `80`
  - Serves UI assets

---

### ### Data Layer

- **\*\*Amazon RDS (PostgreSQL)\*\***
  - Private subnet
  - Port: `5432`
  - Automated backups (7-day retention)
  - Stores persistent application data
- **\*\*Amazon ElastiCache (Redis)\*\***

- Private subnet
- Port: `6379`
- Used for caching leaderboard and session data

---

### ### Container Registry

- **\*\*Amazon ECR\*\***
- Stores backend and frontend container images
- Images pulled securely by EKS worker nodes

---

## ## 3. Request & Data Flow

### ### Request Flow

User  
↓  
Internet  
↓  
Application Load Balancer (Public Subnet)  
↓  
Target Group  
↓  
EKS Worker Node  
↓  
Backend Pod

---

### ### Data Access Flow

Backend Pod  
├→ RDS PostgreSQL (persistent data)  
├→ Redis ElastiCache (cached data)  
└→ Response returned to ALB → User

---

## ## 4. Network Architecture

### ### VPC Layout

**\*\*VPC CIDR:\*\*** `10.0.0.0/16`

#### #### Public Subnets

- `10.0.1.0/24` -- us-east-1a (ALB, NAT Gateway)
- `10.0.2.0/24` -- us-east-1b (ALB)
- `10.0.3.0/24` -- us-east-1c (ALB)

#### #### Private Subnets

- `10.0.11.0/24` -- us-east-1a (EKS, RDS, Redis)
- `10.0.12.0/24` -- us-east-1b (EKS, RDS, Redis)
- `10.0.13.0/24` -- us-east-1c (EKS, RDS, Redis)

### ### Networking Principles

- No direct internet access to workloads
- Outbound access via NAT Gateway
- All databases and caches isolated in private subnets

---

## ## 5. Security Architecture

### ### Security Group Model

Component	Allowed Ingress
ALB	80 / 443 from Internet
Backend Pods	3001 from ALB
RDS	5432 from Backend
Redis	6379 from Backend
EKS	Cluster & node internal communication

---

### ### Security Design Principles

- Least-privilege network access
- No public database endpoints
- Encrypted communication where applicable
- Clear trust boundaries between layers

---

## ## 6. High Availability & Scalability

- Multi-AZ deployment for all core services
- EKS node auto-scaling
- Stateless application pods
- Managed database failover handled by AWS

---

## ## 7. Failure Scenarios & Isolation

Failure	Impact	Mitigation
Pod crash	Minimal	Kubernetes restarts
Node failure	Minimal	Auto-scaling replaces node
AZ outage	Reduced capacity	Multi-AZ design
Cache failure	Performance degradation	App falls back to DB
DB issue	Service degradation	RDS backups & snapshots

---

## ## 8. Architecture Summary

This architecture provides:

- Secure, private-by-default infrastructure
- Clear separation between ingress, compute, and data layers
- Production-grade reliability and scalability
- Infrastructure that is easy to reason about, operate, and reproduce

---

**\*\*Document Owner:\*\*** Infrastructure / DevOps

**\*\*Last Updated:\*\*** YYYY-MM-DD

EOF

## Checkpoint - Did it work?

- ☐ Architecture diagram created
- ☐ Network diagram included
- ☐ Security diagram included

☐ Data flow documented

**Architecture documented!**

## Step 4: Create Runbooks

**What we're doing:** Creating runbooks for common operations.

**Create runbooks directory:**

```
mkdir -p docs/runbooks
```

**Create deployment runbook:**

```
cat > docs/runbooks/deployment.md << 'EOF'
Deployment Runbook

Prerequisites
- AWS CLI configured
- Terraform installed
- kubectl configured

Steps

1. Review changes:
  ```bash
  terraform plan

  Apply changes:
  terraform apply

  Verify deployment:
  terraform output
  kubectl get nodes
  kubectl get pods -n h-game

  Test application:
  ALB_URL=$(terraform output -raw alb_dns_name)
```



```
curl http://$ALB_URL/health
```

```
EOF
```

Create troubleshooting runbook:

```
cat > docs/runbooks/troubleshooting.md << 'EOF'
# Troubleshooting Runbook

## Pods Not Starting

Check Pod status:
```
kubectl get pods -n h-game
kubectl describe pod <pod-name> -n h-game
kubectl logs <pod-name> -n h-game

Common issues:
Image pull errors → Check ECR access
Resource limits → Check resource requests
Node capacity → Check node resources
Can't Connect to RDS
Check security groups:
aws ec2 describe-security-groups --group-ids <sg-id>

Test connection:
kubectl run postgres-test --image=postgres:15-alpine --rm -it -- psql -h
$RDS_ENDPOINT

ALB Not Responding
Check target health:
TG_ARN=$(terraform output -raw target_group_arn)
aws elbv2 describe-target-health --target-group-arn $TG_ARN

Check ALB status:
ALB_ARN=$(terraform output -raw alb_arn)
aws elbv2 describe-load-balancers --load-balancer-arns $ALB_ARN

EOF
```

## Checkpoint - Did it work?

- ☐ Runbooks created
- ☐ Deployment runbook included
- ☐ Troubleshooting runbook included

Runbooks created!

## Step 5: Document Terraform Code

**What we're doing:** Adding comments and descriptions to Terraform code.

**Review and add comments:**

```
Example: Add comments to main files
Most files already have comments, but verify:

Check vpc.tf has comments
head -20 vpc.tf

Check eks-cluster.tf has comments
head -20 eks-cluster.tf

Add variable descriptions if missing
```

```
cat > variables.tf << 'EOF'
variable "environment" {
 description = "Environment name (dev, staging, prod)"
 type = string
 default = "production"
}

variable "region" {
 description = "AWS region"
 type = string
 default = "us-east-1"
}
EOF
```

### Checkpoint - Did it work?

- ☐ Code comments reviewed
- ☐ Variable descriptions added
- ☐ Code is well-documented

### Code documented!

## Step 6: Configure Remote State Backend

 **Note:** Remote state is optional for learning but **required for production**. This step shows you how to set it up when you're ready.

### Why remote state?

- State file stored in S3 (backed up, versioned)
- State locking with DynamoDB (prevents concurrent modifications)
- Team collaboration (multiple people can work on same infrastructure)
- CI/CD integration (automated deployments)
- Security (state encrypted at rest)

### Current setup (local state):

```
State stored locally
terraform apply
Creates: terraform.tfstate (on your laptop)
```

### Production setup (remote state):

```
State stored in S3
terraform apply
Creates: terraform.tfstate in S3 bucket
```

### Important:

- Remote state requires S3 bucket and DynamoDB table to exist FIRST
- You'll need to migrate existing state from local to remote
- This is a one-time setup per project

## Step 6: Remote State Best Practices (Already Configured in T1)

**Note:** If you followed T1 Step 0, you already have the remote state configured! This section covers best practices and troubleshooting.

### Already configured in T1 Step 0:

- S3 bucket for state storage
- DynamoDB table for state locking
- Backend configuration in backend.tf

### This section covers:

- Migrating from local to remote state (if you skipped T1 Step 0)
- Remote state best practices
- Troubleshooting state issues

### Why Remote State:

- **Team collaboration:** Multiple people can work on same infrastructure
- **State locking:** Prevents conflicts (DynamoDB)
- **Backup:** State stored in S3 (won't be lost)
- **Production standard:** All production systems use remote state

### Current state (local):

```
terraform.tfstate (on your local machine)
├─ Can be lost if machine crashes
├─ Can't collaborate with team
├─ No state locking (conflicts possible)
└─ Not production-ready
```

### Remote state (S3 + DynamoDB):

```
terraform.tfstate (in S3)
├─ Backed up in S3
├─ Team can access
├─ DynamoDB locks state (prevents conflicts)
└─ Production-ready
```

## Create S3 bucket for state:

```
Create S3 bucket for Terraform state
Note: S3 bucket names must be globally unique
Replace 'yourname' with your name or initials to make it unique
TIMESTAMP=$(date +%s)
STATE_BUCKET="h-game-tfstate-yourname-${TIMESTAMP}"

echo "Your state bucket: $STATE_BUCKET"
Save this - you'll need it later!

aws s3 mb s3://$STATE_BUCKET --region us-east-1

Enable versioning (for state history)
aws s3api put-bucket-versioning \
 --bucket $STATE_BUCKET \
 --versioning-configuration Status=Enabled

Enable encryption
aws s3api put-bucket-encryption \
 --bucket $STATE_BUCKET \
 --server-side-encryption-configuration '{
 "Rules": [{
 "ApplyServerSideEncryptionByDefault": {
 "SSEAlgorithm": "AES256"
 }
 }]
 }'

echo "State bucket created: $STATE_BUCKET"
```

## Configure Terraform backend:

```
Create backend configuration file
cat > backend.tf << EOF
=====
TERRAFORM REMOTE STATE BACKEND
=====
Stores state in S3 (backed up, team accessible)
```

```

Uses DynamoDB for state locking (prevents conflicts)

terraform {
 backend "s3" {
 bucket = "REPLACE_WITH_YOUR_BUCKET_NAME" # ⚠️ Replace with
your actual bucket name!
 key = "terraform.tfstate" # State file name
 region = "us-east-1" # AWS region
 dynamodb_table = "h-game-terraform-state-lock" # Lock table
 encrypt = true # Encrypt state at
rest
 }
}
EOF

Note: Replace bucket name with your actual bucket name
The bucket name must be globally unique

```

## Migrate state to S3 (Only if you skipped T1 Step 0):

**⚠️ Note:** If you completed T1 Step 0, you already have remote state configured and don't need to migrate. This section is only for students who started with local state and want to migrate to remote state.

```

Step 1: Create S3 bucket and DynamoDB table (if not already done)
Follow T1 Step 0 instructions to create these resources first

Step 2: Configure backend.tf (if not already done)
Follow T1 Step 0.4 to create backend.tf

Step 3: Initialize backend (will prompt to migrate state)
terraform init -migrate-state

Expected output:
Do you want to copy existing state to the new backend?
Enter "yes" to copy and a new "backend" configuration to the
existing "local" backend.

```

```
Type "yes" to migrate

Verify state migrated
terraform state list

Should show: ALL resources (state now in S3!)
```

### Verify remote state:

```
Check state is in S3
aws s3 ls s3://$STATE_BUCKET/

Should show: terraform.tfstate

Test state locking
Try running terraform plan in two terminals simultaneously
Second terminal should wait (locked by first)
```

### Update backend configuration in code:

```
Update backend.tf with actual bucket name
Edit backend.tf and replace placeholder with your bucket name

Example:
bucket = "h-game-terraform-state-1234567890"
```

### Checkpoint - Did it work?

- ☐ S3 bucket created for state
- ☐ DynamoDB table created for locking
- ☐ State migrated to S3
- ☐ State locking working

### Remote state configured!



## Step 7: Terraform Workspaces

**What we're doing:** Using Terraform workspaces to manage multiple environments (dev, staging, production) with the same codebase.

### Why workspaces?

- Same code for different environments
- Separate state files per environment
- Easy environment switching
- Production-ready pattern

### Create workspaces:

```
List current workspace (default: default)
terraform workspace show

Create dev workspace
terraform workspace new dev

Create staging workspace
terraform workspace new staging

Create production workspace
terraform workspace new production

Switch between workspaces
terraform workspace select dev
terraform workspace select production
```

### Use workspace in configuration:

```
Update locals.tf to use workspace
cat >> locals.tf << 'EOF'

Environment-specific values based on workspace
locals {
 # Get current workspace
 current_workspace = terraform.workspace

 # Environment-specific instance counts
```

```

instance_counts = {
 dev = 1
 staging = 2
 production = 3
}

Use workspace-specific count
node_count = lookup(local.instance_counts, local.current_workspace,
2)

Environment-specific tags
workspace_tags = {
 dev = { Environment = "development" }
 staging = { Environment = "staging" }
 production = { Environment = "production" }
}
}
EOF

```

### Apply to different environments:

```

Deploy to dev
terraform workspace select dev
terraform apply

Deploy to production
terraform workspace select production
terraform apply

```

**Note:** Workspaces are useful for managing multiple environments, but for learning, the default workspace is sufficient.

### Workspaces configured!

## Step 8: Resource Import (Troubleshooting)

**What we're doing:** Learning how to import existing AWS resources into Terraform state.

### Why import?

- Bring existing resources under Terraform management
- Migrate from manual AWS Console setup
- Recover from lost state file
- Add existing resources to infrastructure code

### Import existing VPC:

```
Get VPC ID from AWS Console or CLI
VPC_ID=$(aws ec2 describe-vpcs --filters
"Name=tag:Name,Values=h-game-vpc" --query 'Vpcs[0].VpcId' --output
text)

Import into Terraform state
terraform import aws_vpc.main $VPC_ID

Should show: aws_vpc.main: Imported successfully
```

### Import existing RDS:

```
Import RDS instance
terraform import aws_db_instance.postgres h-game-postgres

Should show: aws_db_instance.postgres: Imported successfully
```

### Import existing EKS cluster:

```
Import EKS cluster
terraform import aws_eks_cluster.main h-game-prod

Should show: aws_eks_cluster.main: Imported successfully
```

### **After import:**

```
Refresh state to match actual infrastructure
terraform refresh

Plan to see what Terraform wants to change
terraform plan

Apply to sync Terraform code with actual state
terraform apply
```

**Note:** Import is only needed if you have existing resources not managed by Terraform. For this course, all resources are created fresh.

**Import strategy documented!**

## Step 9: Terraform Graph Visualization (Optional)

**What we're doing:** Generating visual dependency graphs of your infrastructure.

### Why graph?

- ☐ Visualize resource dependencies
- ☐ Understand infrastructure relationships
- ☐ Debug dependency issues
- ☐ Document architecture

### Generate dependency graph:

```
Install Graphviz (if not installed)
macOS: brew install graphviz
Ubuntu: sudo apt-get install graphviz

Generate graph
terraform graph | dot -Tpng > infrastructure-graph.png

View graph
open infrastructure-graph.png # macOS
Or: xdg-open infrastructure-graph.png # Linux
```

### Graph shows:

- Resource dependencies (arrows show relationships)
- Resource types (VPC, Subnet, EKS, etc.)
- Creation order (top to bottom)

### Interactive visualization (optional):

```
Install Blast Radius (interactive graph viewer)
pip install blastradius

Generate interactive graph
blast-radius --serve .

Opens in browser at http://localhost:5000
```

 **Note:** Graph visualization is optional but helpful for understanding complex infrastructure.

## Graph visualization configured!

### Step 10: Pre-commit Hooks (Optional - For Code Quality)

**What we're doing:** Setting up automated checks before committing Terraform code to Git.

#### Why pre-commit hooks?

- ☐ Automatically format code before commit
- ☐ Validate syntax before commit
- ☐ Generate documentation automatically
- ☐ Catch errors before they reach repository

#### Install pre-commit:

```
Install pre-commit
pip install pre-commit
```

#### Create .pre-commit-config.yaml

```
cat > .pre-commit-config.yaml << 'EOF'
repos:
 - repo: https://github.com/antonbabenko/pre-commit-terraform
 rev: v1.83.0
 hooks:
 - id: terraform_fmt # Auto-format code
 - id: terraform_validate # Validate syntax
 - id: terraform_docs # Generate docs
 - id: terraform_tflint # Lint code
 - id: terraform_validate_with_variables # Validate with
variables
EOF
```

```
Install hooks
pre-commit install

Test hooks
```

```
pre-commit run --all-files
```

### How it works:

```
When you commit:
git add .
git commit -m "Add RDS configuration"

Pre-commit automatically runs:
terraform fmt (formats code)
terraform validate (checks syntax)
terraform_docs (updates README)
terraform_tflint (lints code)

If all pass: Commit succeeds
If any fail: Commit blocked, shows errors
```

 **Note:** Pre-commit hooks are optional but recommended for production code quality.

**Pre-commit hooks configured!**

## Step 11: Validate Reproducibility

**What we're doing:** Testing that infrastructure can be recreated from scratch.

**Test reproducibility:**

```
Create a test workspace
terraform workspace new test-reproducibility

Switch to test workspace
terraform workspace select test-reproducibility

Review what would be created
terraform plan

Should show: All resources would be created
(This validates configuration is complete)

Switch back to default
terraform workspace select default
terraform workspace delete test-reproducibility
```

**Create reproducibility checklist:**

```
cat > docs/REPRODUCIBILITY.md << 'EOF'
Reproducibility Checklist

Prerequisites
- [] AWS account with appropriate permissions
- [] Terraform >= 1.6.0 installed
- [] AWS CLI configured
- [] kubectl installed

Steps to Reproduce

1. Clone repository:
   ```
   git clone <repo>
   cd terraform
```


Configure remote state (if not already done):

```
# Create S3 bucket for state
aws s3 mb s3://h-game-terraform-state-<unique-id>

# Create DynamoDB table for locking
aws dynamodb create-table \
  --table-name terraform-state-lock \
  --attribute-definitions AttributeName=LockID,AttributeType=S \
  --key-schema AttributeName=LockID,KeyType=HASH \
  --billing-mode PAY_PER_REQUEST

# Update backend.tf with bucket name
# Then initialize
terraform init
```

Initialize Terraform:

```
terraform init
```

Review configuration:

```
terraform plan
```

Deploy infrastructure:

```
terraform apply
```

Configure kubectl:

```
aws eks update-kubeconfig --name h-game-prod
```

Deploy applications:

```
kubectl apply -f ../k8s/ -n h-game
```

Verification

After deployment, verify:

- ☐ EKS cluster is accessible
- ☐ Nodes are running
- ☐ Pods are running
- ☐ RDS is accessible
- ☐ ElastiCache is accessible
- ☐ ALB is responding
- ☐ Remote state configured (S3 + DynamoDB) EOF

Checkpoint - Did it work?

- Reproducibility tested
- Checklist created
- Infrastructure can be recreated

Reproducibility validated!

Phase 3: Validation & Consolidation (10 min)

Final Validation

Run complete test suite:

```
# 1. Terraform validation
terraform validate && echo "✅ Terraform valid"

# 2. Infrastructure test
terraform plan && echo "✅ No unplanned changes"

# 3. EKS test
kubectl cluster-info && echo "✅ EKS accessible"

# 4. Application test
kubectl get pods -n h-game && echo "✅ Pods running"

# 5. Documentation test
test -f README.md && echo "✅ README exists"
test -f docs/architecture.md && echo "✅ Architecture doc exists"
test -f docs/runbooks/deployment.md && echo "✅ Runbooks exist"
```

Checkpoint - Did it work?

- ☐ All tests passed
- ☐ Documentation complete
- ☐ Infrastructure validated

Complete infrastructure ready!

Checklist: Did You Complete Everything?

Before moving on, check these boxes:

Testing Phase

- ☐ Infrastructure tested (all components)
- ☐ Terraform validation passed
- ☐ Applications verified
- ☐ All services accessible

Documentation Phase

- ☐ README.md created
- ☐ Architecture diagram created
- ☐ Runbooks created
- ☐ Code documented
- ☐ Remote state backend configured (S3 + DynamoDB)
- ☐ Reproducibility validated

Final Validation

- ☐ Complete test suite passed
- ☐ Documentation reviewed
- ☐ Infrastructure production-ready

Git

- ☐ All documentation committed
- ☐ README.md committed
- ☐ Runbooks committed
- ☐ Architecture diagrams committed

Key Takeaways

The Big Picture

What you learned in this task:

| | |
|--|--|
| INFRASTRUCTURE TESTED & DOCUMENTED | |
| - Complete test suite | |
| - Comprehensive documentation | |
| - Architecture diagrams | |
| - Runbooks for operations | |
| - Reproducibility validated | |
| Production-ready, maintainable infrastructure! | |

Critical Concepts

1. Testing:

- Validate infrastructure works
- Test all components
- Verify applications
- Check connectivity

2. Documentation:

- Preserve knowledge
- Enable reproducibility
- Guide operations
- Support troubleshooting

3. Reproducibility:

- Infrastructure can be recreated
- Configuration is complete
- Documentation is sufficient
- Knowledge is preserved

4. Production Readiness:

- Tested infrastructure
- Complete documentation
- Operational runbooks
- Maintainable codebase

Congratulations!

You've completed Section 5: Terraform & Infrastructure as Code!

You now have:

- Complete production AWS infrastructure
- Managed services (EKS, RDS, ElastiCache)
- Production-grade ingress (ALB)
- Private container registry (ECR)
- Auto-scaling configured
- Remote state backend (S3 + DynamoDB)
- Complete documentation
- Reproducible infrastructure

What you've accomplished:

- Built production-grade infrastructure with Terraform
- Migrated from self-managed to managed services
- Implemented auto-scaling and cost optimization
- Created comprehensive documentation
- Validated production readiness

Next Steps:

- Continue to Section 6: DevSecOps
- Or review and optimize your infrastructure
- Or deploy to production!

You're now ready for production infrastructure!

Section 5 Complete!